



Escuela Politécnica

UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA
INFORMÁTICA EN INGENIERÍA DEL
SOFTWARE

TRABAJO FIN DE GRADO

**Infraestructura de servicios para el análisis
de cultivos**

Álvaro Mendo Martín

2025/2026, 2026



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA
INFORMÁTICA EN INGENIERÍA DEL
SOFTWARE

TRABAJO FIN DE GRADO

**Infraestructura de servicios para el análisis
de cultivos**

Autor: Álvaro Mendo Martín

Tutor: Pedro José Clemente Martín

Agradecimientos

Quisiera expresa mi más sincero agradecimiento a todas aquellas personas que han contribuido al desarrollo del proyecto, haciendo posible su progreso y aportando su apoyo a lo largo de todo el proceso. Gracias a su colaboración, este trabajo ha podido materializarse como una propuesta orientada a aportar valor al sector agrícola mediante el uso de soluciones tecnológicas que podrán mejorar la gestión y rendimiento de las explotaciones agrícolas.

Quisiera agradecer a mi tutor, D. Pedro José Clemente Martín, y a sus compañeros Pablo Alonso Portillo y Julio Domínguez Arjona por su constante disposición para ayudarme, así como por sus valiosos conocimientos que me ha transmitido para poder conseguir el resultado esperado.

No puedo olvidar destacar la tesis de Alan Chavez, publicada en mayo de 2025, que me abrió numerosos frentes de ideas para el desarrollo de este proyecto.

Un especial agradecimiento a mi familia, en especial a mi madre, mi padre y mi hermano, grandes entendedores del mundo agrícola y que, gracias a sus experiencias en este sector, pude tener más contexto sobre en qué debía enfocarme a la hora de desarrollar mi proyecto. Su apoyo constante me ha ayudado a no rendirme y ser constante día a día.

El resultado de este proyecto refleja todo el esfuerzo implicado y no podría haber sido posible sin la determinación y conocimientos que me han aportado todas estas personas mencionadas.

Gracias a todos.

Resumen

El sector agropecuario constituye uno de los pilares fundamentales de la economía extremeña y española, siendo la agricultura una actividad clave para el abastecimiento de mercados nacionales e internacionales. Sin embargo, la alteración imprevisible de los patrones climáticos, especialmente en variables como temperatura, precipitaciones y humedad, incrementa la necesidad de herramientas software capaces de prevenir riesgos que afectan al rendimiento de las cosechas, como heladas y plagas.

El objetivo de este Trabajo de Fin de Grado es el diseño e implementación de la infraestructura servidora de un sistema software orientado a la prevención de riesgos asociados a heladas y plagas en cultivos indicados por el usuario. Para ello, la solución se apoya en datos meteorológicos públicos, previsiones climáticas e históricos de sensores proporcionados por el software DTAgro.

El sistema facilita la monitorización de cultivos y la toma de decisiones frente a situaciones climáticas adversas, contribuyendo a una gestión más eficiente de las explotaciones agrícolas.

En cuanto al desarrollo, se ha utilizado principalmente *Python* junto con el microframework ***Flask*** para los servicios de tratamiento de datos y predicción, así como ***Spring Boot*** para la implementación de servicios externos complementarios. Además, el uso de un Protocolo de Contexto para Modelos (*Model Context Protocol, MCP*) ha permitido la interacción de agentes inteligentes externos con funcionalidades propias del sistema.

Finalmente, la adopción de una arquitectura orientada a microservicios junto con el despliegue en contenedores ha permitido mantener un entorno controlado tanto en desarrollo como en producción.

Como resultado, la solución permite optimizar recursos, anticipar riesgos climáticos y

proporcionar a los agricultores una base para la planificación y gestión diaria de cultivos.

Palabras clave: Monitorización Agrícola, Prevención de Riesgos Climáticos, Microservicios, Contenedorización, Model Context Protocol (MCP).

Abstract

The agricultural sector is one of cornerstones of the economies of Extremadura and Spain, with agriculture playing a key role in supplying national and international markets. However, unpredictable changes in weather patterns, particularly in variables such as temperature, precipitation, and humidity, increases the need for software tools capable of preventing risks that affects crop yields, such as frost and plagues.

The objective of this Final Degree Project leads to the design and implementation of the server infrastructure for a software system aimed to prevent these risks on farms. To this end, the solution relies on public meteorological data, weather forecasts, and historical sensor data provided by the DTAgro software.

Furthermore, on the development side of this projects, it has been used *Python* as the main programming language, using its ***Flask***'s microframework for data management services and predictors, as well as ***Spring Boot*** for the implementation of complementaries external services. Also, the use of a Model Context Protocol (MCP) allowed the interaction between intelligent agents and some functionalities of the main project.

Finally, the microservices' architecture implemented with the deployment side using containers allowed the mantenance of a controlled environment for development and production. The system facilities crop monitoring and decision-making in the face of adverse weather conditions, contributing to more efficient management of agricultural operations.

Keywords: Agricultural Monitoring, Agricultural's climate risks prevention, Microservices, Contenerization, Model Context Protocol (MCP).

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Objetivos	2
1.3. Planificación del Proyecto	4
2. Estado del Arte	9
2.1. Conceptos Básicos	9
2.2. Técnicas actuales de mitigación frente a riesgos climáticos	13
2.3. Tecnologías relevantes para la gestión agrícola	14
2.3.1. Maher App [1]	14
2.3.2. Eco Solutions - Riego Predictivo [2]	17
2.3.3. Irristrat [3]	20
2.3.4. Conclusiones y aportaciones adicionales	22
3. Propuesta de desarrollo	26
4. Requisitos	28
4.1. Metodología de identificación y clasificación de requisitos	28
4.1.1. Clasificación de requisitos	28
4.1.2. Criterios de documentación	29
4.1.3. Plantilla de especificación	29
4.2. Identificación y especificación de requisitos	30
4.2.1. Requisitos Funcionales (RF)	30
4.2.2. Requisitos No Funcionales (RNF)	33
5. Diseño y modelado del sistema	35
5.1. Diagramas de contexto y componentes	35

5.2.	Diagrama de casos de uso	38
5.3.	Diagramas de secuencia	39
5.3.1.	Genérico	39
5.3.2.	Heladas Observadas	40
5.3.3.	Heladas Futuras	43
5.3.4.	Plagas Calculadas	44
5.3.5.	Plagas Estimadas	46
5.4.	Modelo de datos	48
5.4.1.	Descripción Tablas Características	50
5.5.	Diseño de la arquitectura del sistema	53
5.6.	Software de diseño utilizado	54
6.	Implementación y desarrollo	56
6.1.	Tecnologías y herramientas utilizadas	56
6.2.	Arquitectura de la implementación	67
6.3.	Desarrollo de los servicios del sistema	68
6.3.1.	Servicios Externos	68
6.3.2.	Servicio de datos	70
6.3.3.	Servicio de ingesta de datos meteorológicos	84
6.3.4.	Servicio IA	86
6.3.5.	Servicios Predictores	87
6.4.	Integración del sistema	95
6.4.1.	Gestión de retardos en las Peticiones entre Servicios	96
6.4.2.	Flujos Asíncronos con el Servicio de IA	99
6.4.3.	Monitorización y Trazabilidad con Logs Estructurados	99
6.4.4.	Seguridad Centralizada	101
6.5.	Despliegue y ejecución	102
6.5.1.	Organización de código fuente y control de versiones	103
6.5.2.	Orquestador general: <code>agro_predict_orchestator</code>	103
6.5.3.	Despliegue en Entorno Cloud	106

6.5.4. Automatización DevOps	107
6.5.5. Uso de Infraestructura como Código (IaC)	115
6.6. Resultados obtenidos	118
6.6.1. Validación del Servicio Predictor de Heladas	119
6.6.2. Validación del Servicio Predictor de Plagas	135
6.6.3. Conclusiones de la validación	144
6.7. Problemas encontrados y soluciones adoptadas	145
6.7.1. Dependencia de datos	145
6.7.2. Ingesta de datos en producción	145
6.7.3. Alcance de la fase de desarrollo	146
6.7.4. Tratamiento de tareas asíncronas en producción	146
6.7.5. Obtención de datos climáticos para predicciones estimadas de plagas	149
7. Conclusiones y trabajos futuros	151
7.1. Declaración de uso de Inteligencia Artificial	151
7.2. Trabajos futuros	158
Anexos	161
A. Especificación Requisitos del Sistema	162
A.1. Requisitos Funcionales	162
A.2. Requisitos No Funcionales	172
B. Manual de instalación	175
B.1. Pasos previos	175
B.2. Proceso de ejecución	176
B.2.1. Insertar fichero env en el repositorio clonado	176
B.2.2. Entorno IA	177
B.2.3. Carga inicial de datos	177
B.3. Contribuciones Futuras	177
B.3.1. <code>data-service</code>	178

B.3.2. climate_risks_predictor	180
B.3.3. mcp-ia-service	181
B.3.4. Aemet	182
B.3.5. SiAR	184
B.3.6. ITACyL	185
B.3.7. agro-predict-orchestator	186
C. Manual de uso	187
C.1. Requisitos previos de uso	187
C.1.1. Servicios desplegados en máquina virtual	187
C.2. Perfiles de usuario y casos de uso	189
C.2.1. Usuario agrícola	189
C.2.2. Aplicativo software	191
C.2.3. Usuario administrador	191
C.3. Flujo de trabajo completo	197
Bibliografía	198

Índice de tablas

2.1. Comparativa de soluciones de gestión y predicción de riego agrícola	23
4.1. Plantilla definición de requisitos <i>ISO/IEC/IEE 29148</i>	29
4.2. Requisitos Funcionales de Agro-Predict	30
4.3. Requisitos No Funcionales de Agro-Predict	33
6.1. Modelos de horas de frío registrados en el sistema	77
6.2. Tabla de etapas fenológicas registradas	77
C.1. Recursos disponibles	195

Índice de figuras

1.1. Nomenclatura UP	6
1.2. Planificación Temporal <i>Project Libre</i> , 1ª parte	7
1.3. Planificación Temporal <i>Project Libre</i> , 2ª parte	8
1.4. Planificación Temporal <i>Project Libre</i> , 3ª parte	8
2.1. Representación Visual de Heladas Blancas Fuente: [4]	12
2.2. Representación Visual de Heladas Negras Fuente: [5]	13
2.3. Maher Mobile UI	16
2.4. Eco Solutions UI	17
2.5. Modelo WRF	19
2.6. Irristrat Mobile UI	21
5.1. Diagrama de Contexto	36
5.2. Diagrama de Componentes	37
5.3. Diagrama de Casos Uso	38
5.4. Diagrama Secuencia Genérico	40
5.5. Diagrama Secuencia Heladas Observadas	41
5.6. Diagrama Secuencia Uso <i>Celery</i>	42
5.7. Diagrama Secuencia Heladas Futuras	43
5.8. Flujo asíncrono agente inteligente	44
5.9. Diagrama Secuencia Plagas Calculadas	45
5.10. Diagrama Secuencia Plagas Estimadas	47
5.11. Modelo E/R	49
5.12. Arquitectura Sistema	54
6.1. Extracto de Informe PDF	59
6.2. Firma Digital con certificado PKCS#12	60
6.3. Broker RabbitMQ	61

6.4. Administrador Web <i>RabbitMQ</i>	62
6.5. Panel Admin Keycloak	66
6.6. Flujo de acciones <i>Celery</i>	67
6.7. Flujo de acciones servicio ingesta	85
6.8. Flujo de acciones servicio IA	87
6.9. Flujo de acciones servicio heladas	90
6.10. Flujo de acciones servicio plagas	94
6.11. Representación Gráfica Flujo CI/CD	115
6.12. Despliegue Componentes Azure	118
6.13. Predicción Heladas Tabla	122
6.14. Predicción Heladas Tabla	123
6.15. Predicción Heladas Tabla	124
6.16. Predicción Heladas Tabla	125
6.17. Predicción Heladas Gráfico	126
6.18. Contexto Cálculo Heladas Observadas	130
6.19. Resultados Generales Heladas Observadas	131
6.20. Registro Tipos Heladas	132
6.21. Gráfica Tipos Heladas	133
6.22. Balance resultante Heladas Observadas	134
6.23. Fragmento tabla predicción plagas calculadas	137
6.24. Fragmento predicción plagas estimadas	141
6.25. Fragmento predicción plagas estimadas	142
6.26. Fragmento predicción plagas estimadas	143
6.27. Fragmento predicción plagas estimadas	144

Índice de listados

6.1. Herramienta - Procesado respuesta IA	62
6.2. Instrucción - Definición comportamiento IA	63
6.3. Ejemplo de Wrapper para reutilizar código	69
6.4. Fragmento predicción riesgo heladas histórica sobre Lauranne (Almendo) .	76
6.5. Asociación Cultivo-Plaga	80
6.6. Implementación del patrón Circuit Breaker	96
6.7. Excepción personalizada para la API	97
6.8. Manejador global de errores en Flask	98
6.9. Fragmento del decorador de logging estructurado	100
6.10. Fragmento del decorador validación token	101
6.11. Configuración resumen <code>compose.yml</code> para <code>no-gpu</code>	104
6.12. Fragmento de configuración para Integración Continua (CI)	108
6.13. Despliegue de cambios en <i>Docker</i> (CD) microservicios	111
6.14. Actualización de etiquetas (CD) orquestador	113
6.15. Cuerpo de la petición <code>/heladas/futuras/provincial</code>	119
6.16. Fragmento de respuesta JSON del predictor de heladas futuras	119
6.17. Cuerpo de la petición <code>/heladas/observadas/provincial</code>	127
6.18. Fragmento de respuesta JSON del predictor de heladas observadas	127
6.19. Fragmento de respuesta JSON de <code>/plagas/calculadas</code>	135
6.20. Cuerpo de la petición <code>/plagas/estimadas</code>	138
6.21. Fragmento de respuesta JSON de <code>/plagas/estimadas</code>	138
6.22. Configuración volumen <code>compose.yml</code> para <code>rabbitmq</code>	147
6.23. Configuración <i>celery</i> para entorno producción	148
6.24. Configuración actualizada de <code>datos_sensores</code>	150
7.1. Fragmento Contexto Repositorio XML	155
C.1. Ejemplo de registro DSL para <i>Drosophila suzukii</i> en cerezo.	193

C.2. Múltiples ventanas temporales en el registro de plagas	195
---	-----

Capítulo 1: Introducción

Los impactos del cambio climático arremeten cada año con más fuerza en el sector agrario, haciendo que muchos agricultores con huertos de autoconsumo y empresas con fincas dedicadas a la explotación agraria se vean continuamente con la incertidumbre sobre la rentabilidad de sus cosechas y la posibilidad de obtener beneficios o asumir pérdidas económicas significativas.

Estos sucesos se pueden observar en mayor medida sobre la Comunidad Autónoma de Extremadura. Acontecimientos climáticos imprevisibles, como heladas tardías, sequías prolongadas o aparición de plagas en estaciones del año inusuales causan estragos sobre plantaciones y árboles frutales en momentos de cosecha, obligando a los responsables agrarios de contar con aseguradoras como **Hacex Seguros** [6] o **Agroseguros**, para cubrir año tras año las pérdidas ocasionadas [7].

Extremadura constituye uno de los pilares económicos más relevantes de España en el ámbito de la producción y exportación agrícola, por lo que resulta fundamental minimizar al máximo las pérdidas derivadas de estos fenómenos y garantizar la sostenibilidad y competitividad del sector agrario [8].

1.1. Contexto

El sector agrario ha mantenido y mantiene metodologías de trabajo basadas en la **experiencia** y en **procesos manuales**. Aunque en las últimas décadas se ha apostado por la inversión en mejoras respecto a la maquinaria utilizada y tecnologías aplicada al campo. Muchas decisiones relacionadas con la planificación y prevención ante riesgos climatológicos continúan dependiendo en gran medida de la experiencia personal y criterio humano, generando **imprecisiones** [9].

La gestión de los riesgos climáticos continúa siendo uno de los principales desafíos del

sector agrario, debido a la incertidumbre inherente a los fenómenos meteorológicos y al impacto del cambio climático, que genera alteraciones constantes en los patrones de temperatura, precipitaciones y humedad. La anticipación de eventos adversos, como heladas, la optimización de los sistemas de riego o el control de plagas, requiere cada vez una mayor precisión y capacidad de análisis sobre datos históricos y predictivos.

En este contexto de transformación digital y evolución de los sistemas inteligentes, la incorporación de herramientas basadas en inteligencia artificial y modelos predictivos se presenta como una solución especialmente prometedora para el sector agrícola. Estas tecnologías permiten anticipar escenarios futuros a partir del análisis de datos históricos, facilitando una toma de decisiones más eficiente, planificada y orientada a la prevención de pérdidas en las explotaciones agrarias [10].

Dentro de este movimiento de digitalización surge **DTAgro** [11], un proyecto orientado al desarrollo de una plataforma de monitorización y análisis para explotaciones agrarias, basada principalmente en la recopilación de datos procedentes de sensores LoRaWAN [12]. Estos dispositivos desplegados en parcelas de campo permiten obtener información en tiempo real sobre variables relevantes del entorno agrícola, como temperatura, humedad, estado del suelo o condiciones ambientales que inferen en las hojas de los cultivos (humedad foliar), proporcionando una base consistente de datos para la supervisión continua de cultivos y la toma de decisiones fundamentadas.

1.2. Objetivos

En este apartado se define el objetivo general del Trabajo de Fin de Grado, así como los objetivos específicos que delimitan su alcance y permiten estructurar el desarrollo de la solución propuesta. Asimismo, se identifican los principales artefactos generados durante el proyecto.

Objetivo general

Diseñar e implementar una infraestructura servidora complementaria del sistema **DTAgro**, orientada a la monitorización de explotaciones agrarias mediante el procesamiento de datos meteorológicos y de sensores IoT (*Internet of Things*), con el fin de generar predicciones que permitan anticipar riesgos de heladas y plagas sobre los cultivos.

Objetivos específicos

1. Diseñar una arquitectura basada en microservicios [13] que permita la integración, procesamiento y gestión de datos procedentes de servicios meteorológicos externos y sensores agrícolas.
2. Desarrollar servicios predictores capaces de analizar información histórica y futura, generando modelos estimados, orientados a la detección temprana de riesgos climáticos, especialmente heladas y plagas o enfermedades.
 - Implementar mecanismos de predicción de heladas a partir de datos meteorológicos históricos y previsiones climáticas futuras, considerando las restricciones climáticas asociadas al cultivo indicado.
 - Incorporar estimaciones basadas en valores de cota de altitud sobre parcelas agrícolas, con el objetivo de mejorar la precisión de las predicciones de heladas.
 - Desarrollar mecanismos de predicción de plagas y enfermedades mediante el análisis conjunto de datos meteorológicos y datos recopilados por sensores IoT, conociendo las condiciones favorables para la aparición de cada caso.
3. Implementar mecanismos de generación de informes que recopilen los resultados obtenidos por los servicios predictores, facilitando la interpretación de la información procesada.

4. Integrar librerías y herramientas que permitan la creación de informes automatizados en distintos formatos, garantizando una correcta visualización de los datos y una presentación clara de los resultados.

1.3. Planificación del Proyecto

El desarrollo del proyecto se estructuró siguiendo una **metodología iterativa e incremental** [14], resaltando la validación progresiva por parte del tutor, sobre los diferentes artefactos que se iban desarrollando. Cabe destacar que esta estrategia de planificación y puesta a prueba tuvo numerosas ventajas en la fase de desarrollo del proyecto. A partir de una idea básica general, a lo largo de todo el ciclo de vida del proyecto, se han ido identificando nuevos requisitos, gracias a reuniones que ha ido teniendo el tutor con **Ingenieros Agrónomos** y la investigación autodidacta abordada, por lo que al trabajar con esta metodología de trabajo, ha supuesto una adaptación ante **cambios**, sencilla y rápida durante las fases del proyecto.

Asimismo, este proyecto está alineado con la consecución del proyecto en desarrollo **TID4AGRO** [11], cuyo enfoque reside en la mejora competitiva y sostenible del sector agroalimentario de la EUROACE [15].

Fases identificadas

Para tener una perspectiva más visual sobre la planificación y desarrollo del proyecto, este se dividió en las siguientes fases:

- **Fase 1 - Definición de alcance y objetivos:** Establecimiento de los objetivos del proyecto junto con el tutor académicos y delimitación del alcance del sistema.
- **Fase 2 - Estado del arte y análisis de soluciones existentes:** Estudio orientado al análisis funcional de herramientas y sistemas similares, caracterizadas por la gestión agraria.

- **Fase 3 - Identificación de los requisitos:** Identificación de necesidades funcionales y no funcionales mediante reuniones con el tutor y potenciales usuarios finales.
- **Fase 4 - Diseño Arquitectónico:** Definición de una arquitectura orientada a microservicios y enfocada hacia la escalabilidad y a la **comunicación síncrona y asíncrona entre servicios**.
- **Fase 5 - Modelado y Persistencia de datos:** Diseño de un esquema relacional y definición de persistencia mediante ORM.
- **Fase 6 - Diseño del flujo del sistema:** Definición de diagramas de contexto, flujo, clase y casos de uso, entre otros, sobre el sistema, para pautar la implementación de cada servicio.
- **Fase 7 - Implementación del sistema:** Desarrollo de los microservicios, integración con servicios externos y procesamiento de datos.
- **Fase 8 - Validación y *Testing*:** Ejecución de pruebas integrales y unitarias sobre cada servicio desarrollado para validar su comportamiento.
- **Fase 9 - Despliegue y Contenedorización:** Preparación del sistema para su despliegue mediante contenedores y configuración del entorno de producción.
- **Fase 10 - Documentación y optimización final:** Redacción de la documentación técnica y mejora final de funcionalidades y rendimiento.

Alcance Temporal

Con el objetivo de representar de forma visual la planificación temporal del proyecto, a continuación, se muestra una parte de la distribución de las distintas tareas identificadas dentro de las fases descritas anteriormente (véase Figura 1.2, Figura 1.3, Figura 1.4). Esta planificación se elaboró una vez definido el alcance del sistema, permitiendo estructurar el desarrollo del trabajo y estimar los tiempos necesarios para completar cada actividad

dentro del tiempo medio establecido por la universidad para el Trabajo de Fin de Grado (aproximadamente 300 horas).

Puesto que la metodología utilizada para el desarrollo del proyecto es **iterativa e incrementa**, es preciso seguir una nomenclatura definida por UP (*Unified Process*) [16], sobre las tareas definidas en el marco de trabajo (véase Figura 1.1).

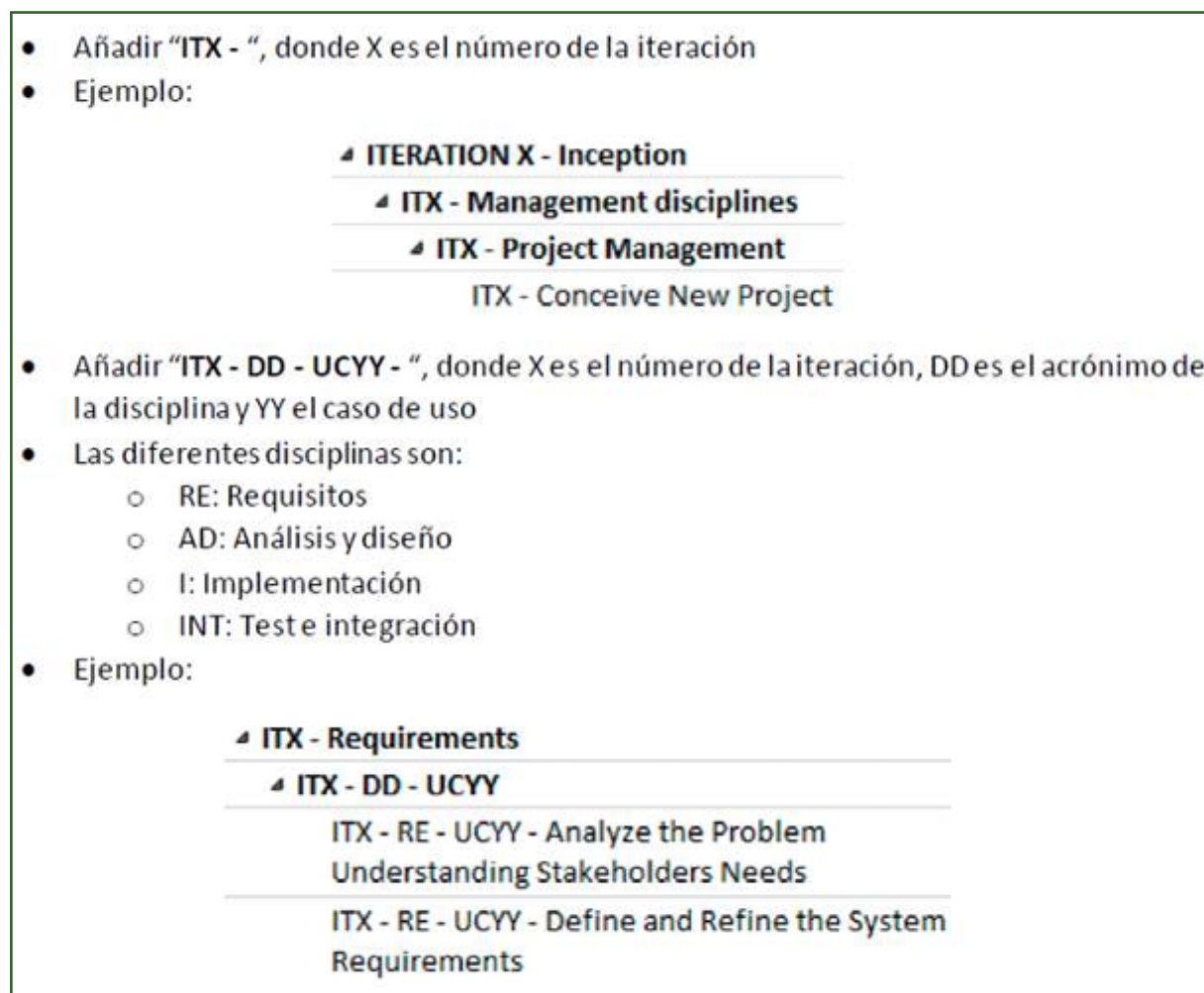


Figura 1.1: Nomenclatura UP.

Fuente:[17]

Para la gestión y organización de esta planificación se ha utilizado el *software ProjectLibre* [18], una herramienta orientada a la administración profesional de proyectos que permite definir tareas, recursos y plazos de ejecución. Además, posibilita la generación de distintos diagramas de seguimiento y control, como el diagrama de *Gantt*,

diagramas de red o estructuras de descomposición del trabajo (WBS), proporcionando una visión clara de la relación temporal entre las diferentes actividades.

El intervalo temporal sobre el que se organiza el proyecto se establece desde la fecha de inicio, 30/06/2025, hasta su fecha de finalización, 25/05/2026, periodo en el que se distribuyen las distintas fases de análisis, diseño, implementación, validación y documentación final del sistema.

	U	Nombre	Duracion	Inicio	Terminado	Trabajo	Predecesores
1	✓	Project - TFG	233 days?	30/6/25, 10:00	21/5/26, 10:00	381,5 horas	
2	✓	ITERATION 1- Alcan y Obj	56,312 days?	30/6/25, 10:00	16/9/25, 13:30	6,5 horas	
3	✓	IT1 - Definición de objetivo	56,312 days?	30/6/25, 10:00	16/9/25, 13:30	6,5 horas	
4	✓	IT - RE - UC01	0,188 days?	30/6/25, 10:00	30/6/25, 11:30	1,5 horas	
5	✓	IT1 - RE - UC01 - Propuesta de proyectos a desarrol...	0,062 days?	30/6/25, 10:00	30/6/25, 10:30	0,5 horas	
6	✓	IT1 - RE - UC01 - Elección de proyecto	0,125 days?	30/6/25, 10:30	30/6/25, 11:30	1 hora	
13	✓	IT1 - RE - UC05	0,125 days?	16/9/25, 9:30	16/9/25, 10:30	1 hora	
14	✓	IT1 - RE - UC05 - Definición inicial de riesgos y restri...	0,125 days?	16/9/25, 9:30	16/9/25, 10:30	1 hora	
7	✓	IT1 - RE - UC02	0,25 days?	16/9/25, 9:30	16/9/25, 11:30	2 horas	
8	✓	IT1 - RE - UC02 - Definición objetivos proyecto	0,25 days?	16/9/25, 9:30	16/9/25, 11:30	2 horas	
9	✓	IT1 - RE - UC03	0,125 days?	16/9/25, 11:30	16/9/25, 13:30	1 hora	
10	✓	IT1 - RE - UC03 - Delimitación del alcance del sistema	0,125 days?	16/9/25, 11:30	16/9/25, 13:30	1 hora	
11	✓	IT1 - RE - UC04	0,125 days?	16/9/25, 11:30	16/9/25, 13:30	1 hora	
12	✓	IT1 - RE - UC04 - Identificación de actores implicados	0,125 days?	16/9/25, 11:30	16/9/25, 13:30	1 hora	
78	✓	ITERACION3 - Análisis de requisitos	4,125 days?	6/11/25, 9:30	12/11/25, 10:30	11 horas	
79	✓	IT3 - RE - UC01	0,5 days?	6/11/25, 9:30	6/11/25, 14:30	4 horas	
80	✓	IT3 - RE - UC01 - Reunión con tutor y partes interesadas	0,25 days?	6/11/25, 9:30	6/11/25, 11:30	2 horas	
81	✓	IT3 - RE - UC01 - Puesta en común de las necesidades ...	0,25 days?	6/11/25, 11:30	6/11/25, 14:30	2 horas	
82	✓	IT3 - RE - UC02	0,625 days?	7/11/25, 10:00	7/11/25, 16:00	5 horas	
83	✓	IT3 - RE - UC02 - Identificación de requisitos funcionales	0,375 days?	7/11/25, 10:00	7/11/25, 14:00	3 horas	
84	✓	IT3 - RE - UC02 - Identificación de requisitos no funci...	0,25 days?	7/11/25, 14:00	7/11/25, 16:00	2 horas	
85	✓	IT3 - RE - UC03	1,312 days?	11/11/25, 8:00	12/11/25, 10:30	2 horas	
86	✓	IT3 - RE - UC03 - Priorización de requisitos	0,125 days?	11/11/25, 8:00	11/11/25, 9:00	1 hora	
87	✓	IT3 - RE - UC05 - Validación de requisitos	0,125 days?	12/11/25, 9:30	12/11/25, 10:30	1 hora	
57	✓	ITERACION 6 - Diseño detallado del sistema	4 days?	10/12/25, 8:00	15/12/25, 17:00	19 horas	
58	✓	IT6 - AD - UC01	4 days?	10/12/25, 8:00	15/12/25, 17:00	19 horas	
59	✓	IT6 - AD - UC01 -Definición diagrama contexto	0,5 days?	10/12/25, 8:00	10/12/25, 13:00	4 horas	
60	✓	IT6 - AD - UC01 - Definición diagrama componentes	0,5 days?	10/12/25, 12:00	10/12/25, 17:00	4 horas	
62	✓	IT6 - AD - UC01 - Definición diagrama casos uso	0,375 days?	11/12/25, 8:00	11/12/25, 11:00	3 horas	
61	✓	IT6 - AD - UC01 - Definición diagrama flujo	1 day?	15/12/25, 8:00	15/12/25, 17:00	8 horas	
63	✓	ITERACION 5 - Modelado y persistencia de datos	2,5 days?	1/1/26, 8:00	5/1/26, 13:00	16 horas	
64	✓	IT5 - AD - UC01	1,5 days?	1/1/26, 8:00	2/1/26, 13:00	8 horas	
65	✓	IT5 - AD - UC01 - Diseño del modelo relacional	0,5 days?	1/1/26, 8:00	1/1/26, 13:00	4 horas	
66	✓	IT5 - AD - UC01 - Definición de entidades y relaciones	0,5 days?	2/1/26, 8:00	2/1/26, 13:00	4 horas	

TFG - pagina1

Figura 1.2: Planificación Temporal *Project Libre*, 1ª parte.

Fuente: Elaboración propia

		Nombre	Duración	Inicio	Terminado	Trabajo	Predecesores
67	✓	IT5 - AD - UC02	1 day?	2/1/26, 12:00	5/1/26, 13:00	8 horas	
68	✓	IT5 - AD - UC02 - Diseño de persistencia mediante ORM	0,5 days?	2/1/26, 12:00	2/1/26, 17:00	4 horas	
69	✓	IT5 - AD - UC02 - Validación de integridad de datos	0,5 days?	5/1/26, 8:00	5/1/26, 13:00	4 horas	
70	✓	ITERACION4 - Diseño Arquitectónico	37,5 days?	20/11/25, 9:00	12/1/26, 14:00	14 horas	
71	✓	IT4 - AD - UC01	37,375 days?	20/11/25, 9:00	12/1/26, 13:00	4 horas	
72	✓	IT4 - AD - UC01 - Definición arquitectura basada en mi...	0,125 days?	20/11/25, 9:00	20/11/25, 10:00	1 hora	
73	✓	IT4 - AD - UC01 - Diseño de comunicación síncrona e...	0,125 days?	20/11/25, 10:00	20/11/25, 11:00	1 hora	
74	✓	IT4 - AD - UC01 - Diseño de comunicación asíncrona	0,25 days?	12/1/26, 10:00	12/1/26, 13:00	2 horas	
75	✓	IT4 - AD - UC03	19,625 days?	16/12/25, 8:00	12/1/26, 14:00	10 horas	
76	✓	IT4 - AD - UC03 - Definición de integración con servi...	0,75 days?	16/12/25, 8:00	16/12/25, 15:00	6 horas	
77	✓	IT4 - AD - UC03 - Remodelación en base a escalabilidad	0,5 days?	12/1/26, 9:00	12/1/26, 14:00	4 horas	
47	✓	ITERACION 7 - Implementación del sistema	93,25 days?	30/10/25, 8:00	10/3/26, 10:00	177 horas	
48	✓	IT7 - I - UC01	92 days?	30/10/25, 8:00	6/3/26, 17:00	140 horas	
49	✓	IT7 - I - UC01 - Desarrollo de servicios externos	2,5 days?	30/10/25, 8:00	3/11/25, 13:00	20 horas	
50	✓	IT7 - I - UC01 - Desarrollo de servicio de datos	7,5 days?	6/1/26, 8:00	15/1/26, 13:00	60 horas	
51	✓	IT7 - I - UC01 - Desarrollo de servicio IA	2,5 days?	4/2/26, 8:00	4/2/26, 13:00	20 horas	
52	✓	IT7 - I - UC01 - Desarrollo de servicios predictores	5 days?	2/3/26, 8:00	6/3/26, 17:00	40 horas	
53	✓	IT7 - I - UC02	45,25 days?	6/1/26, 8:00	10/3/26, 10:00	37 horas	
56	✓	IT7 - I - UC02 - Desarrollo de automatización de proce...	1,5 days?	6/1/26, 8:00	7/1/26, 13:00	12 horas	
54	✓	IT7 - I - UC02 - Integración con sensores	1,875 days?	6/3/26, 8:00	9/3/26, 16:00	15 horas	
55	✓	IT7 - I - UC02 - Implementación de generación de info...	1,25 days?	9/3/26, 8:00	10/3/26, 10:00	10 horas	
40	✓	ITERACION 8 - Validación y testing	5,75 days?	9/3/26, 8:00	16/3/26, 15:00	20 horas	
41	✓	IT8 - INT - UC01	0,75 days?	9/3/26, 8:00	9/3/26, 15:00	10 horas	
43	✓	IT8 - INT - UC01 - Pruebas de integración entre servic...	0,5 days?	9/3/26, 8:00	9/3/26, 13:00	4 horas	
42	✓	IT8 - INT - UC01 - Pruebas funcionales de microservi...	0,75 days?	9/3/26, 8:00	9/3/26, 15:00	6 horas	
44	✓	IT8 - INT - UC02	0,75 days?	16/3/26, 8:00	16/3/26, 15:00	10 horas	
46	✓	IT8 - INT - UC02 - Ajustes de rendimiento	0,5 days?	16/3/26, 8:00	16/3/26, 13:00	4 horas	
45	✓	IT8 - INT - UC02 - Corrección de errores detectados	0,75 days?	16/3/26, 8:00	16/3/26, 15:00	6 horas	
15	✓	ITERACION 2 - Estado del arte y competencia	143,312 da...	1/10/25, 8:00	20/4/26, 10:30	22 horas	
16	✓	IT2 - Análisis software sector agrario	7,5 days?	1/10/25, 8:00	10/10/25, 13:00	14 horas	
17	✓	IT2 - RE - UC01	0,75 days?	1/10/25, 8:00	1/10/25, 15:00	6 horas	
18	✓	IT2 - RE - UC01 - Investigación de soluciones existe...	0,75 days?	1/10/25, 8:00	1/10/25, 15:00	6 horas	
19	✓	IT2 - RE - UC02	5,25 days?	3/10/25, 10:00	10/10/25, 13:00	8 horas	
20	✓	IT2 - RE - UC02 - Comparativa de funcionalidades d...	0,75 days?	3/10/25, 10:00	3/10/25, 17:00	6 horas	
TFG - pagina31							

Figura 1.3: Planificación Temporal *Project Libre*, 2ª parte.

Fuente: Elaboración propia

		Nombre	Duración	Inicio	Terminado	Trabajo	Predecesores
21	✓	IT2 - RE - UC02 - Identificación de ventajas y limitaci...	0,25 days?	10/10/25, 10:00	10/10/25, 13:00	2 horas	
22	✓	IT2 - Conclusiones de análisis y DTAgro	131,062 da...	17/10/25, 10:00	20/4/26, 10:30	8 horas	
23	✓	IT2 - RE - UC01	131,062 da...	17/10/25, 10:00	20/4/26, 10:30	8 horas	
24	✓	IT2 - RE - UC01 - Creación informe análisis resultante	0,875 days?	17/10/25, 10:00	20/10/25, 9:00	7 horas	
25	✓	IT2 - RE - UC01 - Justificación de la propuesta DTAgro	0,125 days?	20/4/26, 9:30	20/4/26, 10:30	1 hora	
34	✓	ITERACION 9 - Despliegue y contenedorización	15,625 days?	6/4/26, 8:00	27/4/26, 14:00	23 horas	
35	✓	IT9 - I - UC01	15,625 days?	6/4/26, 8:00	27/4/26, 14:00	23 horas	
36	✓	IT9 - I - UC01 - Dockerización de los microservicios	0,75 days?	6/4/26, 8:00	6/4/26, 15:00	6 horas	
37	✓	IT9 - I - UC01 - Despliegue en máquina virtual AWS	1 day?	10/4/26, 8:00	10/4/26, 17:00	8 horas	
38	✓	IT9 - I - UC01 - Configuración de dependencias y redes	0,5 days?	13/4/26, 8:00	13/4/26, 13:00	4 horas	
39	✓	IT9 - I - UC01 - Validación de ejecución en entorno final	0,625 days?	27/4/26, 8:00	27/4/26, 14:00	5 horas	
26	✓	ITERACION 10 - Documentación y cierre	36,25 days?	1/4/26, 8:00	21/5/26, 10:00	73 horas	
27	✓	IT10 - RE - UC01	6 days?	1/4/26, 8:00	8/4/26, 17:00	46 horas	
28	✓	IT10 - RE - UC01 - Documentación técnica de microser...	0,75 days?	1/4/26, 8:00	1/4/26, 15:00	6 horas	
29	✓	IT10 - RE - UC01 - Redacción Memoria TFG	5 days?	2/4/26, 8:00	8/4/26, 17:00	40 horas	
30	✓	IT10 - RE - UC02	28,25 days?	13/4/26, 8:00	21/5/26, 10:00	27 horas	
31	✓	IT10 - RE - UC02 - Revisión final del sistema	0,625 days?	13/4/26, 8:00	13/4/26, 14:00	5 horas	

Figura 1.4: Planificación Temporal *Project Libre*, 3ª parte.

Fuente: Elaboración propia

Capítulo 2: Estado del Arte

La actual digitalización del sector agrícola ha impulsado la creación de múltiples soluciones tecnológicas orientadas a la gestión de recursos y la atenuación de riesgos climáticos, sobre una explotación agrícola. En los últimos años, tanto la comunidad científica, como la industrial en todo el mundo, han dedicado sus esfuerzos en la aplicación de modelos predictivos, técnicas de análisis de datos y sistemas inteligentes para la mejora de este sector con la creación de planificadores agrícolas [19].

En este contexto, resulta necesario identificar las herramientas ya desarrolladas para guiar el desarrollo del proyecto hacia una **solución única, innovadora y útil para los clientes finales**.

2.1. Conceptos Básicos

Para hablar de la gestión de recursos y la aplicación de modelos predictivos, que incorporan las diferentes herramientas previamente desarrolladas sobre este sector, se deben de tener conocimientos básicos sobre sus elementos esenciales.

Adquisición y Gestión de Datos Agrícolas

El término **servidor en la nube**, es habitual utilizarlo para la definición arquitectónica de herramientas dedicadas a la gestión de recursos, además de para el entrenamiento de modelos. Estos sistemas, junto con sus modelos integrado, sobre el marco del sector agrícola, deben de consumir una cantidad de datos masiva con el objetivo de funcionar lo más precisos posibles [20]. Existen dos principales vías de obtención de datos: mediante sensores (capaces de producir millones de datos por día) o sobre estaciones meteorológicas. Debido a que el *software* empresarial debe mantener y acceder diariamente a esta

cantidad masiva de datos, las empresas suelen contratar planes *cloud* (en la nube), despreocupándose así por el mantenimiento de estos datos, gracias a que en su mayoría, los proveedores de estos servicios y servidores *cloud*, son los responsables de su mantenimiento.

Los **sensores** son dispositivos tecnológicos diseñados para medir, registrar y analizar variables específicas en tiempo real del entorno agrícola, como el estado del suelo e inferencias climatológicas sobre cultivos [21].

Debido a que los sensores se pueden encontrar dentro de una finca o explotación agraria, se debe establecer una comunicación entre el sensor y el servidor que almacenará sus datos. Para ello, protocolos de red como **LoRaWAN** [12], permiten mantener una conexión entre estos dos puntos mencionados, proporcionando una capacidad de transmisión de datos inalámbrica, de bajo consumo en red y sobre conexiones de larga distancia.

Por otro lado, existe una métrica comúnmente utilizada en gestores de cultivos profesionales, denominada **índice NDVI**, también conocido como **Índice de Vegetación de Diferencia Normalizada** [22].

Es una métrica esencial para evaluar la salud de los cultivos a partir de **imágenes satelitales**. Muestra su estado en función de la cantidad de luz que absorben los cultivos y datos terrestres locales. Los gestores agrícolas que lo implementan permiten a los agricultores supervisar la evolución estacional sobre los cultivos, detectar los primeros signos de estrés hídrico y planificar cualquier intervención antes de que los daños sean críticos.

Conceptos agronómicos

Paralelamente, hay una medida ampliamente utilizada en los sistemas gestores actuales, conocida como **grados día de crecimiento** (GDD). Es una media acumulada del calor necesario para que los cultivos evolucionen en sus etapas del ciclo de vida [23]. Su cálculo (2.1) consta de una temperatura base sacada de la media entre las temperaturas necesarias para las etapas del ciclo de vida en un cultivo específico. A su vez, esta temperatura base se resta a la temperatura media diaria, lo que permite predecir cosechas, plagas y optimizar

riego.

$$GDD = \frac{T_{max} - T_{min}}{2} - T_{base} \quad (2.1)$$

El denominado **estrés hídrico**, es un impacto significativo en la producción agrícola y calidad de la cosecha. Suele producirse por factores clave, como la falta de agua en el terreno del cultivo, ausencia de lluvias, sequías o riegos deficientes [24].

La **etapa fenológica** [25] de un cultivo hace referencia al estado de desarrollo en el que se encuentra dentro de su ciclo biológico. Estas etapas varían según la especie y las condiciones climáticas [26], debido a que el comportamiento fisiológico de cada cultivo responde de forma distinta a factores ambientales como la temperatura, la radiación solar o la disponibilidad hídrica. No obstante, de manera general, pueden identificarse las siguientes fases principales:

- **Dormancia:** Periodo protagonizado por una suspensión temporal del crecimiento vegetativo y una reducción significativa de la actividad metabólica. Suele iniciarse en otoño como mecanismo de adaptación a condiciones climáticas adversas.
- **Brotación:** Fase en la que se produce la prosperación del crecimiento al concluir la dormancia, dando lugar a la aparición de nuevos brotes, yemas y hojas.
- **Floración:** Etapa en la que se desarrollan y abren las flores, permitiendo el proceso de **polinización**.
- **Cuajado:** Fase posterior a la polinización en la que comienza la formación del fruto.
- **Engorde:** Periodo de crecimiento y aumento de tamaño del fruto, suele deberse a la llegada de **clima cálido**, eliminando horas de frío junto con la aparición de más horas de sol a lo largo del día.
- **Maduración:** Fase en la que el fruto alcanza sus características finales de tamaño, textura y composición.
- **Cosecha:** Momento en el que el fruto alcanza el estado óptimo para su recolección.

- **Senescencia:** Etapa final en el ciclo de vida del cultivo, proceso de envejecimiento donde la planta entra en un estado de deterioro.

A la hora de predecir riesgos de heladas, se debe tener en consideración los tipos de heladas que se pueden producir en el sector agrario.

Los principales tipos de heladas junto con sus motivos de aparición y los problemas causantes son los siguientes:

- **Heladas Blancas**

Son heladas que se producen con frío y humedad. Cuando la temperatura se encuentra **por debajo** de 0°C y hay suficiente humedad en el ambiente (rangos de 60% y 70%), aparece hielo en forma de pequeños cristales (véase Figura 2.1) que se adhieren sobre superficies sólidas (hierbas, plantas, frutos, etc.). Este tipo de helada no suele ser dañina para el campo y protegen la parte interna de la planta [27].



Figura 2.1: Representación Visual de Heladas Blancas

Fuente: [4]

- **Heladas Negras**

Son heladas que se producen cuando la temperatura se encuentra por debajo de los 0°C pero no se forma escarcha. Esto sucede porque **el aire está en condiciones muy secas**, imposibilitando la compensación entre la temperatura ambiente y la soportada por la planta afectada, impidiendo la producción de condensación y formación de escarcha que la protege [27].

Estas heladas son temidas en el campo debido a los daños que producen. El frío seco ataca directamente a la estructura interna de la planta, produciendo un aspecto negro (véase Figura 2.2) que adquieren de repente los tejidos vegetales como las hojas o tallos jóvenes pudiendo morir la planta entera si los daños afectan a zonas vitales.



Figura 2.2: Representación Visual de Heladas Negras
Fuente: [5]

2.2. Técnicas actuales de mitigación frente a riesgos climáticos

Dadas las amenazas climáticas que han estado presenciando los agricultores, actualmente se utilizan ciertas técnicas y herramientas para mitigar el impacto negativo de estos sucesos adversos.

La elección óptima del cultivo, la introducción de nuevos cultivos más resistentes a las nuevas condiciones o el desplazamiento en algunos casos de las áreas de cultivo actuales contribuyen a la prevención de efectos negativos [28].

La mejora en la gestión del suelo, diversificando cultivos, el cambio a variedades más resistentes, con distintas necesidades de frío o de ciclos más cortos, los cambios en el manejo (fechas de siembra o aportación de nutrientes).

El empleo de prácticas como las cubiertas vegetales, mejorando la estructura del suelo y su contenido en materia orgánica o la mejora en la gestión de pastos, adaptadas a las

condiciones locales de cultivo [29] [30].

La dificultad en la aplicación de estas técnicas nombradas reside en que solucionan casos de uso locales y normalmente difieren sobre el cultivo aplicado y sistema de producción, por lo que identificar la solución óptima requiere un esfuerzo importante de recogida de información y análisis.

En conclusión, la identificación de las técnicas analizadas ha permitido extraer un concepto clave para el desarrollo de este proyecto: la necesidad de mantener una gestión de cultivos óptima, favoreciendo la precisión de las predicciones ante riesgos de heladas. Sobre este contexto, destaca la **optimización de cultivos mediante la selección de variedades con requerimientos de frío diferenciados y ciclos fenológicos adaptados** [31].

2.3. Tecnologías relevantes para la gestión agrícola

Actualmente, el mercado tecnológico del sector agrario cuenta con centenares de herramientas encargadas de gestionar y mantener una eficiencia de trabajo sobre las explotaciones agrarias, por lo que es conveniente realizar un estudio exhaustivo de las más destacadas para conocer la competencia, qué ofrecen y como innovar la idea de desarrollo sobre la que se parte para implementar el sistema software desarrollado.

2.3.1. Maher App [1]

Descripción general

Maher es una empresa dedicada a la fabricación profesional de programadores de riego y clima agrícola, situada en la provincia de Almería.

Son referentes en el mercado nacional e internacional, contando con más de 20 años de trayectoria profesional en el sector, aportando continuamente innovadoras ventajas tecnológicas que permiten rentabilizar al 100% las cosechas y obtener el máximo rendimiento con una mayor sostenibilidad del medioambiente.

Características técnicas y funcionales

Desde el punto de vista técnico que gestiona y aporta el sector, cabe destacar su creación propia de aplicación web para riego, visualizando su panel de control en la Figura 2.3.

Se encuentra alojada en la nube para garantizar su compatibilidad con cualquier sistema operativo. A su vez, aportan mayor capacidad de almacenamiento y seguridad en el registro de datos.

Registra datos en la nube, almacenando el historial de riegos ejecutados, historial de anomalías registradas, programas de riego, configuración de equipo, lectura de sensores y copia de seguridad de los programas de riego.

Realiza conexiones a estaciones meteorológicas **METOS** [32], permitiendo que el programador de riegos reciba las lecturas de estas estaciones, y mediante decisiones, condicionar el inicio y finalización de los programas de riego.

Incluye un gestor de consultas relacionada con el funcionamiento de los sensores en tiempo real, pudiendo indicar valores máximos y mínimos sobre cada sensor registrado. Además, permite la notificación de alertas a través de correos electrónicos asociados a la cuenta registrada en la web, en caso de que algún sensor desplegado en la explotación agrícola haya alcanzado valores máximos o mínimos límites indicados por el agricultor.



Figura 2.3: Maher Mobile UI.

Fuente: [1]

Análisis crítico

Pese a no tratarse de una empresa ampliamente consolidada en el marco del desarrollo de software orientada a la automatización de riegos, la solución presentada por Maher resulta interesante desde el punto de vista funcional.

Una de sus características por las que destaca es su baja curva de aprendizaje, lo que facilita su adaptación por parte de los usuarios finales, sin requerir de una configuración compleja o conocimientos técnicos avanzados. Esta simplicidad de uso, facilita una implantación ágil de los servicios ofrecidos en explotaciones agrarias.

No obstante, aunque la satisfacción del cliente constituye uno de los principales objetivos, su solución ofrecida al mercado contiene limitaciones. En particular, **carece** de un sistema de predicción basado en modelos de datos entrenados y aprendizaje automático, obteniendo datos únicamente de sus estaciones meteorológicas METOS. Esto produce una

imprecisión importante en la creación y entrenamiento de modelos predictivos, debido a que solo tienen una fuente de consulta de datos.

La integración de fuentes meteorológicas externas y públicas, permitiría al servicio, mejorar la calidad global del *software* desarrollado.

2.3.2. Eco Solutions - Riego Predictivo [2]

Descripción general

Eco Solutions es una empresa cuya misión se centra en revolucionar el campo de la **Eficiencia Energética** a través de soluciones tecnológicas innovadoras. Diseñan, desarrollan e implementan tecnologías avanzadas que optimizan el rendimiento en empresas y ciudades, así como en fincas agrarias e industrias a nivel global, consiguiendo maximizar el ahorro de recursos esenciales como energía y agua.

Estas soluciones están diseñadas de forma adaptables al cliente final, lo que permite su implementación en diferentes contextos agrícolas y tipos de cultivos, como se muestra en la Figura 2.4.

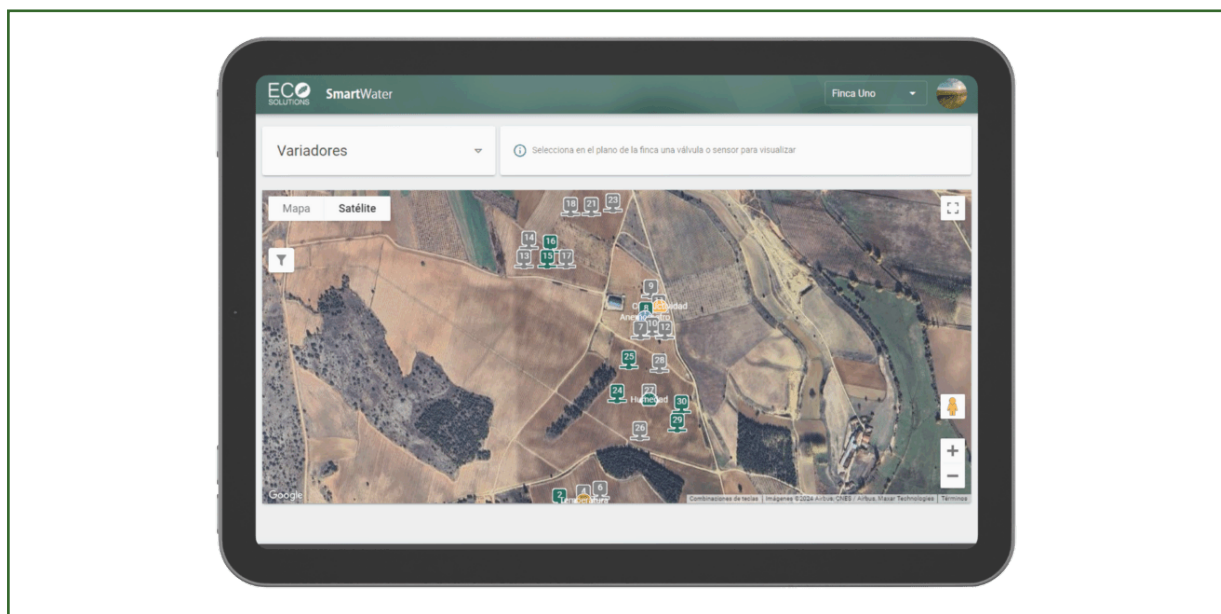


Figura 2.4: Eco Solutions UI.

Fuente: [2]

Características técnicas y funcionales

Desde el punto de vista técnico, las soluciones de riego predictivo desarrolladas por Eco Solutions se basan en el uso de modelos de datos y algoritmos de predicción para anticipar las necesidades hídricas de los tipos de cultivos, como también la sustentación correcta y exacta de estiércol. De entre sus modelos utilizados, se puede destacar el uso de *Random Forest* [33] y *Boosted Regression Tree* [34]. A su vez, utilizan modelos WRF (*Weather Research and Forecasting*) [35]. Se caracteriza por ser un modelo de predicción numérica del tiempo, diseñado para predicciones atmosféricas (véase Figura 2.5).

Como fuente de datos para entrenar sus modelos predictivos, utilizan estaciones meteorológicas (propias o públicas), obteniendo datos tanto históricos como actuales y predicciones futuras.

Por otro lado, consideran el estado fenológico actual del cultivo, teniendo en cuenta sus fechas de etapa de crecimiento, y sus necesidades específicas de agua.

Asimismo, analizan las características del suelo, como la capacidad de retención de agua, que influye directamente en la frecuencia y duración del riego.

Automatizan procesos que involucran el riego, ajustando la cantidad de agua aplicada en función de las predicciones realizadas por el sistema [37].

Orientan el sistema a un enfoque predictivo, anticipando las necesidades reales del cultivo en lugar de reaccionar únicamente ante umbrales predefinidos.

Análisis crítico

A pesar de que Eco Solutions ofrece un enfoque avanzado para el riego inteligente, su solución presenta ciertas limitaciones desde el punto de vista técnico y académico.

La falta de documentación pública sobre los modelos de predicción empleados imposibilita conocer el tratamiento de datos específico que incorpora internamente el *software*. Los modelos mencionados parecen derivarse de una aproximación híbrida, que combina métodos estadísticos tradicionales con técnicas de aprendizaje automático, integrando datos históricos, variables meteorológicas y parámetros fenológicos para la estimación del

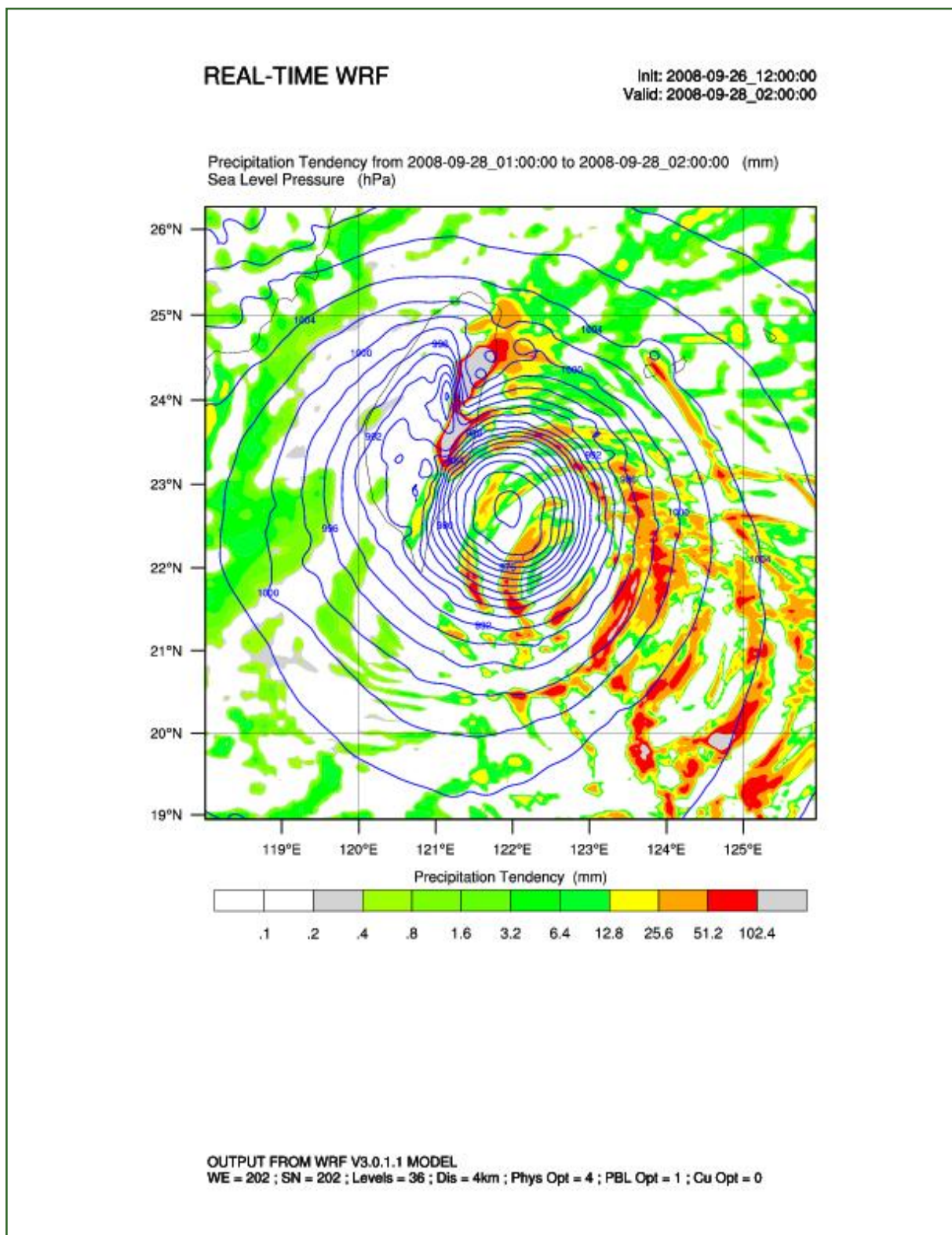


Figura 2.5: Modelo WRF

Fuente: [36]

riego.

Asimismo, al tratarse de soluciones completamente personalizadas al criterio del cliente final, su arquitectura interna y capacidad de escalabilidad no están completamente definidas, pudiendo dificultar su adaptación a nuevos escenarios sin un nuevo proceso de desarrollo.

2.3.3. Irristrat [3]

Descripción general

Irristrat es un *software* de gestión de riegos inteligente avanzado, integrado, potente y accesible desde cualquier dispositivo. Capaz de procesar información de campo en tiempo real y proporcionar a los agricultores indicaciones y acciones precisas para conseguir un riego en sus explotaciones de forma eficiente, contribuyendo a mejorar su rendimiento y resultados (véase Figura 2.6).

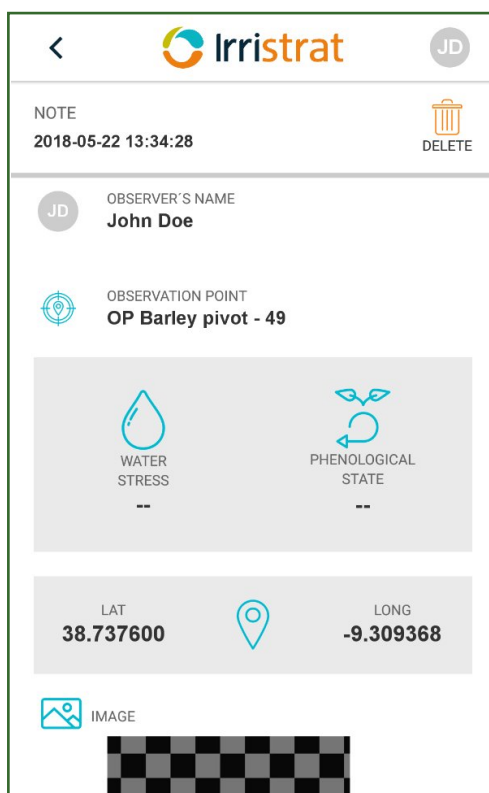


Figura 2.6: Irristrat Mobile UI.
Fuente: [3]

Características técnicas y funcionales

Desde el punto de vista técnico, las soluciones de riego automático que propone Irristrat destacan por disponer de una aplicación web y móvil. Permite la gestión y supervisión de las fincas registradas desde el sistema, de forma remota y centralizada.

Las principales funcionalidades que permite la plataforma se agrupan en los siguientes módulos [38] [39]:

- **Parametrización y gestión**

Permite la configuración de distintas unidades de gestión, así como la visualización de datos mediante gráficos, tablas y mapas interactivos. Incluye opciones para la selección de unidades de medida e idioma, además de la posibilidad de exportar los datos generados por el sistema.

- **Clima**

Ofrece previsiones meteorológicas y acceso a datos climáticos históricos y actuales. Incluye la posibilidad de ajuste manual sobre determinados parámetros y el cálculo de identificadores agronómicos relevantes, como las horas de frío y los Grados Día de Crecimiento (fórmula asociada (2.1)).

- **Información del campo**

Incluye funcionalidades para el registro de notas de campo, el seguimiento de los estados fenológicos sobre los cultivos, así como el almacenamiento de los registros relacionados con el riego y la aplicación de fertilizantes.

- **Informes**

Proporciona la generación de pre-informes globales, comparativas entre planes de riego, análisis de consumo de agua y el cálculo del índice **NDVI** (Índice de Vegetación de Diferencia Normalizada), utilizado para evaluar la salud y el vigor de vegetación en una zona determinada del terreno.

- **Comunicaciones**

Incorpora un sistema de alarmas destinado a notificar incidencias o eventos relevantes para la correcta gestión de explotaciones agrarias.

2.3.4. Conclusiones y aportaciones adicionales

Como resultado del análisis comparativo realizado, se determinó que el *software* desarrollado por **Eco Solutions** constituía una primera aproximación para el desarrollo del presente proyecto, debido a su integración funcional con modelos predictivos y pronósticos climáticos estimados, además de su enfoque orientado a la automatización eficiente de riegos.

Pese a que el estudio se ha centrado en tres soluciones *softwares* altamente demandadas en el mercado actual, este número ha resultado suficiente para identificar la tendencia de

las herramientas actuales desarrolladas y poder tener una idea más clara sobre el paradero final para la solución que se quiere transmitir con este proyecto, así como las principales fortalezas y limitaciones existentes en los sistemas de automatización agrícolas.

Adicionalmente, se ha considerado una cuarta solución, **OpenSprinkler** [40], una plataforma web y móvil de código abierto orientado al control y automatización de riegos en parques y viveros. La iniciativa sobre el análisis de esta herramienta reside en la información que ofrecen al público, dentro de su organización localizada en la plataforma **GitHub** [41], permitiendo investigar internamente cada uno de los repositorios que generan el correcto funcionamiento de la plataforma y conociendo un poco su arquitectura interna modular en entornos profesionales y empresariales.

Si bien no se ha tomado como base principal de proyecto, su estructura y planteamiento arquitectónico han servido como inspiración para determinadas decisiones de diseño sobre el desarrollo propuesto.

A modo resumen de este capítulo, se presenta a continuación una tabla (véase Tabla 2.1), donde se recogen las principales características que aborda cada una de las herramientas analizadas, permitiendo comparar los criterios que cumple cada una de ellas.

Tabla 2.1: Comparativa de soluciones de gestión y predicción de riego agrícola

Criterio	Maher App	Eco Solutions	Irristrat	Agro Predict
Enfoque principal	Automatización de riego basada en sensores	Riego predictivo basado en modelos de datos	Gestión integral del riego y explotación	Predicción de riesgos climáticos basado en datos meteorológicos y sensores

Criterio	Maher App	Eco Solutions	Irristrat	Agro Predict
Uso de modelos predictivos	No dispone	Sí (Random Forest, BRT, WRF)	Parcial (indicadores agronómicos, no ML avanzado)	Parcial (indicadores agronómicos junto con estimaciones, no ML)
Fuentes de datos	Estaciones METOS	Estaciones propias y públicas + datos históricos	Datos climáticos, sensores y registros de campo	Datos climáticos futuros e históricos públicos, sensores y registros de campo
Tratamiento de datos	Basado en reglas y umbrales	Aproximación híbrida (estadística + ML)	Análisis descriptivo y agronómico	Estimaciones basadas en reglas y umbrales
Variables consideradas	Sensores y clima local	Clima, suelo y estado fenológico	Clima, NDVI, GDD y registros de cultivo	Sensores, clima local, GDD, suelo, estado fenológico y registros de cultivo
Automatización	Alta (programadores de riego)	Alta (decisiones predictivas)	Media (soporte a la decisión)	Alta (decisiones predictivas con informes resultantes)

Criterio	Maher App	Eco Solutions	Irristrat	Agro Predict
Escalabilidad	Limitada a su ecosistema	Dependiente de personalización	Moderada (plataforma adaptable)	Alta (mecanismos internos que adaptan la situación a la que se somete la infraestructura)
Facilidad de uso	Alta	Media	Media	Alta
Principales fortalezas	Simplicidad y rápida implantación	Enfoque predictivo avanzado	Gestión integral y visualización de datos	Enfoque predictivo mantenible en el futuro, con visualización de datos haciendo uso de informes con formato PDF
Principales limitaciones	No utiliza modelos predictivos	Falta de transparencia y documentación	Limitado uso de inteligencia artificial	No utiliza modelos de aprendizaje automático para generar predicciones

Capítulo 3:Propuesta de desarrollo

El conjunto de servicios propuesto consiste en el desarrollo de una solución *software* centrada en la gestión y predicción de riesgos climáticos sobre el sector agrícola. Su finalidad consiste en proporcionar ayuda a la toma de decisiones anticipada por los clientes finales, ante fenómenos que puedan afectar negativamente en el desarrollo de los cultivos.

La solución *software* se apoya en una generación de informes en formato PDF dirigida a agricultores e ingenieros agrónomos, desde los cuales, el usuario puede consultar información detallada y concentrada sobre las predicciones realizadas, ayudándose de gráficos en base a datos recopilados para comprender mejor el contexto de cálculo. En el contexto de clientes *software*, como es el caso de DTAgro, se proporcionan respuestas predictivas en formato JSON estructuradas para su adaptación sobre el ecosistema del *software* empresarial existente.

El sistema se compondrá de tres servicios característicos:

1. Predictor de riesgos de heladas

Este módulo permite estimar el riesgo de heladas para el día actual y para un intervalo de 24 horas futuras. La predicción se basa en el análisis de datos meteorológicos históricos y futuros, proporcionados por servicios públicos de terceros. Genera una estimación probabilística en función de las condiciones climáticas registradas, tanto para localidades o cultivos seleccionados. Por otro lado, se puede consultar la predicción de forma general para una comunidad autónoma o provincia especificada.

El objetivo de este módulo es ofrecer una herramienta preventiva que permita evitar posibles daños en variedades de cultivos, sensibles a bajas temperaturas. La inclusión de alertas y recomendaciones textuales programadas, indica al cliente final la mejor decisión a tomar.

2. Predictor de riesgos de plagas y enfermedades

Este módulo permite estimar el riesgo de plagas/enfermedades que pueden llegar a afectar a un cultivo específico, utilizando datos meteorológicos y proporcionados por sensores en DTAgrro, mientras se mantengan registros de sus plagas afectantes internamente. Por otro lado, gracias a los datos obtenidos de servicios públicos de terceros, se pueden conocer calendarios de riesgos anuales, sobre plagas/enfermedades para grupos de cultivo como **cereales o leguminosos**, permitiendo estimar las semanas en las que puede ser crítica la prevención y protección de ciertos cultivos afectados.

El usuario puede seleccionar el cultivo a analizar y el sistema devuelve un nivel de riesgo estimado junto con información de contexto relacionada.

En conjunto, la infraestructura a desarrollar se divide en un conjunto de servicios de apoyo ante decisiones basadas en datos climatológicos y provenientes de sensores IoT, con un enfoque predictivo a corto plazo y orientado a mejorar la planificación agrónoma en entornos con recursos tecnológicos limitados. De este modo, el sistema permitirá su integración y aplicación en entornos empresariales como DTAgrro, proporcionando soporte operativo mediante modelos estimados predictivos que optimizan la gestión de cultivos.

Capítulo 4: Requisitos

La identificación de requisitos representa una fase clave en la **planificación del proyecto**, debido a que define el comportamiento esperado de la infraestructura desarrollada y condiciona su desarrollo a lo largo de todo el ciclo de vida.

En el presente proyecto, los requisitos se han analizado a partir de los diferentes usuarios que interactuarán con los servicios implementado, principalmente agricultores e ingenieros agrónomos. La clasificación de estos clientes finales ha permitido identificar las funcionalidades necesarias para la infraestructura desarrollada, así como las restricciones técnicas que deben considerarse en su implementación.

4.1. Metodología de identificación y clasificación de requisitos

Para la identificación y documentación de los requisitos que influyen en el comportamiento y estructura del *software* desarrollado, se ha decidido seguir una metodología de trabajo integrada en la norma *ISO/IEC/IEEE 29148*. Está especializada en proporcionar un marco metodológico unificado para la gestión de requisitos en el ciclo de vida del sistema *software*.

4.1.1. Clasificación de requisitos

Como se ha mencionado anteriormente, los requisitos identificados se clasifican en dos bloques principales, dependiendo de su tipo de implicación y funcional que designan en el desarrollo del sistema.

- **Requisitos Funcionales (RF):** Definen los comportamientos que el sistema debe proporcionar para satisfacer las necesidades de los potenciales clientes.

- **Requisitos No Funcionales (RNF):** Definen restricciones o cualidades de calidad que debe de cumplir implícitamente el sistema desarrollado (rendimiento, escalabilidad, usabilidad, etc.)

4.1.2. Criterios de documentación

Cada requisito identificado se documenta mediante los siguientes atributos seleccionados de la norma *ISO/IEC/IEE 29148*.

- **Identificador:** Código único (RF-x o RNF-x)
- **Descripción:** Breve descripción que define el requisito.
- **Nivel de abstracción:** Identifica la capa jerárquica de detalle en la que se encuentra un requisito.
- **Referencia:** Fuente de la que se deriva el requisito identificado.
- **Prioridad:** Prioridad que se le ha asignado a cada uno de los requisitos.

El nivel de abstracción se define de la siguiente forma:

- **Nivel Alto:** Define una necesidad general del sistema.
- **Nivel Medio:** Define una funcionalidad concreta, pero todavía no implementable.
- **Nivel Bajo:** Define un comportamiento específico listo para implementación.

4.1.3. Plantilla de especificación

La especificación detallada de los requisitos se formula con la plantilla que presenta la norma ya mencionada (véase Tabla 4.1):

[<Condition>] <Subject><Action><Object>[<Restriction>]
--

Tabla 4.1: Plantilla definición de requisitos *ISO/IEC/IEE 29148*

Esta plantilla permite documentar cada uno de los requisitos de forma clara, estructurada y trazable. No obstante, aunque la norma establece que la especificación debe formalizarse mediante una única frase, se ha optado por desglosar la descripción de cada requisito conforme a las distintas secciones propuestas por la propia norma.

Este enfoque facilita una comprensión más completa por parte del lector y proporciona un formato estructurado que mejora la claridad y la verificabilidad de los requisitos.

Para conocer la especificación detallada de cada uno de los requisitos funcionales y no funcionales identificados, véase el [Anexo A].

4.2. Identificación y especificación de requisitos

Con el objetivo de comprender en detalle los requisitos identificados, su funcionalidad y su relevancia dentro del proyecto, esta sección se dedica a describir cada uno de los requisitos que el sistema debe cumplir.

4.2.1. Requisitos Funcionales (RF)

Los requisitos funcionales identificados se recogen en formato de tabla (véase Tabla 4.2). Se encuentra dividida en diferentes columnas que ayudan a entender de manera general la **funcionalidad** que debe de abarcar el sistema e información de contexto relevante sobre cada uno de los requisitos.

Tabla 4.2: Requisitos Funcionales de Agro-Predict

Requisito	Descripción	Nivel	Referencia	Prioridad
RF-01	Obtener datos meteorológicos desde servicios externos oficiales.	Alto	Necesidad identificada por tutor académico	+++
RF-02	Almacenar datos meteorológicos recopilados.	Medio	Análisis arquitectónico del sistema	+++

Requisito	Descripción	Nivel	Referencia	Prioridad
RF-03	Generar predicciones de riesgo de plagas/enfermedades.	Alto	Definición del alcance del proyecto	+++
RF-04	Generar informes resultantes sobre datos de predicciones.	Medio	Reunión concertada con Ingenieros Agrónomos	++
RF-05	Generar predicciones de riesgo de heladas sobre zonas geográficas.	Medio	Criterios definidos por el autor del proyecto	+++
RF-06	Generar predicciones de riesgo de heladas sobre una variedad de cultivo almacenada.	Medio	Criterios definidos por el autor del proyecto	++
RF-7	Flexibilidad de lectura de datos abastecidos tanto por estaciones meteorológicas como sensores IoT.	Medio	Necesidad identificada por el tutor académico	+
RF-8	Predicciones de riesgos de plagas sobre grupos Leguminosos y de Cebada.	Medio	Limitaciones de datos públicos disponibles	++
RF-9	Mecanismo de monitorización frente a alertas de riesgos por plagas/enfermedades o heladas.	Medio	Necesidades identificadas por el tutor académico.	++
RF-10	Identificación y detección de riesgos por heladas blancas y negras.	Medio	Criterios definidos por el autor del proyecto	++
RF-11	Informar mediante recomendaciones preventivas frente a riesgos de heladas o plagas detectados.	Medio	Criterios definidos por el autor del proyecto	++

Requisito	Descripción	Nivel	Referencia	Prioridad
RF-12	Generar predicciones ante riesgos de heladas para un rango temporal determinado (hoy, mañana).	Medio	Análisis del alcance del proyecto	+++
RF-13	Permitir registrar y gestionar cultivos y sus variedades.	Medio	Criterios definidos por el autor del proyecto	++
RF-14	Permitir registrar y gestionar provincias, comunidades autónomas, localidades y estaciones.	Medio	Criterios definidos por el autor del proyecto	+++
RF-15	Generar gráficos visuales sobre datos de predicciones obtenidos.	Medio	Necesidades identificadas por el tutor académico	+
RF-16	Registrar alertas en los informes resultantes cuando el riesgo de heladas y plagas supere un umbral determinado.	Medio	Comportamiento registrado de la herramienta <i>Irristrat</i>	++
RF-17	Monitorizar las horas-frio acumuladas sobre una variedad de cultivo por zona/parcela.	Medio	Reunión concretada con Ingenieros Agrónomos	++
RF-18	Establecer umbrales de registro horas-frio en base a modelos identificados.	Medio	Reunión concretada con Ingenieros Agrónomos	++
RF-19	Establecer registros y relaciones sobre modelos horas-frio que implementa cada variedad de cultivo.	Medio	Reunión concretada con Ingenieros Agrónomos	++

Requisito	Descripción	Nivel	Referencia	Prioridad
RF-20	Implementar comunicación entre DTAgro y los servicios implementados.	Alto	Necesidad identificada por el tutor del proyecto	+++
RF-21	Mantener un registro interno de parcelas, dispositivos y sensores disponibles por DTAgro.	Alto	Necesidad identificada por el tutor del proyecto	+++
RF-22	Proporcionar contexto de cálculo a los informes predictivos resultantes.	Alto	Necesidad identificada por el tutor del proyecto	+++

4.2.2. Requisitos No Funcionales (RNF)

Los requisitos no funcionales identificados se recogen en formato de tabla (véase Tabla 4.3). Se encuentra dividida en diferentes columnas que ayudan a entender las características implícitas que debe contener el sistema, y aunque el potencial cliente no las haya especificado, son necesarias para cumplir con criterios de calidad.

Tabla 4.3: Requisitos No Funcionales de Agro-Predict

Requisito	Descripción	Nivel	Prioridad
RNF-01	Capacidad de aumentar la carga de trabajo en el sistema, incrementando sus recursos sin perder rendimiento (Escalabilidad).	Alto	+++
RNF-02	Capacidad de mantener el sistema estable ante una carga elevada de tráfico.	Alto	+++
RNF-03	Mantener las actividades sensibles del sistema aplicando mecanismos de seguridad (Seguridad).	Alto	+++
RNF-04	Disponer de una versión del sistema intuitiva y fácil de utilizar para cualquier tipo de usuario (Usabilidad).	Alto	+++

Requisito	Descripción	Nivel	Prioridad
RNF-05	Funcionamiento correcto del sistema, sin fallos durante un tiempo específico (Fiabilidad).	Alto	+++

Capítulo 5:Diseño y modelado del sistema

En este capítulo se presenta el diseño y modelado de la infraestructura desarrollada en el proyecto. El objetivo de esta fase se centra en la definición estructurada de los diferentes componentes que forman parte de la arquitectura interna del sistema, junto con sus relaciones y comportamientos entre ellos, consiguiendo así los hitos definidos.

Para ello, se emplean diferentes formas de modelado, permitiendo representar el sistema desde diferentes perspectivas. La introducción inicial de diagramas de **contexto** y **componentes**, ofrecen una visión global sobre las diferentes piezas que conforma la infraestructura y su relación entre sí. Posteriormente, se ofrecen diagramas de caso de uso, describiendo las funcionalidades que se proporciona a los usuarios.

Finalmente, se incorporan diagramas de **flujo y secuencia**, permitiendo conocer las diferentes interacciones y relaciones entre las funcionalidades que define cada componente durante la ejecución del software.

Estos modelos permiten comprender de forma clara la estructura y el funcionamiento del proyecto desarrollado antes de su implementación, permitiendo así disponer de una guía abstracta para la fase de desarrollo y evolución de la solución propuesta.

5.1. Diagramas de contexto y componentes

Diagrama de contexto

Los diagramas de contexto proporcionan una vista de alto nivel sobre un sistema. Se trata de un medio utilizado para definir una entidad basándose en sus límites de alcance y en su relación con los componentes externos, como las partes interesadas (véase Figura 5.1).

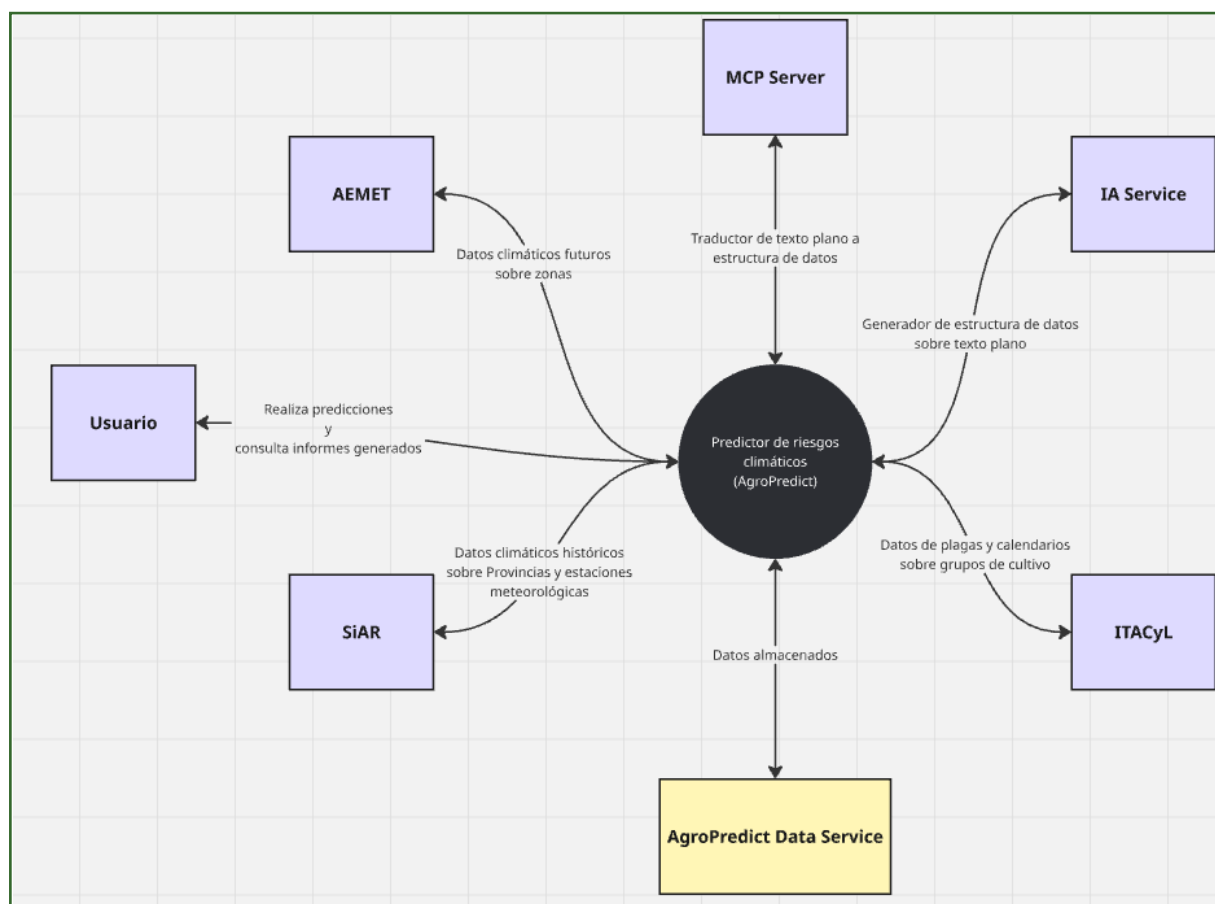


Figura 5.1: Diagrama de Contexto.

Fuente: Elaboración propia

La Figura 5.1 representa el diagrama de contexto sobre el sistema **AgroPredict** desarrollado. Su dominio se representa mediante un círculo, que contiene en su interior todo aquello que se encuentra bajo su control (funciones, responsabilidades de trabajo, procesos).

A partir del núcleo del diagrama, se establecen asociaciones hacia las diferentes entidades externas o, en este caso, servicios, puesto que la infraestructura está basada en una arquitectura orientada a microservicios (véase la Sección 5.5 para conocer más sobre la arquitectura utilizada). Estas entidades representan los sistemas que funcionan de manera externa e interactúan con el producto. Entre los servicios externos representados destacan los proporcionados por AEMET (Agencia Estatal de Meteorología), SiAR (Sistema de Información Agroclimática para el Regadío) ITACyL (Instituto Tecnológico Agrario de Castilla y León), que suministran información meteorológica y agrícola, basada en estaciones y sensores, utilizada por *AgroPredict* para la generación de predicciones de riesgos climáticos.

Las asociaciones entre estas dos representaciones principales representan los flujos de datos que se

intercambian entre el sistema y los servicios externos, los cuales quedan definidos en los diferentes enlaces de asociación.

Cabe destacar el servicio *AgroPredict Data Service*, ya que contiene la base de datos que abastece el sistema, junto con el servicio interno, encargado de su gestión y abastecimiento.

Diagrama de componentes

Los diagramas de componentes representan las **relaciones** entre los componentes individuales del sistema (véase Figura 5.2).

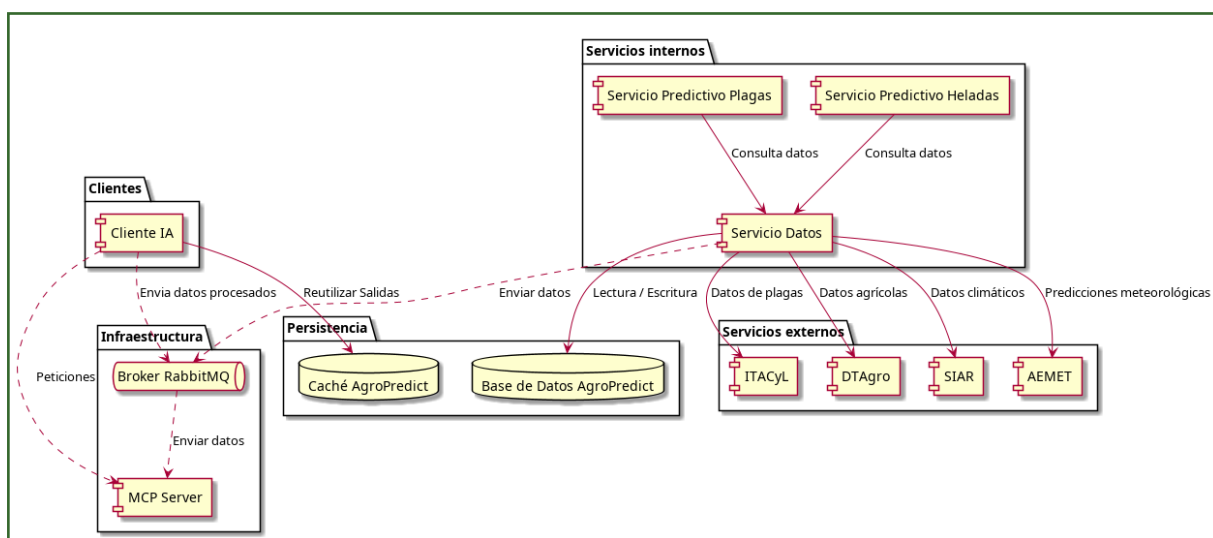


Figura 5.2: Diagrama de Componentes.

Fuente: Elaboración propia

La Figura 5.2 representa el diagrama de componentes sobre el sistema *AgroPredict*. En él, cada uno de los componentes definidos se organizan en distintos **paquetes**, lo que permite clasificar las diferentes partes que tienen comportamientos en común sobre la estructura de la arquitectura del sistema.

La asociación entre componentes refleja cómo se conectan las distintas partes del sistema, así como el flujo de datos que intercambian entre ellas. De este modo, se puede observar la interacción entre los servicios internos, la capa de persistencia y los servicios externos, así como el uso de mecanismos de comunicación asíncrona para la transformación de cierta información.

5.2. Diagrama de casos de uso

Los diagramas de caso de uso representan el modelado de las funcionalidades en un sistema, mostrando las interacciones entre los actores identificados y los procesos. Permiten visualizar desde un nivel alto de abstracción qué hace el sistema sin entrar en detalle (véase Figura 5.3).

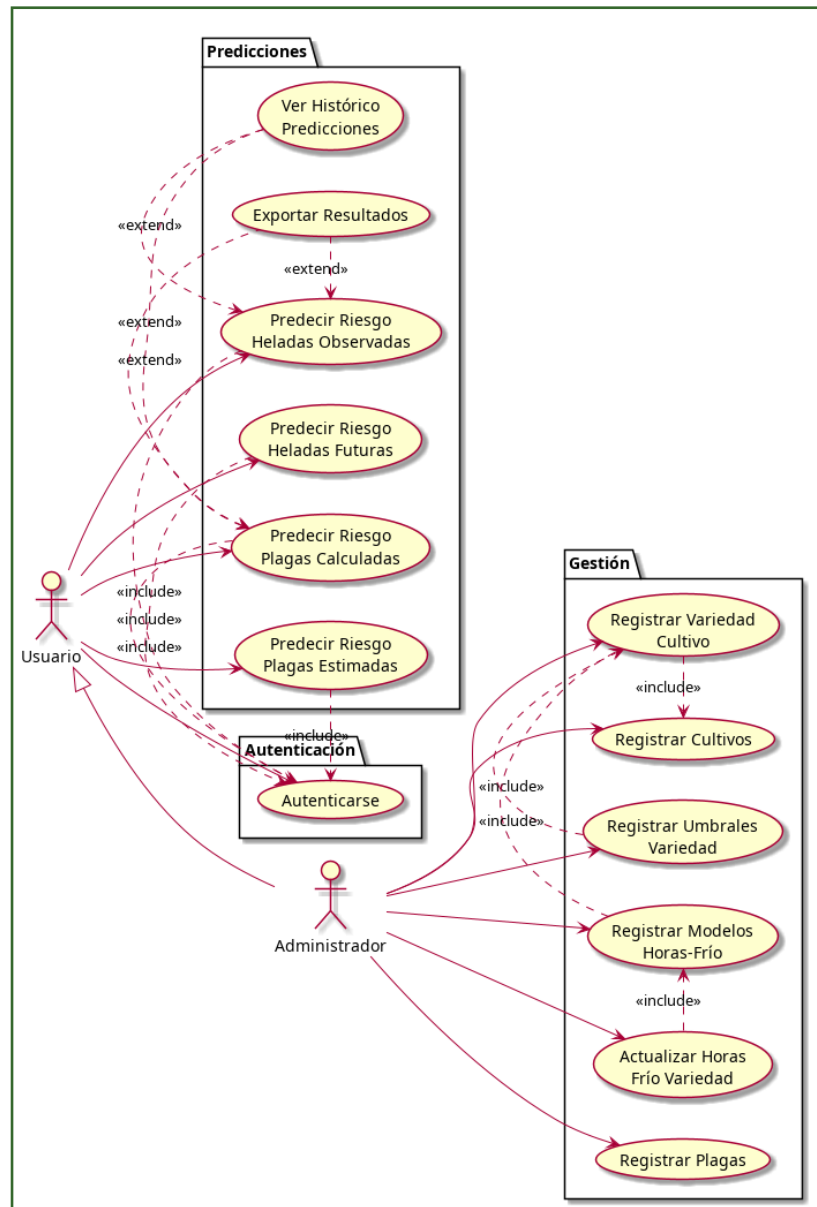


Figura 5.3: Diagrama de Casos Uso.
Fuente: Elaboración propia

Tras la representación del diagrama (Figura 5.3), se puede observar cómo las funcionalidades identificadas

para el sistema se dividen en dos grandes bloques: las funcionalidades de predicción y las funcionalidades de gestión. Los actores definidos interactúan con estos casos de uso en función del rol que presentan en el sistema y de las restricciones de acceso asociadas a cada funcionalidad.

5.3. Diagramas de secuencia

Los diagramas de secuencia representan el flujo de comunicación interna que se produce en el sistema cuando un actor realiza una acción. Permiten conocer el intercambio de mensajes entre los distintos módulos, el tipo de comunicación aplicada y la lógica seguida en el proceso para obtener una solución.

En esta sección se podrían definir una cantidad elevada de diagramas de secuencia para cada funcionalidad interna y externa que presenta el sistema. Sin embargo, se ha decidido representar únicamente aquellos asociados a las funcionalidades predictivas, debido a su importancia de uso y con el objetivo de simplificar la explicación.

5.3.1. Genérico

Antes de introducir los diagramas más específicos relacionados con las funcionalidades predictivas, se presenta una primera visión general que describe los pasos que siguen las distintas lógicas de predicción para obtener el resultado esperado (véase Figura 5.4).

El flujo representado en el diagrama (Figura 5.4), comienza con la solicitud de predicción por parte del usuario que interactúa con el sistema. Esta petición se envía al servicio predictor, encargado de alimentar su lógica interna con los datos necesarios (históricos, futuros, plagas, entre otros), obtenidos a través del **servicio de datos**.

Los parámetros soportados en las solicitudes de los distintos predictores definidos se recogen en la siguiente Subsección 6.3.5.

Finalmente, una vez obtenidos los datos procesados por el servicio de datos, se ejecuta la lógica predictiva interna y se transforma la respuesta resultante al formato definido por el usuario en la solicitud de predicción, ya sea JSON, para facilitar la integración con otras herramientas, o PDF mediante un informe que recopila dicha respuesta predictiva, para su entrega final.

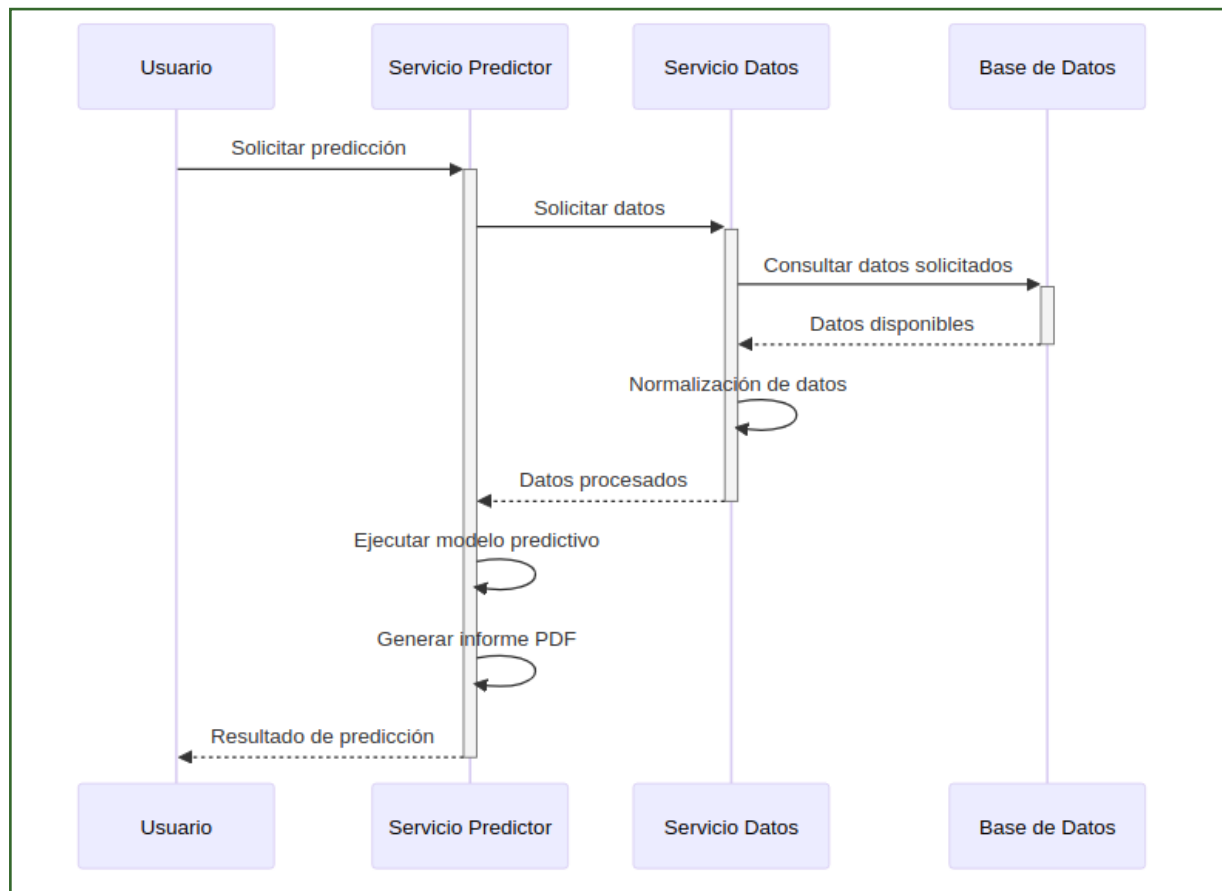


Figura 5.4: Diagrama Secuencia Genérico
Fuente: Elaboración propia

5.3.2. Heladas Observadas

El diagrama de secuencia para la funcionalidad de predicción ante riesgos de heladas por datos observados (heladas observadas), se centra en el uso de datos climáticos históricos para generar una predicción actual, es decir, válida para el día de hoy (véase Figura 5.5).

El flujo representado en el diagrama (Figura 5.5) comienza con la solicitud, por parte del usuario, de predicción ante riesgos de heladas. Esta petición se envía al servicio predictor, encargado de alimentar su lógica interna con los datos necesarios, obtenidos a través del servicio de datos.

Es importante tener en cuenta el procesamiento interno que realiza el servicio de datos al recibir una petición sobre datos almacenados. En primer lugar, consulta la base de datos para comprobar si dispone de los registros asociados a datos climáticos históricos sobre las condiciones incluidas en el cuerpo de la solicitud (véase para visualizar los parámetros requeridos en el cuerpo de la petición). En caso afirmativo,

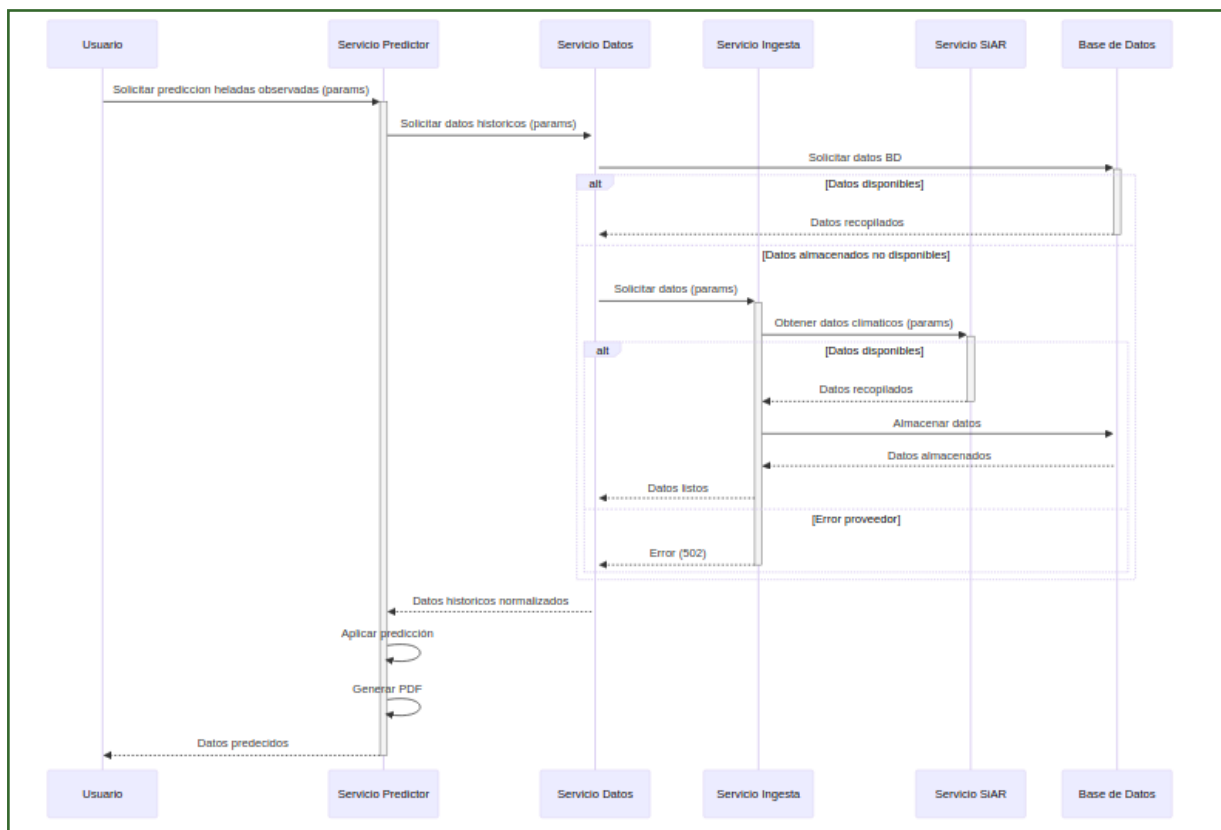


Figura 5.5: Diagrama Secuencia Heladas Observadas
Fuente: Elaboración propia

los normaliza y los envía al servicio predictor.

En caso contrario, si no cuenta con dichos datos, se apoya en el servicio de ingesta para consultar la información necesaria al servicio SiAR, proveedor de datos meteorológicos históricos. De este modo, obtiene los datos históricos requeridos, los almacena en la base de datos y, posteriormente, puede abastecer la solicitud del servicio predictor.

Adicionalmente, se ha implementado una funcionalidad caracterizada por el uso de la herramienta *Celery* [42], encargada de programar y trabajar de manera asíncrona con el reintento de peticiones fallidas. Véase la Sección 6.1 y la Subsección 6.3.2 para conocer más acerca de esta herramienta y su uso interno en la infraestructura para controlar datos meteorológicos históricos.

El flujo de acciones asociados a su comportamiento se representa con el siguiente diagrama de secuencia (véase Figura 5.6).

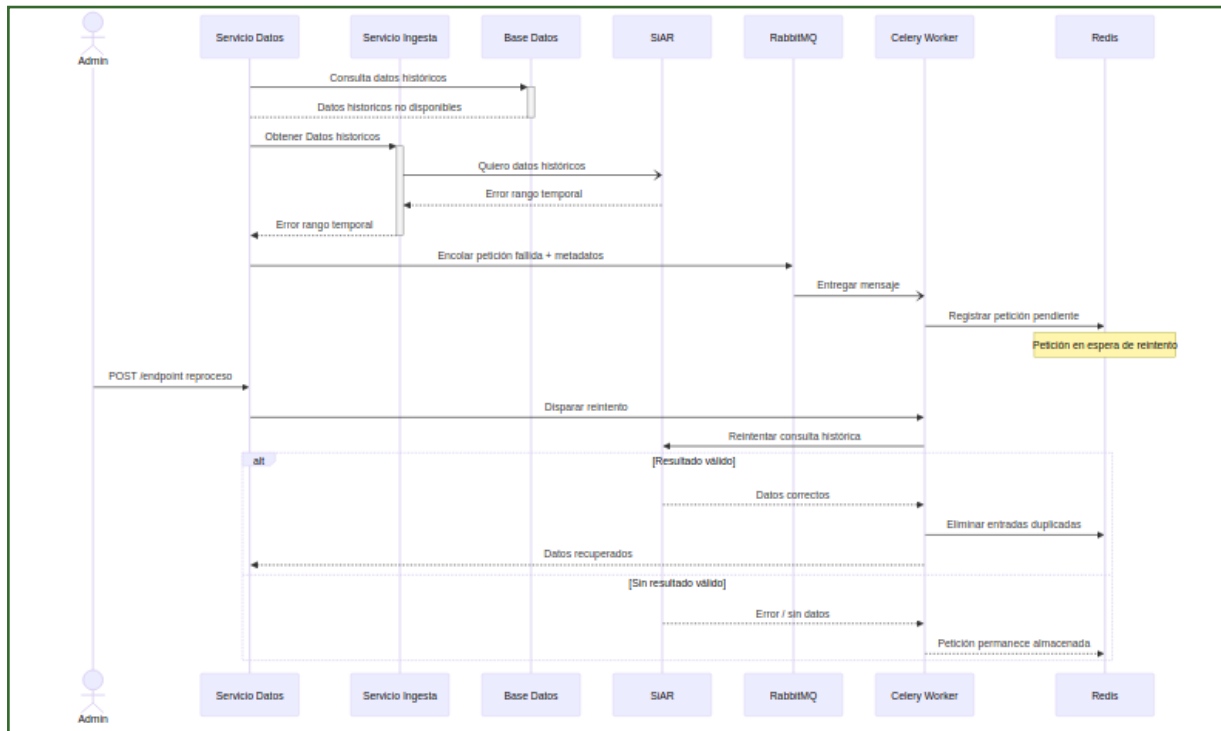


Figura 5.6: Diagrama Secuencia Uso *Celery*

Fuente: Elaboración propia

Finalmente, una vez obtenidos los datos necesarios, se ejecuta la lógica predictiva interna, se genera una respuesta en formato PDF o en formato JSON y se entrega al usuario.

5.3.3. Heladas Futuras

El diagrama de secuencia para la funcionalidad de predicción ante riesgos de heladas futuras, se centra en el uso de datos climáticos futuros y agentes inteligentes para generar una predicción sobre un futuro cercano (mañana, pasado mañana) (véase Figura 5.7).

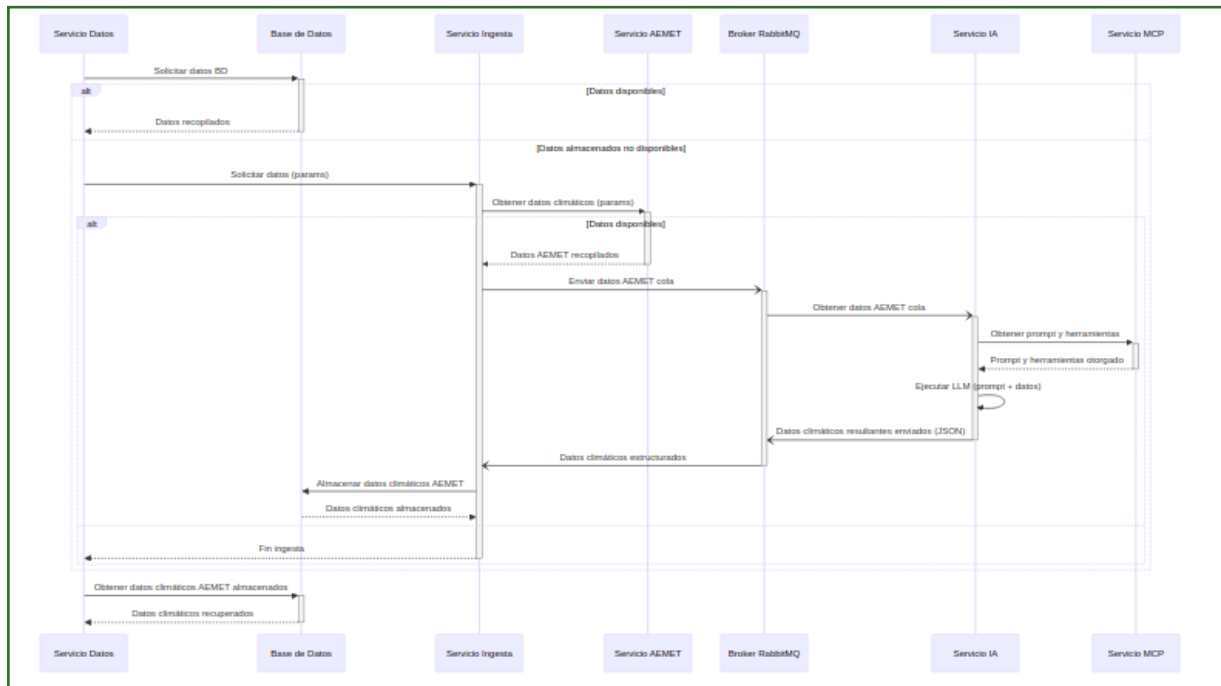


Figura 5.7: Diagrama Secuencia Heladas Futuras

Fuente: Elaboración propia

El flujo representado en el diagrama (Figura 5.7) presenta un comportamiento particular respecto a los diagramas ya analizados. El flujo inicial se mantiene similar al de los casos anteriores, con la diferencia de que el proveedor al que el servicio de ingesta solicita los datos es AEMET.

Debido a que este servicio externo (AEMET) devuelve la información en formato **texto plano**, se implementa un mecanismo de comunicación asíncrona mediante colas **RabbitMQ** [43], a través del componente *broker* [44]. En este proceso, los datos obtenidos de AEMET se envían a un servidor MCP (*Model Context Protocol*) [45], encargado de almacenarlos y de proporcionar a un servicio *IA* las herramientas (*tools*) y las instrucciones (*prompt*) necesarias para transformar el texto plano en un formato más manejable y compatible en la fase de desarrollo.

La respuesta generada por la *IA* se envía de vuelta al servicio de datos, que se encarga de normalizarla y remitirla al servicio predictor (este comportamiento resumido se detallará más adelante) (véase su comportamiento en la Figura 5.8).

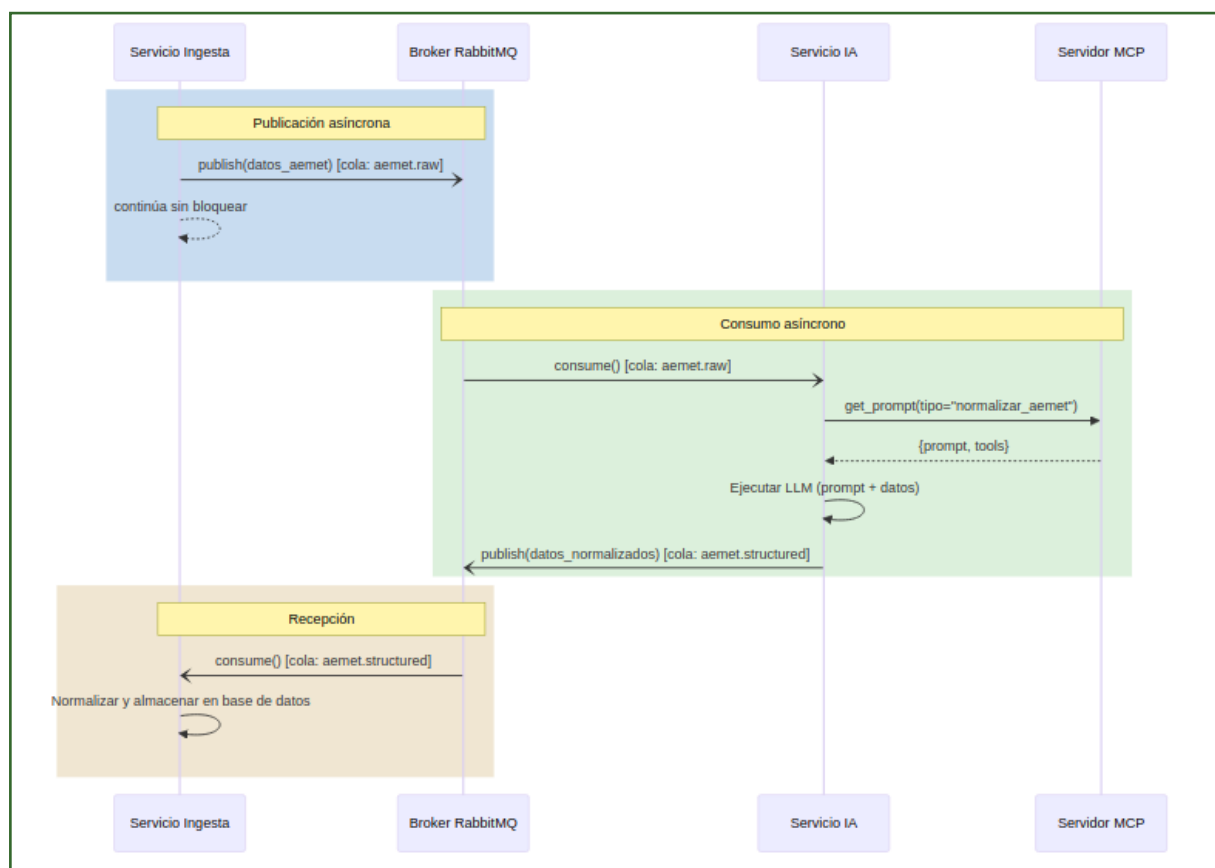


Figura 5.8: Flujo asíncrono que describe la utilización de un sistema basado en IA generativa para analizar datos con carácter de pronóstico por AEMET.

Fuente: Elaboración propia

Finalmente, una vez obtenidos los datos necesarios, se ejecuta la lógica predictiva interna, se genera un informe en formato PDF o una respuesta en formato JSON y se entrega al usuario.

5.3.4. Plagas Calculadas

El diagrama de secuencia para la funcionalidad de predicción ante riesgos de plagas calculadas se centra en el uso de datos relacionados con plagas y enfermedades que afectan a grupos de cultivos tales como **cereales** y **leguminosos**, atendiendo a calendarios de riesgos anuales por cada plaga y enfermedad afectante (véase Figura 5.9).

El flujo representado en el diagrama (Figura 5.9) comienza con la interacción del usuario con el sistema, mediante la consulta de predicciones sobre plagas y enfermedades en cultivos pertenecientes a grupos como **cereales** y **leguminosos** (únicamente soportados).

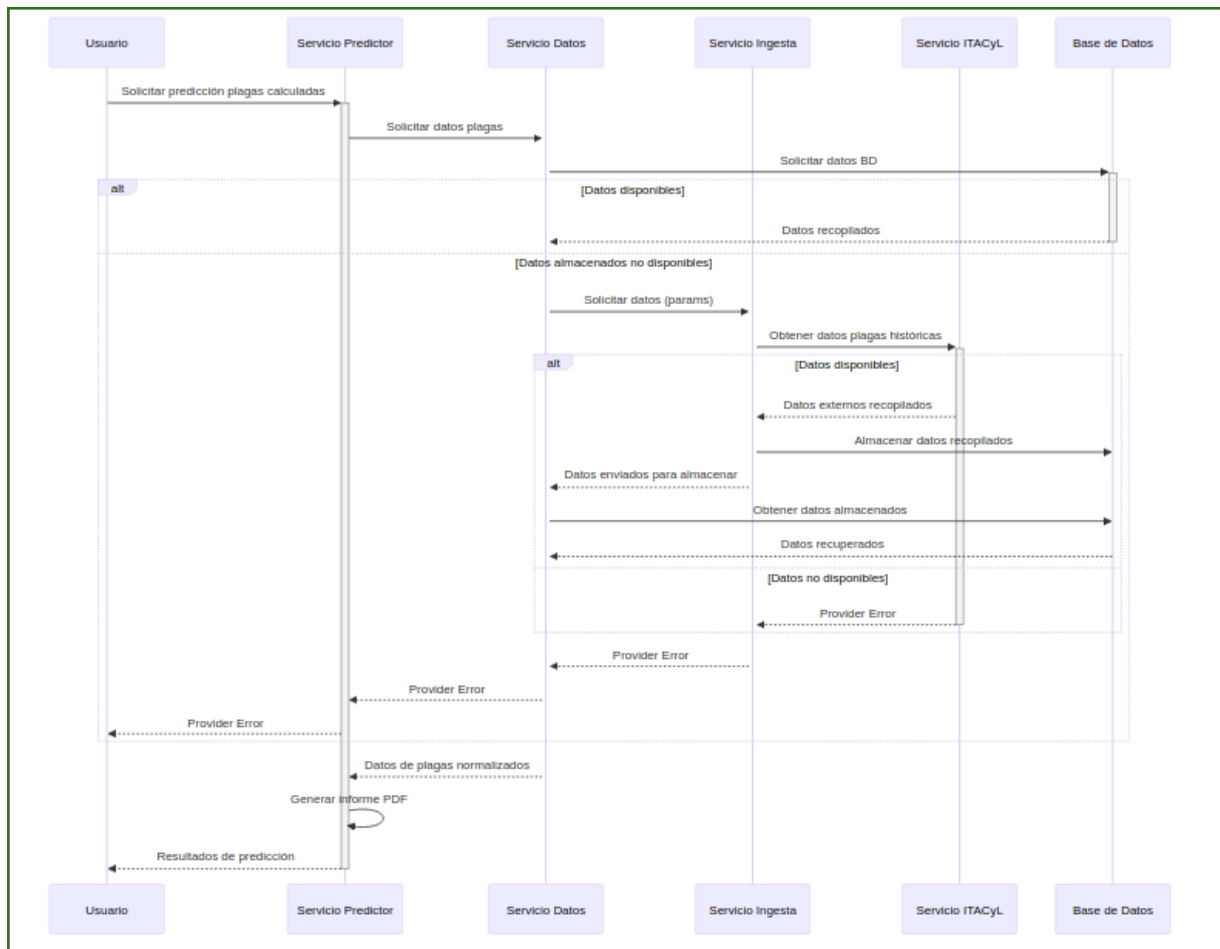


Figura 5.9: Diagrama Secuencia Plagas Calculadas
Fuente: Elaboración propia

El servicio predictor procesa la solicitud y solicita al servicio de datos la información necesaria para generar los resultados.

El servicio de datos, en primer lugar, consulta la base de datos del sistema en busca de plagas **con calendario** asociadas al grupo de cultivo que pertenece el cultivo indicado en la petición. Si los datos están disponibles, los normaliza y los devuelve al servicio predictor. En caso contrario, delega en el servicio de ingesta la obtención de la información requerida, consultando fuentes externas como ITACyL. Una vez obtenidos los datos desde dichos servicios externos, el servicio de ingesta los almacena en la base de datos, permitiendo así su posterior reutilización por el servicio de datos.

Finalmente, con los datos necesarios disponibles, se evalúan los niveles de riesgo asociado a cada calendario de las plaga o enfermedad obtenidas (aspecto que se detallará más adelante), generando un informe en formato PDF o una respuesta en formato JSON que se devuelve al usuario.

5.3.5. Plagas Estimadas

El diagrama de secuencia correspondiente a la funcionalidad de predicción de riesgos de plagas y enfermedades estimadas emplea datos de los cultivos asociados a una parcela, previamente registrados en el sistema, los cuales mantienen una relación con las plagas o enfermedades que pueden afectarles. Asimismo, integra información climática procedente de sensores proporcionados por DTAgro, con el fin de obtener resultados más exactos (véase Figura 5.10).

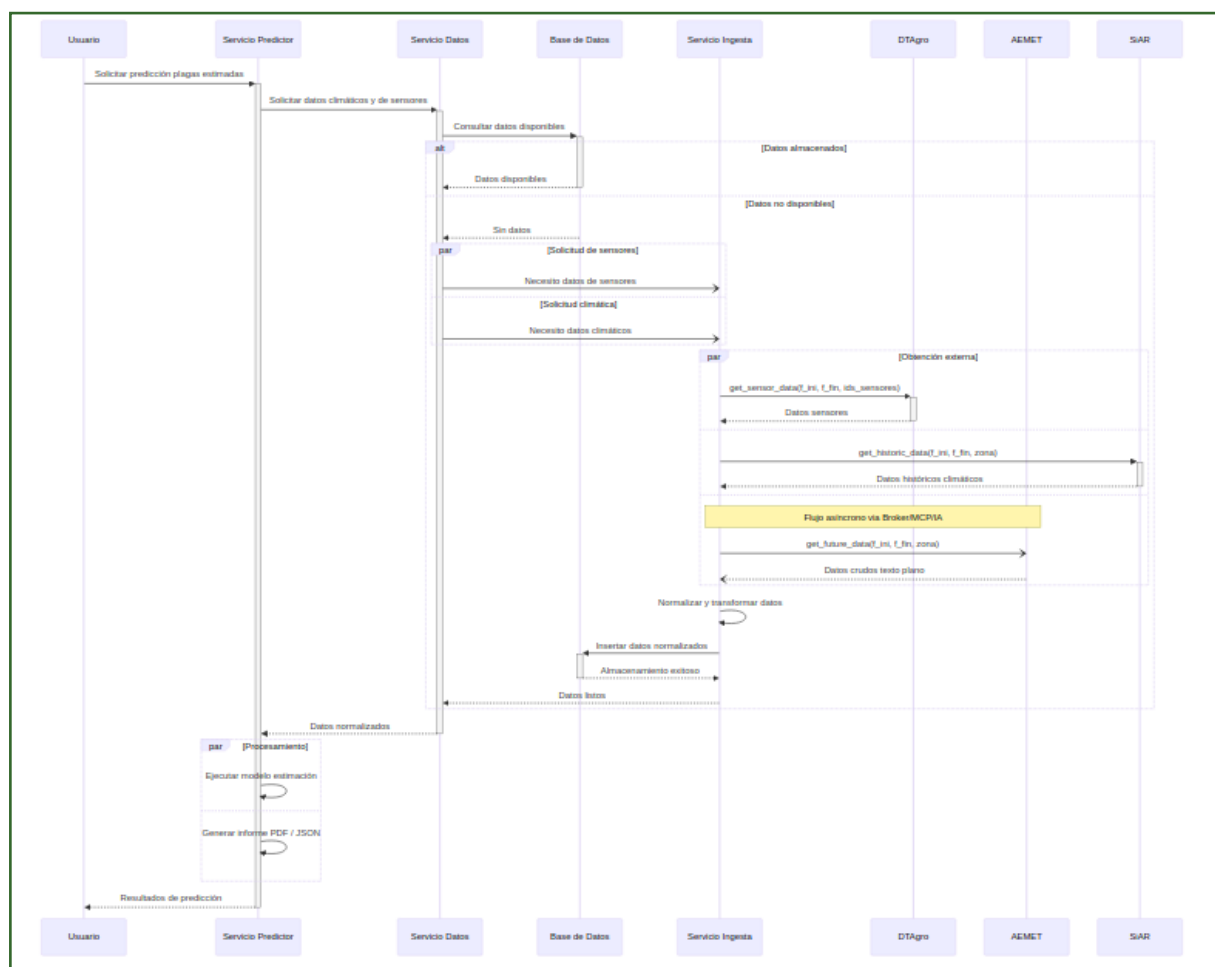


Figura 5.10: Diagrama Secuencia Plagas Estimadas
Fuente: Elaboración propia

El flujo representado en el diagrama comienza con la solicitud del usuario hacia el sistema, para obtener una predicción ante riesgos por plagas o enfermedades sobre un cultivo indicado por parámetros.

El servicio predictor, encargado de proveer una solución al usuario, requiere de datos climáticos específicos sobre las plagas y enfermedades que afectan al cultivo seleccionado. Para ello, consulta estos datos al servicio de datos, que intentará obtenerlos desde la base de datos asociada. En caso de disponer de ellos, los normaliza y los envía al servicio predictor, que procesa la lógica de riesgo interna y genera finalmente un informe en formato PDF o una respuesta en formato JSON, que se devuelve al usuario.

Por el contrario, puede existir la posibilidad en la que no se disponga de datos climáticos sobre plagas y enfermedades para el cultivo especificado. En ese caso, el servicio de datos lanza de forma concurrente dos solicitudes asíncronas al servicio de ingesta: una para la obtención de datos de sensores y otra para la obtención de datos climáticos. El servicio de ingesta, a su vez, realiza en paralelo tres peticiones a

servicios externos: los datos de sensores se solicitan a DTAgro, los datos históricos climáticos a SiAR, y los datos de predicción meteorológica a AEMET. Esta última petición sigue un flujo asíncrono especial a través del componente *broker*, el servidor MCP y el servicio *IA*, encargado de transformar la respuesta en texto plano de AEMET en un formato estructurado, tal y como se detalla en la Figura 5.8.

Una vez recopiladas las respuestas de los tres servicios externos, el servicio de ingesta normaliza y transforma los datos obtenidos, procediéndose a su inserción en la base de datos. Confirmado el almacenamiento, los datos se disponen al servicio de datos, que los envía normalizados al servicio predictor. Finalmente, este ejecuta el modelo de estimación y genera el informe resultante de forma concurrente, devolviendo el resultado al usuario.

5.4. Modelo de datos

Uno de los puntos críticos sobre el conjunto de sistemas desarrollado reside en el tratamiento de datos. Para comprender mejor la estructuración de los datos y como se encuentran relacionados unos con otros, a continuación, se mostrará en la Figura 5.11 el **modelo entidad/relación** visual almacenado en la base de datos y se explicará el motivo de existencia sobre las tablas más relevantes.

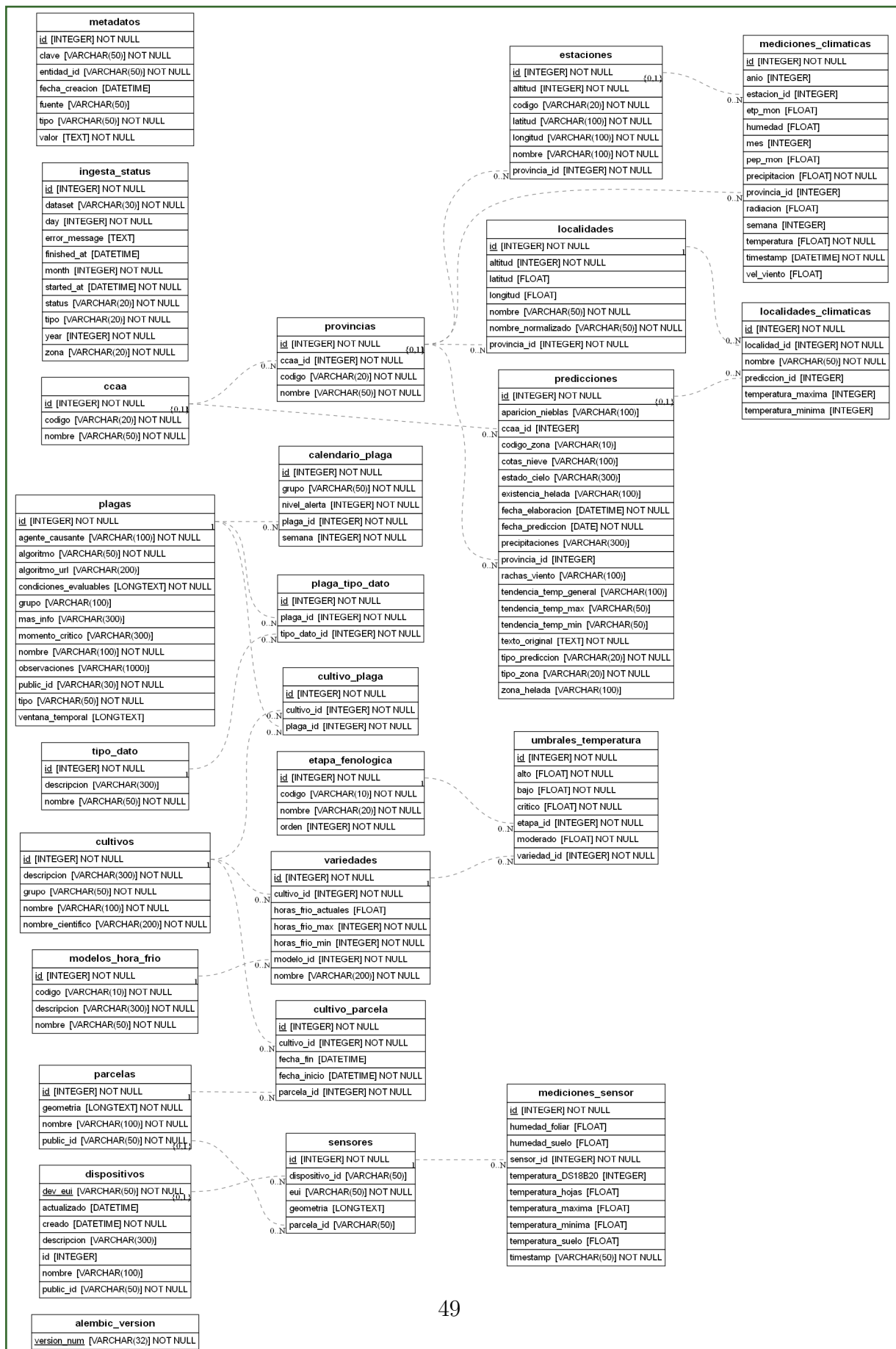


Figura 5.11: Modelo E/R.
Fuente: Elaboración propia.

5.4.1. Descripción Tablas Características

Para conocer de forma precisa el modelo generado en la Figura 5.11, las diferentes tablas disponibles se pueden agrupar en bloques independientes, representando un dominio concreto de almacenamiento dentro del sistema.

1. **Datos Climáticos y Geográficos:** Este bloque gestiona la información meteorológica, tanto histórica como predictiva, junto con el contexto geográfico necesario para su correcta interpretación. Constituye la base sobre la que operan los servicios predictores, proporcionando los datos necesarios para el análisis de riesgos.

Las principales estructuras de almacenamiento que conforman este dominio son:

- Provincias
- Comunidades Autónomas (CCAA)
- Localidades
- Estaciones
- Predicciones
- Localidades_Climáticas
- Mediciones_Climáticas

Dentro de este conjunto, es preciso conocer el motivo de creación sobre las tablas: **localidades_climaticas**, **predicciones** y **mediciones_climaticas**.

En primer lugar, las **localidades_climaticas** surgen como resultado de descomponer la información proporcionada por el servicio de AEMET. Dado que las predicciones realizadas sobre el nivel provincial contienen información específica para distintas localidades, se implementa esta tabla intermedia para asociar cada predicción con los valores concretos de cada localidad.

Por su parte, la tabla de **predicciones** registra información asociada a las predicciones obtenida por AEMET. Desde los valores procesados por el servicio *IA*, hasta el texto plano que devuelve para mantener una trazabilidad de contexto.

Finalmente, las **mediciones_climaticas** almacena los datos históricos asociados a los datos obtenidos por el servicio externo de SiAR. Ya sea sobre una estación o sobre una provincia.

2. **Dominio agronómico:** Este bloque gestiona información relacionada con cultivos y variedades asociadas, junto con las diferentes fases de su ciclo de vida asociadas a umbrales de temperatura que ayudan a controlar el riesgo climático del cultivo. Por otro lado, gestiona los diferentes modelos de horas de frío que registra el sistema para utilizar en los servicios predictores.

Las principales estructuras de almacenamiento que conforman este dominio son:

- Cultivos
- variedades
- Etapa_Fenologica
- Umbrales_Temperatura
- Modelos_Hora_Frio

Dentro de este conjunto, es preciso conocer el motivo de creación sobre las tablas: **etapa_fenologica**, **umbrales_temperatura** y **modelos_hora_frio**.

En primer lugar, la tabla **etapa_fenologica** sirve para almacenar las distintas fases del ciclo de vida del cultivo. Inicialmente, no se consideró implementar una tabla única para almacenar este tipo de datos, pero si el cliente quiere registrar en un futuro, nuevas etapas fenológicas que no se encuentran ya almacenadas o se quiere diferenciar las diferentes etapas que tiene cada cultivo de manera independiente, fue conveniente crear la tabla.

Por su parte, la tabla de **umbrales_temperatura** sirve como tabla intermedia entre la asociación de la entidad **variedades** y la entidad **etapa_fenologica**. Internamente, almacena sobre cada etapa fenológica que registran las variedades, sus umbrales **críticos, altos, moderados o bajos** de temperatura. Sirviendo de ayuda sobre la predicción ante riesgos de helada.

Finalmente, la tabla de **modelos_hora_frio** tiene el mismo contexto de implementación que la tabla **etapa_fenologica**. Almacena los diferentes modelos de horas de frío que pueden utilizar las variedades de cultivo para contabilizar sus respectivas horas de frío y pueden insertarse más modelos en función de las necesidades del cliente.

3. **Plagas:** Este bloque almacena toda la información requerida para conseguir el comportamiento esperado en la lógica relacionada con las plagas y sus derivados. Incorpora información de plagas, calendarios de riesgos sobre cada plaga y requisitos necesarios para predecir el riesgo de una plaga, entre otros.

Las principales estructuras de almacenamiento que conforman este dominio son:

- Plagas
- Tipo_Dato
- Plaga_Tipo_Dato
- Calendario_Plaga
- Cultivo_Plaga

Dentro de este conjunto, es preciso conocer el motivo de creación sobre las tablas: **tipo_dato**, **plaga_tipo_dato**, **cultivo_plaga**.

En primer lugar, la tabla **tipo_dato** se encarga de almacenar las diferentes variables sobre las que se deben requerir datos para predecir el riesgo de una plaga. El motivo de esta implementación reside en la escalabilidad de este comportamiento. El cliente puede registrar más variables a tener en cuenta y se pueden crear asociaciones entre plagas para obtener de una forma precisa, sobre qué variables concretas se debe almacenar información para predecir una plaga.

Una vez comentada la tabla **tipo_dato**, la tabla **plaga_tipo_dato** es una tabla intermedia entre plagas y tipos de datos, asociando a cada plaga las variables requeridas para su predicción de riesgos. Esto servirá de ayuda a los servicios predictores para consultar únicamente datos relacionados con estas variables sobre los servicios externos para predecir el riesgo de una plaga.

Finalmente, se debe mantener un registro de las diferentes plagas que arremeten sobre los cultivos todos los años. Esto se consigue con la implementación de la tabla intermedia entre **cultivo** y **plaga**.

4. **Sensores y Datos Reales:** Con objetivo de mantener registros de datos relacionados con sensores y datos reales, este bloque se centra en el almacenaje de datos sobre las mediciones que se obtienen sobre determinados sensores, los cultivos que pertenecen a una parcela e información relacionada con entidades físicas como sensores, dispositivos y parcelas.

Las principales estructuras de almacenamiento que conforman este dominio son:

- Sensores
- Dispositivos
- Parcelas
- Cultivo_Parcela
- Mediciones_Sensor

Dentro de este conjunto, es preciso conocer el motivo de creación sobre las tablas: **dispositivos**, **cultivo__parcela**, **mediciones__sensor**.

A parte de mantener registros sobre sensores y parcelas en sus respectivas tablas, el motivo de crear la tabla para **dispositivos** reside en la clasificación de los tipos de sensores: virtuales o reales, desplegados en el campo.

Por otro lado, para una mayor trazabilidad y mantenimiento en el progreso de desarrollo sobre el proyecto, se implementó una tabla intermedia, llamada *cultivo__parcela* entre cultivos y parcelas, donde recoge la asociación de ambos.

Finalmente, la tabla **mediciones__sensor** almacena información sobre las mediciones que se obtienen por los servicios externos como DTAgro sobre los sensores registrados en el sistema.

5. **Soporte / Sistema:** Este bloque engloba las estructuras auxiliares que, aun no formando parte directa de la lógica de negocio sobre los servicios predictores, son esenciales para garantizar el correcto funcionamiento, trazabilidad y mantenimiento del sistema.

A diferencia de los dominios explicados, centrados en el modelado agronómico y climático, este conjunto de tablas tiene como objetivo ayudar internamente a procesos como la ingesta de datos, el control de estados y la gestión del sistema.

Las principales estructuras de almacenamiento que conforman este dominio son:

- Metadatos
- Alembic_Version
- Ingesta_Status

En primer lugar, la tabla `metadatos` sirve para almacenar los registros de sensores, dispositivos y parcelas que se han registrado en el sistema, mediante la importación de un fichero. Sirve como ayuda en la trazabilidad sobre ingesta de datos a través de ficheros.

A su vez, la tabla `alembic_version` se autogeneró al incorporar migraciones *Alembic* en el proyecto. Internamente almacena el identificador de la versión más reciente con la que se está trabajando sobre el modelo de datos.

Finalmente, la tabla `ingesta_status` almacena los diferentes estados de ingesta por los que pasa una operación de ingesta de datos. Este aspecto ayuda a comprobar si se ha completado correctamente la ingesta, se han producido errores o si está en proceso.

5.5. Diseño de la arquitectura del sistema

Desde las primeras fases de diseño, se optó por una arquitectura orientada a microservicios. La principal motivación residió en la necesidad de independencias entre funcionalidades, permitiendo la evolución autónoma de cada servicio. Esta decisión de diseño se originó por los constantes cambios que sufren las *APIs* de terceros y la necesidad de disponer de servicios desacoplados, capaces de evolucionar de forma independiente sin afectar al comportamiento general del sistema.

A continuación, se dispone una representación de la arquitectura completa sobre la infraestructura desarrollada, donde se aprecian los diferentes servicios interconectados y las comunicaciones que se encuentra protegidas por la capa de seguridad aplicada (véase Figura 5.12). Todos los servicios representados, sus comunicaciones implementadas y el por qué de su existencia, se encuentran detallados en el Capítulo 6.

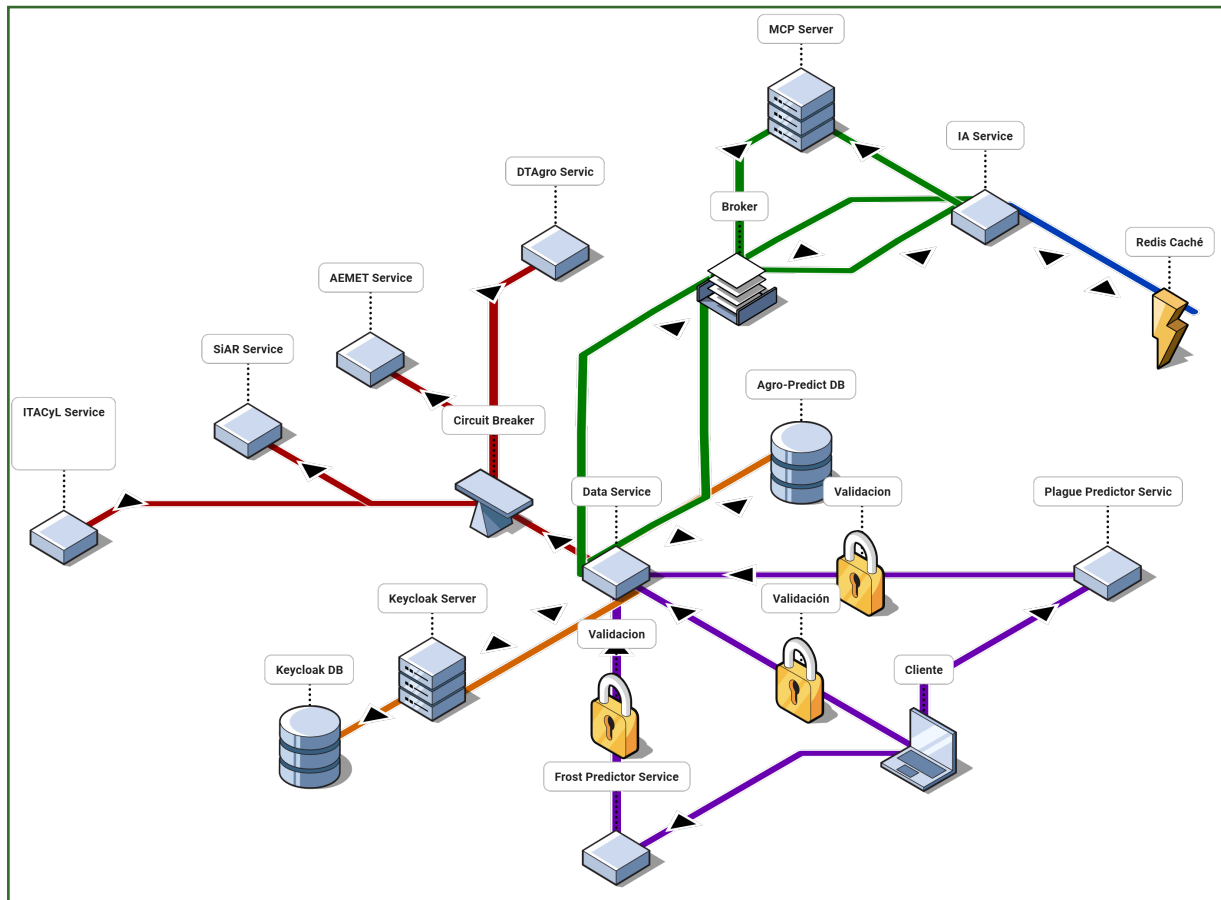


Figura 5.12: Arquitectura Sistema
Fuente: Elaboración propia.

5.6. Software de diseño utilizado

Con el objetivo de documentar las herramientas empleadas en la elaboración de los distintos diagramas presentados en este capítulo, se describen a continuación aquellas que han tenido un papel relevante en el proceso de diseño.

1. **MIRO** [46]: Herramienta web de diseño colaborativo que facilita la creación de diagramas mediante plantillas predefinidas. Su uso ha permitido agilizar la elaboración del diagrama de contexto (véase Figura 5.1), así como mantener una organización visual clara durante la fase de diseño.
2. **MERMAID** [47]: Herramienta que permite crear diagramas y visualizaciones a partir de código de texto simple. A pesar de que su página web incluye un editor de código, se ha decidido utilizar la librería *mmdc-cli*, permitiendo trabajar con la herramienta desde terminal y permitiendo

automatizar la generación de diagramas (véase Figura 5.4, Figura 5.5, Figura 5.7, Figura 5.9, Figura 5.10).

3. **PlantUML** [48]: Lenguaje declarativo de especificación- sobre diagramas UML (*Unified Modeling Language*). Dispone de una versión en la terminal, *plantuml*, capaz de automatizar la generación de diagramas visuales a partir de código (véase Figura 5.2, Figura 5.3).
4. **FossFLOW** [49]: Aplicación web progresiva (PWA) de código abierto para crear diagramas isométricos. Mediante el uso de componentes definidos sobre los principales proveedores en la nube (AWS, Azure, Google Cloud), permite generar diseños de arquitecturas de una manera fácil y entendible para cualquier espectador.
5. **ERAlchemy** [50] La generación de la figura que representa el modelo entidad/relación (véase Figura 5.11) se ha realizado de forma automática mediante una herramienta de *software* libre, basada en *Python* llamada *ERAlchemy*. Su configuración depende de la comunicación interna hacia la base de datos.

Capítulo 6: Implementación y desarrollo

En este capítulo se describe la fase de implementación y desarrollo de la solución propuesta para dar respuesta al problema planteado. En él se detallan las distintas etapas seguidas durante el proceso de construcción del sistema, así como las decisiones técnicas adoptadas y las tecnologías empleadas.

El objetivo principal de esta fase es materializar el diseño previamente definido, transformándolo en un software funcional mediante la integración de sus distintos componentes. Para ello, se abordan aspectos como la configuración del entorno de desarrollo, la implementación de los diferentes módulos de la infraestructura y la interacción entre los mismos.

6.1. Tecnologías y herramientas utilizadas

En esta fase del ciclo de vida del proyecto, uno de los pasos más importantes a la hora de comenzar con el desarrollo es la elección de las distintas tecnologías con las que se va a trabajar. De esta manera, se puede adquirir una visión espacial de las posibilidades y limitaciones que aporta cada una.

A lo largo de la carrera universitaria, se ha intentado trabajar con numerosas herramientas, otorgando conocimientos necesarios para que, a la hora de abordar un proyecto de estas magnitudes, se puedan elegir las decisiones más acordes para su desarrollo.

Comunicación

Dado que se decidió desde un principio que la **arquitectura orientada a microservicios** sería la más apropiada para este tipo de proyecto (véase Sección 6.2), la correcta definición del tipo de comunicación entre los distintos **microservicios**, suponía una de las principales decisiones a tomar, debido a que la configuración de cada microservicio dependía de este aspecto.

Los tipos de comunicaciones estándar en el marco del desarrollo software son:

- Comunicación *RESTful* (*Representational State Transfer*) [51]: También conocida como **comunicación síncrona**
- Comunicación por mensajería: También conocida como **comunicación asíncrona**

En una primera propuesta de solución, solo se consideró la configuración de **comunicación síncrona** entre microservicios. Sin embargo, con el transcurso del desarrollo, evolucionó los aspectos a implementar con el uso de agentes inteligentes (*IA*), incorporando **comunicación asíncrona** en ciertas secciones de la arquitectura. Finalmente, se obtuvo como resultado, una configuración de comunicación **híbrida** (incorporación de ambos mecanismos en la arquitectura).

Servicios externos

Como se ha indicado previamente en el diseño del proyecto, el sistema dispone de una serie de servicios encargados de comunicarse directamente con las *APIs* públicas sobre servicios de terceros.

Estos servicios fueron implementados en una fase temprana del desarrollo, con el objetivo de garantizar una implementación sencilla, mantenible y fácilmente adaptable a cambios, debido a los constantes cambios de este tipo de *APIs*,

El *stack* tecnológico utilizado en cada uno de ellos es el siguiente:

- **Servicios SiAR e ITACyL:** Implementados utilizando el *framework* para Java ***Spring Boot v.3.5.6*** [52], basado en el ecosistema ***Spring***, lo que permite una integración robusta y estructurada con servicios externos.
- **Servicio AEMET:** Implementado en el lenguaje de programación ***Elixir*** [53], un lenguaje funcional y concurrente que se ejecuta sobre la máquina virtual de *Erlang* [54].

La elección de este lenguaje reside en las características que aporta. A nivel funcional, el desempeño de este servicio se centraba en realizar consultas periódicas a la *API-REST* de AEMET. *Elixir* facilita esta tarea mediante la configuración de programadores y relojes lógicos que permiten mantener concurrencia entre procesos.

Además, se utilizó el gestor de paquetes estándar del lenguaje, *Mix* [55]. Inicialmente, se valoró el uso del *framework Phoenix* [56], pero se descartó debido a que añadía complejidad innecesaria en la configuración y dificultaba la adaptación a posibles cambios en la *API* de AEMET.

Persistencia de datos

El sistema en conjunto depende de los datos que se almacenan en la base de datos sobre el sistema gestor de bases de datos elegido, *MariaDB v10.11* [57]. Su configuración y uso, facilitaron la asociación entre ciertos servicios dedicados, con la base de datos que almacena el gestor.

MariaDB destaca por ser un sistema gestor de bases de datos **relacional**, *OpenSource* y compatible con otros sistemas gestores como *MySQL*, ofreciendo buen rendimiento y mejoras funcionales con gestión de código abierto.

Framework por defecto

El *framework* que se ha decidido para implementar la mayor parte de los servicios del sistema (servicio de datos, servicio *IA*, servicios predictivos) es el *microframework* de *Flask* [58]. La elección de su uso se basa en la experiencia de utilización que tiene el autor sobre proyectos educativos y personales, además de que es muy recomendado por personas con experiencia en el sector de **desarrollo web**.

Las características por las que destaca el uso de *Flask* en el proyecto son las siguientes:

- **Trabaja sobre el lenguaje de programación Python**
- **Ligero y minimalista:** Su utilidad está orientada a tipos de trabajos específicos como servicios web y microservicios.
- **Amplia documentación:** Su crecimiento popular desde su lanzamiento en 2010 está siendo exponencial. Desarrolladores con experiencia en el desarrollo web lo están postulando como la sustitución de *frameworks* más robustos (*Django*), para proyectos pequeños y medianos, donde importa el rendimiento y la adaptación en equipos.
- **Compatible con WSGI:** *Flask* está basado en **WSGI** (*Web Server Gateway Interface*) [59], estándar *Python* para el desarrollo de aplicaciones web, gestionando peticiones HTTP síncronas para asegurar la compatibilidad entre servidores y *frameworks*.

Generación de resultados

Cuando el usuario solicita una predicción ante riesgos climáticos, el servicio predictor consultado puede responder (en función de las preferencias de uso) con un informe generado, cuyo contenido refleja los datos obtenidos por la predicción, recomendaciones ante los riesgos, alerta, entre otros.

Para generar estos informes, se ha hecho uso de una librería de código abierto llamada **ReportLab** [60], capaz de crear documentos en formato PDF desde funciones y sentencias definidas con *Python*. Presenta funcionalidades como la generación de gráfico e ilustraciones y formateo de tablas y figuras, entre otros. La amplia documentación asociada a la librería otorga una flexibilidad total al desarrollador para configurar el formato de los ficheros PDF resultantes (véase Figura 6.1).

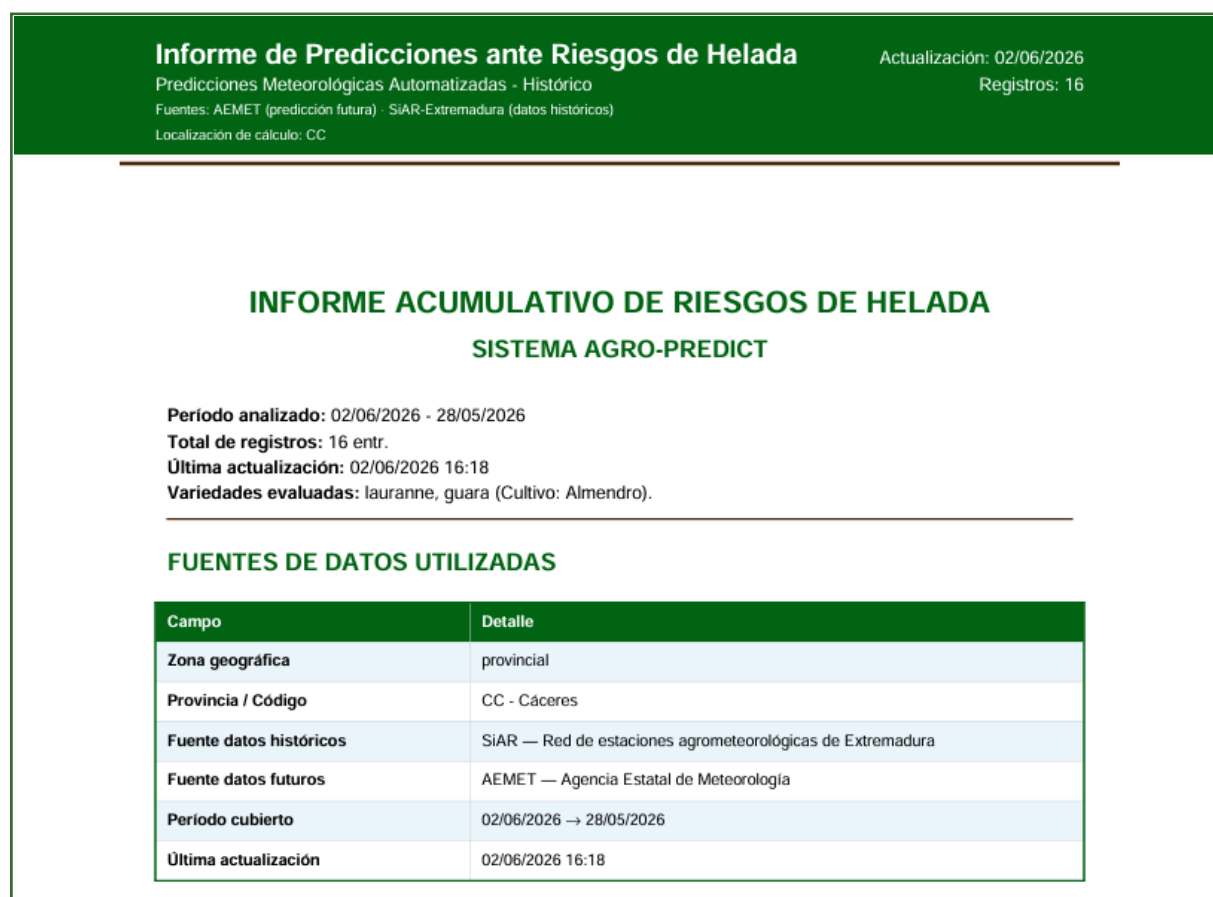


Figura 6.1: Extracto de Informe PDF

Fuente: Elaboración propia

Asimismo, puesto que los usuarios finales en su gran mayoría serán **Ingenieros Agrónomos**, se ha querido incorporar un poco más de profesionalidad en los informes resultantes, añadiendo firmas con certificado digital. Para ello, se ha hecho uso de la librería de código abierto **PyHanko** [61] para *Python*. Dada las credenciales del certificado digital utilizado, disponibles en ficheros con extensión *.key* y *.pem*, se puede generar una firma digital con la posibilidad de incluirse en cualquier parte sobre un documento (véase Figura 6.2).

Además, la generación de certificados digitales por esta librería, mantiene una compatibilidad con los

certificados de la **FNMT** (Fábrica Nacional de Moneda y Timbre), gracias a que es una librería basada en estándares **X.509** y **PKCS#12**, formatos estándar que utiliza la FNMT [62].

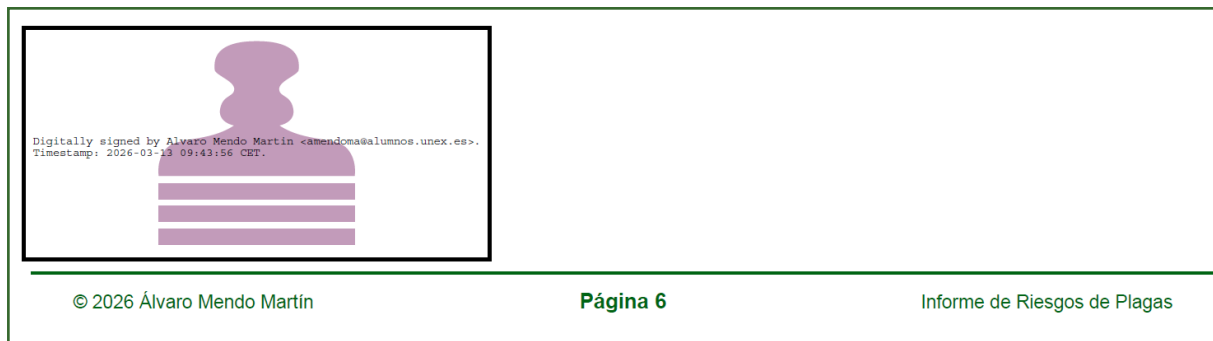


Figura 6.2: Firma Digital con certificado PKCS#12
Fuente: Elaboración propia

Interacciones con la Base de Datos

Para mantener una consistencia de implementación y acceso a datos, se ha decidido implantar en el servicio de datos un **ORM** (*Object-Relational Mapping*), con el objetivo de actuar como *punto* entre la base de datos relacional y el código orientado a objetos. Esta herramienta permite interactuar con bases de datos relacionales mediante programación orientada a objetos, facilitando así el desarrollo y mantenimiento del código.

El *ORM* utilizado por defecto en el servicio de datos es **SQLAlchemy** [63], considerado un estándar dentro del ecosistema de *Python*.

A la hora de decidir su uso en el proyecto, se ha analizado en detalle su función dentro del servicio encargado del tratamiento de datos. En aquellos casos donde la lógica interna es especialmente compleja, el uso de un *ORM* puede no resultar la mejor opción. Sin embargo, tras evaluar que la mayoría de las interacciones entre el servicio y la base de datos se basan en operaciones **CRUD** (*Crear, Leer, Actualizar y Eliminar*), se optó por su utilización.

Por otro lado, se utilizó una herramienta de migración, en otras palabras, un programa que permite gestionar la evolución del esquema de una base de datos, integrada con el *ORM* de **SQLAlchemy**, denominada **Alembic** [64]. Gracias a esta herramienta, cada vez que se realizan modificaciones en el esquema, es posible generar migraciones que permiten **actualizar** de forma controlada la estructura de la base de datos.

Asimismo, **Alembic** mantiene un historial de versiones dentro del propio sistema de archivos del proyecto,

donde se almacena el código que ha ejecutado, necesario para aplicar los cambios en el esquema, pudiendo monitorizar modificaciones sobre el esquema y permitiendo **retroceder** a versiones anteriores registradas.

Conectores comunicación asíncrona

En ciertas acciones internas del sistema (véase Sección 6.3) se ha identificado la necesidad de emplear comunicación asíncrona, debido a los retardos de respuesta derivados del uso de *IA*.

Con el objetivo de gestionar este tipo de comunicación, se ha requerido la configuración y uso de un **Broker**, entendido como una entidad software que actúa como intermediario entre diferentes aplicaciones o sistemas, facilitando la comunicación y el intercambio de mensajes (véase Figura 6.3).

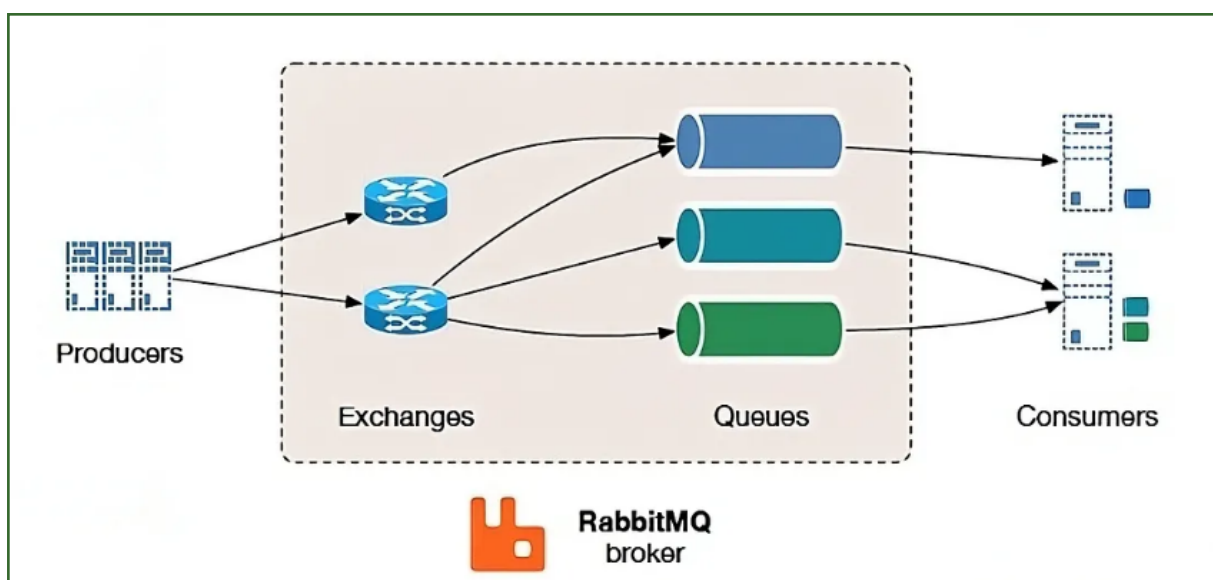


Figura 6.3: Broker RabbitMQ

Fuente: [65]

Este *broker* se ha configurado utilizando el protocolo de comunicación asíncrona **RabbitMQ**, ampliamente adoptado en entornos reales y profesionales. Dado que es necesario realizar tareas de gestión y mantenimiento sobre los mensajes intercambiados entre los distintos puntos finales de la comunicación (servicios), a través de colas, se ha habilitado un *plugin* interno que proporciona una interfaz web para la administración de dichas operaciones (véase Figura 6.4).

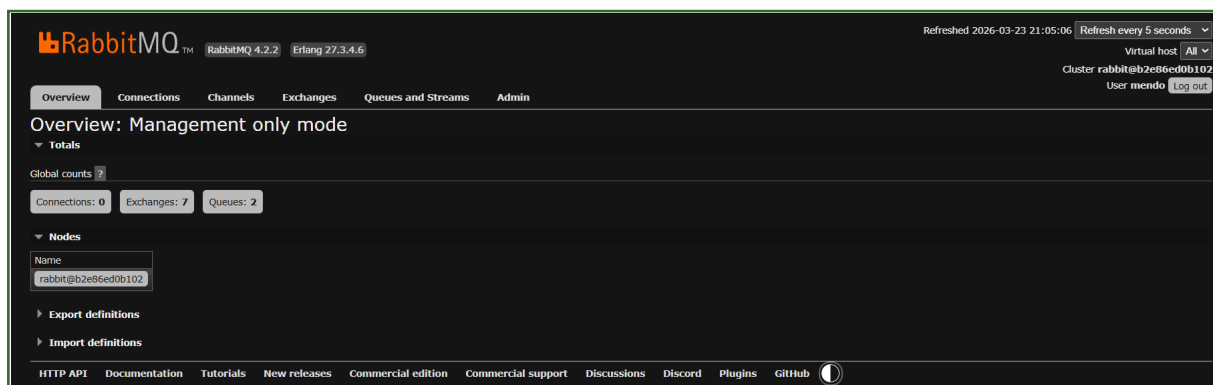


Figura 6.4: Administrador Web *RabbitMQ*
Fuente: Elaboración propia

Acceso a datos por agentes inteligentes

Con el objetivo de que un agente de *IA* se pueda comunicar con datos provenientes de las colas asíncronas desde el servicio de datos, se ha implementado un **servidor intermedio** llamado **Servidor MCP**. Es un programa ligero que actúa como puente, permitiendo a los modelos del lenguaje (LLMs) acceder de forma estandarizada a datos, herramientas e incluso comportamientos predefinidos (*prompts*), entre otros.

Para la creación rápida y sencilla del servidor MCP, se ha utilizado un *framework* desarrollado en *Python*, llamado **FastMCP** [66]. Mediante anotaciones que proporciona el *framework*, sobre las funciones definidas en el servidor, se pueden separar los diferentes recursos disponibles para el modelo del lenguaje utilizado.

Operaciones definidas en el servidor

Profundizando en la configuración interna del servidor MCP, se presenta a continuación la definición de las **instrucciones** (*prompts*) configuradas, junto con la herramienta asociada al servicio de *IA*.

La herramienta `procesar_respuesta_ia` (véase Listado 6.1) actúa como capa de validación y normalización de la salida generada por el servicio *IA*. Dado que los modelos de lenguaje pueden devolver la respuesta envuelta en bloques de código *markdown* o con variaciones en los nombres de los campos, esta herramienta se encarga de limpiar el texto recibido, transformarlo con estructura *JSON* y verificar la presencia de los campos climáticos requeridos, devolviendo un objeto estructurado y homogéneo al servicio de datos.

```
1 @mcp.tool()
```

```

2 def procesar_respuesta_ia(respuesta_ia: str) -> Dict[str, Any]:
3     """Procesa y valida la respuesta JSON de la IA."""
4     if not respuesta_ia:
5         return {"error": "Respuesta vacia", "procesado": False}
6
7     # Eliminar bloques markdown si existen
8     respuesta_limpia = respuesta_ia.strip()
9     for prefijo in ['```json', '```']:
10         if respuesta_limpia.startswith(prefijo):
11             respuesta_limpia = respuesta_limpia[len(prefijo):].strip()
12         if respuesta_limpia.endswith('```'):
13             respuesta_limpia = respuesta_limpia[:-3].strip()
14
15     try:
16         respuesta_json = json.loads(respuesta_limpia)
17         campos_requeridos = [
18             "estado_del_cielo", "aparicion_de_nieblas",
19             "tendencia_de_temperaturas_maximas",
20             "tendencias_de_temperaturas_minimas",
21             "rachas_de_viento", "precipitaciones",
22             "existencias_de_heladas", "zonas_de_heladas",
23             "cotas_de_nieve", "temperaturas_localidades"
24         ]
25         resultado = {
26             c: respuesta_json.get(c) or respuesta_json.get(c.replace('_',
27             ' '))
28             for c in campos_requeridos
29         }
30         return {**resultado, "procesado": True, "valido": True}
31
32     except json.JSONDecodeError as e:
33         return {"error": str(e), "procesado": False}

```

Listado 6.1: Herramienta - Procesado respuesta IA

El *prompt* *clasificacion-climatica* (véase Listado 6.2) define el comportamiento del servicio *IA* como clasificador de textos meteorológicos. Establece las instrucciones necesarias para que el modelo extraiga del texto plano de *AEMET* un conjunto de campos climáticos estructurados en formato *JSON*, imponiendo restricciones explícitas sobre los tipos de datos permitidos y el tratamiento de valores ausentes.

```

1  @mcp.prompt("clasificacion-climatica")
2  async def clasificacion_prompt(texto: str):
3      """Genera el prompt de clasificacion climatica para el servicio IA."""
4      campos = """
5          - estado_del_cielo, aparicion_de_nieblas
6          - tendencia_de_temperaturas_maximas, tendencias_de_temperaturas_minimas
7          - tendencias_de_temperatura_general, rachas_de_viento (incluir zonas)
8          - precipitaciones, existencias_de_heladas, zonas_de_heladas
9          - cotas_de_nieve, temperaturas_localidades (temp_max, temp_min por
10         localidad)
11         """
12
13         instrucciones = """
14             - Responde exclusivamente con JSON, sin texto adicional.
15             - Todos los valores deben ser string, int, float o bool. Sin objetos
16             anidados.
17             - Usa null para campos no mencionados en el texto.
18             - temperaturas_localidades: objeto con temp_max y temp_min (int)
19             por localidad,
20             o null si no se especifican.
21         """
22
23         return [
24             {"role": "system", "content": "Eres un clasificador de datos climaticos."},
25             {"role": "user", "content": (
26                 f"Extrae los siguientes campos en JSON:\n{campos}\n"
27                 f"Restricciones:\n{instrucciones}\n"
28                 f"TEXT0:\n{texto}"
29             )}
30         ]

```

Listado 6.2: Instrucción - Definición comportamiento IA

Definición del servicio IA

Este servicio está caracterizado por el uso de un modelo de lenguaje (LLM) **qwen 2.5** [67] para la generación de respuestas hacia el servicio de datos. Para ello, utiliza los *prompts* y herramientas definidas en el servidor MCP, junto con los datos obtenidos por la cola del *broker*.

El proveedor del modelo de lenguaje utilizado es **OLLAMA** [68], herramienta de código abierto encargada

de proporcionar modelos para descargar, configurar y ejecutar en cualquier entorno de desarrollo.

Seguridad

Desde una perspectiva orientada a entornos de producción, aquellos servicios que gestionan información sensible deben incorporar mecanismos de seguridad que regulen el acceso a los mismos. Este es el caso del servicio de datos, ya que interactúa directamente con la base de datos del sistema.

Para abordar este aspecto crítico, se ha implementado un mecanismo de validación en las peticiones a la *API*, incluyendo procesos de autenticación y autorización. No obstante, en lugar de concentrar toda la lógica asociada a la seguridad dentro del propio servicio de datos, se ha optado por delegar esta responsabilidad en un servicio externo especializado: **Keycloak** [69].

Keycloak es una herramienta de código abierto para la gestión de identidades y accesos, que permite integrar funcionalidades avanzadas de autenticación y autorización sin necesidad de desarrollarlas desde cero. Además, proporciona un sistema de gestión de roles que facilita el control de acceso a los distintos recursos del sistema en función del rol asociado al usuario/cliente.

Asimismo, dispone de una *API RESTful* que permite al servicio de datos interactuar con el sistema de autenticación para validar credenciales, gestionar tokens y controlar los permisos de acceso, entre otras funcionalidades.

La forma en la que esta herramienta está configurada en el sistema es a modo de servidor con protocolo HTTPS, ofreciendo un panel de control (véase Figura 6.5) para que usuarios con privilegios de administración, puedan realizar configuraciones internas oportunas.

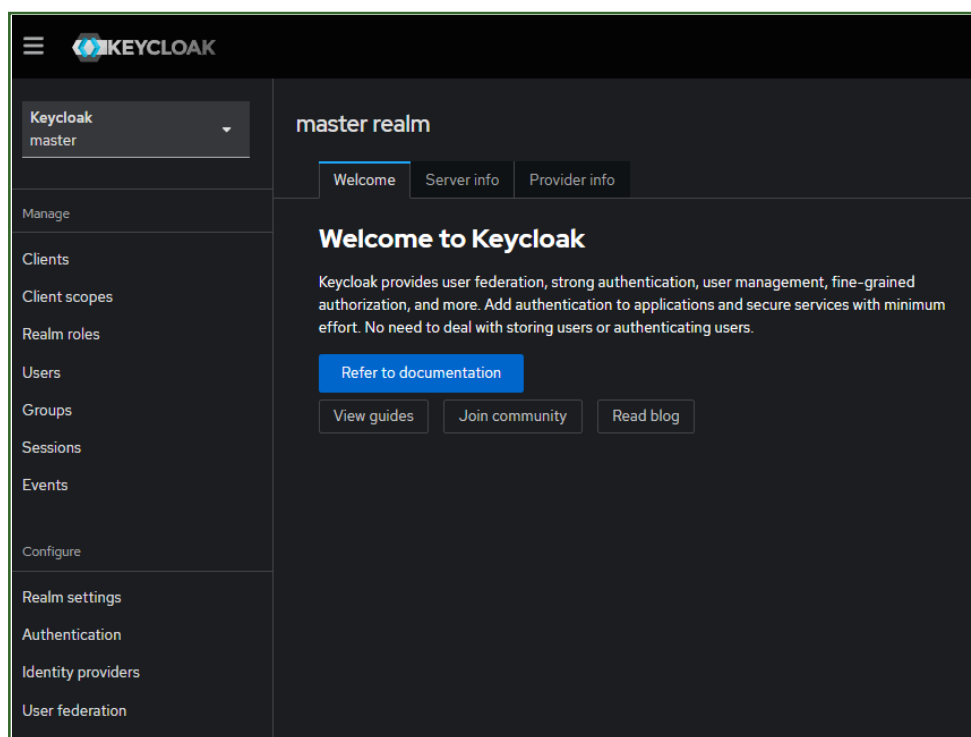


Figura 6.5: Panel Admin Keycloak

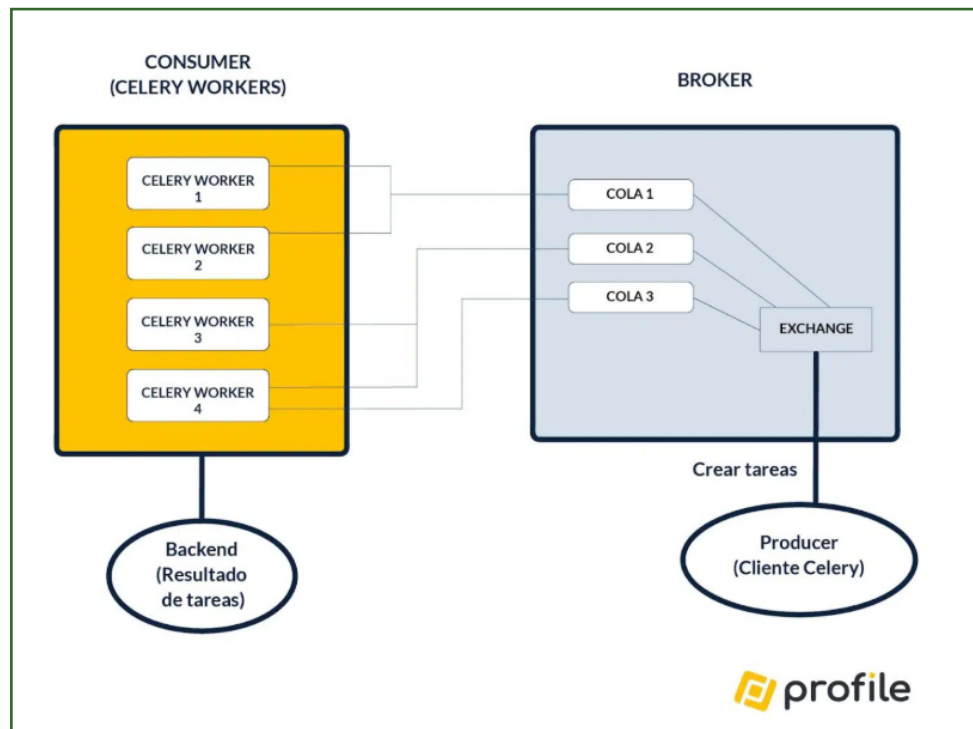
Fuente: Elaboración propia

Tareas programadas

Para conseguir un comportamiento de programación de tareas sobre ciertas acciones internas que se producen dentro de los servicios, se ha decidido implementar y utilizar la herramienta **Celery**, encargada de ejecutar trabajos de forma asíncrona para no bloquear la ejecución normal del programa, mejorando la experiencia del usuario.

Para su funcionamiento, **Celery** requiere el uso de componentes adicionales orientados a la gestión distribuida de tareas, como un *broker* de mensajería y procesos trabajadores encargados de consumir y ejecutar las tareas pendientes. En este proyecto, se ha utilizado **RabbitMQ** como sistema de colas y **Redis** [70] como mecanismo de almacenamiento temporal para determinadas operaciones internas.

El flujo de acciones que implementa internamente es el mostrado en la Figura 6.6. Para más información sobre su uso dentro de la infraestructura desarrollada véase la Subsección 6.3.2 bajo el apartado de **Históricos**, y los errores detectados y solventados en entornos de producción en la Subsección 6.7.3.

Figura 6.6: Flujo de acciones *Celery*

Fuente: [71]

6.2. Arquitectura de la implementación

En secciones anteriores del presente documento se ha ido introduciendo de manera concisa el tipo de arquitectura utilizada en el proyecto desarrollado: **arquitectura orientada a microservicios**.

Existen numerosos tipos de arquitecturas, cada uno con sus ventajas y desventajas en función del contexto del proyecto. La principal motivación para la elección de este tipo de arquitectura reside en la independencia de implementación que se consigue en cada servicio.

Internamente, los microservicios disponen de una arquitectura basada en **capas** (N Capas) [72]:

- Capa de datos (*APIs* de terceros)
- Capa de persistencia (ORM, Objetos de consulta y *Data Access Objects* (DAO) o Repositorios)
- Capa de modelo (Entidades, Colecciones, Eventos, Servicios)
- Capa de aplicación (Controladores, Servicios y *Event Listeners*)
- Capa de presentación (Informes generados, Vistas de modelos, controladores de entrada)

- Capa de cliente (Petición HTTP hacia el sistema)

Estas capas varían dependiendo de la funcionalidad que se quiere conseguir con la implementación de cada microservicio.

6.3. Desarrollo de los servicios del sistema

Una vez descrito el contexto de desarrollo, resulta necesario detallar las funcionalidades internas que desempeña cada uno de los servicios que conforman el sistema. Este análisis permite comprender la lógica que implementa cada microservicio de manera independiente, facilitando su mantenimiento, evolución y adaptación a distintos entornos de producción.

6.3.1. Servicios Externos

Desde la definición del objetivo del sistema, se consideró fundamental el tratamiento adecuado de los datos climáticos como base para la generación de predicciones. Sin embargo, durante las fases iniciales del desarrollo, la obtención de datos climáticos resultó compleja, debido a la limitada disponibilidad de entidades públicas que proporcionaran historiales completos de información procedente de sensores o estaciones meteorológicas en explotaciones agrarias.

Ante esta problema, se optó por integrar servicios encargados de consumir *APIs* públicas de entidades que sí ofrecen acceso a datos meteorológicos, como SiAR, AEMET e ITACyL. Estos servicios son denominados como **servicios externos**.

Durante el proceso de diseño, se valoró la posibilidad de integrar estas funcionalidades directamente en el servicio de datos. No obstante, se identificaron ciertas ventajas en mantener estos servicios de forma independiente. Por un lado, permite realizar un filtrado y limpieza sobre los datos obtenidos por estas *APIs* públicas, debido a que varios valores proporcionados por estas entidades no son relevantes para los cálculos realizados por los servicios de predicción. A su vez, las *APIs* públicas presentan limitaciones en el número de peticiones, por lo que delegar su gestión en servicios específicos reduce la complejidad del servicio de datos y mejora la robustez del sistema.

En pocas palabras, estos servicios externos se encargan de procesar, filtrar y adaptar la información obtenida de las *APIs* públicas climatológicas, así como de gestionar los límites de acceso a las mismas, proporcionando datos ya preparados para el servicio de datos, siendo el único encargado de consumirlos.

Estructura interna de capas que implementa

De acuerdo con la arquitectura de N Capas definida en Sección 6.2, estos servicios implementan las siguientes capas:

- Capa de datos
- Capa de aplicación
- Capa de cliente

No se incluyen las capas de persistencia ni de modelo, debido a que la función principal de estos servicios no es almacenar información, sino actuar como intermediarios en la obtención y transformación de datos.

Tecnología de desarrollo

Los servicios de SiAR e ITACyL han sido desarrollados con el *framework* de *Spring Boot*. El motivo de su elección reside en las anotaciones propias del *framework* que ayudan a crear *APIs RESTful* de una manera fácil y rápida, cumpliendo con los objetivos de eficiencia buscados.

Por su parte, el servicio de AEMET está desarrollado en base al lenguaje *Elixir*, gracias a que permite gestionar peticiones concurrentes ante la *API* de AEMET. Cabe destacar que esta funcionalidad ha quedado en **desuso** durante la evolución del sistema, al no resultar necesaria en la versión final.

Adicionalmente, estos servicios emplean envolturas (*wrappers*) (véase Listado 6.3) para estructurar las respuestas generadas. Este enfoque permite encapsular la lógica de transformación de datos, favoreciendo la reutilización de código y mejorando la interoperabilidad entre los distintos componentes del sistema. Asimismo, se han definido plantillas orientadas a la reutilización de funciones comunes.

```
1  JsonInclude(JsonInclude.Include.NON_NULL)
2  @JsonIgnoreProperties(ignoreUnknown = true)
3  public class ResponseWrapper<T> {
4      JsonProperty("datos")
5      private List<T> datos;
6
7      JsonProperty("MensajeRespuesta")
8      private String mensajeRespuesta;
9
10     public List<T> getDatos() {
11         return datos;
```



```
12     }  
13  
14     public void setDatos(List<T> datos) {  
15         this.datos = datos;  
16     }  
17  
18     public String getMensajeRespuesta() {  
19         return mensajeRespuesta;  
20     }  
21 }
```

Listado 6.3: Ejemplo de Wrapper para reutilizar código

Puntos finales implementados

Los puntos finales (*endpoints*) implementados por cada uno de los servicios externos descritos siguen la misma estructura que las *APIs* públicas de las que obtienen los datos. Esto permite que la interacción entre el servicio de datos y dichas *APIs*, a través del servicio externo intermediario, se realice de forma natural, sin necesidad de redefinir parámetros ni requisitos adicionales, reduciendo así la complejidad de la lógica interna de cada servicio externo.

Los puntos finales que implementa el servicio externo SiAR, se encuentran disponibles en su manual público de *API* [73]. Por otro lado, los puntos finales que implementa el servicio externo AEMET se encuentran definidos en su especificación OpenAPI, bajo la etiqueta `predicciones-normalizadas-texto` [74]. Finalmente, los puntos finales que implementa el servicio externo ITACyL se encuentran definidos en su documentación pública de *API*, denominada **Sativm**, bajo la etiqueta `Plagas` [75].

6.3.2. Servicio de datos

Como se ha comentado previamente en el documento, uno de los puntos críticos del sistema desarrollado reside en el almacenamiento y tratamiento de los datos. Para abordar este punto, se decidió implementar un servicio de datos cuya finalidad principal es obtener información procedente de los servicios externos, almacenarla en la base de datos y normalizarla, aplicando procesos de filtrado y limpieza en función de los requisitos de los servicios predictores.

Ahora bien, surge una cuestión relevante: ¿qué sentido tiene almacenar los datos procedentes de servicios

externos cuando este servicio podría actuar únicamente como intermediario entre los servicios predictores y las *APIs* externas, aplicando filtros en tiempo real?

La respuesta es concisa, mejora la eficiencia y la escalabilidad del sistema. Si cada vez que un usuario solicita una predicción de riesgo de heladas sobre un cultivo, se realizara una consulta directa a las *APIs* de terceros, se generaría una dependencia constante de estos servicios, además de un elevado número de peticiones que pueden exceder el máximo de peticiones por tiempo, restringido por ciertos servicios externos.

Por ello, la estrategia adaptada consiste en almacenar los datos obtenidos en la base de datos y trabajar posteriormente sobre ellos. De este modo, se reduce la carga sobre los servicios externos, se evita la congestión de consultas y se mejora significativamente el rendimiento y la eficiencia del sistema.

Estructura interna de capas que implementa

De acuerdo con la arquitectura de N Capas definida en la Sección 6.2, estos servicios implementan las siguientes capas:

- Capa de datos
- Capa de persistencia
- Capa de modelo
- Capa de aplicación

A pesar de que su principal funcional resida en el tratamiento de datos, el servicio contiene lógica de negocio, procesando información compleja a enviar como respuesta sobre la petición del cliente.

Estructura interna del servicio

Para comprender mejor la estructura interna que maneja el servicio de datos, se divide la explicación en apartados referentes a los diferentes módulos de datos definidos:

1. **Históricos:** Controla el almacenamiento y manejo de datos históricos procedentes del servicio externo SiAR. Sobre este módulo se puede consultar y obtener información sobre datos meteorológicos históricos (definidos entre rangos de tiempo), horarios, diarios, semanales o mensuales sobre una **estación meteorológica** o sobre la media de los datos recopilados sobre las estaciones registradas en una **provincia**. Además, debe tenerse en cuenta que SiAR mantiene

un registro histórico de datos con un rango temporal variable, que comienza aproximadamente el 01/03/2025 y se extiende hasta la fecha actual. No obstante, mediante la realización de pruebas de carga, se ha observado que este intervalo de consulta no es constante. Debido a que la documentación oficial del servicio no especifica el motivo de esta variabilidad de disponibilidad de datos, se plantea la posible existencia de limitaciones internas en el almacenamiento y disponibilidad de datos históricos por parte de SiAR.

Las pruebas realizadas sobre SiAR durante la fase de desarrollo, permitieron identificar todas las principales limitaciones que presenta este servicio tras consultar datos meteorológicos históricos. En determinadas ocasiones, el servicio devuelve errores al solicitar información correspondiente a fechas anteriores a la fecha de comienzo variable de disponibilidad de datos (01/03/2025), o a intervalos temporales concretos sobre estaciones o provincias específicas. Estos fallos detectados no son provocados por una formalización de petición errónea, sino porque todavía no se encuentran disponibles para su consulta y descarga a través de la plataforma.

Dado que puede darse el caso de que estos datos se encuentren disponibles a futuro, se decidió implementar un mecanismo asíncrono de reintentos basado en colas de mensajes. De este modo, se evita que errores temporales en la disponibilidad de la información por el servicio SiAR, provoquen bloqueos en el flujo principal del programa.

Cuando el servicio encargado de consultar datos meteorológicos recibe un error asociado al rango temporal solicitado sobre SiAR, la petición fallida se envía automáticamente a una cola gestionada por un *broker* **RabbitMQ**, junto con los metadatos necesarios para poder reintentar la operación posteriormente.

Una vez almacenada la petición en el *broker*, el *Celery Worker* [76] consume los mensajes pendientes y registra temporalmente la información asociada en *Redis*, permitiendo mantener un control centralizado de las consultas pendientes de reintento.

Posteriormente, el administrador de la infraestructura **puede ejecutar un punto final específico** encargado de desencadenar el reprocesamiento de las peticiones almacenadas. Cuando esto ocurre, el *Celery Worker* reintenta automáticamente las consultas históricas pendientes sobre el servicio externo.

Adicionalmente, el mecanismo incorpora un control interno de peticiones duplicadas. De esta forma, si una consulta previamente fallida obtiene resultados válidos durante un reintento posterior, todas las entradas repetidas asociadas a dicha petición son eliminadas automáticamente de *Redis*. En caso contrario, la petición permanece almacenada hasta futuros intentos de recuperación.

Aunque inicialmente se contempló el uso de **Celery Beat** [77] para automatizar periódicamente el reprocesamiento de peticiones fallidas, finalmente se decidió no incorporarlo a la infraestructura desplegada. *Celery Beat* permite programar tareas recurrentes mediante expresiones **crontab** [78],

posibilitando la ejecución automática de reintentos sin intervención manual. Sin embargo, debido a las limitaciones de recursos presentes en algunos entornos de despliegue, especialmente en memoria disponible, se optó por mantener un mecanismo manual de reintento con el objetivo de reducir el consumo global de recursos de la infraestructura.

Para conocer los puntos finales definidos sobre este módulo, véase sus siguientes especificaciones:

GET /historical/provincias

GET /climate/historical/provincias Obtiene datos climáticos históricos agregados por provincia

Campo	Tipo	Descripción
provinceCode	string (query)	Código identificativo de la provincia
type	string (query)	Tipo de predicción a realizar
startDate	string (query)	Fecha inicial de recogida de datos
endDate	string (query)	Fecha final de recogida de datos

GET /historical/estacion

GET /climate/historical/estacion Obtiene datos climáticos históricos de una estación concreta

Campo	Tipo	Descripción
estacionCode	string (query)	Código identificativo de la estación
type	string (query)	Tipo de predicción a realizar
startDate	string (query)	Fecha inicial de recogida de datos
endDate	string (query)	Fecha final de recogida de datos

POST /historical/reintentar-pendientes

POST /climate/historical/reintentar-pendientes Ejecuta peticiones programadas		
Campo	Tipo	Descripción
		Esta petición no espera ni parámetros de entrada ni cuerpo

2. **Forecast:** Controla el almacenamiento y manejo de datos sobre predicciones meteorológicas a corto plazo (hoy y mañana) procedentes del servicio AEMET. Gracias a este módulo se puede consultar y obtener información sobre datos meteorológicos futuros en función de unas zonas geoespaciales indicadas como una comunidad autónoma o una provincia.

Para conocer los puntos finales definidos sobre este módulo, véase sus siguientes especificaciones:

GET /pronostico

GET /climate/pronostico Obtiene datos meteorológicos predictivos		
Campo	Tipo	Descripción
zona	string (path)	Tipo de zona sobre la que se realiza la predicción
prediccion	string (path)	Tipo de predicción a realizar
ccaaId	string (path)	Identificador de comunidad autónoma (opcional)
provinciaId	string (path)	Identificador de la provincia

3. **Plagas:** Controla el procesamiento y manejo de datos sobre las plagas que se registran manualmente en el sistema o las procedentes del servicio externo ITACyL. Cabe destacar que cada una de las plagas procedentes del servicio externo incorporan un calendario de nivel de riesgo para cada semana del año, pudiendo generar predicciones sobre los cultivos pertenecientes a grupos como **cereales o leguminosos**, pero no tienen en cuenta datos meteorológicos para generar los calendarios de plaga.

Para conocer los puntos finales definidos sobre este módulo, véase sus siguientes especificaciones:

GET /plagas

GET /climate/plagas Obtiene plagas que tienen asociado un calendario de riego		
Campo	Tipo	Descripción
grupo	string (query)	Grupo de cultivo al que afecta la plaga/enfermedad
tipo	string (query)	Indica si es plaga o enfermedad
plaga_id	string (query)	Identificador único de la plaga/enfermedad (opcional)

POST /plagas

POST /climate/plagas Registra una nueva plaga en el sistema		
Campo	Tipo	Descripción
public_id	string	Identificador público de la plaga o enfermedad
nombre	string	Nombre de la plaga o enfermedad
agente_causante	string	Agente causante de la plaga o enfermedad
momento_critico	string	Momento crítico de aparición
observaciones	string	Observaciones adicionales
mas_info	string	Enlace a información detallada
tipo	string	Tipo de plaga o enfermedad
grupo	string	Grupo de cultivo al que pertenece
recursos	list(string)	Variables necesarias para detectar el riesgo
condiciones_evaluables	list(dict)	Condiciones iniciales que incrementan el riesgo
ventana_temporal	list(dict)	Condiciones temporales que incrementan el riesgo

4. **Cultivos:** Este módulo representa la parte más compleja del servicio, encargado de gestionar y procesar la información asociada a datos generados sobre cultivos.

Internamente, los cultivos se relacionan con sus parcelas y variedades, estableciendo asociaciones que aportan información sobre las variables climáticas relevantes a considerar específicas de cada

variedad, así como sobre los tipos de sensores disponibles para la captura de datos. La decisión de incorporar variedades se basa en la necesidad de definir con mayor precisión los umbrales climáticos de cada cultivo, permitiendo así obtener resultados predictivos más precisos (véase Listado 6.4).

```
1      {
2          "variedad": "Lauranne",
3          "nombre_cientifico": null,
4          "etapa_fenologica": "maduracion",
5          "temperatura_evaluada": 0.441,
6          "porcentaje_riesgo": 69.99,
7          "nivel_riesgo": "alto",
8          "umbrales": {
9              "critico": 0.0,
10             "alto": 2.0,
11             "moderado": 20.0,
12             "bajo": 25.0
13         },
14         "alertas": []
15     }
```

Listado 6.4: Fragmento predicción riesgo heladas histórica sobre Lauranne (Almendro)

Para mantener un estado saludable y controlar el riesgo frente a heladas, los cultivos registran las horas de frío a las que están sometidos. Cada cultivo requiere un número determinado de horas de frío para mantener un estado fenológico adecuado a lo largo de su ciclo de vida. Por este motivo, cada variedad de cultivo se asocia a un modelo de horas de frío específico.

Estos modelos definen las condiciones climáticas necesarias para iniciar el recuento de horas de frío. La existencia de distintos modelos se debe a las diferencias en el comportamiento de cada cultivo y a los detalles de cada método de cálculo. De este modo, se evita aplicar un mismo modelo de forma generalizada, garantizando un cálculo más preciso y adaptado a cada tipo de cultivo [79].

Por otro lado, en los registros de variedades contra el servicio, se especifican los límites máximos y mínimos de horas de frío necesarias. El sistema, paralelamente se encarga de registrar sobre la variedad registrada, las horas de frío acumuladas, en base a los registros históricos climáticos anuales que se almacenan en el sistema.

A continuación se indican los diferentes modelos de horas de frío registrados en el sistema [79] (véase Tabla 6.1).

Tabla 6.1: Modelos de horas de frío registrados en el sistema

Nombre	Código	Descripción
$T \leq 7,2^{\circ}\text{C}$ (Clásico)	CM7	Acumula las horas en que la temperatura es inferior a $7,2^{\circ}\text{C}$.
Clásico 0-7	C07	Acumula las horas en que la temperatura se encuentra entre 0 y 7°C .
Unidad de Frío	UF	Acumula horas con temperatura entre $2,5$ y $9,1^{\circ}\text{C}$. Temperaturas entre $1,4$ y $15,9^{\circ}\text{C}$ no computan; valores entre 16 y 18°C restan unidades.
Porciones de Frío (dinámico)	PF	Basado en la ley de Arrhenius. Se consideran 12 PF para especies de baja demanda de frío y 90 PF para especies de alta demanda.

Para incrementar la precisión, sobre cada variedad de cultivo registrada se asocian umbrales de temperatura climáticos que deben mantener estables durante cada etapa fenológica en su ciclo de vida [80]. Cada variedad de cultivo cuenta con diferentes etapas fenológicas, pero para simplificar este concepto, se han definido etapas comunes para cada variedad, independientemente de si forman parte de su ciclo de vida o no, y para cada una de estas etapas se registran sus umbrales. Los registros almacenados en la base de datos respecto a las etapas fenológicas se realizó de forma manual, incorporando los datos definidos en la Tabla 6.2 sobre un fichero JSON y generando un proceso encargado de leer los datos del fichero en segundo plano y almacenarlos en la correspondiente tabla de la base de datos.

A continuación se indican las etapas fenológicas generales registradas en el sistema [26] (véase Tabla 6.2).

Tabla 6.2: Tabla de etapas fenológicas registradas

Nombre	Código	Orden	Descripción
Dormancia	DOR	1	Reposo invernal de la planta, suspendiendo el crecimiento y la baja actividad metabólica.
Brotancia	BRO	2	Fin de dormancia y emergimiento de las primeras flores.

Nombre	Código	Orden	Descripción
Floración	FLO	3	Fase reproductiva del ciclo de vida, desde la aparición de los brotes a la apertura de la flor.
Cuajado	CU	4	Flores fecundadas se transforman en fruto.
Engorde	ENG	5	Los frutos aumentan rápidamente de tamaño y peso.
Maduración	MAD	6	Fase final de los frutos, alcanzando su máximo potencial de desarrollo.
Cosecha	COS	7	Fase final del ciclo del cultivo, comienzo de recolección.

Para conocer los puntos finales definidos sobre esta primera parte del módulo, véase sus siguientes especificaciones.

GET /crop/variedades/umbrales

GET /crop/variedades/umbrales Obtiene los umbrales climáticos definidos sobre una variedad de cultivo

Campo	Tipo	Descripción
nombre	string (path)	Nombre de la variedad de cultivo

GET /crop/

GET /crop/ Obtiene información sobre el campo indicado

Campo	Tipo	Descripción
campo	string (path)	Campo a consultar. Valores: variedades, modelos, etapas_fenologicas, cultivos

POST /crop/variedades/umbrales

POST /crop/variedades/umbrales Registra umbrales climáticos para una etapa fenológica, sobre una variedad de cultivo

Campo	Tipo	Descripción
etapa_fenologica	string	Código de la etapa fenológica asociada al umbral
nombre	string (path)	Nombre de la variedad de cultivo asociada al umbral
critico	float	°C para que el riesgo sea crítico
alto	float	°C para que el riesgo sea alto
moderado	float	°C para que el riesgo sea moderado
bajo	float	°C para que el riesgo sea bajo

POST /crop/

POST /crop/ Registra información sobre el campo indicado

Campo	Tipo	Descripción
campo	string (path)	Campo a consultar. Valores: variedades, modelos, etapas_fenologicas, cultivos
requestBody	json	Cuerpo de la petición específico del campo elegido

PATCH /crop/variedades/horas_frio

PATCH /crop/variedades/horas_frio Actualiza los umbrales de horas de frío asociados a la variedad indicada

Campo	Tipo	Descripción
nombre	string (path)	Nombre de la variedad de cultivo
frio_max	int (query)	Límite máximo de umbral para las horas de frío
frio_min	int (query)	Límite mínimo de umbral para las horas de frío

Con el objetivo de mejorar la predicción del riesgo de plagas sobre los cultivos, se ha implementado una asociación entre ambas entidades. De esta forma, es posible consultar para cada cultivo las plagas que potencialmente le afectan de manera recurrente.

Esta asociación se ha establecido mediante un campo común, denominado **grupo de cultivo**, presente tanto en el registro de cultivos como en el de plagas. Este campo representa una agrupación de plantas agrícolas que comparten características agronómicas similares, lo que permite su clasificación.

Gracias a esta implementación, es posible obtener resultados como la respuesta JSON proporcionada por su respectivo punto final de consulta (véase Listado 6.5).

```

1      [
2          {
3              "cultivo": {
4                  "nombre": "Berenjena",
5                  "nombre_cientifico": "Solanum melongena",
6                  "grupo": "hortaliza_fruto"
7              },
8              "plagas": [
9                  {
10                     "id_publico": "PLAGA-PRUEBA-01",
11                     "nombre": "Mildiu",
12                     "tipo": "hongo",
13                     "grupo": "hortaliza_fruto"
14                 }
15             ]
16         }

```

17

]

Listado 6.5: Asociación Cultivo-Plaga

Para conocer los puntos finales definidos sobre la asociación cultivo-plaga, véase su siguiente especificación.

GET /crop/plague

GET /crop/plagues Obtiene información de las plagas asociadas a un cultivo dado		
Campo	Tipo	Descripción
cultivos	list(string) (query)	Nombre de los cultivos a analizar, pueden ser varios

Continuando con la mejora de la lógica interna de los cultivos, el sistema gestiona un registro de parcelas. Durante el proceso de registro de un nuevo cultivo, es posible indicar el identificador público de una parcela previamente registrada, permitiendo así asociar los cultivos a una localización concreta.

Esta asociación facilita una mejor organización de la información, permitiendo gestionar los cultivos en función de su localización y posibilitando futuras ampliaciones, como el análisis de condiciones climáticas específicas o la obtención de datos por sensores específicos sobre la parcela asociada.

Para conocer los puntos finales definidos sobre la asociación cultivo-parcela, véase su siguiente especificación:

GET /crop/parcel

GET /crop/parcel Obtiene información de las parcelas asociadas a un cultivo dado		
Campo	Tipo	Descripción
cultivo	string (query)	Nombre del cultivo a analizar
parcela	string (query)	Identificador público de la parcela (opcional)

5. **Metadatos:** Durante las primeras versiones en la generación de los informes predictivos resultantes por los servicios correspondientes, el tutor del proyecto indicó un concepto clave a registrar en

los informes y fue la incorporación de contexto de cálculo. Cada informe generado contenía la información precisa de los resultados predictivos obtenidos, pero se perdía el trasfondo que contenía el resultado derivado.

Para ello, se incorporaron dos ingenieros agrónomos al proyecto que proporcionaron información para completar esta información de contexto. Los datos proporcionados se incluían dentro de ficheros con extensión **.csv**, traducidos independientemente hacia el sistema para mantener persistencia sobre su información.

La información contenida en los ficheros proporcionados, se relacionó con las principales entidades que se identificaron:

- **Parcelas:** Almacenan información sobre sus identificadores públicos, nombre, localización referenciada por sus geo coordenadas, entre otros.
- **Dispositivos:** Almacenan información sobre sus identificadores públicos, sensor asociado y los tipos de datos que registran, entre otros.
- **Sensores:** Almacenan información sobre sus identificadores públicos, nombre y localización referenciada por sus geo coordenadas. Este último dato, permitió asociar los sensores proporcionados con las parcelas registradas, mediante consultas geoespaciales.

Asimismo, debido a que los clientes que utilicen este sistema desarrollado pueden tener diferentes necesidades e incorporar nuevos datos sobre parcelas, dispositivos y sensores que registran para sus explotaciones agrarias, se han implementado puntos finales capaces de leer el contenido de ficheros adjuntados a la petición y registrarlos en el sistema, comprobando siempre criterios de validación. Las especificaciones de sus puntos finales se encuentran a continuación:

POST /metadatos/registro

POST /metadatos/registro Registra el contenido de un fichero de metadatos adjuntado en la petición

Campo	Tipo	Descripción
tipo	string (path)	Tipo de metadato a registrar: parcelas, dispositivos, sensores
file	File (requestBody)	Ficho con información a insertar

GET /metadatos

GET /metadatos Obtiene información de metadatos registrada en función del tipo indicado

Campo	Tipo	Descripción
tipo	string (path)	Tipo de metadato a consultar: parcelas, sensores, dispositivos
entidad_id	string (query)	Filtros sobre entidades relacionadas con los metadatos

6. **Sensores:** Con motivo de gestionar la información que se obtiene a través del servicio externo DTAgro en relación con sensores, se ha decidido implementar un módulo dentro del servicio de datos encargado de controlar esta lógica.

Para conocer los puntos finales definidos sobre el módulo de sensores, véase su siguiente especificación:

GET /sensores

GET /sensores Recupera datos almacenados sobre DTAgro, con posibilidad de aplicar filtros.

Campo	Tipo	Descripción
eui	string (query)	Identificador de sensor público para recopilar sus datos
fecha_inicio	string (query)	Fecha inicial para la recopilación de datos
fecha_fin	string (query)	Fecha final para la recopilación de datos
nombre_dt_agro	string (query)	Nombre del campo de dato sobre el que se quiere obtener información referente al sensor indicado
nombre_predictor	string (query)	Traducción del valor <code>nombre_dt_agro</code> sobre el lenguaje entendido por el predictor que consulta

En conclusión, todos estos módulos explican el comportamiento interno de este servicio, proporcionando datos de calidad a los servicios predictores y permitiendo así obtener resultados predictivos más precisos.

6.3.3. Servicio de ingesta de datos meteorológicos

Durante el desarrollo del servicio de datos, surgió la incertidumbre sobre cómo realizar la población de datos en entornos de producción. En el entorno local de desarrollo, las pruebas se realizaban mediante la inserción manual de información en la base de datos para validar el funcionamiento del sistema. Sin embargo, este comportamiento debía automatizarse en la fase de despliegue para poder desplegar la infraestructura implementada sobre cualquier sistema mitigando problemas de falta de datos en la base de datos.

Por este motivo, se planteó la implementación de un servicio de ingesta de datos integrado en el servicio de datos. Su principal función es gestionar la **ingesta automática** de información en la base de datos. Este planteamiento permitió dar respuesta a varias cuestiones iniciales:

- ¿Qué ocurre si el servicio predictor solicita datos al servicio de datos que no se encuentran en la base de datos?
- En caso de no existir los datos solicitados, ¿se interrumpe el flujo de respuesta hacia el cliente?
- ¿Es escalable mantener un mecanismo manual para poblar la base de datos?

El servicio de ingesta opera de forma autónoma dentro del servicio de datos (véase Figura 6.7), mediante un hilo de ejecución independiente del hilo principal. Esto permite que, de forma paralela, se verifique la existencia de los datos solicitados (en la base de datos) y, en caso de no estar disponibles, se obtengan desde servicios externos u otras fuentes de información (dependiendo del tipo de dato requerido). Posteriormente, estos datos son almacenados en la base de datos para que el servicio pueda completar la respuesta.

Dado que este servicio realiza peticiones a sistemas externos, se ha incorporado un mecanismo de **control de estado** que informa tanto al cliente como al propio servicio de datos sobre el estado actual del proceso de ingesta. Los estados definidos son los siguientes:

- **PENDING**: Estado inicial que indica el comienzo del proceso de ingesta.
- **PENDING RETRY**: Petición de datos históricos sobre SiAR fallida por limitaciones de fecha. Asignada para su re-ejecución con *Celery Worker*.
- **LOADING**: Estado en el que se realiza la obtención, procesamiento y validación de los datos.
- **READY**: Estado final que indica que los datos ya están disponibles en la base de datos.
- **FAILED**: Estado que indica que se ha producido un error durante el proceso de ingesta.

Esta solución presenta un inconveniente relacionado con el tiempo de respuesta del servicio de datos. Cuando la información requerida ya se encuentra en la base de datos, la respuesta es inmediata. Sin embargo, si es necesario activar el servicio de ingesta, el tiempo de respuesta aumenta debido a la obtención y procesamiento de los datos.

Para mitigar este problema, el sistema expone al cliente el estado de la ingesta junto con metadatos adicionales (como el inicio y fin del proceso) en formato JSON. Esto permite al cliente realizar consultas periódicas y monitorizar la evolución del proceso hasta la obtención de una respuesta satisfactoria o la detección de un error.

Por último, con el objetivo de facilitar futuros procesos de telemetría y monitorización, se ha diseñado una tabla específica en la base de datos que registra las ejecuciones del servicio de ingesta.

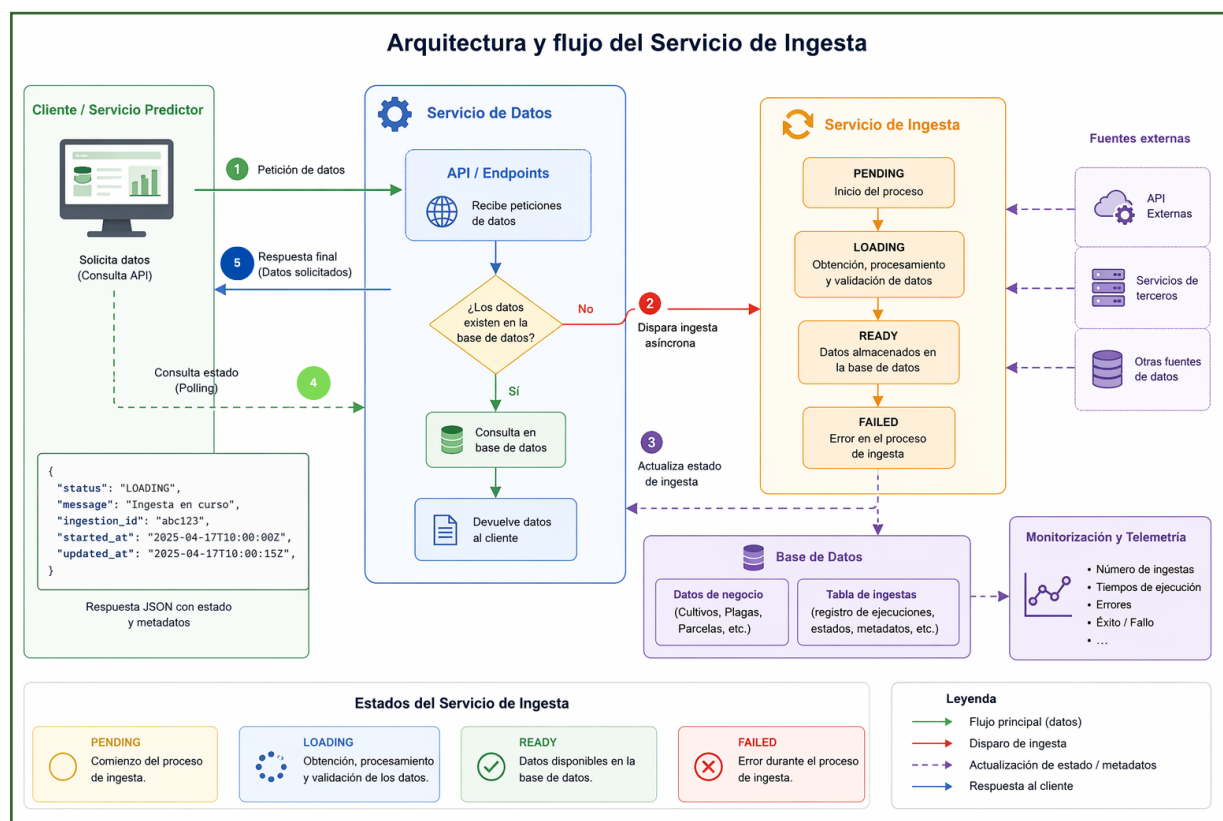


Figura 6.7: Flujo de acciones servicio ingesta.

Fuente: Elaboración propia.

6.3.4. Servicio IA

En el proceso de obtención de datos futuros procedentes de AEMET para almacenarlos y disponerlos a los servicios predictores con el objetivo de generar predicciones futuras en base a dichos datos, se obtuvieron respuestas en formato **texto simple** que debían ser tratadas dentro del sistema de forma diferente para recabar su información contenida de una manera estructurada.

Como primera solución, se planteó y desarrolló un analizador de texto. Mediante expresiones regulares definidas, era capaz de obtener información estructurada en formato JSON sobre las respuestas que devolvía AEMET. Esta solución a primera vista puede ser válida, pero se pierde precisión en la recopilación de datos debido a las expresiones regulares que controla. Por ello, el uso del analizador sobre las respuestas procedentes de AEMET se utilizó como salvaguarda ante fallos del mecanismo de IA que se implementó.

Para solventar el problema identificado y obtener datos estructurados más precisos sobre las respuestas de AEMET, se decidió delegar esta tarea a un servicio de IA configurado únicamente para cubrir las necesidades indicadas. Al trabajar con modelos de IA, sus respuestas sobre las tareas solicitadas pueden conllevar largas esperas. Es por eso que se implementó este flujo de acciones de manera asíncrona, utilizando el *broker* y el servidor MCP, descritos en secciones previas (véase 6.1) para no interrumpir el flujo general del servicio de datos a la espera de la obtención de la respuesta por el servicio de IA.

Finalmente, con esta implementación y buena configuración de herramientas e instrucciones almacenadas en el servidor MCP y utilizadas por el servicio de IA, se consiguen resultados meteorológicos más precisos y bien estructurados para manejarlos mejor desde el servicio de datos (véase su flujo de comportamiento en la Figura 6.8).

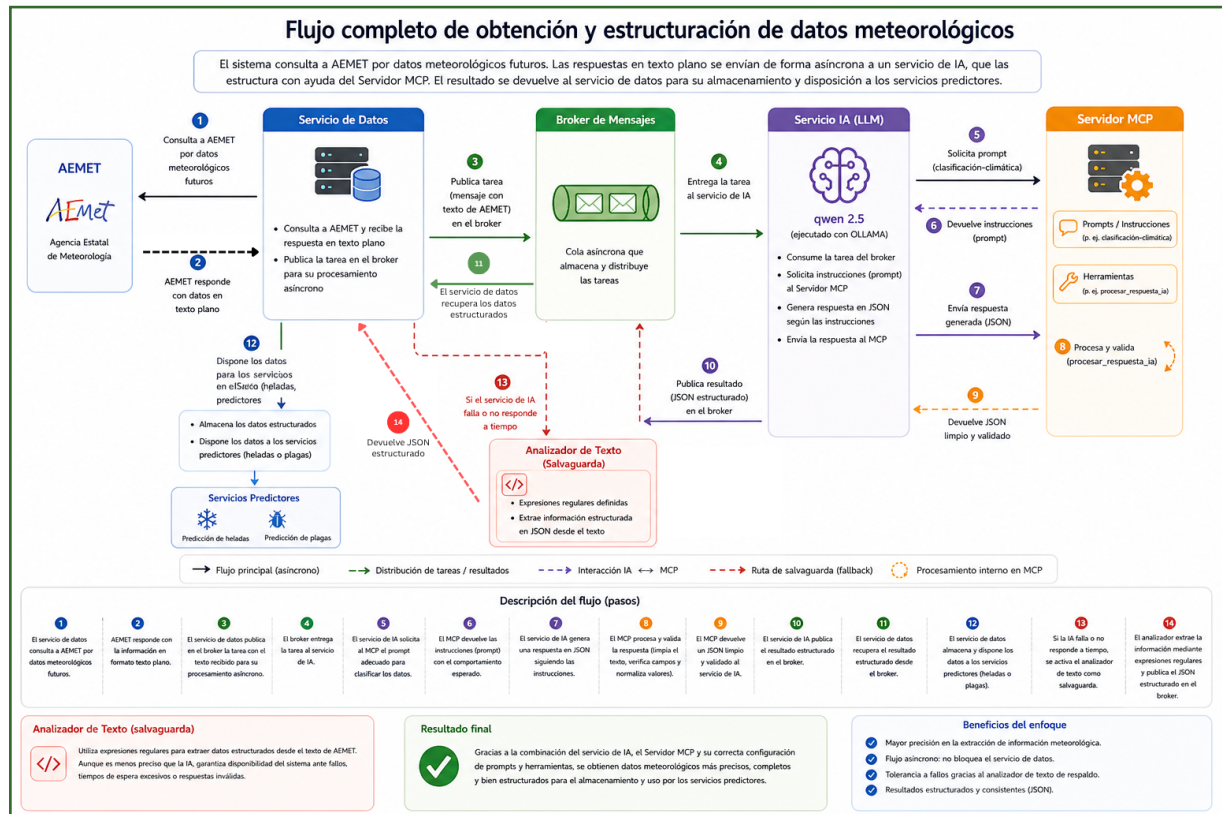


Figura 6.8: Flujo de acciones servicio IA.

Fuente: Elaboración propia.

6.3.5. Servicios Predictores

Una vez el sistema global es capaz de recopilar, estructurar y almacenar datos climáticos sobre los distintos puntos comentados en el servicio de datos, el paso crítico a definir e implementar recae en la lógica de los servicios predictores que deben cumplir con sus requisitos definidos. Para ello, se han diseñado dos servicios predictores especializados. Estos servicios actúan como el cerebro del sistema, tratando internamente los datos recopilados del servicio de datos y generando sobre su lógica interna de procesamiento, alertas y recomendaciones comprensibles para el cliente final. No se limita a mostrar información numérica, sino a contextualizar la información para prever riesgos ante heladas y plagas sobre cultivos indicados.

Predicción de riesgo ante heladas

El servicio de predicción ante heladas se diseñó con un objetivo claro: cuantificar los riesgos que suponen los descensos térmicos sobre las distintas etapas fenológicas en las que se encuentra cada cultivo. La

complejidad de esta implementación reside en que no todas las heladas son iguales, ni afectan de la misma manera a todos los cultivos. Una temperatura de 2°C puede ser insignificante para un cultivo en etapa fenológica de dormancia, pero devastador para un frutal en plena etapa de floración. Por este motivo, no se debe conformar la lógica ante heladas con la temperatura mínima histórica, diaria y futura de una zona indicada, sino que calcula un **nivel de riesgo agronómico ponderado**.

El flujo de trabajo implementado en la función principal del servicio `obtener_predicciones_heladas_futuras` y su equivalente para **datos observados** reflejan esta forma de pensar. En primer lugar, se recopila sobre el servicio de datos la información meteorológica relevante, ya sea de los datos históricos abastecidos por el SiAR o de las predicciones de AEMET. Con estos datos obtenidos, el servicio realiza el siguiente análisis:

1. **Análisis de Temperatura y Umbral Fenológico:** El sistema consulta dinámicamente al servicio de datos con el objetivo de obtener los umbrales de sensibilidad sobre cada variedad de cultivo solicitada. El método `evaluar_nivel_por_umbral` se encarga de relacionar la temperatura prevista por AEMET con la etapa fenológica en la que se encuentra el cultivo. Así, un mismo valor de temperatura puede generar una alerta de riesgo **crítica** para una variedad y una **moderada** para otra, ajustándose a la realidad biológica del cultivo.
2. **Incorporación de Variables Ambientales Secundarias:** Se añade una capa extra de precisión evaluando factores como la humedad relativa y el viento. Mediante el método `calcular_nivel_riesgo_porcentaje`, se indica la probabilidad de ocurrencia ante **heladas blancas** (temperaturas cercanas a 0°C y humedad ambiental alta) o **negras** (viento, poca humedad y temperaturas cercanas a 0°C). Esta división de tipo de helada es agradecida para el agricultor, ya que las medidas de protección varían drásticamente entre un tipo y otro.
3. **Evaluación de Localidades por Altitud y Cota de Nieve:** Cuando se realizan predicciones a futuro, la información de AEMET suele incluir una descripción de la cota de nieve e información climática general sobre las localidades situadas en una provincia solicitada. Conociendo las altitudes máximas y mínimas de la cota y la altitud de cada localidad, el sistema puede generar riesgos elevados incluso si la temperatura genérica no es esencialmente baja, pero la localidad se encuentra dentro del rango de la cota. Esta lógica, demuestra cómo se combinan datos geoespaciales con predicciones para ajustar las alertas resultantes.
4. **Horas de Frío:** Uno de los aspectos más diferenciados del servicio es la integración de las horas de frío acumuladas. A través del método `aplicar_condiciones_horas_frio`, el sistema consulta las horas de frío acumuladas de cada variedad y escala el nivel de riesgo de heladas en función del grado de vernalización completado: si una variedad ha acumulado menos del 50 % de sus horas

mínimas requeridas, el riesgo sube dos niveles; si se encuentra entre el 50 % y el 100 % sube un nivel. Las variedades que no han completado su vernalización son más vulnerables frente a heladas.

5. **Redefinición de la Fecha para Datos Históricos:** Un aspecto clave en la generación de predicciones de riesgo de helada en el momento actual es la correcta definición del rango de datos climáticos históricos que se emplean como contexto.

El sistema utiliza internamente una representación del día como número ordinario dentro del **año agrícola**, cuyo inicio se fija el 1 de octubre, comienzo convencional de la campaña de cultivos de secano. Esta referencia, denominada en el código **día juliano agrícola**, permite calcular en qué punto del ciclo invernal se encuentra cada registro climático histórico recogido, independientemente del año natural que marca. Todos los datos históricos de la provincia de Cáceres comprendidos dentro del período activo de riesgo de heladas se emplea como **base** para el cálculo de la predicción **actual**.

El resultado final es un informe de riesgos en formato PDF que recopila un resumen de la situación analizada, una tabla que recopila una acumulación histórica de cultivos predichos junto con su riesgo porcentual ante heladas, una lista de alertas junto con sus recomendaciones a tener en cuenta para prevenir los riesgos y un gráfico de líneas donde se registra de manera visual el progreso de riesgos que ha ido teniendo cada cultivo analizado.

A continuación, en la Figura 6.9, se puede observar visualmente el flujo seguido por este servicio predictor.

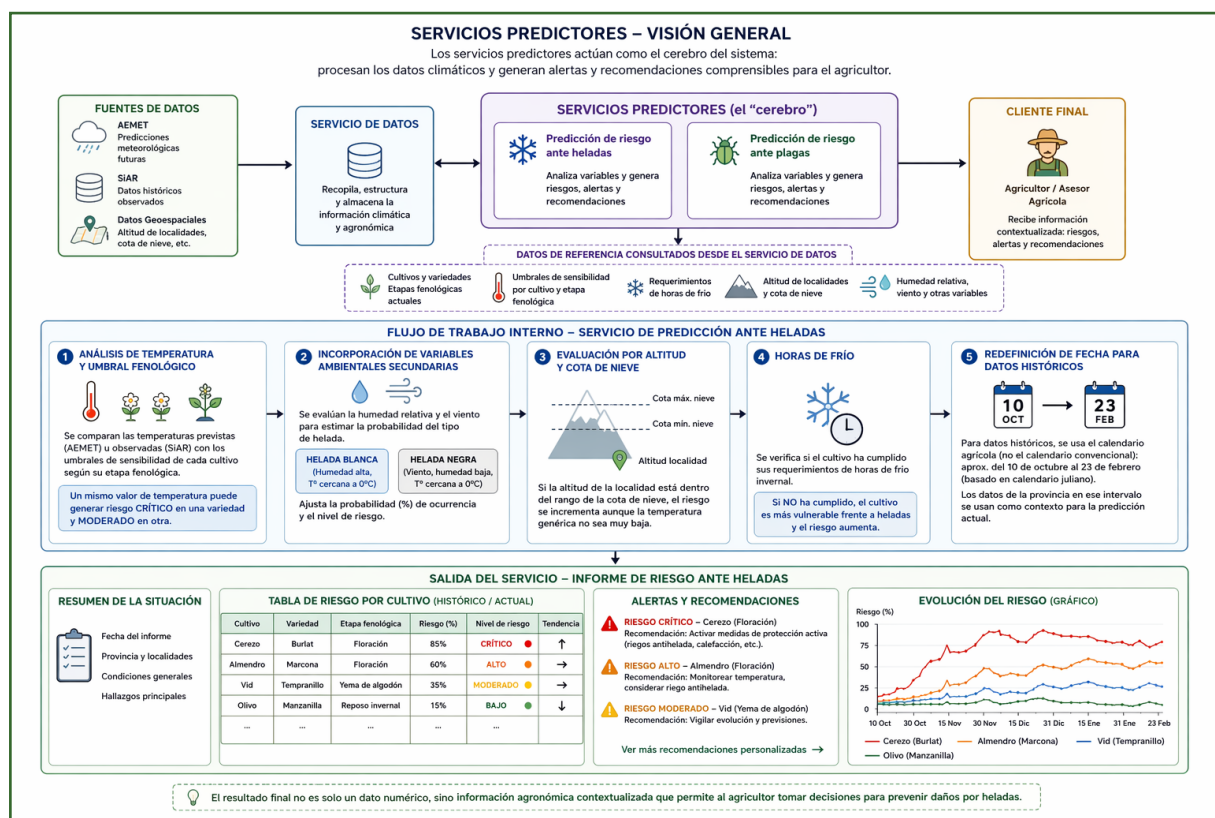


Figura 6.9: Flujo de acciones servicio heladas.
Fuente: Elaboración propia.

Con el objetivo de conocer los puntos finales que proporciona de manera pública la *API* de este servicio predictor, se muestra a continuación su especificación:

POST /heladas/observadas/

POST /heladas/observadas/ Genera una predicción de riesgo ante heladas basada en datos históricos para el día de hoy.

Campo	Tipo	Descripción
format	string (query)	Formato de respuesta (json, pdf)
tipo	string (path)	Tipo de fecha a realizar la predicción a día de hoy (Hora, Día, Semana)
codigo_provincia	string	Código de la provincia a obtener datos meteorológicos (opcional)
codigo_estacion	string	Código de la estación a obtener datos meteorológicos (opcional)
evaluacion	boolean	Indica si se incluye una evaluación predictiva sobre variedades o no
variedades	list(string)	Listado de nombre de variedades de cultivo a evaluar (opcional)

POST /heladas/futuras/

POST /heladas/futuras/ Genera una predicción de riesgo ante heladas basada en datos futuros para el día de mañana.

Campo	Tipo	Descripción
format	string (query)	Formato de respuesta (json, pdf)
zona	string (path)	Tipo de zona geográfica sobre la que se quiere realizar la predicción (provincial, ccaa, nacional)
codigo_provincia	string	Código de la provincia a obtener datos meteorológicos (opcional)
codigo_ccaa	string	Código de la comunidad autónoma a obtener datos meteorológicos (opcional)
evaluacion_var	boolean	Indica si se incluye una evaluación predictiva sobre variedades o no
evaluacion_loc	list(string)	Indica si se incluye una evaluación predictiva sobre localidades o no
variedades	list(string)	Listado de nombre de variedades de cultivo a evaluar (opcional)
localidades	list(string)	Listado de nombre de localidades provinciales a evaluar (opcional)

Predicción de Riesgo por Plagas y Enfermedades

A diferencia del módulo de heladas, el servicio predictor ante riesgos de plagas y enfermedades necesita que se den una serie de condiciones durante un periodo de tiempo concreto para desarrollarse.

Para abordar esto, se implementaron dos modos de predicción complementarios en el servicio predictor.

- **Modo Calculado (Calendario de Riesgos):** Este modo, se basa en datos históricos proporcionados por el servicio ITACyL, almacenados y dispuestos en el servicio de datos. Funciona como un calendario de riesgo teórico que indica, para cada semana del año, cuál es el nivel de riesgo de aparición sobre una plaga concreta en un cultivo dado. Este modo sirve como primera solución

predictiva que permite al agricultor sabe que “estamos en fechas de...”.

Cabe destacar que las mismas restricciones de cultivo que se indicaron al describir el servicio externo ITACyL, se aplican en este modo predictivo, obteniendo predicciones sobre calendarios de plaga hacia cultivos que pertenezcan a los grupos de cultivo de **Cereales** y **Leguminosos**.

- **Modo Estimado:** Aquí es donde reside el verdadero potencial del servicio. El método `obtener_prediccion_plagas_estimadas` implementa una lógica de **ventana móvil** sobre los cultivos proporcionados en la petición. En lugar de evaluar datos sobre un solo día para generar la predicción, itera sobre un rango de fechas definido en la consulta. Para cada día, se obtienen datos climáticos especificados en la consulta predictiva asociados a los sensores en DTAgro sobre los que se quieren consultar dichos datos (para conocer más sobre los problemas de implementación y cómo se solventaron, véase la Subsección 6.7.5).

La clave de este diseño es el motor de evaluación **genérico** definido en `evaluar_plaga_generica`. Esta función actúa como una máquina de reglas configurables. En lugar de tener una función a evaluar independiente por plaga, por ejemplo, una para la plaga Mildu y otra para la Brotitis, se le pasa un objeto JSON de configuración (“condiciones_evaluables”) que actúa como receta.

La receta define una serie de requisitos sobre cada plaga asociada al cultivo indicado, que se deben de cumplir para obtener un riesgo elevado sobre la misma, como por ejemplo: “`temperatura_media >= 15.0`” y “`humedad_foliar <= 80.0`”. El sistema extrae el valor real capturado por los sensores para ese día, aplica el operador correspondiente y determina si la condición se cumple o no. Dependiendo de cuántas condiciones se cumplan, se asigna un nivel de alerta (**PREVENTIVA** o **CRITICA**).

Esta arquitectura basada en configuración (Strategy Pattern simplificado) es mantenible y escalable; añadir una nueva plaga en el sistema no requiere que se implemente una funcionalidad nueva en este servicio predictor para evaluar sus condiciones, solo se requiere la acción de añadir una nueva entrada en la base de datos con sus condiciones asociadas.

En definitiva, ambos servicios predictores manejan todos los datos almacenados y mantenidos por el servicio de datos, proporcionando predicciones que no solo devuelven datos resultantes, sino que ayudan a los clientes, mediante recomendaciones asociadas a las alertas generadas, para prevenir posibles daños sobre sus plantaciones.

A continuación, en la Figura 6.10, se puede observar visualmente el flujo seguido por este servicio predictor.

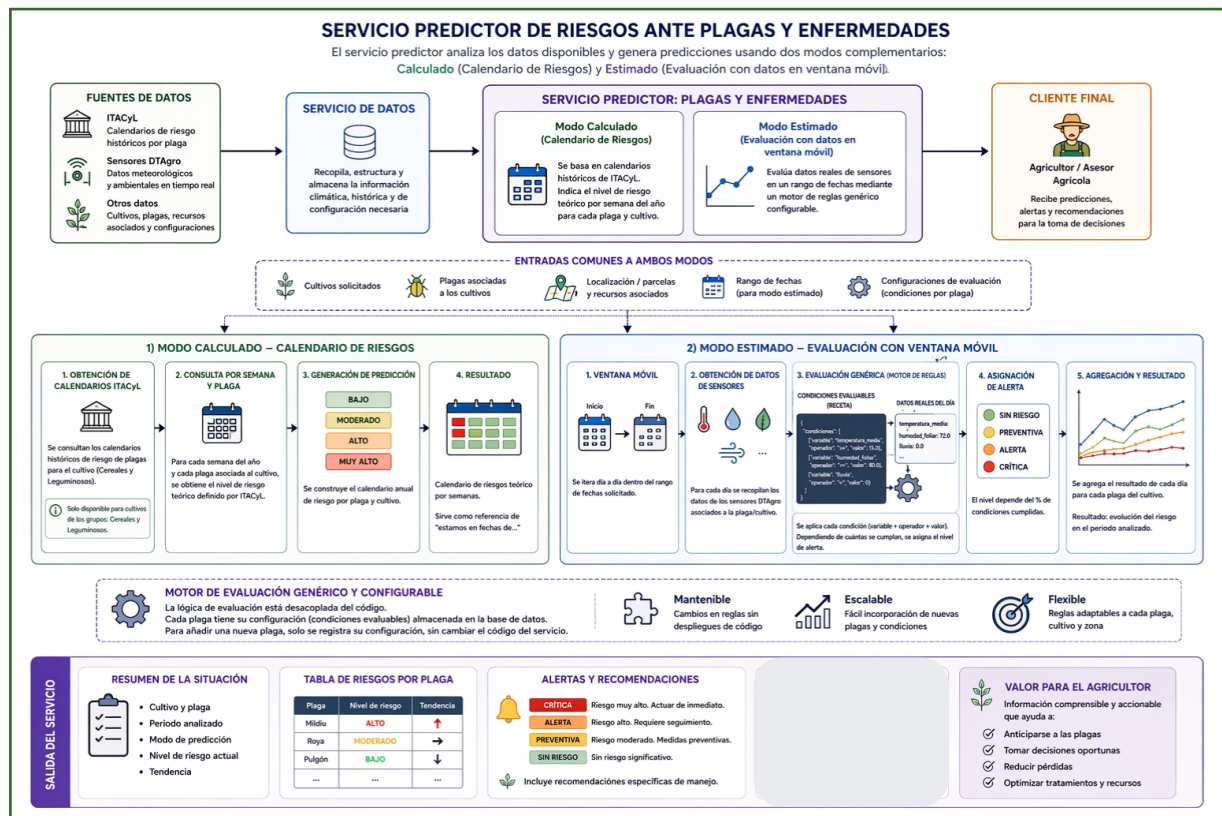


Figura 6.10: Flujo de acciones servicio plagas.

Fuente: Elaboración propia.

Con el objetivo de conocer los puntos finales que proporciona de manera pública la *API* de este servicio predictor, se muestra a continuación su especificación:

POST /plagas/calculadas

POST /plagas/calculadas Genera una predicción anual de riesgo ante plagas basada en datos calculados sobre grupos de cultivos específicos (cereales, leguminosas).

Campo	Tipo	Descripción
format	string (query)	Formato de respuesta (json, pdf)
cultivo	string (query)	Nombre del cultivo a analizar

POST /plagas/estimadas

POST /plagas/estimadas/ Genera una predicción de riesgo ante plagas mediante estimación temporal de datos meteorológicos y de sensores.

Campo	Tipo	Descripción
format	string (query)	Formato de respuesta (json, pdf)
cultivo	string	Nombre del cultivo a analizar
fecha_inicio	string	Fecha de comienzo de la predicción
fecha_fin	string	Fecha de finalización de la predicción
id_plaga	string	Identificador público de la plaga asociada al cultivo sobre la que se quiere aplicar el análisis
datos_sensores	list(json)	Lista de objetos de sensores a recopilar datos climáticos
sensor	string (datos_sensores)	Identificador público del sensor sobre el que se quiere recuperar datos
nombre_dt_agro	string (datos_sensores)	Nombre del campo de dato sobre el que se quiere obtener información referente al sensor indicado
nombre_predictor_plaga	string (datos_sensores)	Traducción del valor nombre_dt_agro sobre el lenguaje entendido por el predictor
codigo_provincia	string	Código de la provincia a obtener datos meteorológicos (opcional)
codigo_estacion	string	Código de la estación a obtener datos meteorológicos (opcional)
parcela	string	Identificador de la parcela asociada al cultivo indicado (opcional)

6.4. Integración del sistema

Una vez descrita la implementación interna de cada uno de los microservicios que componen la arquitectura, es necesario conocer en detalle el desafío que supone su integración. En el sistema distribuido

desarrollado, el correcto funcionamiento del conjunto de servicios no depende solo de la lógica aislada que contiene cada uno, sino de la fiabilidad de las interacciones entre ellos. Este apartado detalla los mecanismos utilizados para asegurar una comunicación tolerante a fallos y monitorizada entre los distintos módulos, especialmente en las interacciones entre el Servicio de Datos y los Servicios Predictores.

6.4.1. Gestión de retardos en las Peticiones entre Servicios

En el flujo de trabajo síncrono utilizado, el Servicio Predictor realiza una petición HTTP hacia el Servicio de Datos y se bloquea a la espera de una respuesta. Si el Servicio de Datos experimenta una alta latencia debido a una consulta compleja, una sobrecarga de peticiones puntual por distintos clientes o un fallo temporal, el hilo de ejecución del Servicio Predictor queda a la espera de la respuesta, lo que puede provocar un efecto en cascada de *timeouts* y, en el peor de los casos, detener el servidor de desarrollo Flask.

Para solventar este riesgo, se implementó una estrategia de protección en la capa de integración basada en dos conceptos esenciales: *timeouts* configurables y el patrón **Circuit Breaker** [81].

El Patrón Circuit Breaker

El patrón *Circuit Breaker* (o disyuntor) actúa como un interruptor automático en las llamadas a servicios externos. Su objetivo es prevenir que un sistema realice llamadas a un servicio que se sabe que está fallando, dándole tiempo para recuperarse. En lugar de reintentar consultas hacia el servicio, consumiendo recursos innecesariamente, el *breaker* abre el circuito y devuelve un error controlado de forma inmediata, informando al servicio consultante que pare de enviar solicitudes hasta que se recupere el servicio caído.

Este patrón se implementó tanto el Servicio de Datos como en los Servicios Predictores. Se utilizó la librería `circuitbreaker`, integrándola como un decorador sobre las funciones del cliente HTTP que se comunican con servicios ajenos al propio. Asimismo, la librería mencionada proporciona por defecto un *Circuit Breaker* ya configurado y listo para utilizar, pero se le realizaron ciertos cambios personales.

```
1 from circuitbreaker import CircuitBreaker
2 from circuitbreaker import circuit
3
4 # Configuración personalizada del Circuit Breaker
5 class CircuitBreakerPersonalizado(CircuitBreaker):
6     FAILURE_THRESHOLD = 7 # Número máximo de fallos consecutivos
7     # antes de abrir el circuito
```

```

8     RECOVERY_TIMEOUT = 60 # Segundos que permanece abierto el
9     #circuito antes de pasar a semi-abierto
10    EXPECTED_EXCEPTION = requests.exceptions.RequestException
11
12    # Uso del Circuit Breaker
13    @circuit(cls=CircuitBreakerPersonalizado)
14    def get_itacyl_data()

```

Listado 6.6: Implementación del patrón Circuit Breaker

La lógica de funcionamiento es la siguiente:

1. **Estado Cerrado:** El flujo de peticiones es normal. Si una petición falla, se incrementa un contador de fallos. Si el contador supera el umbral `FAILURE_THRESHOLD`, el circuito se abre.
2. **Estado Abierto:** Todas las peticiones entrantes son rechazadas automáticamente, lanzando una excepción `RequestException`. Esto evita saturar a los servicios caídos mediante se recupera. Tras completarse el `RECOVERY_TIMEOUT`, el circuito pasa a un estado semi abierto.
3. **Estado Semi-Abierto:** Se permite el paso de una única petición de prueba. Si esta petición
4. tiene éxito, el circuito se cierra y se reanuda el flujo normal. Si falla, el circuito vuelve a abrirse.

Este mecanismo proporciona al sistema de una **degradación controlada** de servicios. Si un servicio consultor recibe información de que el servicio consultante no está disponible, este servicio consultor no se bloquea indefinidamente, sino que informa al cliente de la indisponibilidad temporal del sistema para completar la operación iniciada.

Manejo Centralizado de Errores y Excepciones Personalizadas

Para complementar el *Circuit Breaker* y garantizar que los errores se gestionan de igual manera en todas las capas de servicios `Flask`, se implementó un sistema de **excepciones personalizadas** y **manejadores globales de errores**.

Se definió una clase base de excepción, `APIException`, que extiende de la clase `Exception` de *Python* y permite encapsular un mensaje de error, un código de estado HTTP y un tipo de error interno.

```

1    class APIException(Exception):
2        """Excepcion base para errores de la API"""
3        def __init__(self, message, status=400, error=None):

```

```
4     super().__init__()
5     self.message = message
6     self.status = status
7     self.error = error if error else 'Error'
```

Listado 6.7: Excepción personalizada para la API

A nivel de aplicación *Flask*, se registró un *error handler* global que captura cualquier excepción lanzada durante el ciclo de vida de una petición y la transforma en una respuesta JSON estructurada para informar al cliente del error producido.

```
1     @app.errorhandler(APIException)
2     def handle_api_exception(e):
3         return jsonify({
4             'status' : e.status,
5             'error' : e.error,
6             'message' : e.message,
7         }), e.status
8
9     @app.errorhandler(Exception)
10    def handle_generic_exception(e):
11        logger.exception("Unhandled exception")
12
13        return jsonify({
14            'success': False,
15            'status': 500,
16            'message': 'Internal Server Error'
17        }), 500
```

Listado 6.8: Manejador global de errores en Flask

Esta estrategia de manejo de errores proporciona tres beneficios claves:

1. **Consistencia en la API:** El cliente siempre recibe respuestas de error con la misma estructura JSON, independientemente si el fallo fue ocasionado por validación de datos, fallo interno del sistema o un servicio externo caído.
2. **Desacoplamiento:** La lógica de negocio de los servicios implementados en el sistema no necesita preocuparse de cómo formatear una respuesta de error HTTP.

3. **Ocultación interna de complejidad:** Uno de los graves problemas que se producen si no se manejan correctamente los errores, es el conocimiento de lógica interna que contiene el sistema. `Flask` por defecto, devuelve la traza del error al cliente para conocer las diferentes iteraciones internas que se han procesado para manejar la consulta recibida. Este aspecto genera problemas de seguridad.

6.4.2. Flujos Asíncronos con el Servicio de IA

Como se ha mencionado en previas secciones, la integración con el Servicio de IA presenta latencias de respuesta debido al procesamiento interno del modelo de lenguaje.

La solución de integración adoptada fue un modelo de **Comunicación Asíncrona basada en colas (Message Queueing)**. El flujo, como se puede apreciar en la Figura 5.12, se divide en tres pasos:

1. **Publicación de Tarea:** El Servicio de Datos, al necesitar un análisis de texto de AEMET, no llama directamente al Servicio de IA. En su lugar, publica el contenido del texto a procesar en un mensaje sobre una cola específica del *broker* `RabbitMQ`, junto con un identificador de seguimiento `/correlation_id`.
2. **Procesamiento en Segundo Plano:** El Servicio de IA que actúa como un *worker* o consumidor, está constantemente suscrito a la cola entrante de `RabbitMQ`. Al recibir un mensaje, obtiene las instrucciones del Servidor MCP, lo procesa, realiza las inferencias con el modelo `qwen 2.5` y, al final, publica su resultado en una cola de respuesta.
3. **Obtención de resultados:** El Servicio de Datos, mientras tanto, se encuentra suscrito a la cola de respuesta, donde obtiene con el tiempo la respuesta procedente del Servicio de IA.

Este patrón de integración aplicado (*Worker Pattern*) permite desacoplar temporalmente a los servicios, permitiendo que el sistema escale de manera más efectiva y no se bloquee ante tareas de larga duración.

6.4.3. Monitorización y Trazabilidad con Logs Estructurados

Para conocer el comportamiento del sistema distribuido implementado requiere aplicar estrategias de monitorización que permitan seguir el camino de una petición a través de los microservicios. Para ello, se implementó un decorador personalizado, `@log`, que se asocia a cada punto final de las *APIs* en cada microservicio.

```
1 import logging
2 import uuid
3 from flask import request, g
4
5 def log(log_file):
6     def decorator(func):
7         def wrapper(*args, **kwargs):
8             g.request_id = str(uuid.uuid4()) # ID unico para la traza
9             logger = logging.getLogger('api')
10
11             # Log de la peticion entrante (omitido por brevedad)
12             # ...
13
14             response = func(*args, **kwargs)
15
16             # Log de la respuesta saliente con metadatos
17             log_entry = {
18                 "request_id": g.request_id,
19                 "endpoint": request.path,
20                 "method": request.method,
21                 "status_code": response.status_code,
22                 "processing_time_ms": processing_time
23             }
24             logger.info(json.dumps(log_entry))
25
26             return response
27         return wrapper
28     return decorator
```

Listado 6.9: Fragmento del decorador de logging estructurado

Este decorador no solo registra la petición en un archivo JSON, sino que genera un **Identificador de Petición** único (`request_id`). Este identificador se propaga en las cabeceras HTTP en las cabeceras HTTP de las llamadas entre microservicios. De esta manera, si un usuario reporta un error en el resultado de una predicción, es posible filtrar los archivos de log de todos los servicios involucrados por ese `request_id` y reconstruir la **traza completa** de la petición.

6.4.4. Seguridad Centralizada

Finalmente, con motivo de cubrir requisitos de seguridad en el sistema, se ha implementado una integración externalizada sobre un Servicio de Autenticación e Identidades llamado *Keycloak*. De esta manera, la lógica asociada al control y generación de credenciales de autenticación y el control de roles en el sistema para restringir accesos sobre ciertas partes de este en función del rol asociado, no lo implementan los microservicios del sistema, sino que se delega a *Keycloak*.

Para ello, se ha implementado un decorador personalizado, `@token_required`. Se implanta sobre los puntos finales que se quieran proteger en las *APIs* de cada microservicio, como el decorador `@log`.

```
1  def token_required(roles: list = None):
2      """
3      Decorador para proteger endpoints con JWT
4      """
5      def decorator(f):
6          @wraps(f)
7          def decorated(*args, **kwargs):
8              # Obtencion de datos de autenticacion en la request
9              # ...
10             try:
11                 # Verificar formato
12                 if not auth_header.startswith('Bearer '):
13                     return jsonify({'message':
14                                     'Formato invalido. Usa: Bearer <token>'}), 401
15
16                 token = auth_header.split(" ")[1]
17
18                 # Obtener configuracion
19                 #keycloak_url = current_app.config.
20                 get('KEYCLOAK_SERVER_URL', 'http://localhost:8090')
21
22                 #realm = current_app.config.
23                 get('KEYCLOAK_REALM', 'undersounds')
24
25                 client_id = current_app.config.
26                 get('KEYCLOAK_CLIENT_ID')
27
28                 # Obtener JWKS
29                 jwks = get_jwks_keys()
```



```
30
31     # Obtener clave publica especifica
32     public_key = get_public_key(token, jwks)
33
34     # Decodificar y verificar JWT
35     current_app.logger.info("Decodificando token...")
36     claims = jwt.decode(
37         token=token,
38         key=public_key,
39         algorithms=['RS256'],
40         audience=client_id,
41         options={
42             "verify_aud": True,
43             "verify_exp": True,
44             "verify_iss": True,
45             "verify_signature": True
46         }
47     )
48     token_roles = []
49     # Validar roles si se especificaron
50     # ....
51
52     # Guardar claims para usar en el endpoint
53     request.user_claims = claims
54
55     # Control de excepciones
56     # ...
57     return f(*args, **kwargs)
58
59     return decorated
60 return decorator
```

Listado 6.10: Fragmento del decorador validación token

6.5. Despliegue y ejecución

Con el objetivo de garantizar que el sistema desarrollado sea accesible y fácilmente desplegable por cualquier persona o entidad interesada, se ha prestado especial atención a la configuración del entorno

de ejecución. Para ello, se ha estructurado un ecosistema de despliegue que permite levantar toda la arquitectura de microservicios con un único comando.

6.5.1. Organización de código fuente y control de versiones

Todo el código fuente del proyecto se encuentra centralizado en una **organización** dedicada dentro de la plataforma de *GitHub* [82]. Cada uno de los microservicios descritos en secciones anteriores cuentan con su propio repositorio. Esta separación no solo facilita el mantenimiento y la evolución individual de cada módulo, sino que también permite un control de versiones independiente por repositorio, donde una actualización de un servicio concreto no necesita recompilar o redesplegar el resto del sistema.

6.5.2. Orquestador general: `agro_predict_orchestator`

Para unificar y simplificar el despliegue del sistema, se ha creado un repositorio central de configuración denominado **`agro_predict_orchestator`**. Este repositorio sirve para desplegar todo el sistema, abstrayendo al usuario final de la necesidad de conocer los puertos, las dependencias y las variables de entorno específicas de cada microservicio.

En función de los recursos hardware que se tenga en el equipo donde se va a desplegar el sistema, el orquestador cuenta con dos configuraciones independientes de despliegue:

`docker/`

Este directorio contiene el fichero principal de orquestación: `compose.yml`. En él se define la configuración completa del sistema, incluyendo todos los microservicios, la base de datos **`MariaDB`**, el *broker* **`RabbitMQ`** y el servidor de autenticación **`Keycloak`**. Este es el modo de despliegue **completo y recomendado**, que hace uso del Servicio de *IA* para obtener la máxima precisión en el tratamiento de datos textuales de AEMET.

`no-gpu/`

Teniendo en cuenta que no todos los entornos de ejecución disponen de una unidad de procesamiento gráfico (GPU) dedicada, se ha preparado una configuración alternativa. En este escenario, el Servicio de *IA* y el motor **`OLLAMA`** se omiten del despliegue (véase Listado 6.11).

La motivación de esta variante se basa en la siguiente circunstancia: la ejecución del modelo de lenguaje **`qwen2.5`** íntegramente sobre CPU resulta en tiempos de inferencia largos para un servicio que debe ser ágil. Para garantizar la operatividad del sistema en estos casos, se activa automáticamente el mecanismo de salvaguarda basado en el **parser de expresiones regulares**.

Es importante destacar que, si bien esta versión “ligera”, permite ejecutar el sistema en equipos menos potentes, la precisión de los datos estructurados extraídos de AEMET se ve reducida, asumiendo las limitaciones de un analizador sintáctico frente a la comprensión contextual de un LLM.

```
1 name: agro-predict
2 services:
3   mariadb:
4     image: mariadb:10.11
5     environment:
6       MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
7       MYSQL_DATABASE: ${MYSQL_DATABASE}
8       MYSQL_USER: ${MYSQL_USER}
9       MYSQL_PASSWORD: ${MYSQL_PASSWORD}
10    ports:
11      - "3307:3306"
12    volumes:
13      - mariadb_data:/var/lib/mysql
14    healthcheck:
15      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
16      interval: 10s
17      timeout: 5s
18      retries: 5
19
20  cache:
21    image: redis:7.4-alpine
22    ports:
23      - "6379:6379"
24    command: redis-server --requirepass ${REDIS_PASS}
25    volumes:
26      - cache:/data
27
28  aemet-service:
29    image: mendo1/agro-predict:aemet-service-1.0
30    environment:
31      - AEMET_API_KEY=${AEMET_API_KEY}
32      - PORT=4000
33    ports:
34      - "4000:4000"
35
```

```
36  siar-service:
37      image: mendo1/agro-predict:siar-service-1.0
38      environment:
39          - SIAR_API_KEY=${SIAR_API_KEY}
40      ports:
41          - "8086:8086"
42
43  itacyl-service:
44      image: mendo1/agro-predict:itacyl-service
45      environment:
46          - ITACyL_API_KEY=${ITACyL_API_KEY}
47      ports:
48          - "8087:8087"
49
50  rabbitmq:
51      image: rabbitmq:latest
52      container_name: broker
53      restart: unless-stopped
54      ports:
55          - "5673:5672"
56          - "15673:15672"
57      environment:
58          RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER}
59          RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASS}
60      volumes:
61          - ./rabbitmq/conf.d:/etc/rabbitmq/conf.d
62
63  worker:
64      image: mendo1/agro-predict:data-service-latest
65      container_name: worker
66      command: celery -A celery_worker.celery_app worker --loglevel=info
67      env_file:
68          - .env
69      depends_on:
70          - data-service
71          - cache
72          - rabbitmq
73      restart: unless-stopped
74
75  data-service:
```

```
76     image: mendo1/agro-predict:data-service-latest
77     ports:
78       - "9002:5000"
79     depends_on:
80       - mariadb
81       - cache
82       - aemet-service
83       - siar-service
84       - itacyl-service
85
86     predictors:
87       image: mendo1/agro-predict:predictors-1.0
88       ports:
89         - "10000:10000"
90       depends_on:
91         - data-service
92
93     volumes:
94       mariadb_data:
95       aemet-data:
96       cache:
```

Listado 6.11: Configuración resumen `compose.yml` para `no-gpu`

En definitiva, esta estrategia de despliegue permite su puesta en marcha tanto en servidores *cloud* de altas prestaciones como en máquinas de desarrollo locales, adaptándose al contexto de quien lo utilice sin eliminar lógica principal del sistema (más información sobre la instalación y uso de la infraestructura en Anexo B y Anexo C)

6.5.3. Despliegue en Entorno Cloud

Con el objetivo de probar que el despliegue puede realizarse sobre cualquier máquina o equipo, se proporcionó accesos sobre una máquina virtual de AWS (Amazon Web Service) para configurar la implementación en este entorno de producción.

Las acciones necesarias para el despliegue, lejos de mantener una comunicación `SSH` desde el equipo local de desarrollo hacia la máquina virtual proporcionada, fue la clonación desde la máquina del repositorio orquestador público (`agro_predict_orchestator`) y la ejecución del comando `docker compose up` sobre

la carpeta *no-gpu*, debido a los problemas de memoria que mantenía la máquina. Al tener la población de datos iniciales (necesarios para el arranque del sistema) automatizada con el Servicio de Ingesta frente a la base de datos, no fue necesario realizar ninguna otra acción.

6.5.4. Automatización DevOps

Tras el desarrollo de la infraestructura *software* del proyecto y su despliegue en entornos *cloud*, resulta necesario mantener un entorno controlado y fiable ante futuros cambios en los repositorios de *GitHub*, donde se aloja el código fuente de cada componente del sistema.

Para ello, se han aplicado prácticas de automatización de tareas propias de la disciplina *DevOps* (*Development and Operations*), basadas principalmente en los procesos de **Integración Continua** (CI) y **Despliegue Continuo** (CD).

Integración Continua (CI): La integración continua es un proceso de automatización que permite a los desarrolladores integrar cambios en el código de forma frecuente en una rama principal o de desarrollo. Cada integración desencadena la ejecución automática de pruebas con el objetivo de garantizar que los cambios incorporados no introducen errores ni inconsistencias en el sistema.

Este proceso se considera exitoso cuando las modificaciones se integran correctamente, la aplicación compila de forma adecuada y todas las pruebas automatizadas se ejecutan sin fallos.

La implementación de este mecanismo se ha realizado mediante *GitHub Actions*, configurado en los distintos repositorios de los microservicios. Esta herramienta permite definir flujos de trabajo que se ejecutan automáticamente ante eventos específicos del repositorio.

En este caso, los flujos se activan principalmente ante eventos de *push* o la creación de una *pull request*. Cuando esto ocurre, se activan una secuencia de pasos encargados de integrar y validar el código antes de su incorporación a la rama principal. Si alguno de los pasos falla, el flujo se detiene automáticamente y los cambios no se fusionan hasta que los errores sean corregidos.

El flujo de trabajo, también denominado *workflow*, comienza con la selección del sistema operativo sobre el que se ejecutarán las acciones. Esto es posible debido a que *GitHub* proporciona máquinas virtuales efímeras (*runners*) que ejecutan los procesos automatizados, permitiendo al desarrollador elegir el entorno de ejecución. En este proyecto se ha seleccionado **Ubuntu** como sistema operativo para la ejecución de dichos flujos de trabajo.

Posteriormente, se identifican los pasos (*steps*) que se van a lanzar cuando se produzca un evento de los



ya mencionados. Cada paso presenta la misma estructura de configuración base, pudiendo personalizarla o ampliarla según las necesidades a cubrir:

- **uses** (opcional): Sirve para ejecutar una acción de código abierto (*open-source*), que en esencia es una pieza de código empaquetada y reutilizable disponible en *GitHub*. Su uso es opcional, pero su aplicación permite definir acciones más compactas y legibles para futuras modificaciones y mantenimientos. La nomenclatura de referencia utilizada mantiene la siguiente estructura: `actions/nombre-publico-de-la-action@vX`, la identificación vX hace referencia a la versión que se quiera utilizar de la acción pública elegida.
- **name**: Nombre identificativo del paso configurado. Sirve de ayuda para describir en pocas palabras **de qué** se encarga el paso asociado, permitiendo mantener una trazabilidad en tiempos de ejecución.
- **with** (opcional): Al utilizar una acción ya configurada, identificada con el valor del atributo *uses*, la persona o entidad encargada de su mantenimiento dispone al público una serie de especificaciones de uso sobre la acción implementada. Sobre esas especificaciones se definen una serie de atributos con valores de ejemplos, permitiendo personalizar el comportamiento de la acción a utilizar. Con los valores definidos sobre este atributo *with*, indica directamente a la acción definida en el atributo *uses*, cómo se debe comportar.
- **run**: Definición de comandos a ejecutar sobre el sistema operativo configurado para completar el trabajo asociado o ciertos pasos dentro del trabajo.

Para comprender mejor la configuración básica de una automatización para un proceso de integración continua, a continuación se presenta un fragmento de código dentro de una *GitHub Action* (véase Listado 6.12).

```
1 name: Python application
2
3 on:
4   push:
5     branches: [ "main" ]
6   pull_request:
7     branches: [ "main" ]
8
9 jobs:
10   build:
11
12     runs-on: ubuntu-latest
```

```

13
14     steps:
15     - uses: actions/checkout@v6
16     - name: Set up Python 3.13
17       uses: actions/setup-python@v6
18       with:
19         python-version: "3.13"
20     - name: Install system dependencies
21       run: sudo apt-get update && sudo apt-get install -y libmariadb-dev
22     - name: Install dependencies
23       run: |
24         python -m pip install --upgrade pip
25         pip install flake8 pytest
26         if [ -f requirements.txt ]; then pip install -r requirements.txt;fi
27     - name: Lint with flake8
28       run: |
29         flake8 . --count --select=E9,F63,F7,F82 --show-source
30         --statistics
31         flake8 . --count --exit-zero --max-complexity=10 --max-line-length=127
32         --statistics
33     - name: Test with pytest
34       env:
35         PYTHONPATH: .
36       run: pytest app/tests/ -v

```

Listado 6.12: Fragmento de configuración para Integración Continua (CI)

Como se puede observar en el código (véase Listado 6.12), en el atributo *on* se definen los eventos sobre los que se quiere que se ejecute la automatización y la rama del repositorio restringiendo el uso de esta acción, puesto que *GitHub* cobra por minutos de ejecución utilizados. Seguidamente, se definen los trabajos o *jobs*, donde se configuran los distintos pasos necesarios para validar correctamente el código fuente y considerar la ejecución como exitosa.

El flujo de trabajo comienza con la preparación del entorno de ejecución sobre una máquina virtual basada en *Ubuntu*. Posteriormente, se realiza la obtención del código fuente del repositorio y la configuración de la versión de *Python* requerida por el proyecto.

Una vez preparado el entorno, se instalan las dependencias necesarias tanto del sistema operativo como de la propia aplicación, permitiendo disponer de todas las librerías y herramientas requeridas para la validación del servicio.

Finalmente, el flujo ejecuta procesos automáticos de análisis y validación del código mediante herramientas de comprobación y ejecución de pruebas automatizadas. Este procedimiento permite detectar posibles errores de sintaxis, incumplimientos de calidad o fallos funcionales antes de integrar los cambios sobre la rama principal del repositorio.

Despliegue Continuo (CD): El despliegue continuo consiste en la automatización del proceso de publicación de nuevas versiones del sistema en un entorno de producción, de forma que cada cambio que supera correctamente las fases de integración y validación se despliega automáticamente sin intervención manual.

En este proyecto, el despliegue continuo se centra en la construcción y actualización automática de imágenes *Docker* asociadas a los distintos microservicios. Una vez que el flujo de integración continua finaliza de manera satisfactoria, se desencadena un proceso automatizado que genera una nueva imagen del servicio modificado y la publica en **Docker Hub**, plataforma organizada por repositorios públicos y centralizados donde el servicio orquestador del proyecto, **agro-predict-orchestator**, podrá consultar las actualizaciones disponibles y desplegar el sistema utilizando los contenedores *Docker* más recientes.

El etiquetado de las imágenes generadas sigue una convención que distingue el entorno de destino: si la actualización proviene de la rama **main**, la imagen recibe la etiqueta **latest** (p.ej., **data-service-latest**), indicando que se trata de una versión estable lista para producción. En cambio, si proviene de cualquier otra rama, la imagen se etiqueta con el hash corto del **commit** correspondiente (p.ej., **data-service-a1b2c3d**), lo que permite identificar la versión de la imagen de producción y la imagen de despliegue.

Como paso final del flujo de cada microservicio, se lanza un evento personalizado (*repository dispatch*) hacia el repositorio **agro-predict-orchestator**. Este evento transporta tres datos clave al actuarse: el nombre del servicio actualizado, la etiqueta de la imagen recién generada y la rama de origen. De esta forma, el servicio orquestador no actúa de forma autónoma, sino que únicamente se activa cuando recibe dicha señal, asegurando que cualquier despliegue esté siempre vinculado a una actualización real de alguno de los microservicios.

El comportamiento que sigue el **servicio orquestador** una vez recibido el evento depende en gran medida de la rama desde la que se originó la actualización. Para comprender esta distinción, es necesario introducir brevemente conceptos básicos del uso de ramas utilizadas en repositorios de *GitHub*.

Todo repositorio en *GitHub* dispone de una rama principal, denominada **main**, que contiene la versión estable del proyecto y es la que se despliega en el entorno de producción. A partir de ella, los desarrolladores pueden crear tantas ramas paralelas como sean necesarias para el desarrollo de nuevas funcionalidades,

correcciones o mejoras, manteniendo el código base aislado hasta su validación. Una vez finalizado el trabajo en una rama secundaria, los cambios se integran en `main` mediante un proceso de revisión, habitualmente a través de *pull requests*, garantizando así el control de calidad antes de su incorporación al entorno de producción.

Con este modelo en mente, el **servicio orquestador** define dos comportamientos diferenciados en función de la rama de origen:

1. **Actualización originada en una rama de desarrollo** (paralela a `main`): el orquestador realiza o **crea** dicha rama en su propio repositorio si no existía previamente, actualiza en ella la etiqueta de la imagen del servicio modificado dentro de los ficheros `compose.yml` correspondientes, y confirma el cambio mediante un `commit`. No se produce ningún despliegue sobre las máquinas virtuales definidas en producción, ya que los cambios aún no han sido validados para producción. Este comportamiento ayuda a que todo desarrollador trabajando en dicha rama, pueda mantener una versión de contenedores *Docker* en desarrollo, actualizada al último `commit` realizado sobre el servicio modificado hasta que se validen los cambios para producción.
2. **Actualización originada en la rama `main`**: el orquestador actualiza directamente la etiqueta de la imagen en los ficheros `compose.yml` de la rama principal y, si se detectan cambios reales respecto al estado anterior, procede a desplegar el sistema en las máquinas virtuales configuradas. El despliegue se realiza mediante conexión SSH, descargando las nuevas imágenes y recreando los contenedores afectados.

Para comprender mejor la configuración de automatización explicada en el proceso de despliegue continuo aplicado en el proyecto, a continuación se muestran dos fragmentos de código dentro de las *GitHub Actions* para los microservicios (véase Listado 6.13) y para el servicio orquestador (véase Listado 6.14).

```
1
2 job:
3     docker:
4
5     needs: [build]
6     if: github.event_name == 'push'
7
8     runs-on: ubuntu-latest
9     steps:
10     - name: checkout
11       uses: actions/checkout@v6
12
```

```

13     - name: Extract Docker image metadata
14       id: meta
15       uses: docker/metadata-action@v6
16       with:
17         images: ${{ vars.DOCKER_USERNAME }}/agro-predict
18         tags: |
19             type=sha,prefix=data-service-,format=short,
20             enable=${{ github.ref != 'refs/heads/main' }}
21             type=raw,value=latest,prefix=data-service-,
22             enable=${{ github.ref == 'refs/heads/main' }}
23
24     - name: Login in to Docker Hub
25       uses: docker/login-action@v4
26       with:
27         username: ${{ vars.DOCKER_USERNAME }}
28         password: ${{ secrets.DOCKER_PASSWORD }}
29
30     - name: Set Up Docker Build
31       uses: docker/setup-buildx-action@v4
32
33
34     - name: Build and push to Docker Hub
35       uses: docker/build-push-action@v7
36       with:
37         context: .
38         push: true
39         tags: ${{ steps.meta.outputs.tags }}
40         annotations: ${{ steps.meta.outputs.annotations }}
41         provenance: true
42         sbom: true
43         cache-from: type=gha
44         cache-to: type=gha,mode=max
45
46     - name: Trigger the result to orchestrator repository
47       uses: peter-evans/repository-dispatch@v4
48       with:
49         token: ${{ secrets.PAT }}
50         repository: agro-predict-tfg-2026/agro-predict-orchestrator
51         event-type: deploy
52         client-payload: |

```

```

53     {
54         "service": "data-service",
55         "tag": "${ steps.meta.outputs.version }",
56         "branch": "${ github.ref_name }"
57     }

```

Listado 6.13: Despliegue de cambios en *Docker* (CD) microservicios

Sobre este fragmento de código cabe destacar ciertos atributos:

- **needs**: Identifica los trabajos (*jobs*) de los que depende para efectuarse, sino no resultan satisfactorios, este no se ejecutará.
- **client-payload**: Conjunto de datos que se pasarán al repositorio indicado en el atributo **repository** cuando se efectúe el evento personalizado definido en el atributo **event-type** / **uses**.

```

1 jobs:
2   update_tags:
3     runs-on: ubuntu-latest
4     outputs:
5       changed: ${ steps.check.outputs.changed }
6       is_main: ${ steps.check.outputs.is_main }
7
8   steps:
9     - uses: actions/checkout@v4
10      with:
11        fetch-depth: 0
12
13     - name: Setup branch
14       id: branch
15       run: |
16         BRANCH="${ github.event.client_payload.branch }"
17         IS_MAIN=${ github.event.client_payload.branch == 'main' }
18         echo "is_main=$IS_MAIN" >> $GITHUB_OUTPUT
19
20         if [ "$BRANCH" != "main" ]; then
21           git fetch origin $BRANCH 2>/dev/null && git checkout $BRANCH \
22             || git checkout -b $BRANCH
23         fi
24

```

```

25     - name: Update image tags
26       run: |
27         sed -i
28           "s|mendo1/agro-predict:${{ github.event.client_payload.service }}
29           -[^ ]*|mendo1/agro-predict:${{ github.event.client_payload.tag }}|g"
30         docker/compose.yml
31         sed -i
32           "s|mendo1/agro-predict:${{ github.event.client_payload.service }}
33           -[^ ]*|mendo1/agro-predict:${{ github.event.client_payload.tag }}|g"
34         no-gpu/compose.yml
35
36     - name: Check for changes
37       id: check
38       run: |
39         git add docker/compose.yml no-gpu/compose.yml
40         if git diff --cached --quiet; then
41           echo "changed=false" >> $GITHUB_OUTPUT
42           echo "is_main=${{ steps.branch.outputs.is_main }}" >> $GITHUB_OUTPUT
43         else
44           echo "changed=true" >> $GITHUB_OUTPUT
45           echo "is_main=${{ steps.branch.outputs.is_main }}" >> $GITHUB_OUTPUT
46         fi
47
48     - name: Commit and push
49       if: steps.check.outputs.changed == 'true'
50       run: |
51         git config user.name 'github-actions[bot]'
52         git config user.email 'github-actions[bot]@users.noreply.github.com'
53         git remote set-url origin https://x-access-token:
54         ${ secrets.GITHUB_TOKEN }
55         @github.com/agro-predict-tfg-2026/agro-predict-orchestator.git
56         git commit -m "chore: update ${github.event.client_payload.service}}
57         to ${ github.event.client_payload.tag }}"
58         git push origin HEAD

```

Listado 6.14: Actualización de etiquetas (CD) orquestador

Con el objetivo de comprender el flujo completo de automatización que se ha implementado, a continuación se dispone de una representación gráfica donde se comprenden los pasos seguidos desde que un usuario sube una modificación de código a un repositorio, hasta que se despliegan exitosamente

los cambios sobre el entorno de producción (véase Figura 6.11).

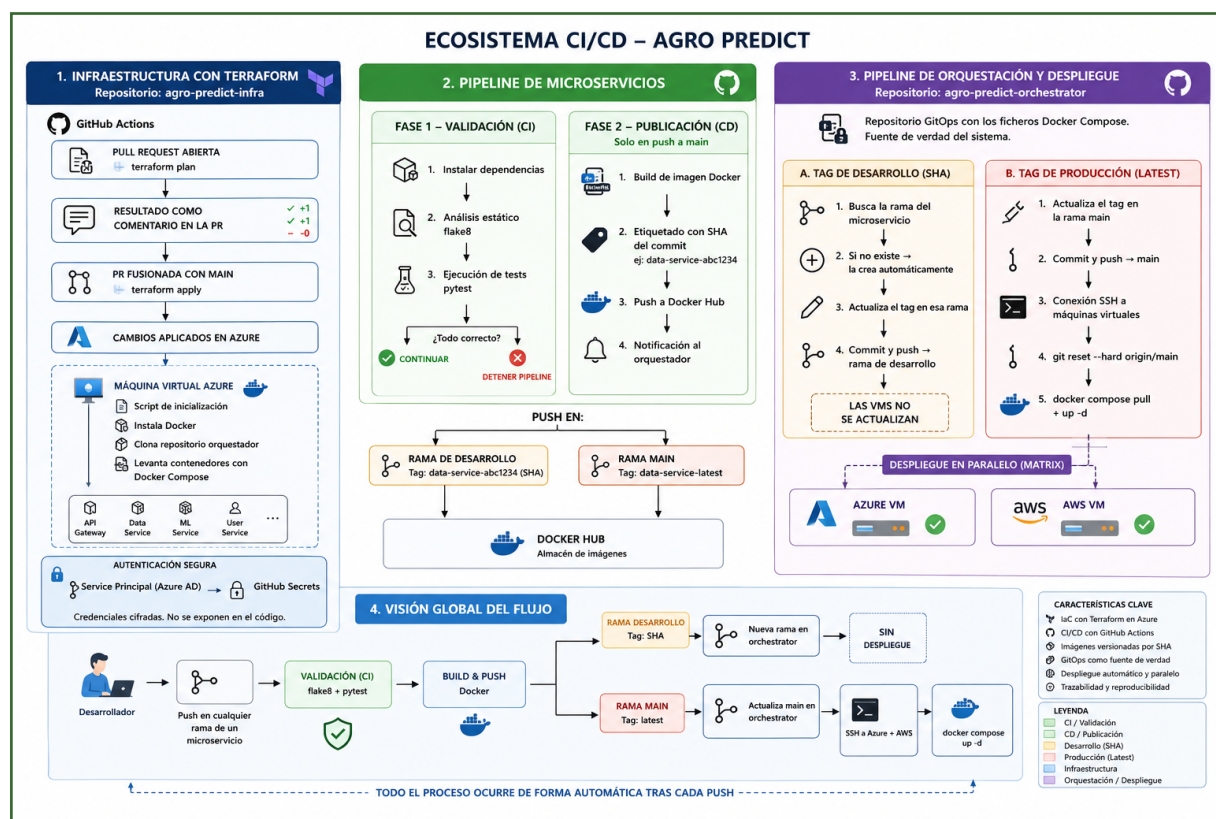


Figura 6.11: Representación Gráfica Flujo CI/CD

Fuente: Elaboración propia

6.5.5. Uso de Infraestructura como Código (IaC)

Tras realizar el despliegue *cloud* de la infraestructura desarrollada sobre una máquina virtual de AWS (véase Subsección 6.5.3) y aplicar prácticas propias de la disciplina *DevOps* para mantener un ecosistema automatizado y mantenible, se decidió ampliar este enfoque mediante la incorporación de prácticas de **Infraestructura como Código (IaC)** [83].

El objetivo principal de esta incorporación es permitir la definición, configuración y gestión de los componentes de infraestructura mediante código, facilitando su despliegue sobre distintos proveedores *cloud* como AWS, Google Cloud o Microsoft Azure, entre otros.

Este enfoque proporciona una mayor flexibilidad en la administración de la infraestructura, permitiendo replicar entornos de forma consistente y simplificando posibles procesos de migración entre proveedores en función de las necesidades técnicas o económicas del proyecto.

Herramienta utilizada

La herramienta seleccionada para la definición y aprovisionamiento de infraestructura ha sido *Terraform* [84].

Los principales motivos que justifican su utilización son los siguientes [85]:

- **Compatibilidad multi-proveedor:** *Terraform* permite gestionar infraestructura sobre múltiples proveedores *cloud* mediante una sintaxis unificada, facilitando la portabilidad y reutilización de configuraciones.
- **Lenguaje declarativo:** La herramienta utiliza el lenguaje declarativo HCL (*HashiCorp Configuration Language*) [86], diseñado para definir infraestructura de forma clara, legible y mantenible por futuros desarrolladores o administradores del sistema.
- **Escalabilidad y flexibilidad:** Permite gestionar desde pequeños entornos hasta infraestructuras distribuidas de gran tamaño, adaptándose al crecimiento y evolución de las necesidades del proyecto.
- **Versionado:** Los planes de infraestructura a desplegar generados con *Terraform* a través de su comando en la terminal, `terraform plan`, pueden versionarse para mantener respaldos de producción en casos donde la versión actual disponible presente errores y deban ser solventados mientras la infraestructura de la aplicación afectada sigue disponible a cualquier usuario.

Componentes desplegados

Para el despliegue de la infraestructura mediante *Terraform*, se ha mantenido el enfoque basado en una máquina virtual desplegada sobre *Microsoft Azure* [87]. Junto a la configuración de dicha máquina virtual, también se han definido recursos de red virtual con el objetivo de mantener un entorno de comunicación controlado entre los distintos componentes de la infraestructura.

Adicionalmente, se han configurado otros servicios complementarios orientados a mejorar la gestión y mantenimiento del sistema. Entre ellos destacan:

- **Azure Key Vault:** Servicio utilizado para la gestión segura de secretos del proyecto, permitiendo almacenar claves de acceso, credenciales y otra información sensible [88].
- **Azure Blob Storage:** Servicio de almacenamiento utilizado para centralizar y persistir registros (*logs*) generados por el sistema [89].

Aunque estos servicios adicionales han sido definidos y configurados mediante *Terraform*, se decidió no desplegarlos finalmente en el entorno de producción. Esta decisión se debe, por un lado, a las modificaciones adicionales que sería necesario realizar sobre los distintos microservicios para integrar completamente dichos componentes y, por otro, las limitaciones de recursos presentes en la cuenta de estudiantes utilizada en *Azure*.

Automatización interna de la máquina virtual

Con el objetivo de mantener un entorno de despliegue completamente automatizado, fue necesario definir una serie de acciones que se ejecutan internamente sobre la máquina virtual desplegada en *Azure*.

Estas acciones permiten preparar automáticamente el entorno de ejecución y desplegar la infraestructura contenerizada basada en *Docker*. Para ello, se configuró un *script* de inicialización que se ejecuta automáticamente cada vez que la máquina virtual arranca.

A continuación, se describen las principales tareas realizadas por dicho *script*:

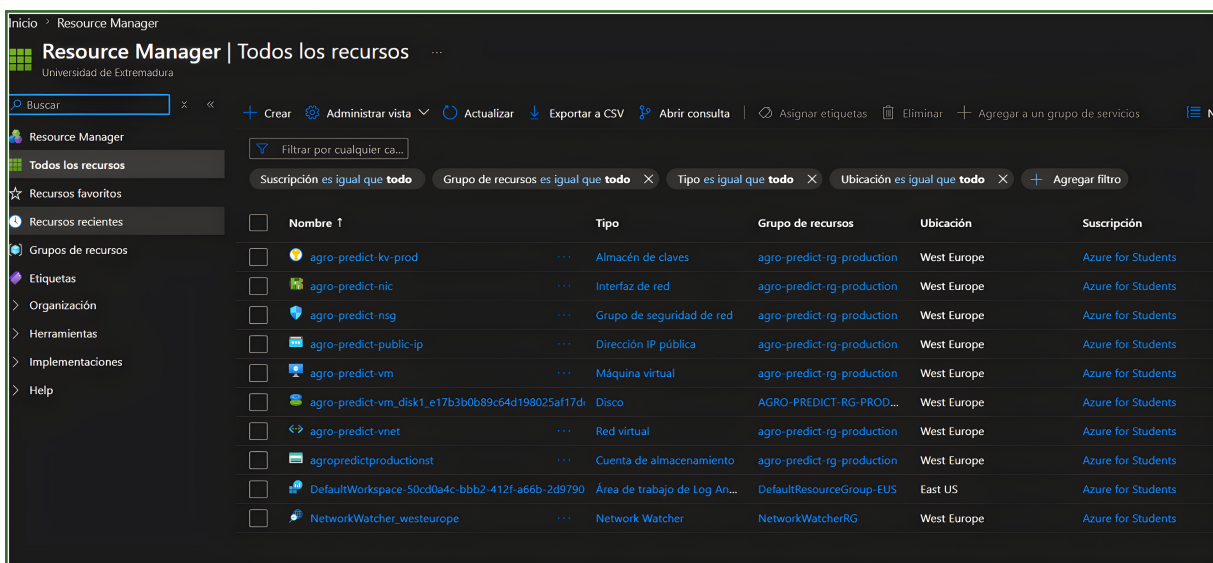
1. **Instalación de dependencias de *Docker*:** Debido a que la máquina virtual se inicializa sin herramientas adicionales instaladas, es necesario instalar *Docker* y sus dependencias siguiendo las recomendaciones oficiales proporcionadas por la documentación de su página oficial [90].
2. **Clonación del repositorio de orquestación:** La organización del proyecto, como ya se ha comentado previamente, dispone de un repositorio denominado **agro-predict-orchestrator**, encargado de almacenar la configuración necesaria para el despliegue de los contenedores *Docker*. Por ello, el *script* realiza automáticamente la clonación del repositorio sobre la máquina virtual.
3. **Generación de variables de entorno:** Por motivos de seguridad, el repositorio de orquestación no almacena directamente las variables de entorno necesarias para el funcionamiento de los servicios desplegados. Para solucionar este problema, *Terraform* genera dinámicamente un fichero **.env** durante el proceso de aprovisionamiento, incorporándolo posteriormente al entorno de despliegue correspondiente.

En este caso, el fichero se incorpora sobre la configuración **no-gpu**, debido a que la máquina virtual desplegada no dispone de gráfica dedicada para ejecutar determinados procesos asociados a agentes inteligentes.
4. **Despliegue automático de contenedores:** Finalmente, tras completar la preparación del entorno, se ejecutan los comandos necesarios para descargar y desplegar automáticamente los contenedores definidos mediante *Docker Compose*:


```
docker compose pull
docker compose up -d
```

Representación visual de componentes desplegados

Con el objetivo de representar visualmente los distintos componentes generados automáticamente mediante la ejecución de la configuración desarrollada en *Terraform*, a continuación se presenta una representación gráfica de la infraestructura desplegada (véase Figura 6.12).



Nombre	Tipo	Grupo de recursos	Ubicación	Suscripción
agro-predict-kv-prod	Almacén de claves	agro-predict-rg-production	West Europe	Azure for Students
agro-predict-nic	Interfaz de red	agro-predict-rg-production	West Europe	Azure for Students
agro-predict-nsg	Grupo de seguridad de red	agro-predict-rg-production	West Europe	Azure for Students
agro-predict-public-ip	Dirección IP pública	agro-predict-rg-production	West Europe	Azure for Students
agro-predict-vm	Máquina virtual	agro-predict-rg-production	West Europe	Azure for Students
agro-predict-vm_disk1_e17b3b0b89c64d198025af17d	Disco	AGRO-PREDICT-RG-PROD...	West Europe	Azure for Students
agro-predict-vnet	Red virtual	agro-predict-rg-production	West Europe	Azure for Students
agropredictproductionst	Cuenta de almacenamiento	agro-predict-rg-production	West Europe	Azure for Students
DefaultWorkspace-50cd0a4c-bbb2-412f-a66b-2d9790	Área de trabajo de Log An...	DefaultResourceGroup-EUS	East US	Azure for Students
NetworkWatcher_westeurope	Network Watcher	NetworkWatcherRG	West Europe	Azure for Students

Figura 6.12: Despliegue Componentes Azure

Fuente: Elaboración propia

6.6. Resultados obtenidos

Una vez completada la fase de implementación y desplegado el sistema en su totalidad, se procedió a evaluar su comportamiento. El objetivo de esta sección no es realizar un análisis exhaustivo de las soluciones obtenidas, sino **validar cualitativamente** que la integración de los distintos módulos produce resultados coherentes, útiles y alineados con los requisitos y objetivos iniciales planteados.

Para ello, se realizaron una serie de peticiones hacia los puntos finales de los Servicios Predictores, simulando el flujo de acciones que un cliente real puede realizar para consultar riesgos de heladas y plagas sobre cultivos y localizaciones concretas de la provincia de Cáceres. Los resultados que se presentan a

continuación corresponden a las salidas generadas por el sistema y almacenadas en su historial de logs e informes.

6.6.1. Validación del Servicio Predictor de Heladas

Para probar el Servicio Predictor de Heladas, se realizó una petición progresiva en el tiempo, consultando el riesgo de heladas sobre las variedades del cultivo **Almendro** registradas en el sistema: **Guara** y **Lauranne**.

Predicción futura provincial — POST /heladas/futuras/provincial La petición se realizó sobre el siguiente *endpoint*: `/heladas/futuras/provincial` con los siguientes valores en el cuerpo de la petición (véase Listado 6.15).

```
1 {  
2     "codigo_provincia" : "CC",  
3     "evaluacion_var" : true,  
4     "evaluacion_loc" : false,  
5     "variedades" : [  
6         "Lauranne",  
7         "Guara"  
8     ]  
9 }
```

Listado 6.15: Cuerpo de la petición `/heladas/futuras/provincial`

Como se ha aplicado el *endpoint* futuro, realiza la predicción para el día de mañana.

Respuesta de la API y alertas generadas El sistema devolvió una respuesta HTTP 200 con un cuerpo JSON estructurado. Un fragmento relevante de la respuesta, que muestra el nivel de riesgo general y las alertas asociadas, se presenta a continuación:

```
1     "tipo_prediccion": "tomorrow",  
2     "evaluaciones_variedades": {  
3         "total_variedades_evaluados": 2,  
4         "variedades_en_riesgo_critico": 0,  
5         "variedades_en_riesgo_alto": 0,
```

```
6      "variedades_en_riesgo_moderado": 2,
7      "variedades_en_riesgo_debil": 0,
8      "variedades_sin_riesgo": 0,
9      "evaluaciones": [
10         {
11             "variedad": "lauranne",
12             "nombre_cientifico": null,
13             "etapa_fenologica": "maduracion",
14             "temperatura_evaluada": 11,
15             "porcentaje_riesgo": 34.99999140045284,
16             "nivel_riesgo": "moderado",
17             "umbrales": {
18                 "critico": 0.0,
19                 "alto": 2.0,
20                 "moderado": 20.0,
21                 "bajo": 25.0
22             },
23             "alertas": []
24         },
25         {
26             "variedad": "guara",
27             "nombre_cientifico": null,
28             "etapa_fenologica": "maduracion",
29             "temperatura_evaluada": 11,
30             "porcentaje_riesgo": 34.99999140045284,
31             "nivel_riesgo": "moderado",
32             "umbrales": {
33                 "critico": 0.0,
34                 "alto": 2.0,
35                 "moderado": 20.0,
36                 "bajo": 25.0
37             },
38             "alertas": []
39         }
40     ]
41 },
42 "riesgos_heladas_blancas": [],
43 "riesgos_heladas_negras": [],
```

Listado 6.16: Fragmento de respuesta JSON del predictor de heladas futuras

Este fragmento mostrado valida la estructura de la respuesta JSON ajustado a los **objetos de transferencia de datos** definidos en el servicio, lo que garantiza que un futuro cliente *frontend* pudiera ajustar y mostrar la información generada sin ambigüedades.

Paralelamente a la respuesta en formato JSON, el servicio activó la generación de un informe en segundo plano mediante la librería *ReportLab*. El resultado que se muestra en las figuras: Figura 6.13, Figura 6.14, Figura 6.15, Figura 6.16, está comprendido por tablas que representan el análisis por variedad obtenido y un gráfico de evolución temporal (véase Figura 6.17). La inclusión de una firma digital configurada con la librería *pyHanko* certifica la trazabilidad del documento generado de forma automática.

Informe de Predicciones ante Riesgos de Helada		Actualización: 02/06/2026
Predicciones Meteorológicas Automatizadas - Histórico		Registros: 16
Fuentes: AEMET (predicción futura) - SIAR-Extremadura (datos históricos)		
Localización de cálculo: CC		

INFORME ACUMULATIVO DE RIESGOS DE HELADA	
SISTEMA AGRO-PREDICT	
Período analizado: 02/06/2026 - 28/05/2026 Total de registros: 16 entr. Última actualización: 02/06/2026 16:18 Variedades evaluadas: lauranne, guara (Cultivo: Almendro).	
FUENTES DE DATOS UTILIZADAS	
Campo	Detalle
Zona geográfica	provincial
Provincia / Código	CC - Cáceres
Fuente datos históricos	SIAR — Red de estaciones agrometeorológicas de Extremadura
Fuente datos futuros	AEMET — Agencia Estatal de Meteorología
Período cubierto	02/06/2026 → 28/05/2026
Última actualización	02/06/2026 16:18

CUERPO DE LA PETICIÓN - PARÁMETROS	
Parámetro	Valor
Provincia objetivo	CC — Cáceres
Evaluación sobre variedades	✓ Si
Evaluación sobre localidades	✗ No
Variedades a evaluar	lauranne, guara
Localidades a evaluar	—

Figura 6.13: Predicción Heladas Tabla. Fragmento 1-4
Fuente: Elaboración propia

</

Figura 6.14: Predicción Heladas Tabla. Fragmento 2-4
Fuente: Elaboración propia



Figura 6.15: Predicción Heladas Tabla. Fragmento 3-4
Fuente: Elaboración propia

CC105	Mirabel	SiAR Extremadura (Oficial)
CC107	Rebollar	SiAR Extremadura (Oficial)
CC108	Tiétar	SiAR Extremadura (Oficial)
CC109	Valdeínigos II	SiAR Extremadura (Oficial)
CC11	Jarandilla de la Vera	SiAR Extremadura (Oficial)
CC12	Gargantilla	SiAR Extremadura (Oficial)
CC13	Talayuela	SiAR Extremadura (Oficial)
CC14	Coria-Puebla de Argeme	SiAR Extremadura (Oficial)
CC16	Moraleja	SiAR Extremadura (Oficial)
CC17	Valdastillas	SiAR Extremadura (Oficial)
CC18	Valdesalor	SiAR Extremadura (Oficial)
CC19	Zarza de Granadilla	SiAR Extremadura (Oficial)

Figura 6.16: Predicción Heladas Tabla. Fragmento 4-4
Fuente: Elaboración propia

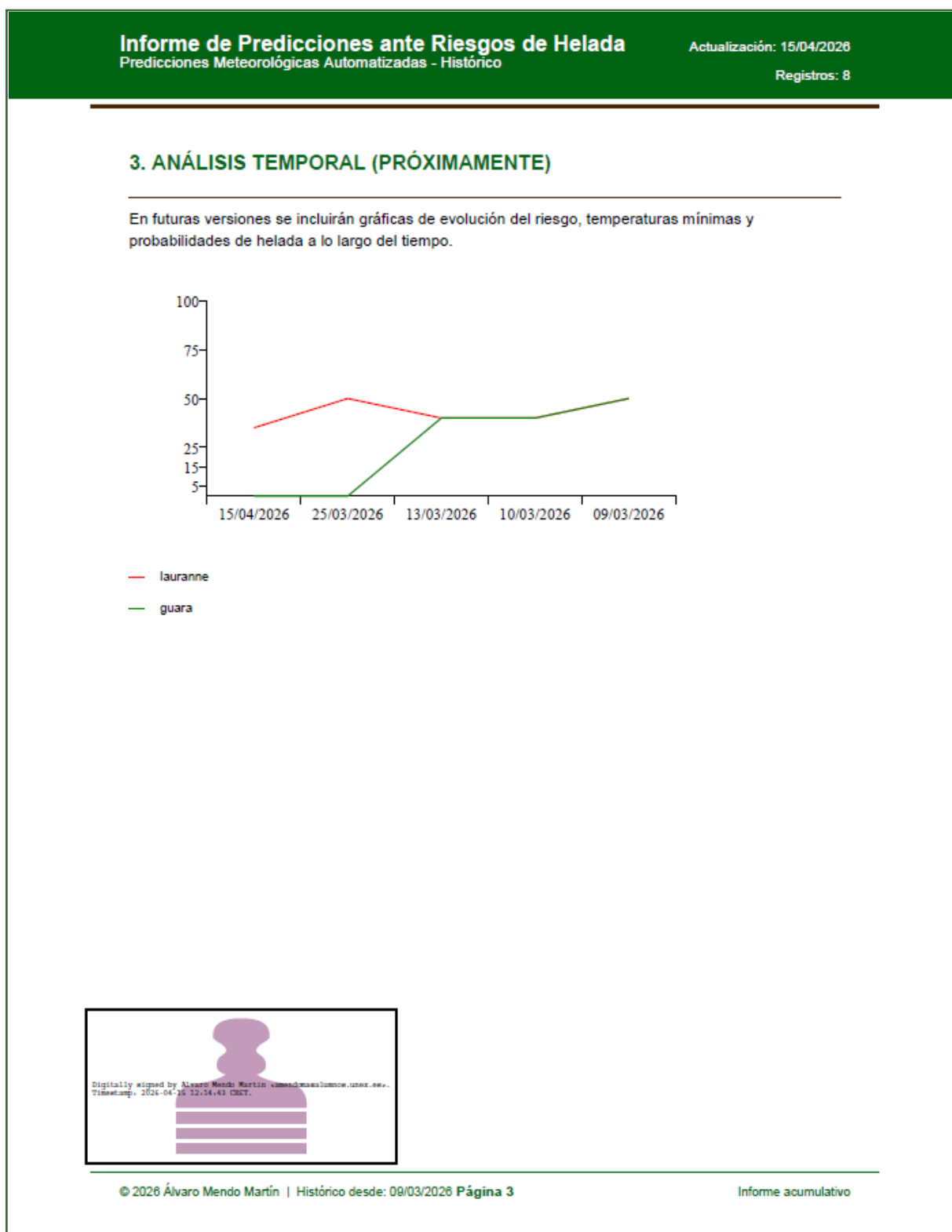


Figura 6.17: Predicción Heladas Gráfico
Fuente: Elaboración propia

Predicción observadas provincial — POST /heladas/observadas/provincial

La petición se realizó sobre el siguiente *endpoint*: /heladas/observadas/provincial con los siguientes valores en el cuerpo de la petición (véase Listado 6.17).

```
1 {
2   "codigo_provincia" : "CC",
3   "estacion" : null,
4   "evaluacion" : true,
5   "variedades" : [
6     "Lauranne",
7     "Guara"
8   ]
9 }
```

Listado 6.17: Cuerpo de la petición /heladas/observadas/provincial

Respuesta de la API y alertas generadas El sistema devolvió una respuesta HTTP 200 con un cuerpo JSON estructurado. Un fragmento relevante de la respuesta, que muestra el nivel de riesgo general y las alertas asociadas, se presenta a continuación:

```
1 "tipo_prediccion": "hoy",
2   "evaluaciones_variedades": {
3     "total_variedades_evaluados": 2,
4     "variedades_en_riesgo_critico": 2,
5     "variedades_en_riesgo_alto": 0,
6     "variedades_en_riesgo_moderado": 0,
7     "variedades_en_riesgo_debil": 0,
8     "variedades_sin_riesgo": 0,
9     "evaluaciones": [
10       {
11         "variedad": "Lauranne",
12         "nombre_cientifico": null,
13         "etapa_fenologica": "brotancia",
14         "temperatura_evaluada": -4.775,
15         "porcentaje_riesgo": 89.99999990052763,
16         "nivel_riesgo": "critico",
17         "umbrales": {
18           "critico": -3.0,
```

```
19         "alto": 0.0,
20         "moderado": 5.0,
21         "bajo": 7.0
22     },
23     "alertas": []
24 },
25 {
26     "variedad": "Guara",
27     "nombre_cientifico": null,
28     "etapa_fenologica": "brotancia",
29     "temperatura_evaluada": -4.775,
30     "porcentaje_riesgo": 89.99999990052763,
31     "nivel_riesgo": "critico",
32     "umbrales": {
33         "critico": -3.0,
34         "alto": 0.0,
35         "moderado": 5.0,
36         "bajo": 7.0
37     },
38     "alertas": []
39 }
40 ]
41 },
42 ...
```

Listado 6.18: Fragmento de respuesta JSON del predictor de heladas observadas

Este fragmento mostrado valida la estructura de la respuesta JSON ajustado a los **objetos de transferencia de datos** definidos en el servicio, lo que garantiza que un futuro cliente *frontend* pudiera ajustar y mostrar la información generada sin ambigüedades.

Paralelamente a la respuesta en formato JSON, el servicio activó la generación de un informe en segundo plano mediante la librería *ReportLab*. Su contenido se encuentra representado en 4 secciones características:

- **Contexto de cálculo:** Proporciona información referente al proceso de cálculo utilizado para obtener los resultados de la predicción (véase Figura 6.18).
- **Resultados generales:** Proporciona información sobre resultados generales obtenidos, desde

temperaturas mínimas alcanzadas para el periodo de evaluación, hasta alertas y recomendaciones identificadas (véase Figura 6.19).

- **Heladas registradas:** Proporciona información en formato de tabla y gráfica sobre las fechas dentro del rango calculado en las que se ha detectado un riesgo de helada blancas y negras, pudiendo observar una evolución temporal sobre dichas heladas (véase Figura 6.20) (véase Figura 6.21).
- **Balance resultante:** Proporciona información en formato de tabla y gráfica sobre los riesgos de heladas temporales que se han detectado para las variedades de cultivos analizadas (véase Figura 6.22).

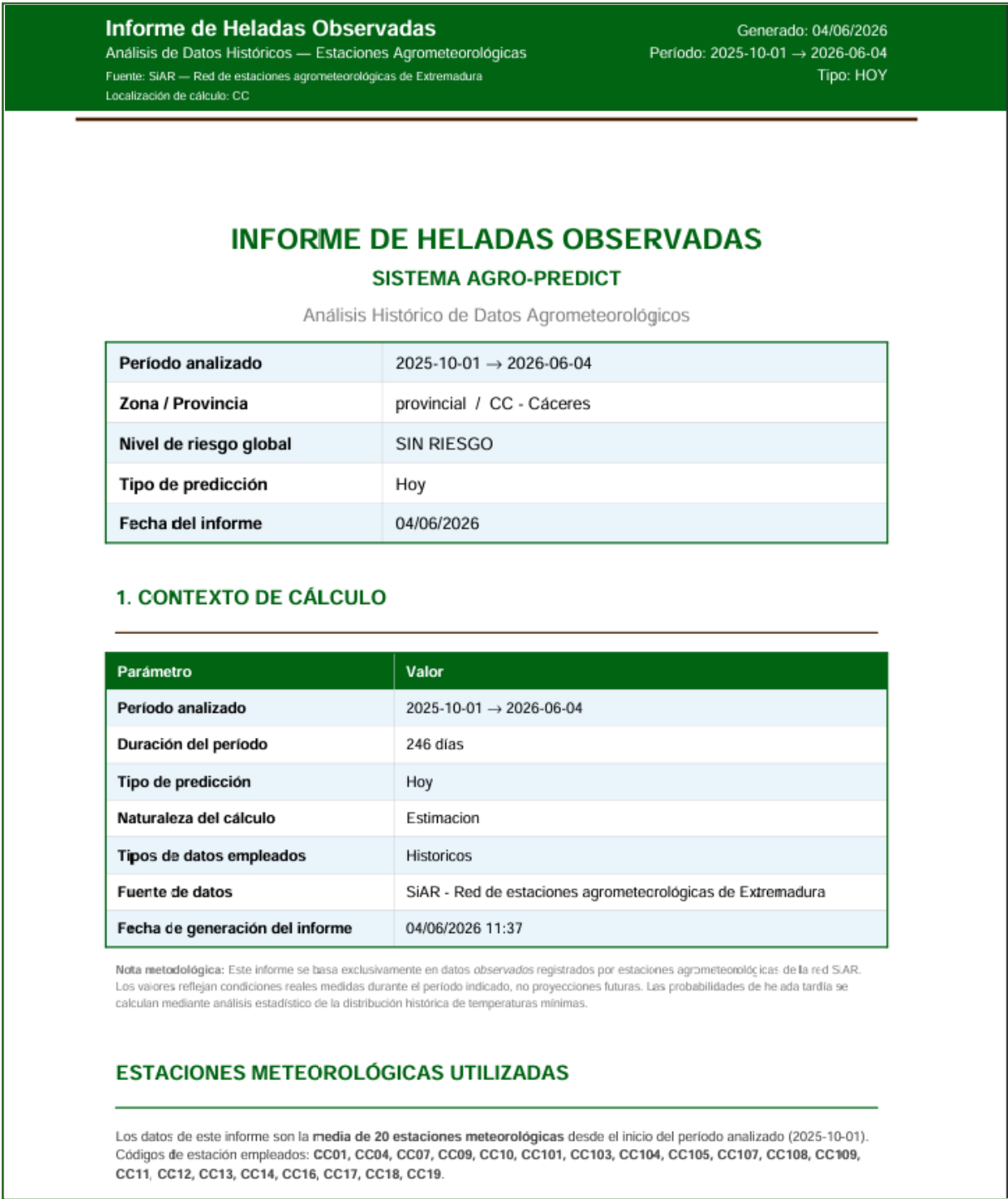


Figura 6.18: Contexto de Cálculo Heladas Observadas
Fuente: Elaboración propia

4. EPISODIOS DE HELADA REGISTRADOS

Heladas Blancas ($T \leq 2^{\circ}\text{C} + \text{HR} \geq 60\%$)

Fecha	Temperatura ($^{\circ}\text{C}$)	Humedad (%)	Estación (Temp.)	Estaciones (HR)
2025-11-21	-0.882	72.40	13	15, 10
2025-11-22	-1.953	76.10	13	15, 10
2025-11-23	1.131	82.10	13	15, 10
2025-11-26	-0.277	80.80	13	15, 10
2025-11-27	-1.484	81.00	13	15, 10
2025-11-28	0.323	86.40	13	15, 10
2025-11-29	-0.549	89.20	13	15, 10
2025-12-01	-1.421	81.20	13	15, 10
2025-12-02	0.660	89.70	13	15, 10
2025-12-24	1.399	88.00	13	15, 10
2025-12-25	-0.283	87.60	13	15, 10
2025-12-26	-0.687	91.50	13	15, 10
2025-12-27	-1.421	83.10	13	15, 10
2025-12-30	0.592	98.00	13	15, 10
2025-12-31	0.390	99.60	13	15, 10
2026-01-01	0.860	95.70	13	15, 10
2026-01-05	-1.751	75.40	13	15, 10
2026-01-06	-4.775	73.30	13	15, 10
2026-01-07	-3.434	83.70	13	15, 10
2026-01-10	0.253	72.70	13	15, 10
2026-01-11	-0.687	88.30	13	15, 10
2026-01-15	1.467	99.20	13	15, 10
2026-01-17	0.323	91.40	13	15, 10
2026-01-18	-1.084	90.30	13	15, 10

Figura 6.20: Registro Tipos Heladas
Fuente: Elaboración propia

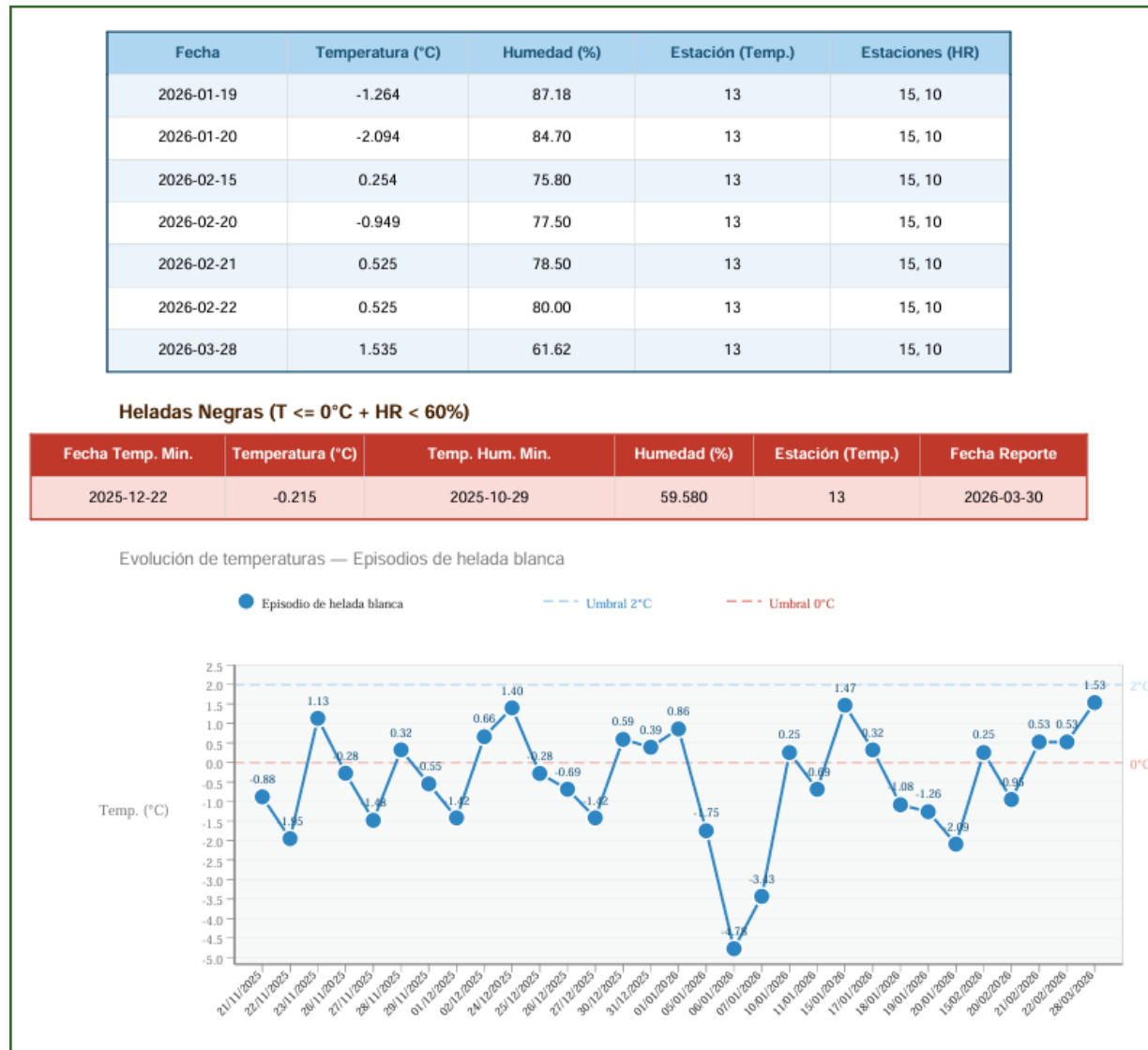


Figura 6.21: Gráfica Tipos Heladas
Fuente: Elaboración propia

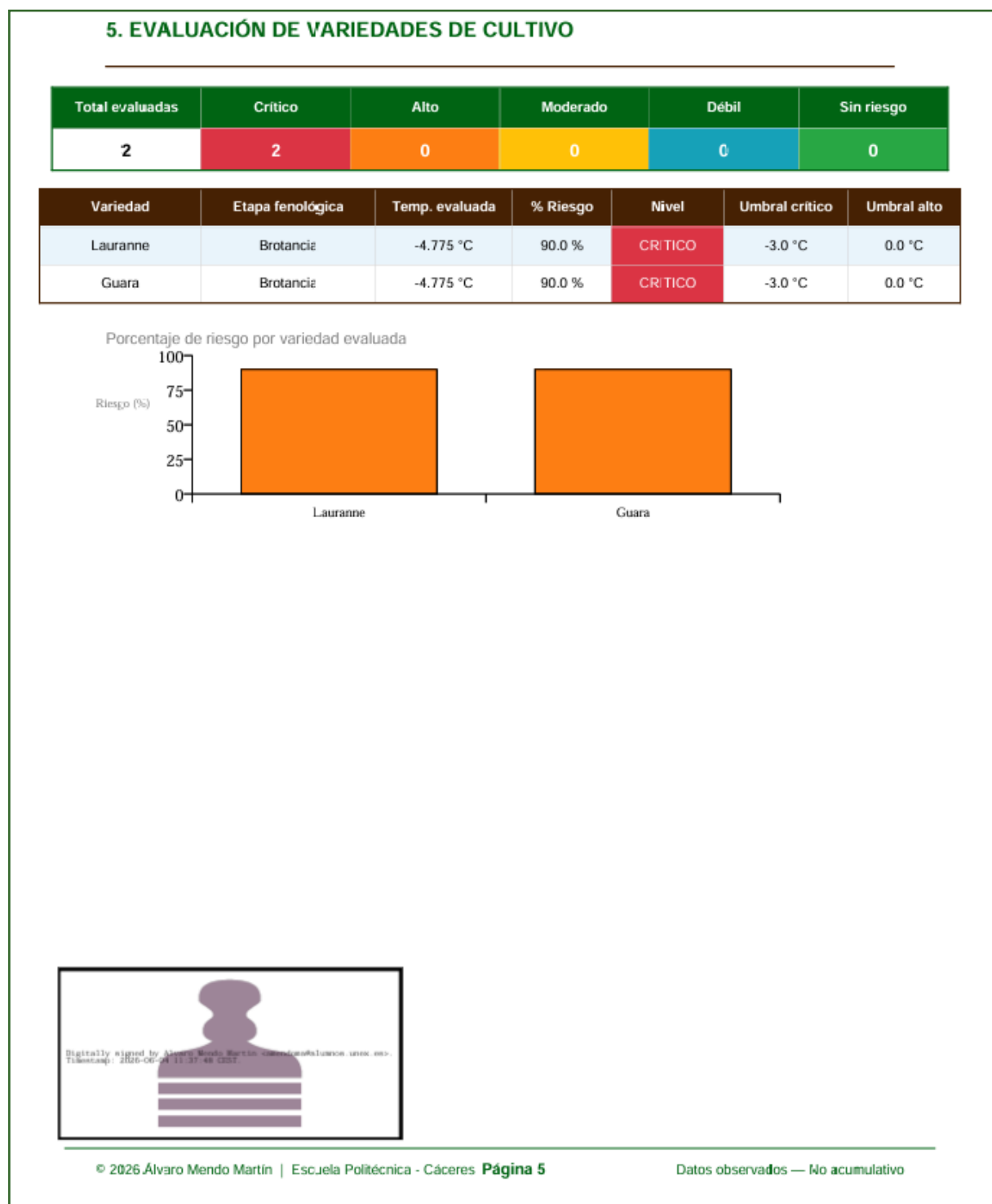


Figura 6.22: Balance resultante Heladas Observadas
Fuente: Elaboración propia

6.6.2. Validación del Servicio Predictor de Plagas

Para probar el Servicio Predictor de Plagas se realizaron peticiones sobre los dos *endpoints* que ofrece el servicio: `/plagas/calculadas`, orientado a predicciones anuales sobre grupos de cultivos específicos, y `/plagas/estimadas`, orientado a predicciones temporales mediante datos meteorológicos y de sensores IoT.

Predicción calculada — POST `/plagas/calculadas` La petición se realizó sobre el *endpoint* `/plagas/calculadas` (véase Sección 6.3.5), enviando como parámetro de consulta el cultivo a analizar. Tal y como se ha explicado previamente su especificación de punto final, pese a ser una petición de tipo POST, no requiere datos en el cuerpo.

Respuesta de la API El sistema devolvió una respuesta HTTP 200 con un cuerpo JSON estructurado. Un fragmento relevante de la respuesta se presenta a continuación (véase Listado 6.19):

```
1  [
2      {
3          "cultivo": {
4              "nombre": "Maiz",
5              "grupo": "cereales"
6          },
7          "plagas": [
8              {
9                  "nombre": "CHINCHES",
10                 "agente_causante": "Aelia spp., Eurygaster spp., ...",
11                 "momento_critico": "Primavera (especialmente hacia final
12                 del enrejado)",
13                 "observaciones": "Con temperaturas sobre 22 grados inician la
14                 migracion hacia las zonas de cereal",
15                 "mas_info": "https://plagas.itacyl.es/chinches-de-los-
16                 cereales",
17                 "tipo": "plaga",
18                 "nivel_riesgo": "MEDIA"
19             },
20             {
21                 "nombre": "CONCHUDOS",
22                 "agente_causante": "Tropinota spp., Oxythyrea spp.",
```

```
23         "momento_critico": "Primavera a primeros de julio",
24         "observaciones": "Periodo critico de primavera a primeros de juli
25         "mas_info": "https://plagas.itacyl.es/conchudos",
26         "tipo": "plaga",
27         "nivel_riesgo": "ALTA"
28     },
29 },
30 ]
```

Listado 6.19: Fragmento de respuesta JSON de `/plagas/calculadas`

Este fragmento valida la estructura de la respuesta JSON ajustada a los **objetos de transferencia de datos** definidos en el servicio, lo que garantiza que un futuro cliente *frontend* pudiera parsear y mostrar la información generada sin ambigüedades.

Paralelamente a la respuesta en formato JSON, el servicio activó la generación de un informe en segundo plano mediante la librería *ReportLab*. El resultado que se muestra en la Figura 6.23 es un fragmento del informe donde se representa una tabla con el análisis anual por cultivo. La inclusión de una firma digital configurada con la librería *pyHanko* certifica la trazabilidad del documento generado de forma automática.



Figura 6.23: Fragmento tabla predicción plagas calculadas
Fuente: Elaboración propia

Predicción estimada — POST /plagas/estimadas La petición se realizó sobre el *endpoint* /plagas/estimadas (véase Sección 6.3.5), especificando el cultivo a analizar, el rango temporal mediante *fecha_inicio* y *fecha_fin*, los identificadores de sensores meteorológicos y, opcionalmente, el identificador de la parcela asociada (véase Listado 6.20).

```
1 {
2     "cultivo": "Tomate",
3     "fecha_inicio": "2025-06-01",
4     "fecha_fin": "2025-09-27",
5     "id_plaga" : "PLAGA-TOMATE-02",
6     "codigo_estacion" : "BA205",
7     "datos_sensores": [
8         {
9             "sensor" : "a840417271882c39",
10            "nombre_dt_agro" : "temp_SOIL",
11            "nombre_predictor_plaga" : "temperatura_suelo"
12        }
13    ]
14 }
```

Listado 6.20: Cuerpo de la petición /plagas/estimadas

Respuesta de la API El sistema devolvió una respuesta HTTP 200 con un cuerpo JSON estructurado. Un fragmento relevante de la respuesta se presenta a continuación:

```
1 {
2     "cultivo": "Tomate",
3     "fecha_inicio": "2025-06-01",
4     "fecha_final": "2025-09-27",
5     "plagas_evaluadas": [
6         {
7             "plaga_id": "PLAGA-TOMATE-02",
8             "nombre": "Meloidogyne spp.",
9             "tipo": "nematodo",
10            "ventana_temporal": [
11                {
12                    "modo": "acumulacion_gdd",
13                    "temperatura_base": 10,
```

```
14         "gdd_objetivo": 600,
15         "dias_ventana": 28,
16         "nivel_si_cumple": "CRITICA"
17     }
18 ],
19     "condiciones_evaluables": [
20     {
21         "tipo": "temperatura_suelo",
22         "valor": 15,
23         "operador": ">="
24     }
25 ],
26     "datos_probabilidad": [
27     {
28         "fecha": "2025-06-01",
29         "nivel_riesgo": "sin_riesgo",
30         "mensaje": {
31             "gdd_acumulado": 266.89,
32             "gdd_objetivo": 600
33         },
34         "condiciones_cumplidas": [
35         {
36             "variable": "temperatura_suelo",
37             "valor_real": 24.34,
38             "operador": ">=",
39             "umbral": 15,
40             "fuente": "sensor"
41         }
42         ],
43         "condiciones_pendientes": [
44         {
45             "gdd_acumulado": 266.89,
46             "gdd_objetivo": 600
47         }
48         ]
49     },
50     ...
51 ],
52 }
```

```
53     ],  
54 }
```

Listado 6.21: Fragmento de respuesta JSON de `/plagas/estimadas`

Este fragmento valida la estructura de la respuesta JSON ajustada a los **objetos de transferencia de datos** definidos en el servicio, lo que garantiza que un futuro cliente *frontend* pudiera parsear y mostrar la información generada sin ambigüedades.

Paralelamente a la respuesta en formato JSON, el servicio activó la generación de un informe en segundo plano mediante la librería *ReportLab*. El resultado que se muestra en la Figura 6.24, Figura 6.25, Figura 6.26 y Figura 6.27 son fragmentos del informe resultante donde se representa una tabla con el análisis por rango temporal y sensores. La inclusión de una firma digital configurada con la librería *pyHanko* certifica la trazabilidad del documento generado de forma automática.



Figura 6.24: Fragmento predicción plagas estimadas 1-4
 Fuente: Elaboración propia



Figura 6.25: Fragmento predicción plagas estimadas 2-4
Fuente: Elaboración propia

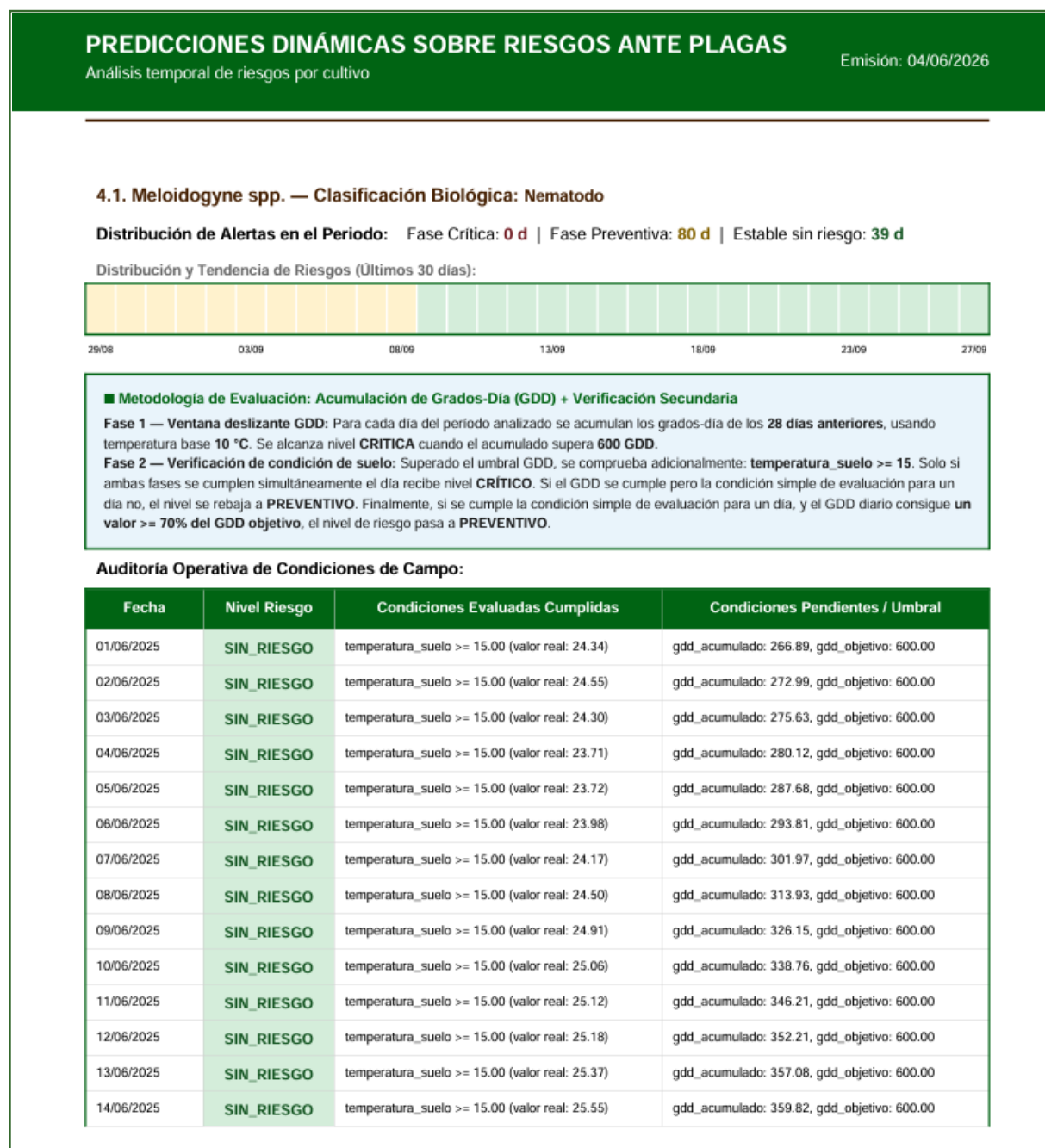


Figura 6.26: Fragmento predicción plagas estimadas 2-4
Fuente: Elaboración propia

PREDICCIONES DINÁMICAS SOBRE RIESGOS ANTE PLAGAS			
Análisis temporal de riesgos por cultivo			Emisión: 04/06/2026
Fecha	Nivel Riesgo	Condiciones Evaluadas Cumplidas	Condiciones Pendientes / Umbral
27/09/2025	SIN_RIESGO	temperatura_suelo >= 15.00 (valor real: 23.58)	gdd_acumulado: 341.08, gdd_objetivo: 600.00
Cláusula de Exención y Nota Técnica: Este documento automatizado compila estimaciones matemáticas bio-climáticas asociadas a los marcos de trabajo del DSL de Agro-Predict. Las alertas CRÍTICAS determinan exclusivamente que las variables de campo coinciden con los rangos óptimos de desarrollo fisiológico del patógeno/insecto (p.ej. acumulación acumulada de GDD). No sustituye la diagnosis visual de un técnico agrónomo calificado.			

Figura 6.27: Fragmento predicción plagas estimadas 4-4

Fuente: Elaboración propia

6.6.3. Conclusiones de la validación

Los resultados obtenidos en las pruebas de ejecución demuestran que el sistema cumple con los objetivos de funcionalidad propuestos:

- **Integración efectiva:** Se ha demostrado la capacidad del sistema para orquestar la comunicación entre microservicios desarrollados en diferentes tecnologías (*Python/Flask*, *Java/Spring Boot*, *Elixir*) y con diferentes propósitos.
- **Generación de datos que aportan valor:** El sistema no se limita a exponer datos crudos, sino que los transforma en **alertas**, **recomendaciones** y **visualizaciones** que facilitan la toma de decisiones en el ámbito agrícola.
- **Arquitectura estable:** La implementación de mecanismos como el *Circuit Breaker* y el manejo centralizado de errores demostró su utilidad durante las pruebas, degradando el servicio de forma controlada cuando el Servicio de Datos simulaba una latencia alta.

En definitiva, el comportamiento general del sistema puede generar soluciones predictivas sencillas de entender por los usuarios finales, pudiendo anticiparse a riesgos climáticos en función de las alertas generadas y recomendaciones asociadas. Además de contar internamente con un mecanismo complejo de servicios distribuidos que se adaptan fácilmente a cambios futuros, pudiendo ser escalado y adaptado a cualquier entorno de trabajo que requiera su uso.

6.7. Problemas encontrados y soluciones adoptadas

Esta sección está dedicada a la explicación de los diferentes problemas que se han ido produciendo durante todo el ciclo de vida del proyecto y las soluciones aproximadas que se han aportado para solventarlos.

6.7.1. Dependencia de datos

Uno de los principales inconvenientes que integra la infraestructura desarrollada reside en la dependencia sobre el correcto funcionamiento de las *APIs* públicas que consultan los servicios externos para la recopilación de datos meteorológicos. A lo largo de todo el proceso de desarrollo, estas *APIs*, como la de SiAR, AEMET o ITACyL, han estado en constantes cambios de versión y disponibilidad, lo que provocaban fallos internos repentinos en la infraestructura, sin conocer el origen del problema, debido a que no se proporcionaban notificaciones informativas sobre cambios en sus versiones o caídas temporales.

A fecha de hoy, gracias al servicio de ingesta interno en el servicio de datos, aunque se produzca un fallo repentino de disponibilidad sobre la *API* de SiAR para la recopilación de datos meteorológicos históricos, existe en la base de datos suficiente información histórica almacenada como para seguir aportando una predicción de **heladas observadas** precisa para el día de hoy. Sin embargo, la predicción de heladas futuras se rige por los datos actuales y futuros que proporciona AEMET a día de hoy, por lo que si no dispone de estos datos, la infraestructura genera un error interno, imposibilitando devolver una respuesta predictiva.

Ante esta limitación en la predicción de riesgos de heladas, no se ha propuesto una solución definitiva, ya que sería necesario disponer de fuentes meteorológicas alternativas o datos futuros de estaciones accesibles que permitan garantizar la continuidad del servicio y salvaguardar la infraestructura en caso de que las *APIs* de los servicios externos dejen de suministrar datos en un momento determinado.

6.7.2. Ingesta de datos en producción

Otro problema derivado del despliegue de la infraestructura en la máquina virtual *cloud* se detectó al poner en funcionamiento el conjunto completo de servicios. Tras su despliegue, y al ejecutar pruebas de integración y carga, se observó que ninguna de las funcionalidades del servicio de datos ni de los servicios predictores respondía correctamente. Pese a contar con un mecanismo automatizado de ingesta inicial de datos al instanciar el servicio de datos en cualquier entorno, se identificó que existían ciertos datos críticos cuya presencia en la base de datos era imprescindible y que no estaban siendo contemplados por

dicho proceso de ingesta. En el entorno de desarrollo este problema no se manifestó, ya que los datos se fueron incorporando de forma progresiva conforme avanzaba el desarrollo.

Como solución, se optó por automatizar completamente la ingesta de todos los datos iniciales necesarios para garantizar el correcto funcionamiento de los servicios. Para ello, se generaron distintos ficheros JSON que recopilan la información esencial, los cuales son procesados por el servicio de ingesta durante la inicialización del sistema (véase el Anexo C para conocer mejor este punto).

6.7.3. Alcance de la fase de desarrollo

A medida que avanzaba la fase de desarrollo del proyecto, fueron surgiendo nuevos requisitos y propuestas de mejora orientadas a incrementar la precisión, fiabilidad y escalabilidad de los servicios implementados.

La incorporación progresiva de nuevas funcionalidades y soluciones técnicas más complejas provocó inicialmente ciertos problemas relacionados con la integración entre componentes, el mantenimiento del código y la estabilidad de algunos servicios.

Para afrontar estas dificultades, se adoptó un enfoque de desarrollo basado en la modularización de componentes, la separación de responsabilidades y la reutilización de código. Esta metodología permitió reducir el acoplamiento entre servicios, facilitar el mantenimiento de la infraestructura y simplificar la incorporación de nuevas funcionalidades durante la evolución del proyecto.

6.7.4. Tratamiento de tareas asíncronas en producción

Un problema bastante pesado de solventar en la fase de despliegue del proyecto fue la correcta comunicación entre el componente *Celery Worker*, el servicio de datos y el *broker RabbitMQ*.

En entornos de desarrollo, la configuración de la herramienta *Celery* se utiliza un conjunto de hilos (*threads*) unitario, es decir, el hilo principal de ejecución, compartiendo recursos entre el servicio de datos desplegado en una terminal y el *worker* desplegado en otra, indicando en la propia configuración de *Celery* dentro del proyecto su campo `worker_pool` a `solo`, pero si se quiere aplicar un rendimiento óptimo de *Celery Worker* con *RabbitMQ*, no es conveniente mantener ese valor.

A diferencia del entorno de desarrollo, para cumplir las necesidades del entorno de producción, *Celery* tiene que mantener un comportamiento más profesional y completo, por lo que es necesario configurar en su campo `worker_pool` el valor `prefork`. De esta manera, se permite trabajar con multiprocesos en tiempo real, mantener un control de colas definidas en el *broker* y mantener un registro en el *broker* de dos

tipos nuevos de cola: *reply_queues*, encargada de enviar respuestas por el *broker* dado los resultados que obtiene cada proceso *worker* y *broadcast_queues* que está más dedicada a el agrupamiento de todas las respuestas que se obtienen sobre cada proceso *worker* y enviarla sobre una sola cola *RabbitMQ*, consiguiendo eficiencia en el lado del cliente/servidor, pero un ancho de banda elevado. Estos dos tipos de cola, *RabbitMQ* las denomina **colas legacy** y por defecto su configuración base no las soporta, arrojando los siguientes errores por consola:

Error

```

1  amqp.exceptions.InternalError:
2  Queue.declare: (541) INTERNAL_ERROR -
3  Feature `transient_nonexcl_queues` is deprecated
4
5  Frequent restarts detected: RestartFreqExceeded

```

Al detectar este error, se solventó siguiendo los siguientes pasos de configuración interna sobre el entorno de producción para el contenedor que aloja el *broker RabbitMQ*:

- **Permitir temporalmente la característica deprecated en RabbitMQ:** Sobre el directorio donde se encuentra el fichero `compose.yml` para desplegar los contenedores, es necesario mantener registrado la siguiente ruta relativa que contiene un fichero con extensión `.conf`: `rabbitmq/conf.d/10-compact.conf`. Su contenido debe ser el siguiente: `deprecated_features.permit.transient_nonexcl_queues = true`.
- **Montar correctamente la configuración en Docker:** Con el motivo de no tener que estar configurando el paso 1 cada vez que se instancia el contenedor del *broker RabbitMQ*, es necesario configurar su apartado de servicio dentro del fichero `compose.yml` con un volumen apuntando al fichero configurado en el paso anterior para tomarlo como referencia cada vez que se levante el servicio (véase Listado 6.22).

```

1  rabbitmq:
2      image: rabbitmq:3-management
3      volumes:
4          - ./rabbitmq/conf.d:/etc/rabbitmq/conf.d

```

Listado 6.22: Configuración volumen `compose.yml` para `rabbitmq`

- **Reinicio de contenedores dependientes de *RabbitMQ*:** Tras completar la configuración del paso 1 y 2, es necesario reiniciar los componentes dependientes de *RabbitMQ* debido a las nuevas características aplicadas:

```
docker compose down
docker compose up -d
```

- **Confirmación de características aplicadas:** Como último paso, es recomendable acceder internamente al entorno de *rabbitmq*, comprobando que se ha aplicado correctamente la configuración:

Entrada al entorno de *rabbitmq*: `docker exec -it rabbitmq rabbitmqctl environment`
 Debe aparecer una línea con el contenido parecido a:
`deprecated_features.permit.transient_nonexcl_queues = true`

Adicionalmente se aporta la nueva configuración en producción de la herramienta *Celery* dentro del código fuente del servicio sobre el que se quiera aplicar (en este caso sobre el servicio de datos) (véase Listado 6.23)

```
1 from kombu import Queue # Se instala cuando te instalas la libreria Celery
2
3 CELERY = {
4     "broker_url": os.getenv("RABBITMQ_CONNECTION"),
5     "result_backend": os.getenv("REDIS_URL"),
6
7     "worker_pool": "prefork",
8
9     # Evitar control-plane problematico
10    "worker_disable_mingle": True,
11    "worker_enable_remote_control": False,
12
13    # Evitar colas dinamicas
14    "task_create_missing_queues": False,
15
16    # Cola explicita
17    "task_default_queue": "default",
18    "task_queues": (
19        Queue("default", durable=True),
20    ),
21 }
```

Listado 6.23: Configuración *celery* para entorno producción

6.7.5. Obtención de datos climáticos para predicciones estimadas de plagas

Durante la puesta en marcha del **modo estimado** sobre las predicciones de plagas asociadas a un cultivo en el entorno de producción previamente tratado, surgió un problema importante a solucionar con el objetivo de conseguir un resultado predictivo más aproximado, flexible y controlado.

Inicialmente, durante la fase de implementación del modo estimado en el servicio predictor de plagas, fue necesario obtener datos climáticos tanto de sensores IoT proporcionados por DTAgro como de datos meteorológicos históricos, utilizados como apoyo en la generación de predicciones sobre riesgos de plagas para un cultivo indicado en la petición.

Debido a que existen numerosos sensores dentro del ecosistema DTAgro y cada uno de ellos se encarga de leer y proporcionar datos climáticos específicos e independientes entre sí (sensor 1: temperatura ambiental, sensor 2: temperatura del suelo, etc.), se decidió como primera aproximación definir un campo en el cuerpo de la petición de predicción denominado **datos_sensores**. Este campo permitía que el usuario o cliente externo introdujese los identificadores públicos de los sensores registrados en DTAgro cuyos valores fuesen requeridos para el análisis interno de la plaga seleccionada.

De esta manera, el servicio predictor se encargaría de leer los identificadores de sensores proporcionados y haría uso del servicio de datos para consultar la información temporal asociada a cada uno de ellos, pudiendo utilizar posteriormente cada dato recopilado durante el procesamiento algorítmico del nivel de riesgo.

Una vez desplegado este comportamiento en entornos de producción y tras ser utilizado por DTAgro, se identificaron nuevas necesidades funcionales a incorporar sobre el comportamiento inicialmente definido:

- Una plaga concreta puede requerir datos procedentes de sensores diferentes.
- Capacidad de filtrar los datos proporcionados por DTAgro en función de las necesidades específicas del predictor.

Durante la obtención de datos desde el servicio de datos, encargado de comunicarse directamente con DTAgro, se observó que cada sensor devolvía información adicional no necesaria para el procesamiento interno del predictor. Como consecuencia, era necesario aplicar un proceso previo de limpieza y selección de datos antes de devolver la información al servicio predictor.

Para solucionar este problema, el campo inicialmente definido, **datos_sensores**, fue ampliado a una lista de objetos, donde cada objeto mantiene la siguiente estructura:

- Una plaga a analizar puede que requiera datos de sensores diferentes.
- Capacidad de filtrar datos proporcionados por DTAgro en función de las necesidades del predictor.

A la hora de obtener datos de sensores desde el servicio de datos, encargado de comunicarse con DTAgro, cada sensor devuelve información adicional a la información objetivo que utiliza el servicio predictor por lo que era necesario aplicar una limpieza de datos antes de devolverse por el servicio de datos hacia el servicio predictor. Para ello, se amplió el campo definido inicialmente, `datos_sensores`, como una lista de objetos, donde cada objeto contiene la siguiente estructura (véase Listado 6.24).

```

1 {
2     "sensor" : "string",
3     "nombre_dt_agro" : "string",
4     "nombre_predictor_plaga" : "string"
5 },
```

Listado 6.24: Configuración actualizada de `datos_sensores`

Esta nueva estructura se compone de un campo `sensor`, que mantiene el identificador del sensor sobre el que se realizará la lectura de datos; un campo `nombre_dt_agro`, encargado de almacenar el nombre específico del atributo que debe recuperarse desde DTAgro; y finalmente un campo `nombre_predictor_plaga`, utilizado como mecanismo de traducción interno entre la nomenclatura utilizada por DTAgro y la comprendida por el servicio predictor.

Gracias a esta nueva aproximación, el servicio de datos pasó a enviar únicamente la información estrictamente necesaria para el análisis predictivo, evitando transferencias de datos innecesarias y permitiendo adaptar dinámicamente los recursos climáticos requeridos por cada plaga analizada. De esta manera, se consiguió resolver el problema planteado manteniendo un modelo flexible y fácilmente extensible para futuras necesidades del sistema.

Capítulo 7: Conclusiones y trabajos futuros

Una vez completadas todas las fases del ciclo de vida del proyecto, puede afirmarse que se han alcanzado los objetivos planteados en la fase de inicio. La infraestructura desarrollada cubre de forma satisfactoria las necesidades identificadas: la gestión y monitorización de recursos meteorológicos y agrarios, la detección temprana de riesgos procedentes de heladas, plagas y enfermedades, y la generación automatizada de informes certificados que facilitan la toma de decisiones en explotaciones agrícolas. Todo ello orientado a minimizar el impacto del cambio climático que genera anualmente mucho daño y pérdidas en este sector.

Desde el punto de vista técnico, se ha construido una infraestructura modular, contenerizada y desacoplada, lo que facilita tanto su mantenimiento como su adaptación a entornos *software* empresariales reales, como es el caso de **DTAgro**. La adopción del *Unified Process* como metodología de desarrollo ha permitido iterar de forma controlada sobre los requisitos, reduciendo la incertidumbre inherente a un proyecto de esta naturaleza.

Finalmente, los resultados obtenidos sientan las bases de una herramienta con potencial de crecimiento, tanto en la incorporación de nuevos modelos predictivos como en la integración con futuras capas de presentación. Se espera que este trabajo aporte valor no solo a agricultores y técnicos de campo, sino también a empresas del sector agroindustrial interesadas en la digitalización y en la precisión de sus resultados.

7.1. Declaración de uso de Inteligencia Artificial

Durante el desarrollo de este proyecto se ha hecho uso de herramientas basadas en inteligencia artificial como apoyo a distintas fases del ciclo de vida del *software*.

Herramientas utilizadas A continuación se listan las diferentes herramientas de inteligencia artificial utilizadas, junto con su contexto de uso:

- **ChatGPT** [91]: Su principal uso se ha derivado en la ayuda con la documentación de esta memoria. Uno de los objetivos que se quería conseguir en la documentación del proyecto era dotar al estilo

de escritura un aspecto técnico y profesional. Para conseguirlo, se mantuvo una metodología de trabajo donde se priorizaba el contenido escrito manualmente por el autor, aplicando retoques, por el agente, sobre la redacción de secciones para mantener las explicaciones claras y concisas.

Prompt genérico utilizado

Hola, estoy realizando la documentación sobre mi Trabajo de Fin de Grado para Ingeniería Informática de Software. Me podrías ayudar con la redacción del mismo, aportando una mayor comprensión y claridad del contenido que redacto? IMPORTANTE: sin perder mi estilo de redacción. La documentación la estoy desarrollando en L^AT_EX.

A continuación te adjunto tu primer bloque de trabajo:

«TEXT»

Asimismo, se mantuvieron ciertas conversaciones para organizar el contenido del Capítulo 6, dado que iba a recabar información importante asociada a explicaciones y detalles en la fase de desarrollo. El agente propuso la estructuración (secciones, subsecciones) de este capítulo tal y como se presenta, ajustando ciertas secciones de manera personal.

Prompt genérico utilizado

Hola, tengo que desarrollar un capítulo para la documentación de mi Trabajo de Fin de Grado en Ingeniería Informática de Software. Este capítulo contendrá la información más importante del ciclo de vida de mi proyecto: Implementación y Desarrollo. La arquitectura seguida para esta iteración del ciclo de vida está orientada al uso de microservicios, para conocer mejor los componentes de la arquitectura, visita la siguiente organización pública en GitHub: <https://github.com/agro-predict-tfg-2026>. Todos los repositorios que contiene son públicos por lo que puedes acceder. La estructura de este capítulo quiero que tenga coherencia y aporte seriedad técnica.

Finalmente, cuando se disponía de la versión premium desde el modo gratuito, se mantuvieron debates sobre la implementación de los módulos más complejos en la infraestructura desarrollada, los cultivos y plagas. Esto garantizó afrontar y entender con mayor claridad, con qué enfoque debía afrontar el desarrollo de sus características.

Prompt genérico utilizado

Contexto: Estoy desarrollando el microservicio de datos para mi TFG, centrado en la predicción de riesgos agrícolas. El sistema debe manejar información sobre cultivos y plagas de forma segura, fiable y mantenible.

Objetivo principal: Necesito diseñar un servicio predictor de riesgos de heladas y plagas basado en reglas de estimación (sin Machine Learning), que sea lo más preciso posible.

Requisito clave de arquitectura: Quiero implementar un único algoritmo genérico capaz de predecir riesgos para cualquier combinación de cultivo y plaga que el usuario especifique en la petición (ej. 500 cultivos y 500 plagas), sin tener que crear un algoritmo distinto para cada uno.

Preguntas concretas:

- *¿Cómo puedo estructurar este algoritmo genérico para que utilice dinámicamente la información almacenada de cada cultivo o plaga (umbrales de temperatura, humedad, ciclo de vida, etc.) en el momento de la predicción?*
- *¿Qué patrón de diseño o estrategia de modelado de datos me recomiendas para que el sistema sea extensible y mantenible a largo plazo?*
- *¿Debería actualizar los campos asociados a las entidades en la base de datos tras cada predicción, o es mejor mantener un registro de eventos/histórico separado? ¿Qué implicaciones tiene cada enfoque en rendimiento y trazabilidad?*

Restricciones:

- *Sin uso de Machine Learning.*
- *Alta precisión en las predicciones.*
- *Escalabilidad para manejar cientos de cultivos y plagas con una sola función predictora.*
- *Debate conmigo, quiero entender primero como afrontar el desafío antes de desarrollarlo.*

A continuación te paso el contexto del servicio de datos, los servicios predictores y los campos de las entidades en la base de datos:

«TEXTO»

- **Claude AI** [92]: Su principal uso se derivó a mejorar la implementación de la infraestructura desarrollada. Conociendo las capacidades de este agente, se le proporcionaron bloques de código ya desarrollados, junto con contexto asociado para obtener un código mejorado y mantenible a largo plazo.

Prompt genérico utilizado

Rol: Eres un desarrollador senior de Python utilizando el microframework de Flask. Además, tienes los conocimientos necesarios para aplicar buenas prácticas, eficiencia y reutilización sobre las funcionalidades a desarrollar.

Contexto: Hola, estoy realizando «X TAREA». La visión sobre la que he desarrollado la funcionalidad es la siguiente: «EXPLICACIÓN». A continuación te voy a pasar el código que he desarrollado y me gustaría que lo evaluaras y aplicaras los cambios necesarios para completar la funcionalidad si es que no lo está. También, implementa revisa si es necesario controlar excepciones de límite que no haya tenido en cuenta con mi implementación. Por último, te he adjuntado un fichero .xml que contiene la estructura y los detalles del proyecto en general para que lo tomes como referencia y puedas generar las mejoras de manera más precisa.

Restricciones:

- *Antes de mejorar el código, aplica el rol que te he definido y realiza un planning de lo que vas a hacer.*
- *Una vez has implementado las mejoras en el código, explica paso a paso lo que has hecho y las decisiones tomadas para que pueda aprender para futuras implementaciones.*

Código: «CODIGO»

El fichero .xml que se menciona en el contexto del *prompt* ha sido generado automáticamente con la librería **repomix** [93], es una potente herramienta que se ejecuta en el directorio raíz del

proyecto y empaqueta todo el repositorio dentro de un fichero *AI-friendly* (véase un fragmento del contenido del fichero en Listado 7.1).

```
1      This file is a merged representation of the entire codebase,  
2      combined into a single document by Repomix.  
3  
4      <file_summary>  
5      This section contains a summary of this file.  
6  
7      <purpose>  
8      This file contains a packed representation of the entire  
9      repository's contents.  
10     It is designed to be easily consumable by AI systems for  
11     analysis, code review,  
12     or other automated processes.  
13     </purpose>  
14  
15     <file_format>  
16     The content is organized as follows:  
17     1. This summary section  
18     2. Repository information  
19     3. Directory structure  
20     4. Repository files (if enabled)  
21     5. Multiple file entries, each consisting of:  
22     - File path as an attribute  
23     - Full contents of the file  
24     </file_format>  
25  
26     <usage_guidelines>  
27     - This file should be treated as read-only. Any changes should  
28     be made to the  
29     original repository files, not this packed version.  
30     - When processing this file, use the file path to distinguish  
31     between different files in the repository.  
32     - Be aware that this file may contain sensitive information.  
33     Handle it with  
34     the same level of security as you would the original  
35     repository.  
36     </usage_guidelines>  
37
```

```

38     <notes>
39     - Some files may have been excluded based on .gitignore rules
40     and Repomix's configuration
41     - Binary files are not included in this packed representation.
42     Please refer to the Repository Structure section for a complete
43     list of file paths, including binary files
44     - Files matching patterns in .gitignore are excluded
45     - Files matching default ignore patterns are excluded
46     - Files are sorted by Git change count
47     (files with more changes are at the bottom)
48     </notes>
49
50     </file_summary>
51
52     <directory_structure>
53     .dockerignore
54     .github/workflows/pipeline.yml
55     .gitignore
56     app/__init__.py
57     app/auth/__init__.py
58     app/auth/keycloak_service.py
59     ....
60

```

Listado 7.1: Fragmento Contexto Repositorio XML

Por otro lado, en el proceso de documentar la memoria del proyecto, ha ayudado a proporcionar un estilo visual técnico y profesional sobre listados y tablas.

Prompt genérico utilizado

Hola, estoy realizando la documentación sobre mi TFG para Ingeniería Informática de Software y me gustaría que aplicaras un estilo bonito, técnico y resaltante sobre los siguientes fragmentos de código / Tablas desarrolladas. La configuración de estos estilos quiero que sea global y reutilizable para cualquier otro componente del estilo que incorpore en la documentación. La documentación la estoy realizando en $\text{\LaTeX}$ por lo que a continuación te voy a pasar como contexto la configuración de la plantilla base y después los fragmentos sobre los que quiero que apliques el estilo.

Plantilla: «tfeEPCC.cls»

Fragmento a estilar: «TEXTO»

- **DALL-E** [94]: Siendo un sistema de inteligencia artificial desarrollado por la empresa *OpenAI*, permite generar imágenes en base a texto.

Su uso se redujo a generar imágenes que representaran el flujo interno de acciones y comportamientos que implementan los servicios predictores para obtener la propia predicción. Incluyendo en las conversaciones el contexto redactado de cada servicio predictor, consiguió generar resultados muy precisos y entendibles (véase las figuras generadas por este agente: Figura 6.7, Figura 6.8, Figura 6.9, Figura 6.10).

Prompt genérico utilizado

Hola, me gustaría que generaras una representación visual del flujo de acciones que se describen en el siguiente texto adjuntado. Me gustaría que lo representaras utilizando el modelo DALL-E-3 para obtener una mayor precisión.

Texto descriptivo del flujo de acciones a representar: «TEXTO»

- **QODO** [95]: Plataforma basada en inteligencia artificial orientada a asistir a desarrolladores en tareas relacionadas con la generación, evaluación y validación de código fuente [96].

Su utilización dentro del proyecto se ha centrado principalmente en el desarrollo de pruebas automatizadas sobre los microservicios que componen la infraestructura implementada, permitiendo agilizar la creación de pruebas unitarias y de integración.

Adicionalmente, la herramienta se ha integrado como mecanismo de apoyo para la verificación de calidad del código en nuevas modificaciones incorporadas a los repositorios del proyecto. Para ello, se ha configurado mediante una **acción de GitHub** (*GitHub Action*), ejecutándose automáticamente durante los procesos de integración continua (CI) comentados anteriormente (véase Subsección 6.5.4) [97].

Durante dicha ejecución, la herramienta analiza aspectos relacionados con la calidad y consistencia del nuevo código antes de su incorporación al repositorio remoto, permitiendo detectar posibles incidencias o incumplimientos de buenas prácticas de desarrollo.

Debido a que este agente no dispone de un chat web, se ha integrado como extensión dentro del Entorno de Desarrollo Integrado (IDE). Con dicha extensión, el agente puede mantener un contexto de todos los ficheros disponibles en el directorio raíz del microservicio sobre el que se está aplicando y las respuestas generadas son más precisas y encajan con la compatibilidad del código

base.

Prompt genérico utilizado

*Hola, quiero implementar unas pruebas unitarias y de integración sobre este proyecto en el que estoy trabajando. Como puedes observar, en el directorio raíz del proyecto tengo un fichero `pyproject.toml`. En dicho fichero podrás encontrar las restricciones de ejecución que se aplican a las pruebas que desarrolles y la ruta relativa donde se encuentra la carpeta de tests que ejecutará el proyecto. Además, mantengo una restricción de al menos 80 % de cobertura de código para considerar unas pruebas válidas así que tienes que cumplir esta condición si o si. Una vez hayas accedido a la carpeta de tests, encontrarás un fichero llamado `conftest.py`, ahí deberás de introducir todas las **fixtures** de clientes imaginarios requeridos para las pruebas de integración. Por otro lado, para realizar las pruebas de integración utiliza **MagicMock**.*

Supervisión humana Pese a las capacidades de cómputo de las herramientas mencionadas, el flujo de trabajo adoptado durante el desarrollo estuvo orientado al aprendizaje y la mejora continua. El proceso seguido consistió en analizar, diseñar e implementar cada solución de forma autónoma, empleando los asistentes de inteligencia artificial en una fase posterior como herramienta de validación y contraste. Esta aproximación permitió debatir las decisiones tomadas, identificar posibles mejoras respecto a la solución base y consolidar el conocimiento adquirido en cada iteración.

Como consecuencia, el ritmo de desarrollo fue pausado, priorizando la comprensión profunda de cada componente desarrollado frente a la velocidad de desarrollo, debido a que se tratan conceptos y términos del sector agrario nunca antes aprendidos. Se considera que esta forma de trabajo ha contribuido de manera significativa a la formación técnica obtenida a lo largo del proyecto.

7.2. Trabajos futuros

A pesar de los resultados obtenidos, existen diversos frentes abiertos y extensiones que podrían abordarse en iteraciones futuras del proyecto.

Mejora de la resolución temporal en predicciones provinciales

Actualmente, la *API* de **AEMET** utilizada para la obtención de datos climáticos predictivos a nivel provincial presenta una limitación en su contexto temporal: el *endpoint* de predicción futura por provincia únicamente proporciona datos para el día actual, sin ofrecer previsiones para días posteriores.

Esta restricción genera una descompensación en el Servicio Predictor de Heladas, ya que, aunque el sistema permite al cliente solicitar predicciones para el día siguiente, los datos climáticos proporcionados por el servicio externo de AEMET, corresponden en realidad al día actual. Cabe destacar que esta limitación afecta exclusivamente al ámbito provincial, puesto que el *endpoint* equivalente a nivel de comunidad autónoma sí ofrece datos con mayor horizonte temporal.

Como trabajo futuro, se plantea explorar fuentes de datos alternativas o complementarias a **AEMET** que permitan obtener predicciones meteorológicas provinciales con un pronóstico futuro de datos climáticos mínimo de 24 horas, dotando así al servicio de una capacidad predictiva real y coherente con las opciones ofrecidas al cliente.

Mejora en el uso de parcelas para la precisión de cálculo

Actualmente, las parcelas registradas en el sistema sirve como referente de contexto para el Servicio Predictor de Plagas y Enfermedades, pero no se utilizan en el Servicio Predictor de Heladas.

Inicialmente, el Servicio Predictor de Heladas utilizaría únicamente datos meteorológicos proporcionados por los servicios externos implementados, pero al finalizar el proyecto, se observó una nueva alternativa a la solución proporcionada por este servicio.

Como trabajo futuro, se plantea adaptar la lógica predictiva de heladas a un contexto geográfico más específico, como es el de las parcelas. Obteniendo sus valores de sensores registrados sobre ellas y analizando si es suficiente para generar una predicción de heladas. De esta manera, se obtendrían resultados mucho más precisos y específicos para el cliente final.

Seguridad globalizada

Como se ha podido apreciar en la sección 6.1, existe un gestor de credenciales configurado de forma externalizada, delegando en él la validación de autenticación y generación de credenciales.

En un primer lugar, se decidió proteger los puntos finales importantes del servicio de datos, debido a que es este servicio el encargado de interactuar con la base de datos, por lo que mantener mecanismos de seguridad para su acceso solventaba problemas de seguridad. El motivo por el que no se adaptó esta prevención sobre todos los puntos finales del servicio y solo se implementó sobre un punto final, `/climate/historical/provincias`, reside en la complejidad inherente asociada al manejo de credenciales de autenticación por parte del usuario. Al no existir una interfaz gráfica para poder iniciar sesión o registrarse, el usuario o cliente final debía de comunicarse primeramente con el servicio *Keycloak*, obteniendo así la clave de autenticación a insertar en los servicios predictores para completar la predicción. Finalmente, se mantuvo el mecanismo de seguridad sobre el punto final indicado con objeto de cumplir con los requisitos de seguridad.

Como trabajo futuro, se plantea generar una interfaz gráfica que complemente a la infraestructura *back-end* desarrollada, permitiendo iniciar sesión y la posibilidad de registros en base a roles, completando así la integración del mecanismo de seguridad desarrollado sobre todos los puntos finales críticos de los servicios en el *software*.

Anexos

Capítulo A: Especificación Requisitos del Sistema

El presente anexo contiene la especificación detallada sobre los requisitos identificados del sistema. La estructura de las especificaciones sigue el formato definido en la norma *ISO/IEC/IEEE 29148*, principal estándar para guiar a los ingenieros en la documentación detallada y sistematizada.

A.1. Requisitos Funcionales

RF-01 – Obtención de datos meteorológicos

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando el sistema inicie el proceso de predicción.

Acción: El sistema debe obtener datos meteorológicos actualizados desde servicios externos oficiales.

Objeto / Resultado: Los datos se almacenan en la base de datos para su posterior procesamiento.

Restricciones: Los datos deben obtenerse desde la *API RESTful* del servicio externo. Tener en cuenta la indisponibilidad temporal de la API.

Criterio de aceptación: La obtención de datos debe completarse en menos de 30 segundos y sin errores.

Dependencias: Ninguna.

RF-02 – Almacenamiento de datos

Prioridad: Alta

Nivel de abstracción: Medio

Condición: Tras la recopilación de datos meteorológicos.

Acción: El sistema debe almacenar la información recopilada en una base de datos persistente.

Objeto / Resultado: Los datos quedan registrados y disponibles para procesos predictivos posteriores.

Restricciones: Se debe garantizar integridad, consistencia y evitar duplicidad de registros.

Criterio de aceptación: Los datos recuperados tras su almacenamiento coinciden exactamente con los obtenidos desde la fuente externa.

Dependencias: RF-01

RF-03 – Predicción de riesgo de plagas y enfermedades

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando el usuario solicite una predicción de riesgo de plagas y enfermedades sobre un cultivo.

Acción: El sistema debe ejecutar el modelo predictivo de riesgo de plagas específico para abastecer la condición.

Objeto / Resultado: Se genera un nivel de riesgo asociado al cultivo solicitado.

Restricciones: Solo se aplicará a cultivos soportados por disponibilidad de datos.

Criterio de aceptación: Se genera una predicción de plagas y enfermedades afectantes sobre el cultivo solicitado.

Dependencias: RF-01, RF-02

RF-04 – Informes resultantes sobre predicciones

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando se obtengan resultados predictivos.

Acción: Si el usuario lo indica en la predicción, el sistema genera un informe PDF resultante con los datos de la predicción.

Objeto / Resultado: Se genera un informe PDF resultante que recopila los datos de la predicción.

Restricciones: Se deben asociar recursos de cómputo a esta acción solo si el usuario lo indica en la petición.

Criterio de aceptación: El sistema genera un informe PDF con los resultados de la predicción.

Dependencias: RF-03

RF-05 - Predicciones heladas en zonas geográficas

Prioridad: Alta

Nivel de abstracción: Medio

Condición: Cuando el usuario solicite una predicción de heladas sobre una zona geográfica con nivel de granularidad concreto.

Acción: El sistema debe generar una predicción ante riesgo de heladas sobre esa zona geográfica.

Objeto / Resultado: El usuario recibe una predicción de riesgo de heladas.

Restricciones: La zona geográfica indicada por el usuario debe estar registrada en el sistema.

Criterio de aceptación: Se genera una predicción de riesgo de helada sobre la zona geográfica indicada.

Dependencias: RF-01, RF-02

RF-06 - Predicción de helada sobre variedad de cultivo

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando el usuario solicite una predicción de heladas sobre una variedad de cultivo.

Acción: El sistema debe generar una predicción de riesgo de heladas sobre esa variedad de cultivo.

Objeto / Resultado: Predicción de heladas sobre la variedad de cultivo solicitada.

Restricciones: La variedad de cultivo indicada en la predicción, debe estar registrada en el sistema.

Criterio de aceptación: Se genera una predicción de riesgo de heladas sobre la variedad de cultivo indicada.

Dependencias: RF-01, RF-02, RF-05

RF-07 - Flexibilidad en lectura de datos

Prioridad: Baja

Nivel de abstracción: Medio

Condición: Cuando se solicite una predicción.

Acción: El sistema debe identificar cuándo necesita leer datos de sensores IoT y cuándo leer datos meteorológicos de servicios externos.

Objeto / Resultado: Precisión añadida en el resultado de la predicción.

Restricciones: Deben disponerse datos climáticos de sensores y de servicios externos.

Criterio de aceptación: Adaptación de cálculo en función de tipo de dato necesario.

Dependencias: RF-03, RF-04

RF-08 - Predicciones de riesgos de plagas sobre grupos Leguminosos y de Cebada

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando el usuario solicite una predicción de riesgo de plagas sobre cultivos pertenecientes a los grupos de Leguminosos o Cebada.

Acción: El sistema debe generar una predicción de riesgo de plagas basada en los datos disponibles para dichos grupos de cultivos.

Objeto / Resultado: Predicción de riesgo de plagas sobre el grupo de cultivo solicitado.

Restricciones: La predicción quedará limitada a los datos públicos disponibles para los grupos de Leguminosos y Cebada.

Criterio de aceptación: Se genera una predicción de riesgo de plagas para el grupo de cultivo indicado a partir de los datos disponibles.

Dependencias: RF-13

RF-09 - Mecanismo de monitorización frente a alertas de riesgos por plagas, enfermedades o heladas

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando el sistema detecte un riesgo por plagas, enfermedades o heladas sobre un cultivo en una predicción.

Acción: El sistema debe notificar el riesgo detectado.

Objeto / Resultado: Alerta de riesgo generada y registrada en el sistema.

Restricciones: El proceso de alertas requiere que el cultivo afectado esté registrado en el sistema y que existan datos climáticos disponibles para su evaluación.

Criterio de aceptación: El sistema genera y registra una alerta cuando se supera el umbral de riesgo definido para plagas, enfermedades o heladas.

Dependencias: RF-06, RF-08, RF-10, RF-16

RF-10 - Identificación y detección de riesgos por heladas blancas y negras

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando el sistema evalúe el riesgo de heladas sobre un cultivo o zona.

Acción: El sistema debe identificar y clasificar el tipo de helada detectada, diferenciando entre helada blanca y helada negra en función de los criterios definidos.

Objeto / Resultado: Clasificación del tipo de helada identificada asociada a la predicción generada.

Restricciones: La clasificación del tipo de helada se realizará conforme a los criterios definidos por el autor del proyecto.

Criterio de aceptación: El sistema identifica y distingue correctamente entre heladas blancas y negras en las predicciones generadas.

Dependencias: RF-06, RF-12

RF-11 - Informar mediante recomendaciones preventivas frente a riesgos de heladas o plagas detectados

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando el sistema detecte un riesgo de heladas o plagas sobre un cultivo monitorizado.

Acción: El sistema debe incluir en el informe resultante recomendaciones preventivas asociadas al riesgo detectado.

Objeto / Resultado: Conjunto de recomendaciones preventivas incluidas en el informe de predicción.

Restricciones: Las recomendaciones generadas estarán basadas en los criterios definidos por el autor del proyecto y serán específicas al tipo de riesgo detectado.

Criterio de aceptación: El informe generado incluye recomendaciones preventivas coherentes con el tipo y nivel de riesgo detectado.

Dependencias: RF-06, RF-08, RF-09, RF-10

RF-12 - Generar predicciones ante riesgos de heladas para un rango temporal determinado

Prioridad: Alta

Nivel de abstracción: Medio

Condición: Cuando el usuario solicite una predicción de riesgo de heladas especificando un rango temporal (hoy o mañana).

Acción: El sistema debe generar la predicción de riesgo de heladas correspondiente al rango temporal indicado.

Objeto / Resultado: Predicción de riesgo de heladas acotada al rango temporal solicitado.

Restricciones: El rango temporal de predicción queda limitado a los tipos temporales soportados por las fuentes de datos climáticos disponibles (hoy y mañana).

Criterio de aceptación: El sistema genera predicciones de riesgo de heladas diferenciadas para los rangos temporales de hoy y mañana.

Dependencias: RF-05, RF-06

RF-13 - Permitir registrar y gestionar cultivos y sus variedades

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando el usuario necesite registrar o gestionar un cultivo o alguna de sus variedades en el sistema.

Acción: El sistema debe permitir crear, consultar, modificar y eliminar cultivos y sus variedades asociadas.

Objeto / Resultado: Cultivo o variedad registrada y gestionada correctamente en el sistema.

Restricciones: No podrán registrarse variedades sin estar asociadas a un cultivo previamente registrado en el sistema.

Criterio de aceptación: El sistema permite gestionar el ciclo de vida completo de cultivos y variedades, reflejando los cambios de forma persistente.

Dependencias: RF-01, RF-02

RF-14 - Permitir registrar y gestionar provincias, comunidades autónomas, localidades y estaciones

Prioridad: Alta

Nivel de abstracción: Medio

Condición: Cuando el usuario necesite registrar o gestionar una entidad geográfica o una estación meteorológica en el sistema.

Acción: El sistema debe permitir crear, consultar, modificar y eliminar provincias, comunidades

autónomas, localidades y estaciones meteorológicas.

Objeto / Resultado: Entidad geográfica o estación registrada y gestionada correctamente en el sistema.

Restricciones: Las entidades geográficas deberán respetar la jerarquía territorial definida: comunidad autónoma → provincia → localidad.

Criterio de aceptación: El sistema permite gestionar el ciclo de vida completo de las entidades geográficas y estaciones, manteniendo la coherencia jerárquica entre ellas.

Dependencias: RF-01, RF-02

RF-15 - Generar gráficos visuales sobre datos de predicciones obtenidos

Prioridad: Baja

Nivel de abstracción: Medio

Condición: Cuando el sistema genere un informe de predicción de heladas o plagas.

Acción: El sistema debe incluir en el informe resultante representaciones gráficas visuales de los datos de predicción obtenidos.

Objeto / Resultado: Gráficos visuales integrados en el informe de predicción generado.

Restricciones: Los gráficos generados estarán basados exclusivamente en los datos de predicción calculados por el sistema.

Criterio de aceptación: El informe generado incluye al menos un gráfico visual que representa la evolución temporal de los datos de predicción obtenidos.

Dependencias: RF-06, RF-08, RF-22

RF-16 - Registrar alertas en los informes cuando el riesgo de heladas o plagas supere un umbral determinado

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando el nivel de riesgo calculado en una predicción de heladas o plagas supere el umbral definido para la variedad o cultivo evaluado.

Acción: El sistema debe registrar una alerta en el informe resultante indicando el tipo de riesgo superado y el umbral correspondiente.

Objeto / Resultado: Alerta registrada en el informe de predicción, asociada al umbral de riesgo superado.

Restricciones: Los umbrales de alerta estarán definidos en base al comportamiento registrado de la

herramienta *Irristrat* y serán específicos por tipo de riesgo.

Criterio de aceptación: El informe generado incluye las alertas correspondientes cuando el nivel de riesgo calculado supera los umbrales establecidos.

Dependencias: RF-06, RF-08, RF-09, RF-15

RF-17 - Monitorizar las horas-frío acumuladas sobre una variedad de cultivo por zona o parcela

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando el usuario solicite la monitorización de horas-frío acumuladas sobre una variedad de cultivo en una zona o parcela determinada.

Acción: El sistema debe calcular y registrar las horas-frío acumuladas para la variedad de cultivo indicada en la zona o parcela especificada.

Objeto / Resultado: Registro de horas-frío acumuladas asociado a la variedad de cultivo y zona o parcela indicadas.

Restricciones: La variedad de cultivo y la zona o parcela deben estar previamente registradas en el sistema. Los datos climáticos necesarios para el cálculo deben estar disponibles.

Criterio de aceptación: El sistema calcula y registra correctamente las horas-frío acumuladas para la variedad y zona indicadas.

Dependencias: RF-13, RF-14, RF-18, RF-19, RF-21

RF-18 - Establecer umbrales de registro de horas-frío en base a modelos identificados

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando se requiera configurar los umbrales de horas-frío asociados a un modelo de cálculo identificado.

Acción: El sistema debe permitir definir y registrar los umbrales de horas-frío correspondientes a cada modelo de cálculo disponible.

Objeto / Resultado: Umbrales de horas-frío registrados y asociados al modelo de cálculo correspondiente.

Restricciones: Los umbrales definidos deberán estar basados en los modelos de horas-frío identificados

en colaboración con los Ingenieros Agrónomos consultados.

Criterio de aceptación: El sistema registra correctamente los umbrales de horas-frío para cada modelo disponible y los aplica en los cálculos de monitorización.

Dependencias: RF-19

RF-19 - Establecer registros y relaciones sobre modelos de horas-frío que implementa cada variedad de cultivo

Prioridad: Media

Nivel de abstracción: Medio

Condición: Cuando se requiera asociar un modelo de cálculo de horas-frío a una variedad de cultivo registrada en el sistema.

Acción: El sistema debe permitir registrar y gestionar la relación entre cada variedad de cultivo y el modelo de horas-frío que implementa.

Objeto / Resultado: Relación registrada entre la variedad de cultivo y su modelo de horas-frío asociado.

Restricciones: La variedad de cultivo debe estar previamente registrada en el sistema. Cada variedad podrá estar asociada únicamente a un modelo de horas-frío.

Criterio de aceptación: El sistema registra y mantiene correctamente la relación entre variedades de cultivo y sus modelos de horas-frío correspondientes.

Dependencias: RF-13, RF-18

RF-20 - Implementar comunicación entre DTAgro y los servicios implementados

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando la plataforma **DTAgro** necesite interactuar con cualquiera de los servicios desarrollados en la infraestructura.

Acción: El sistema debe exponer los mecanismos de comunicación necesarios para que **DTAgro** pueda consumir los servicios implementados de forma integrada.

Objeto / Resultado: Canal de comunicación operativo entre **DTAgro** y los servicios de la infraestructura desarrollada.

Restricciones: La comunicación deberá realizarse a través de los puntos finales definidos en cada servicio, respetando los contratos de *API* establecidos.

Criterio de aceptación: DTAgro es capaz de realizar peticiones y recibir respuestas correctas de los servicios implementados a través de los *endpoints* definidos.

Dependencias: RF-01, RF-02, RF-03, RF-04

RF-21 - Mantener un registro interno de parcelas, dispositivos y sensores disponibles por DTAgro

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando el sistema necesite acceder a información sobre las parcelas, dispositivos o sensores gestionados por DTAgro.

Acción: El sistema debe mantener un registro interno actualizado de las parcelas, dispositivos y sensores disponibles proporcionados por DTAgro.

Objeto / Resultado: Registro interno de parcelas, dispositivos y sensores sincronizado con la información de DTAgro.

Restricciones: El registro interno deberá mantenerse coherente con los datos gestionados por DTAgro, siendo esta la fuente de verdad para dichas entidades.

Criterio de aceptación: El sistema dispone de un registro interno accesible y actualizado de las parcelas, dispositivos y sensores proporcionados por DTAgro.

Dependencias: RF-14, RF-20

RF-22 - Proporcionar contexto de cálculo a los informes predictivos resultantes

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando el sistema genere un informe predictivo de heladas o plagas.

Acción: El sistema debe incluir en el informe resultante el contexto de cálculo utilizado, detallando los parámetros, datos de entrada y criterios aplicados durante la generación de la predicción.

Objeto / Resultado: Informe predictivo enriquecido con el contexto de cálculo asociado a la predicción generada.

Restricciones: El contexto de cálculo incluido deberá ser suficientemente detallado para garantizar la trazabilidad y reproducibilidad de los resultados obtenidos.

Criterio de aceptación: El informe generado incluye el contexto de cálculo completo, permitiendo

identificar los parámetros y datos utilizados en la predicción.

Dependencias: RF-06, RF-08, RF-15, RF-16

A.2. Requisitos No Funcionales

RNF-01 Escalabilidad

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando aumenta la carga de trabajo o el número de usuarios concurrentes.

Acción: El sistema debe permitir incrementar sus recursos (CPU, memoria, instancias) sin afectar negativamente el rendimiento.

Objeto /Resultado: El sistema mantiene tiempos de respuesta estables bajo crecimiento de demanda.

Restricciones: Debe implementarse mediante arquitectura modular o desplegable en entornos escalables, como por ejemplo contenedores docker.

Criterio de aceptación: El sistema soporta un incremento del 50 % en carga concurrente sin aumentar más de un 20 % el tiempo medio de respuesta.

Dependencias: Despliegue de los componentes de la arquitectura.

RNF-02 Rendimiento bajo demanda

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando existe una carga elevada de tráfico simultáneo via cliente-servidor o entre servicios internos del sistema.

Acción: El sistema debe mantener estabilidad operativa sin perjudicar el rendimiento general. **Objeto / Resultado:** No se producen caídas ni bloques del sistema.

Restricciones: Debe implementarse mecanismos de control de concurrencia y gestión de peticiones entre servicios internos.

Criterio de aceptación: El sistema mantiene tiempos de respuesta por acción de al menos 1 segundo.

Dependencias: RNF-01.

RNF-03 Seguridad

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando se ejecuten operaciones sensibles (acceso a servicios de terceros para obtener datos, modificación de datos para realizar predicciones).

Acción: El sistema debe aplicar mecanismos de prevención contra datos de seguridad, aplicando restricciones de acceso a los datos y controlando tokens de autenticación.

Objeto / Resultado: Protección frente a accesos no autorizados.

Restricciones: Almacenamiento seguro de credenciales.

Criterio de aceptación: No se permite el acceso a funcionalidades restringidas sin autenticación válida.

Dependencias: Ninguna.

RNF-04 Viabilidad

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando el usuario interactúe con el sistema.

Acción: El sistema debe presentar una interfaz intuitiva y comprensible.

Objeto / Resultado: El usuario puede probar diferentes funcionalidades del sistema sin aprendizaje previo.

Restricciones: Diseño coherente y consistente en navegación.

Criterio de aceptación: Un usuario nuevo debe poder realizar una predicción completa sin asistencia externa.

Dependencias: Ninguna.

RNF-05 Fiabilidad

Prioridad: Alta

Nivel de abstracción: Alto

Condición: Cuando el usuario interactúe con el sistema.

Acción: El sistema debe proporcionar datos para el cumplimiento de la acción realizada en tiempos mínimos.

Objeto / Resultado: El usuario puede interactuar con el sistema de una forma fluida y sin interrupciones

de uso.

Restricciones: Tiempos de respuestas entre 1 y 2 segundos como máximo.

Criterio de aceptación: Un usuario puede realizar acciones en el sistema sin necesidad de esperar a la obtención de una respuesta.

Dependencias: Ninguna.

Capítulo B:Manual de instalación

Con el objetivo de que cualquier persona tenga la posibilidad de desplegar y adaptar esta infraestructura desarrollada en entornos empresariales, este anexo sirve para detallar su configuración a seguir para entornos Windows y Linux.

B.1. Pasos previos

Antes de comenzar con los pasos a seguir, se detallan unos requisitos previos a tener en consideración:

1. **Instalación de Docker:** El servicio se va a desplegar en contenedores de Docker, por lo que es necesario tener instalado esta herramienta en el equipo. Se puede comprobar su integración en el equipo utilizando el siguiente comando en la terminal del equipo: `docker -version`.

Si al ejecutarlo no muestra una versión instalada de Docker, significa que no se encuentra instalado en el equipo y se deben seguir las siguientes especificaciones de instalación:

- **Windows:** En la página oficial de Docker [98], se encuentra el instalador de la herramienta. Se debe seleccionar la opción a instalar en base a la arquitectura que se disponga en el equipo y se siguen los pasos por defecto que indique el instalador. Esta instalación incorpora al equipo una interfaz *docker* a modo panel de control, llamada **Docker desktop**, pudiendo controlar los contenedores creados, lanzados o pausados. Asimismo, permite aplicar una gestión de recursos sobre los contenedores ejecutados en base a las capacidades computacionales que tenga el equipo local. Por otro lado, la instalación incorpora la versión más reciente de **docker** y **docker-compose**, otro comando útil para poner en ejecución sobre contenedores, servicios configurados en ficheros especiales, llamados `compose.yml`.

Nota: Si no se conoce la arquitectura del equipo, se puede consultar introduciendo el siguiente comando en una terminal: `systeminfo`, identificado en el atributo **Tipo de arquitectura**.

- **Linux:** En la página oficial de Docker [90], se dispone de una sección dedicada a la instalación de *docker* para equipos Linux. En primer lugar, se debe elegir la distribución disponible en el equipo (Ubuntu, Debian, Fedora, entre otros) y posteriormente, se siguen las pautas indicadas en la propia página web.

A diferencia de Windows, se requiere instalar un *plug-in* especializado que da soporte al comando **docker-compose**.

Nota: Si no se conoce la distribución Linux del equipo, se puede consultar introduciendo el siguiente comando en una terminal: `cat /etc/*release`.

2. **Instalación de Git:** Puesto que para el despliegue de la infraestructura es necesario clonar el repositorio previamente nombrado en el documento, **agro-predict-orchestator**, se precisa tener instalada una versión de git en el equipo, pudiendo así mantener una comunicación entre la plataforma GitHub y el equipo local. A continuación se detalla la instalación para los dos sistemas operativos tratados:
 - **Windows:** A través de su página oficial [99], se selecciona el instalador necesario en función de la arquitectura disponible en el equipo de trabajo.
 - **Linux:** A través de su página oficial [100], se encuentran los comandos requeridos a ejecutar en función de la distribución de Linux que disponga el equipo de trabajo.
3. **Instalación del repositorio en entorno local:** A través del siguiente enlace, se debe descargar, utilizando comandos git en la terminal, el repositorio en el equipo local: `git clone https://github.com/agro-predict-tfg-2026/agro-predict-orchestator.git`. Este repositorio contiene el fichero **compose.yml** encargado de orquestar y configurar los diferentes servicios que contiene la arquitectura a desplegar. Como se ha comentado previamente en la Subsección 6.5.2, este repositorio consta de dos carpetas, **docker** y **no-gpu**, el usuario debe elegir qué **compose.yml** ejecutar en función de las recomendaciones que se indicaron.

B.2. Proceso de ejecución

Tras haber cumplido con los requisitos previos, se indica a continuación una guía de configuración para poder ejecutar todos los servicios en el equipo sin problemas asociados.

B.2.1. Insertar fichero env en el repositorio clonado

Una vez se ha conocido que el repositorio clonado contiene los fichero **compose.yml**, requeridos para la ejecución de la infraestructura, esta configuración depende de variables de entorno que por problemas de seguridad no se encuentran disponibles en el repositorio. Es necesario ponerse en contacto con alvaromendo2004@gmail.com para que provea un fichero **env**, con las variables de entorno cargadas. Este fichero se debe insertar en la carpeta donde se encuentre el fichero **compose.yml** a ejecutar.

Finalmente, se debe ejecutar el comando sobre la terminal en la ruta de carpetas donde se encuentre el fichero **compose.yml** a ejecutar el siguiente comando: `docker compose up -d`.

B.2.2. Entorno IA

En el caso de haber elegido la ejecución de la infraestructura que contiene los componentes necesarios para el funcionamiento de la *IA*, su fichero de configuración **compose.yml**, ya contiene todas las dependencias integradas para ejecutar el **broker**, **ollama** y **mcp**. Además, el servicio de datos tiene una versión configurada que adapta su flujo de comportamiento al aportado por la *IA*.

Como complemento, es necesario configurar manualmente, en el proveedor **ollama**, la instalación de su modelo integrado **qwen 2.5**. En primer lugar, se ejecutan todos los servicios configurados en contenedores docker con el comando en la terminal, **docker compose up**, desde la ruta de archivos **docker**, dentro del repositorio clonado. Tras ejecutar todos los servicios, es necesario acceder internamente al proveedor **ollama**, utilizando el comando en la terminal **docker exec ollama-service**. Una vez se ha accedido al contenedor (se puede comprobar viendo la modificación del preámbulo de la terminal, pasando a ser ahora el símbolo **#**), se procede a la instalación del modelo, ejecutando el siguiente comando en la terminal: **ollama pull qwen2.5:7b**. Finalmente, Tras completar estos pasos, el entorno dependiente de *IA* se encuentra disponible.

B.2.3. Carga inicial de datos

Gracias a la implementación del servicio interno de ingesta en el servicio de datos, al ejecutar por primera vez el servicio de datos, realiza una carga (en segundo plano) inicial, donde recopila y almacena datos esenciales para el correcto funcionamiento de la aplicación sobre la base de datos. Posteriormente, a medida que se vayan realizando consultas sobre los puntos finales definidos en su *API*, ya sea por el cliente final o por los servicios predictores, se irá poblando la base de datos progresivamente con datos meteorológicos y de sensores IoT.

B.3. Contribuciones Futuras

Con el objetivo de facilitar la incorporación de futuros colaboradores al proyecto, esta sección presenta una visión estructurada de los repositorios que conforman la organización **Agro Predict**. Todos ellos se encuentran alojados públicamente en *GitHub* bajo la organización **agro-predict-tfg-2026**¹.

¹<https://github.com/agro-predict-tfg-2026>

Cada repositorio es autónomo e independiente, respondiendo al principio de separación de responsabilidades propio de la arquitectura de microservicios.

Nota para colaboradores: cada repositorio contiene un `README.md` que documenta de forma autónoma los siguientes aspectos: instalación de dependencias, configuración de variables de entorno, ejecución en entorno local, construcción de la imagen Docker y despliegue en producción. Esta memoria no duplica dicha información.

B.3.1. data-service

Repositorio: <https://github.com/agro-predict-tfg-2026/data-service>

Tecnología: Python (Flask, SQLAlchemy, Celery)

Servicio central de la plataforma. Gestiona la persistencia de todos los datos climáticos, de cultivos, sensores y plagas. Expone una API REST documentada mediante Swagger y coordina los procesos de ingesta de datos procedentes de fuentes externas (AEMET, SIAR, ITACyL y sensores IoT propios) a través de tareas asíncronas Celery.

Estructura — data-service

```
data-service/
├── app/
│   ├── auth/
│   ├── clients/
│   ├── cultivos/
│   ├── forecast/
│   ├── historicos/
│   ├── ingesta/
│   ├── metadata/
│   ├── plagas/
│   └── sensores/
```

```
├── external_services/
├── external_communication/
├── globals/
├── decorator/
├── data/
├── tests/
├── config/
├── helpers/
├── migrations/
├── scripts/
├── swagger/
├── dockerfile
├── requirements.txt
└── README.md
```

Descripción de módulos

Directorio / Fichero	Descripción
<code>app/</code>	Núcleo de la aplicación Flask. Organizado en subdominios funcionales (cultivos, forecast, historicos, ingesta, etc.), cada uno con su propia capa de rutas, servicios y DAOs.
<code>app/ingesta/</code>	Orquestación del proceso ETL. Coordina la ingesta de datos desde AEMET, SIAR, ITACyL y sensores mediante hilos independientes.
<code>app/external_communica</code>	Interfaces de mensajería asíncrona mediante RabbitMQ para comunicación entre microservicios.
<code>config/</code>	Clases de configuración por entorno (desarrollo, producción, test).
<code>migrations/</code>	Historial de migraciones de base de datos gestionadas con Alembic.
<code>swagger/</code>	Especificación OpenAPI 3.0 de la API REST del servicio.
<code>scripts/</code>	Scripts de inicialización y carga de datos semilla.

B.3.2. `climate_risks_predictor`

Repositorio: https://github.com/agro-predict-tfg-2026/climate_risks_predictor

Tecnología: Python

Microservicio orientado a la detección y notificación temprana de riesgos climáticos sobre explotaciones agrarias, con especial énfasis en heladas y plagas asociadas a condiciones meteorológicas adversas. Las predicciones se basan en umbrales técnicos contrastados con fuentes oficiales y en el cálculo de horas-frío acumuladas por variedad de cultivo, sin emplear modelos de aprendizaje automático. Los datos de entrada son suministrados por el `data-service`.

Estructura — `climate_risks_predictor`

```
climate_risks_predictor/
├── app/
```

```

├── config/
├── swagger/
├── main.py
├── dockerfile
├── requirements.txt
└── README.md

```

Descripción de módulos

Directorio / Fichero	Descripción
app/	Lógica principal del servicio: cálculo de riesgos, evaluación de umbrales climáticos y generación de alertas y recomendaciones.
config/	Configuración del servicio: parámetros de conexión con data-service y umbrales de riesgo configurables.
swagger/	Especificación OpenAPI de los endpoints expuestos para consulta de riesgos por provincia, localidad o comunidad autónoma.
main.py	Punto de entrada de la aplicación.

B.3.3. mcp-ia-service

Repositorio: <https://github.com/agro-predict-tfg-2026/mcp-ia-service>

Tecnología: Python (FastMCP)

Servicio de integración con inteligencia artificial generativa mediante el protocolo MCP (*Model Context Protocol*). Actúa como servidor FastMCP que consume datos en texto plano del **data-service**, construye el prompt adecuado y lo transmite a un cliente de IA para su análisis y síntesis. Permite delegar en modelos de lenguaje la interpretación de datos climáticos con carácter de pronóstico.

Estructura — mcp-ia-service

```
mcp-ia-service/
├── mcp_ia_server/
├── services/
└── README.md
```

Descripción de módulos

Directorio / Fichero	Descripción
mcp_ia_server/	Implementación del servidor FastMCP: definición de herramientas (<i>tools</i>) MCP y gestión del ciclo de vida del servidor.
services/	Lógica de negocio del servicio: obtención de datos del data-service , construcción del prompt y comunicación con el cliente de IA.

B.3.4. Aemet

Repositorio: <https://github.com/agro-predict-tfg-2026/Aemet>

Tecnología: Elixir (OTP)

Servicio de recopilación periódica de datos meteorológicos a partir de la API oficial de la Agencia Estatal de Meteorología (AEMET). Implementado sobre la plataforma OTP de Erlang/Elixir, emplea una arquitectura basada en procesos supervisados que garantiza la tolerancia a fallos y la ejecución programada de las tareas de ingesta.

Estructura — Aemet

```
Aemet/
└── aemet/
```

```

├── lib/aemet/
│   ├── application.ex
│   ├── supervisor.ex
│   ├── scheduler.ex
│   ├── orchestrator.ex
│   ├── client.ex
│   ├── parser.ex
│   ├── config/
│   └── utils/
└── README.md

```

Descripción de módulos

Módulo	Descripción
<code>application.ex</code>	Punto de entrada OTP de la aplicación. Define el árbol de supervisión inicial.
<code>supervisor.ex</code>	Supervisor principal. Gestiona el ciclo de vida de los procesos hijos con estrategia de reinicio.
<code>scheduler.ex</code>	GenServer encargado de lanzar las tareas de ingesta de forma periódica y configurable.
<code>orchestrator.ex</code>	Coordina el flujo de datos entre el cliente HTTP y el parser.
<code>client.ex</code>	Cliente HTTP para el consumo de la API pública de AEMET.
<code>parser.ex</code>	Transformación y validación de la respuesta JSON de AEMET antes de su envío al <code>data-service</code> .

B.3.5. SiAR

Repositorio: <https://github.com/agro-predict-tfg-2026/SiAR>

Tecnología: Java (Spring Boot, Maven)

Microservicio de ingesta de datos procedentes del Sistema de Información Agroclimática para el Regadío (SiAR), red de estaciones agrometeorológicas gestionada por el Ministerio de Agricultura, Pesca y Alimentación. El servicio actúa como adaptador entre la API pública de SiAR y el `data-service`.

Estructura — SiAR

```
SiAR/
├── src/
│   ├── main/java/
│   └── test/java/
├── pom.xml
├── dockerfile
└── mvnw
```

Descripción de módulos

Directorio / Fichero	Descripción
<code>src/main/java/</code>	Código fuente principal: controladores REST, servicios de negocio y clientes HTTP para la API de SiAR.
<code>src/test/java/</code>	Pruebas unitarias e integración del servicio.
<code>pom.xml</code>	Descriptor Maven con dependencias, plugins y configuración del proyecto.
<code>dockerfile</code>	Imagen Docker basada en JDK para el despliegue contenerizado.

B.3.6. ITACyL

Repositorio: <https://github.com/agro-predict-tfg-2026/ITACyL>

Tecnología: Java (Spring Boot, Maven)

Servicio REST que actúa como intermediario frente a la API pública de ITACyL (Sativum) para la consulta del calendario de plagas y enfermedades asociadas a cultivos agrícolas. Permite filtrar los resultados por código de cultivo o por grupo de cultivos, y gestiona la autenticación mediante API Key. La respuesta se estandariza a través de un wrapper `ApiResponse` antes de ser consumida por el `data-service`.

Estructura — ITACyL

```

ITACyL/
├── ITACyL/
│   ├── src/main/java/
│   └── src/test/java/
├── README.md
└── LICENSE
  
```

Descripción de módulos

Directorio / Fichero	Descripción
<code>src/main/java/</code>	Controladores Spring Boot, servicio de consulta a ITACyL, cliente HTTP con autenticación por API Key y wrapper de respuesta <code>ApiResponse</code> .
<code>src/test/java/</code>	Pruebas unitarias de los componentes del servicio.
<code>README.md</code>	Documentación de instalación, endpoints disponibles y ejemplos de uso.

B.3.7. agro-predict-orchestator

Repositorio: <https://github.com/agro-predict-tfg-2026/agro-predict-orchestator>

Tecnología: Docker Compose

Repositorio de orquestación del sistema completo. Contiene las definiciones de *Docker Compose* necesarias para levantar el conjunto de microservicios de forma coordinada, tanto en entornos con soporte GPU como sin él. Es el punto de entrada recomendado para el despliegue integral de la plataforma en un entorno local o de producción.

Estructura — agro-predict-orchestator

```
agro-predict-orchestator/
├── docker/
├── no-gpu/
└── LICENSE
```

Descripción de módulos

Directorio	Descripción
docker/	Configuración de Docker Compose para entornos con soporte GPU (p. ej., para tareas de inferencia acelerada).
no-gpu/	Configuración alternativa de Docker Compose para entornos sin GPU, adecuada para la mayoría de despliegues en producción y desarrollo local.

Capítulo C:Manual de uso

El presente apéndice constituye una guía de referencia para usuarios finales de la infraestructura desarrollada. Se describen los requisitos previos a la utilización del software, el proceso de acceso, la navegación por las diferentes funcionalidades y la interpretación de los datos obtenidos por los predictores de plaga y heladas.

El manual está orientado a tres perfiles de usuarios finales:

- **Usuarios agrícolas:** Personal de campo y técnicos que consultan predicciones y visualizan datos agroclimáticos.
- **Aplicativo software:** Aplicativo o entidad software que abastecerá sus nuevas funcionalidades de su sistema empresarial con las funcionalidades incorporadas en la infraestructura desarrollada.
- **Usuario administrador:** Responsable de la configuración del sistema y del registro de plagas y cultivos en el sistema.

C.1. Requisitos previos de uso

Antes de poder probar el funcionamiento del software, es necesario mantener en ejecución en el equipo local los diferentes servicios que forman la infraestructura del sistema. En el Anexo B se detallan los pasos necesarios para su configuración y puesta en marcha.

Dado que este software desarrollado no dispone de una capa de presentación (interfaz gráfica) accesible desde el navegador, se recomienda utilizar una plataforma integral de desarrollo y pruebas de *APIs*, como **Postman** [101] o **Bruno** [102]. Estas herramientas proporcionan una interfaz gráfica desde la que realizar peticiones sobre los distintos puntos finales (*endpoints*) de los servicios descritos en este documento.

C.1.1. Servicios desplegados en máquina virtual

En el caso de que los contenedores de la infraestructura estén desplegados en una máquina virtual sin entorno gráfico, se recomienda seguir los pasos indicados a continuación para poder interactuar con los servicios desde el equipo local.

1. **Verificación e instalación de SSH:** Para establecer la comunicación con la máquina virtual es necesario disponer del cliente SSH en el equipo local. Para comprobar si ya se encuentra instalado, se ejecuta el siguiente comando en la terminal:

```
ssh -V
```

Si el comando devuelve la versión instalada (por ejemplo, `OpenSSH_9.0p1`), SSH ya está disponible y se puede pasar al siguiente paso. En caso contrario, su proceso de instalación según el sistema operativo del equipo local es el siguiente:

- **Linux (Ubuntu/Debian):** `sudo apt update && sudo apt install openssh-client`

- **Windows 11:** SSH está disponible como característica opcional del sistema. Para activarlo, se accede a *Configuración* → *Aplicaciones* → *Características opcionales* y se busca *Cliente OpenSSH*. Alternativamente, se puede instalar desde PowerShell con permisos de administrador:

```
PS>Add-WindowsCapability -Online -Name OpenSSH.Client 0.0.1.0
```

2. **Conexión entre el equipo local y la máquina virtual:** El objetivo de este paso es establecer un túnel SSH que redirija el tráfico de un puerto del equipo local hacia el puerto correspondiente del servicio desplegado en la máquina virtual. De este modo, las peticiones realizadas desde herramientas de prueba mencionadas anteriormente, serán enrutadas de forma transparente hacia la infraestructura remota.

Para ello es necesario conocer previamente:

- La **dirección IP** de la máquina virtual.
- El **puerto** en el que escucha el servicio con el que se desea interactuar.

Con esta información, ejecuta el siguiente comando desde la terminal del equipo local:

```
ssh ubuntu@51.44.230.198 -L 10000:localhost:10000
```

Los parámetros del comando tienen el siguiente significado:

- `ubuntu@51.44.230.198`: usuario e IP de la máquina virtual. Se sustituyen por los valores correspondientes al entorno remoto de trabajo.
- `-L 10000:localhost:10000`: indica que el puerto 10000 del equipo local quedará redirigido al puerto 10000 de la máquina virtual. Se deben modificar estos valores según el puerto del servicio con el que desee comunicar.

Una vez establecida la conexión, las peticiones dirigidas a `http://localhost:10000` en el equipo local serán enrutadas automáticamente hacia el servicio remoto.

C.2. Perfiles de usuario y casos de uso

A continuación se detallan las funcionalidades disponibles para cada uno de los tres perfiles de usuario identificados a usar el software desarrollado. La interacción con la infraestructura se realiza exclusivamente a través de peticiones HTTP sobre los *endpoints* documentados, configurando el entorno de pruebas con el túnel SSH descrito en la sección anterior, si es necesario.

C.2.1. Usuario agrícola

El perfil de usuario agrícola está orientado a personal de campo y técnicos de cooperativas que necesitan consultar predicciones de riesgo frente a heladas o plagas en sus cultivos. Estos usuarios consumen los resultados principalmente en formato **PDF**, listo para su impresión o envío.

Consulta de predicción de heladas

El servicio de predicción de heladas expone dos modalidades de consulta en función de la base temporal de los datos empleados (más información en Subsección 6.3.5).

Predicción basada en datos observados Este *endpoint* permite obtener una predicción de riesgo ante heladas para el día actual, procesando datos históricos. Se corresponde con el *endpoint* detallado en Sección 6.3.5.

Para realizar una consulta como usuario agrícola, deben seguirse los siguientes pasos:

1. En la herramienta de peticiones, crear una nueva solicitud de tipo **POST** hacia la URL base del servicio predictor de heladas, incluyendo la ruta `/heladas/observadas/`.
2. En los parámetros de la petición (*query params*), establecer:
 - **format=PDF**, para obtener el informe en formato descargable.
 - El resto de parámetros requeridos según el *endpoint* (**tipo**, **province**, **estacion**, **evaluacion**, y opcionalmente **variedades**).
3. Enviar la petición. El servicio devolverá un documento PDF con la predicción de riesgo de helada para el día actual en la zona geográfica especificada, incluyendo, si se solicitó, la evaluación sobre variedades concretas de cultivo.

Predicción basada en datos futuros Para predicciones de cara al día siguiente, se emplea el *endpoint* correspondiente a `/heladas/futuras/`, documentado en Sección 6.3.5.

El procedimiento es igual al anterior, con la diferencia de que el parámetro de ruta, `zona`, determina la especificación geográfica de la predicción (`provincial`, `ccaa` o `nacional`). Los parámetros `evaluacion_var` y `evaluacion_loc` controlan, respectivamente, si se desea evaluación sobre variedades y sobre localidades específicas.

Consulta de predicción de plagas

El servicio predictor de plagas ofrece dos variantes de predicción: calculada y estimada.

Predicción calculada anual Utiliza el *endpoint* `POST /plagas/calculadas` (Sección 6.3.5). Está pensado para obtener una predicción anual sobre grupos de cultivos específicos (cereales, leguminosas, etc.). El usuario debe indicar:

- `format=PDF`
- `cultivo`: nombre del cultivo a analizar.

El sistema devolverá un informe PDF con la evaluación anual del riesgo de plagas para el cultivo solicitado.

Predicción estimada temporal El *endpoint* `POST /plagas/estimadas` (Sección 6.3.5) permite acotar la predicción a un intervalo temporal concreto y a una parcela específica. Los parámetros requeridos son:

- `format=PDF`
- `cultivo`: nombre del cultivo.
- `fecha_inicio` y `fecha_fin`: intervalo de la predicción en formato `YYYY-MM-DD`.
- `datos_sensores`: lista de identificadores de sensores de los que se recopilarán datos climáticos.
- `parcela` (opcional): identificador de la parcela asociada.

C.2.2. Aplicativo software

Este perfil está destinado a sistemas empresariales o aplicaciones de terceros que deseen integrar las funcionalidades predictivas de **Agro-Predict** en sus propias soluciones. A diferencia del perfil agrícola, estos consumidores utilizan el formato de respuesta **JSON** (*application/json*), que permite el procesamiento automatizado de los resultados.

Formato de respuesta JSON

Cuando se especifica el parámetro `format=JSON` en cualquiera de los *endpoints* de predicción, el servicio devuelve una respuesta estructurada en formato *JavaScript Object Notation* (JSON). Este formato está diseñado para ser interpretado por aplicaciones software sin intervención humana directa.

La estructura genérica de respuesta incluye:

- `nivel_riesgo`: cadena que indica el nivel de alerta (`PREVENTIVA`, `CRITICA` o `SIN_RIESGO`).
- `mensaje`: texto descriptivo del resultado, o `null` si no aplica.
- `condiciones_cumplidas`: lista de cadenas con las condiciones climáticas que se han verificado y han contribuido al nivel de riesgo.
- `condiciones_pendientes`: lista de cadenas con las condiciones climáticas que no se han alcanzado.
- `recomendacion`: recomendación agronómica asociada al nivel de riesgo, o `null` si no aplica.

Los aplicativos software pueden consumir estos *endpoints* de forma análoga al usuario agrícola, modificando únicamente el valor del parámetro `format`.

C.2.3. Usuario administrador

El usuario administrador es el responsable de poblar el sistema con los datos de referencia necesarios para que los predictores puedan ofrecer resultados. Sus funciones principales son el registro de cultivos y el registro de plagas y enfermedades.

Registro de cultivos

Para que los servicios predictores puedan evaluar el riesgo sobre un determinado cultivo, éste debe estar previamente registrado en la base de datos. La inserción se realiza mediante el método `crear_cultivo`, cuyos campos requeridos son:

- **nombre:** nombre común del cultivo (por ejemplo, *Tomate*, *Cerezo*, *Trigo*).
- **nombre_cientifico:** denominación científica binomial (*Solanum lycopersicum*, *Prunus avium*, *Triticum aestivum*).
- **descripcion:** breve descripción agronómica del cultivo, incluyendo características relevantes para la predicción de riesgos.
- **grupo:** categoría agronómica a la que pertenece (por ejemplo, *hortícolas*, *frutales*, *cereales*, *leguminosas*).

Es responsabilidad del administrador mantener actualizado el catálogo de cultivos, ya que la ausencia de un cultivo en el sistema impedirá que los usuarios puedan solicitar predicciones sobre él.

Registro de plagas y enfermedades mediante DSL

Adicionalmente, se ha definido un **Lenguaje de Dominio Específico (DSL)** para el registro estructurado de plagas y enfermedades. Este DSL, documentado en profundidad en el repositorio del proyecto, establece la plantilla JSON que el administrador debe utilizar para dar de alta nuevas plagas en el sistema.

Estructura general del registro Cada registro de plaga o enfermedad se compone de un documento JSON con los siguientes campos obligatorios y opcionales:

- **Campos obligatorios:**
 - **public_id:** identificador público único con patrón `PLAGA-<CULTIVO>-<CÓDIGO>` (ejemplo: `PLAGA-CEREZO-01`).
 - **nombre:** denominación común de la plaga o enfermedad.
 - **agente_causante:** descripción del organismo causante.
 - **momento_critico:** descripción de la estación o condiciones climáticas en las que la plaga prospera.

- **tipo:** clasificación taxonómica. Valores admitidos: `acaros`, `nematodos`, `insectos`, `hongos`, `bacterias`, `virus`, `plagas`, `enfermedades`.
 - **recursos:** lista de variables climáticas requeridas para la predicción. Deben corresponderse exactamente con los nombres definidos en la tabla de recursos disponibles del DSL (véase Tabla C.1).
 - **algoritmo:** tipo de algoritmo que evaluará el riesgo. Valores admitidos: `por_defecto` (emplea el motor de predicción genérico del sistema) o `adhoc` (requiere especificar una URL externa en el campo `algoritmo_url`).
- **Campos opcionales:**
- **observaciones:** notas adicionales sobre la plaga.
 - **mas_info:** enlace web con información complementaria.
 - **condiciones_evaluables:** lista de condiciones diarias que, de cumplirse, elevan el riesgo a nivel PREVENTIVA. Cada condición se compone de un **tipo** (recurso climático), un **valor** umbral y un **operador** de comparación (`>=`, `<=`, `<`, `>`, `!=`, `==`).
 - **ventana_temporal:** lista de condiciones acumulativas o consecutivas que determinan el nivel de riesgo CRITICA. Soportan dos modos de evaluación:
 - **consecutivo:** requiere que las condiciones se mantengan durante un número determinado de días consecutivos.
 - **acumulacion_gdd:** evalúa la acumulación de grados-día (*Growing Degree Days*) sobre una temperatura base hasta alcanzar un objetivo en un periodo máximo de días.

Ejemplo de registro válido A continuación se muestra un ejemplo completo de documento JSON para el registro de la plaga *Drosophila suzukii* en cultivo de cerezo:

```

1 {
2   "public_id": "PLAGA-CEREZO-01",
3   "nombre": "Drosophila suzukii",
4   "agente_causante": "Mosca de alas manchadas de la familia
5                       Drosophilidae",
6   "momento_critico": "Pre-cosecha con fruta sensible en el arbol",
7   "observaciones": "Umbral termico inferior estimado en 4.7 grados",
8   "mas_info": "https://croplifela.org/es/plagas/...",
9   "tipo": "insecto",
10  "recursos": [
11    "temperatura_media", "temperatura_max",

```

```

12         "temperatura_min", "humedad_relativa"
13     ],
14     "condiciones_evaluables": [
15         { "tipo": "temperatura_media", "valor": 18, "operador": ">=" },
16         { "tipo": "humedad_relativa", "valor": 70, "operador": ">=" }
17     ],
18     "ventana_temporal": [
19         {
20             "modo": "acumulacion_gdd",
21             "temperatura_base": 4.7,
22             "gdd_objetivo": 934,
23             "dias_ventana": 365,
24             "nivel_si_cumple": "PREVENTIVA"
25         },
26         {
27             "modo": "consecutivo",
28             "dias_consecutivos_requeridos": 3,
29             "nivel_si_cumple": "CRITICA"
30         }
31     ],
32     "algoritmo": "por_defecto",
33     "algoritmo_url": null
34 }

```

Listado C.1: Ejemplo de registro DSL para *Drosophila suzukii* en cerezo.

Algoritmos ad-hoc Cuando una plaga requiere una lógica de predicción que no puede ser cubierta por el motor genérico, el campo **algoritmo** debe establecerse a **ad-hoc** y proporcionar en **algoritmo_url** la dirección del servicio externo encargado de la evaluación. El sistema enviará a dicha URL una petición HTTP POST con los datos climáticos del periodo de análisis, y esperará una respuesta JSON estructurada con el nivel de riesgo y las condiciones evaluadas. La especificación completa de los formatos de petición y respuesta se encuentra en la documentación técnica del DSL.

Consideraciones para el administrador

- Los nombres de recursos en el campo **recursos** deben coincidir exactamente con los definidos en la tabla de recursos disponibles del sistema (véase Tabla C.1). Cualquier discrepancia provocará

errores en la predicción.

Tabla C.1: Recursos disponibles

Identificador	Nombre	Descripción
1	temperatura_aire	Temperatura del aire ambiental (°C)
2	humedad_relativa	Humedad relativa del aire (%)
3	precipitacion	Precipitación acumulada (mm)
4	velocidad_viento	Velocidad del viento (m/s)
5	direccion_viento	Dirección del viento (°)
6	radiacion_solar	Radiación solar global (W/m ²)
7	presion_atmosferica	Presión atmosférica (hPa)
8	punto_rocio	Temperatura de punto de rocío (°C)
9	evapotranspiracion	Evapotranspiración de referencia ETo (mm/día)
10	horas_frio	Horas acumuladas por debajo del umbral térmico
11	grados_dia	Grados-día acumulados sobre un umbral base
12	humedad_suelo	Contenido volumétrico de agua en el suelo (%)
13	temperatura_suelo	Temperatura del suelo (°C)
14	ph_suelo	pH del suelo
15	humedad_hoja	Humedad superficial de la hoja (mojadura foliar)
16	ndvi	Índice de vegetación normalizado (salud del cultivo)
17	compuestos_volatiles	COVs emitidos por la planta (detección precoz de plagas)

- Las `condiciones_evaluables` y la `ventana_temporal` son complementarias. Las primeras definen el umbral de riesgo preventivo diario, y las segundas elevan el riesgo a crítico cuando se producen acumulaciones o recurrencias temporales.
- Es posible registrar múltiples ventanas temporales para una misma plaga o enfermedad, combinando modos `consecutivo` y `acumulacion_gdd` según la epidemiología de la misma (véase Listado C.2).

```

1      {
2          "public_id": "PLAGA-CEREZO-01",
3          "nombre": "Drosophila suzukii",
4          "agente_causante": "Mosca de alas manchadas de la familia
5          Drosophilidae",
6          "momento_critico": "Pre-cosecha con fruta sensible en el
7          arbol y temperaturas medias
8          entre 18-25C con alta humedad relativa",
9          "observaciones": "Umbral termico inferior de desarrollo
10         estimado en 4.7C.
11         Requiere 934 grados-dia para emergencia mediana.
12         Maximo riesgo reproductivo con
13         T media 18-25C y HR > 70-80%",
14         "mas_info": "https://croplifela.org/es/plagas/
15         listado-de-plagas/drosophila-suzukii",
16         "tipo": "insecto",
17         "recursos": ["temperatura_media", "temperatura_max",
18         "temperatura_min", "humedad_relativa"],
19         "condiciones_evaluables": [
20             { "tipo": "temperatura_media", "valor": 18,
21             "operador": ">=" },
22             { "tipo": "humedad_relativa", "valor": 70,
23             "operador": ">=" }
24         ],
25         "ventana_temporal": [
26             {
27                 "modo": "acumulacion_gdd",
28                 "temperatura_base": 4.7,
29                 "gdd_objetivo": 934,
30                 "dias_ventana": 365,
31                 "nivel_si_cumple": "PREVENTIVA"
32             },
33             {
34                 "modo": "consecutivo",
35                 "dias_consecutivos_requeridos": 3,
36                 "nivel_si_cumple": "CRITICA"
37             }
38         ],
39         "algoritmo" : "por_defecto",

```

```
40         "algoritmo_url" : null,  
41     }
```

Listado C.2: Múltiples ventanas temporales en el registro de plagas

- La calidad de las predicciones depende directamente de la precisión con que se definan estos parámetros. Se recomienda consolidar los valores con fuentes fitosanitarias actualizadas.

C.3. Flujo de trabajo completo

A modo de resumen, el flujo de trabajo típico del sistema es el siguiente:

1. El **usuario administrador** registra en el sistema los cultivos (mediante su punto final en el servicio de datos) y las plagas o enfermedades (mediante documentos JSON conformes al DSL).
2. Una vez poblada la base de datos con los nuevos registros, los **usuarios agrícolas** pueden solicitar predicciones a través de los *endpoints* de heladas o plagas, especificando **format=PDF** para obtener informes descargables.
3. Paralelamente, los **aplicativos software** pueden consumir los mismos *endpoints* con **format=JSON** para integrar los resultados en sus propios sistemas de información.

Este diseño permite que la infraestructura actúe como un sistema abierto, donde la base de crítica de los servicios predictores (cultivos y plagas) es mantenida por un perfil administrador especializado, mientras que los servicios de predicción son accesibles tanto para consumo personal como para integración sobre entornos software.

Bibliografía

- [1] Electrónica, M. Programador de riego profesional, agrícola | maher electrónica, 2025. Último acceso: 22-02-2026. URL: <https://www.maherelectronica.com/>.
- [2] Ahorrar energía y agua: Tecnologías ECO Solutions. Eco solutions, 2026. Último acceso: 22-02-2026. URL: <https://eco.es.eco/>.
- [3] Irristrat. Irristrat, s. f. Último acceso: 22-02-2026. URL: <https://irriestrat.com/>.
- [4] Verdecora Blog. Recuperar planta después de helada, 2019. Último acceso: 22-02-2026. URL: <https://verdecora.es/blog/wp-content/uploads/2019/01/recuperar-planta-despues-helada.jpg>.
- [5] Portillo, G. Helada negra: qué es, temperatura y consecuencias, 2024. Último acceso: 22-02-2026. URL: <https://ecologiaverde.elperiodico.com/helada-negra-que-es-temperatura-y-consecuencias-4882.html>.
- [6] Entidad Estatal de Seguros Agrarios (ENESA). Plan de seguros agrarios y subvenciones, 2026. Consultado el 4 de mayo de 2026. URL: https://www.mapa.gob.es/es/enesa/plan_de_seguros_agrarios_y_subvenciones/.
- [7] Agroseguro. Informe anual 2025. Technical report, Agroseguro, Madrid, España, 2026. Consultado el 4 de mayo de 2026. URL: https://agroseguro.es/wp-content/uploads/2026/03/Informe-anual-2025_.pdf.
- [8] Extremadura Agro. Extremadura supone en torno al 5% del valor añadido bruto en el sector agrario en España, 2025. Consultado el 4 de mayo de 2026. URL: <https://extremaduraagro.com/extremadura-supone-en-torno-al-5-del-valor-anadido-bruto-en-el-sector-agrario-en-espana/>.
- [9] CEIGRAM - Universidad Politécnica de Madrid. Estrategias de gestión de riesgos agrarios y cambio climático, 2025. Consultado el 4 de mayo de 2026. URL: <https://ceigram.upm.es/estrategias-de-gestion-riesgos-agrarios-y-cambio-climatico/>.
- [10] Ariel Ortiz-Bobea, Toby R. Ault, Carlos M. Carrillo, Robert G. Chambers, and David B. Lobell. Anthropogenic climate change has slowed global agricultural productivity growth. *Nature Climate Change*, 8(4):306–312, 2018. Consultado el 4 de mayo de 2026. URL: <https://www.sciencedirect.com/science/article/pii/S1474667017569542>, doi:10.1038/s41558-018-0089-9.

- [11] TID4AGRO Consortium. Tid4agro: Tecnologías avanzadas, innovadoras y digitales para el sector agroalimentario de la euroace, 2026. Consultado el 4 de mayo de 2026. URL: <https://tid4agro.eu/>.
- [12] Amazon Web Services (AWS). ¿qué es lorawan?, 2026. Consultado el 4 de mayo de 2026. URL: https://docs.aws.amazon.com/es_es/iot-wireless/latest/developerguide/what-is-lorawan.html.
- [13] Atlassian. Microservices architecture, 2026. Accedido: 5 de mayo de 2026. URL: <https://www.atlassian.com/es/microservices/microservices-architecture>.
- [14] Danh Nguyen-Cong and De Tran-Cao. A review of effort estimation studies in agile, iterative and incremental software development. In *2013 RIVF International Conference on Computing and Communication Technologies – Research, Innovation, and Vision for Future (RIVF)*, pages 27–30. IEEE, 2013. doi:10.1109/RIVF.2013.6719861.
- [15] Euroregión EUROACE. ¿qué es la eurorregión euroace?, 2023. Accedido: 5 de mayo de 2026. URL: <https://www.euro-ace.eu/que-es>.
- [16] M. Priestley and M. H. Utt. A unified process for software and documentation development. In *Proceedings of the 18th Annual Conference on Computer Documentation (IPCC/SIGDOC 2000). Technology and Teamwork*, pages 221–238. IEEE, 2000. doi:10.1109/IPCC.2000.887279.
- [17] Richard Helm. Rational unified process, 2003. Recurso docente en línea. Accedido a través de Wayback Machine: https://web.archive.org/web/*/http://sce.uhcl.edu/helm/RationalUnifiedProcess/. URL: <http://sce.uhcl.edu/helm/RationalUnifiedProcess/>.
- [18] Softonic. Projectlibre para windows, 2025. Consultado el 4 de mayo de 2026. URL: <https://projectlibre.softonic.com/>.
- [19] Raman Kumar, Harpreet Kaur Channi, Santosh Kumar Srivastava, Durgesh Srivastava, Kahraman Cengiz, and Pankaj Gaur. Data analytics in agriculture: Predictive models and real-time decision-making. In *Smart Agritech: Robotics, AI, and Internet of Things (IoT) in Agriculture*, pages 169–200. Scrivener Publishing LLC (Wiley), 2024. Consultado el 4 de mayo de 2026. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781394302994.ch7>, doi:10.1002/9781394302994.ch7.
- [20] AgriSmart Data. idatacloud: plataforma cloud para la gestión de datos en agricultura de precisión, 2026. Consultado el 4 de mayo de 2026. URL: <https://agrismartdata.com/idatacloud/>.

- [21] Zeek GPS. Sensores en agricultura de precisión: qué son y sus funciones, 2025. Consultado el 4 de mayo de 2026. URL: <https://zeekgps.com/sensores-agricultura-de-precision-que-son-y-sus-funciones/>.
- [22] EOS Data Analytics. Índice de vegetación de diferencia normalizada (ndvi): qué es y cómo funciona, 2025. Consultado el 4 de mayo de 2026. URL: <https://eos.com/es/blog/indice-de-vegetacion-de-diferencia-normalizada-o-ndvi/>.
- [23] Arable Labs. Una guía para los grados día de crecimiento (gdd): vinculando la temperatura y las etapas de crecimiento de los cultivos, 2023. Consultado el 4 de mayo de 2026. URL: <https://www.arable.com/blog/una-guia-para-los-grados-dia-de-crecimiento-gdd-vinculando-la-temperatura-y-las-etapas-de-crecimiento-de-los-cultivos/>.
- [24] Seipasa. Estrés hídrico en las plantas: soluciones para reducir su impacto en la producción agrícola, 2023. Consultado el 4 de mayo de 2026. URL: <https://www.seipasa.com/es/blog/estres-hidrico-en-las-plantas/>.
- [25] Comarca Agrícola. Comarca agrícola 04, 2024. Consultado el 4 de mayo de 2026. URL: <https://comarcaagricola.com/comarca-agricola-04/>.
- [26] Fertilagro de Colombia SAS. Etapas fenológicas de las plantas, 2025. Consultado el 4 de mayo de 2026. URL: <https://www.fertilagro.com.co/etapas-fenologicas>.
- [27] ISAGRI. Tipos de heladas en agricultura: aprende a identificarlas, 2025. Accedido: 5 de mayo de 2026. URL: <https://www.isagri.es/blog/tipos-de-heladas-en-agricultura>.
- [28] European Environment Agency (EEA). Use of adapted crops and varieties, 2019. Consultado el 4 de mayo de 2026. URL: <https://climate-adapt.eea.europa.eu/es/metadata/adaptation-options/use-of-adapted-crops-and-varieties>.
- [29] European Environment Agency (EEA). Crop diversification and improved soil management for adaptation to climate change in segovia (spain), 2019. Consultado el 4 de mayo de 2026. URL: <https://climate-adapt.eea.europa.eu/es/metadata/case-studies/crop-diversification-and-improved-soil-management-for-adaptation-to-climate-change-in-segovia-spain>.
- [30] Plataforma Tierra. Cubiertas vegetales: estrategia para mejorar la fertilidad, el secuestro de carbono de los suelos y el control de plagas, 2020. Consultado el 4 de mayo de 2026. URL: <https://www.plataformatierra.es/innovacion/cubiertas-vegetales-estrategia-mejorar-fertilidad-secuestro-carbono-suelos-control-plagas>.
- [31] J. E. Chaar. Resistencia a heladas en plantas frutales. *Avances en Investigación Agropecuaria*, 17(3):109–121, 2013. URL: <https://revistasacademicas.ucol.mx/index.php/agropecuaria/article/view/425/316>.

- [32] Pessl Instruments. imetos vws (virtual weather station), 2026. Consultado el 4 de mayo de 2026. URL: <https://metos.global/es/imetos-vws/>.
- [33] IBM. ¿qué es random forest?, 2026. Consultado el 4 de mayo de 2026. URL: <https://www.ibm.com/es-es/think/topics/random-forest>.
- [34] William C. Skamarock, Joseph B. Klemp, Jimy Dudhia, David O. Gill, Dale M. Barker, Wei Wang, and Jordan G. Powers. A description of the advanced research wrf version 3. Technical report, National Center for Atmospheric Research (NCAR), 2008. Documento técnico NCAR; consultado el 4 de mayo de 2026. URL: https://www.researchgate.net/publication/242446432_Prototypes_for_the_WRF_Weather_Research_and_Forecasting_model.
- [35] William C. Skamarock, Joseph B. Klemp, and Jimy Dudhia. Prototypes for the wrf (weather research and forecasting) model. Technical report, National Center for Atmospheric Research (NCAR), 2001. Presentado en la 18th Conference on Weather Analysis and Forecasting, 14th Conference on Numerical Weather Prediction and 9th Conference on Mesoscale Processes; consultado el 4 de mayo de 2026. URL: https://www.researchgate.net/profile/Jimy-Dudhia/publication/242446432_Prototypes_for_the_WRF_Weather_Research_and_Forecasting_model/links/0c9605314d71c38bae000000/Prototypes-for-the-WRF-Weather-Research-and-Forecasting-model.pdf.
- [36] National Center for Atmospheric Research (NCAR). Wrf visualization using vapor, 2023. Accedido: 5 de mayo de 2026. URL: <https://www.ncl.ucar.edu/Applications/wrfvapor.shtml>.
- [37] William C. Skamarock, Joseph B. Klemp, and Jimy Dudhia. Prototypes for the wrf (weather research and forecasting) model. Technical report, National Center for Atmospheric Research (NCAR), 2001. Presentado en la 18th Conference on Weather Analysis and Forecasting and 14th Conference on Numerical Weather Prediction; consultado el 4 de mayo de 2026. URL: https://www.researchgate.net/profile/Jimy-Dudhia/publication/242446432_Prototypes_for_the_WRF_Weather_Research_and_Forecasting_model/links/0c9605314d71c38bae000000/Prototypes-for-the-WRF-Weather-Research-and-Forecasting-model.pdf.
- [38] Hidrosoph. Irristrat: sistema inteligente de gestión del riego, 2026. Consultado el 4 de mayo de 2026. URL: <https://web.hidrosoph.com/ES/irriistrat.html>.
- [39] Jornada Cultiva. Hidrosoph sigue liderando la gestión del riego: 230.000 hectáreas ya utilizan su tecnología inteligente irriistrat, 2023. Consultado el 4 de mayo de 2026. URL: <https://jornadacultiva.com/hidrosoph-sigue-liderando-la-gestion-del-riego-230-000-hectareas-ya-utilizan-su-tecnologia-inteligente-irriistrat/>.
- [40] OpenSprinkler. Opensprinkler: smart irrigation controller, 2026. Consultado el 4 de mayo de 2026. URL: <https://opensprinkler.com/>.

- [41] OpenSprinkler. Opensprinkler github repository, 2026. Consultado el 4 de mayo de 2026. URL: <https://github.com/OpenSprinkler>.
- [42] Celery Project. Introduction to celery. <https://docs.celeryq.dev/en/stable/getting-started/introduction.html>, 2026. Accedido: 25-may-2026.
- [43] Arsys. Rabbitmq, qué es y cuáles son sus funcionalidades, 2024. Consultado el 4 de mayo de 2026. URL: <https://www.arsys.es/blog/rabbitmq-mensajeria>.
- [44] Aprendiendo Arquitectura Software. Patrón broker, 2018. Consultado el 4 de mayo de 2026. URL: <https://aprendearquitecturasoftware.wordpress.com/2018/10/08/grupo-4-patron-broker/>.
- [45] IBM. ¿qué es el model context protocol (mcp)?, 2026. Consultado el 4 de mayo de 2026. URL: <https://www.ibm.com/es-es/think/topics/model-context-protocol>.
- [46] Miro. Miro: Collaborative online whiteboard platform, 2026. Consultado el 4 de mayo de 2026. URL: <https://miro.com/es/>.
- [47] Mermaid. Mermaid: Diagrams and charts from text, 2026. Consultado el 4 de mayo de 2026. URL: <https://mermaid.js.org/>.
- [48] PlantUML. Plantuml: Open-source tool to create uml diagrams from plain text, 2026. Consultado el 4 de mayo de 2026. URL: <https://plantuml.com/es/>.
- [49] Stan Smith. Fossflow - documentación (readme en español), 2026. Consultado el 4 de mayo de 2026. URL: <https://github.com/stan-smith/FossFLOW/blob/master/docs/README.es.md>.
- [50] ERAIchemy. Eralchemy: Entity relation diagrams generation tool, 2026. Consultado el 4 de mayo de 2026. URL: <https://github.com/eralchemy/eralchemy>.
- [51] Arsys. Guía completa para el diseño de restful api: conceptos y mejores prácticas, 2024. Consultado el 4 de mayo de 2026. URL: <https://www.arsys.es/blog/guia-completa-para-el-diseno-de-restful-api-conceptos-y-mejores-practicas>.
- [52] VMware, Inc. Spring boot 3.5.6 available now, 2025. Accedido: 5 de mayo de 2026. URL: <https://spring.io/blog/2025/09/18/spring-boot-3-5-6-available-now>.
- [53] Elixir Core Team. Elixir programming language, 2026. Accedido: 5 de mayo de 2026. URL: <https://elixir-lang.org/>.
- [54] Marcelo Andrader. Elixir y la máquina virtual de erlang, 2021. Accedido: 5 de mayo de 2026. URL: <https://marceloandrader.github.io/blog/2021/10/01/elixir-y-la-m%C3%A1quina-virtual-de-erlang/>.

- [55] Elixir School. Mix, 2024. Accedido: 5 de mayo de 2026. URL: <https://elixirschool.com/es/lessons/basics/mix>.
- [56] Phoenix Framework Community. Phoenix framework, 2026. Accedido: 5 de mayo de 2026. URL: <https://www.phoenixframework.org/>.
- [57] MariaDB Foundation. Mariadb 10.11.5 release notes (community server), 2023. Consultado el 4 de mayo de 2026. URL: <https://mariadb.com/docs/release-notes/community-server/10.11/10.11.5>.
- [58] Pallets Projects. Flask documentation, 2026. Consultado el 4 de mayo de 2026. URL: <https://flask-es.readthedocs.io/>.
- [59] Phillip J. Eby. PEP 3333 – Python Web Server Gateway Interface v1.0.1. Python Enhancement Proposal 3333, Python Software Foundation, 2010. Status: Final. Informational PEP. Replaces PEP 333. URL: <https://peps.python.org/pep-3333/>.
- [60] ReportLab. Reportlab documentation (unofficial readthedocs), 2026. Consultado el 4 de mayo de 2026. URL: <https://reportlab-docs-inofficial.readthedocs.io/en/stable/>.
- [61] pyHanko Developers. pyhanko documentation, 2026. Consultado el 4 de mayo de 2026. URL: <https://docs.pyhanko.eu/en/latest/>.
- [62] Aitor Cuartango. Firma digital de un archivo pdf desde línea de comandos, 2022. Consultado el 4 de mayo de 2026. URL: <https://gist.github.com/acuartango/4ce393c37072df11fe1bdbc38ee9960f>.
- [63] SQLAlchemy Project. Sqlalchemy orm documentation (2.0), 2026. Consultado el 4 de mayo de 2026. URL: <https://docs.sqlalchemy.org/en/20/orm/>.
- [64] SQLAlchemy Project. Alembic documentation, 2026. Consultado el 4 de mayo de 2026. URL: <https://alembic.sqlalchemy.org/en/latest/>.
- [65] Diego Coder. ¿qué es un broker de mensajería?, 2023. Accedido: 7 de mayo de 2026. URL: <https://medium.com/diego.coder/que-es-un-broker-de-mensajer%C3%ADa-8aa8ab7988e8>.
- [66] Prefect, Inc. Welcome to fastmcp, 2026. Accedido: 5 de mayo de 2026. URL: <https://gofastmcp.com/getting-started/welcome>.
- [67] Ollama. Qwen2.5 model library, 2026. Accedido: 5 de mayo de 2026. URL: <https://ollama.com/library/qwen2.5>.
- [68] Ollama, Inc. Ollama api: Introduction, 2026. Accedido: 5 de mayo de 2026. URL: <https://docs.ollama.com/api/introduction>.

- [69] Keycloak. Keycloak documentation, 2026. Consultado el 4 de mayo de 2026. URL: <https://www.keycloak.org/documentation>.
- [70] IBM. What is redis explained? <https://www.ibm.com/think/topics/redis>, October 2021. Accedido: 25-may-2026.
- [71] Miriam Martínez Canelo. Celery: qué es, cómo funciona y mejores prácticas. <https://profile.es/blog/que-es-celery/>, June 2022. Accedido: 24-may-2026.
- [72] La Última Neurona. Arquitectura de n capas, 2023. Consultado el 4 de mayo de 2026. URL: <https://laultimaneurona.com/posts/arquitectura-de-n-capas/>.
- [73] Ministerio de Agricultura, Pesca y Alimentación (MAPA). Manual técnico de la web api del siar. Technical report, Gobierno de España, 2025. Accedido: 5 de mayo de 2026. URL: https://servicio.mapa.gob.es/siarweb/documentos/manuales/ManualTecnico_WEB_API_SIAR.pdf.
- [74] Agencia Estatal de Meteorología (AEMET). Aemet opendata api - introducción, 2026. Accedido: 5 de mayo de 2026. URL: <https://opendata.aemet.es/dist/index.html>.
- [75] Instituto Tecnológico Agrario de Castilla y León (ITACyL). Portal de apis del siar - sistema de información agroclimática para el regadío, 2026. Documentación de autenticación y uso de la API; consultado el 4 de mayo de 2026. URL: <https://portal.api.itacyl.es/portal/apis/69a926a658a57400012f09fd/documentation/#auth>.
- [76] Celery Project. Workers guide. <https://docs.celeryq.dev/en/main/userguide/workers.html>, 2026. Accedido: 25-may-2026.
- [77] Celery Project. Periodic tasks (celery 4.0). <https://docs.celeryq.dev/en/4.0/userguide/periodic-tasks.html>, 2016. Accedido: 25-may-2026.
- [78] Josiah Carlson. crontab: Parse and use crontab schedules in python. <https://pypi.org/project/crontab/>, July 2025. Accedido: 25-may-2026.
- [79] Agro360. Modelos de acumulación de horas frío: un análisis descriptivo y comparativo, 2025. Consultado el 4 de mayo de 2026. URL: <https://www.agro360.cl/post/modelos-de-acumulaci%C3%B3n-de-horas-fr%C3%ADo-un-an%C3%A1lisis-descriptivo-y-comparativo>.
- [80] E. A. Damario and R. Rodríguez. Métodos para estimar valores agroclimáticos de "unidades de frío". *Revista de la Facultad de Agronomía*, 12(3):253–263, 1991. URL: <https://ri.agro.uba.ar/files/download/revista/facultadagronomia/1991damarioea2.pdf>.
- [81] Pablo Barrales Cartes. Patrones de diseño: Circuit breaker, 2022. Accedido: 5 de mayo de 2026. URL: <https://medium.com/@psbarrales/patrones-de-dise%C3%B1o-circuit-breaker-5b87194a5723>.

- [82] Álvaro Mendo. Agro predict tfg 2026. <https://github.com/agro-predict-tfg-2026>, 2026. Repositorio de código fuente. Accedido: 5 de mayo de 2026.
- [83] Amazon Web Services. ¿qué es la infraestructura como código? - explicación de iac. <https://aws.amazon.com/es/what-is/iac/>, 2026. Explicación del proceso de infraestructura como código. Accedido: 24-may-2026.
- [84] HashiCorp. Terraform: Automate infrastructure on any cloud. <https://developer.hashicorp.com/terraform>, 2026. Lenguaje de configuración en Terraform. Accedido: 24-may-2026.
- [85] David Martínez Bonaque. Los beneficios de utilizar terraform. <https://www.deloitte.com/es/es/services/consulting/blogs/todo-tecnologia/los-beneficios-de-utilizar-terraform.html>, June 2021. Accedido: 24-may-2026.
- [86] HashiCorp. Configuration syntax - terraform language. <https://developer.hashicorp.com/terraform/language/syntax/configuration>, November 2025. Accedido: 25-may-2026.
- [87] Microsoft. Servicios de informática en la nube | microsoft azure. <https://azure.microsoft.com/es-es>, 2026. Accedido: 25-may-2026.
- [88] Microsoft. Conceptos básicos de azure key vault. <https://learn.microsoft.com/es-es/azure/key-vault/general/basic-concepts>, April 2026. Accedido: 25-may-2026.
- [89] Microsoft. Introducción a azure blob storage. <https://learn.microsoft.com/es-es/azure/storage/blobs/storage-blobs-introduction>, May 2026. Accedido: 25-may-2026.
- [90] Docker, Inc. Install docker engine, 2026. Accedido: 5 de mayo de 2026. URL: <https://docs.docker.com/engine/install/>.
- [91] OpenAI. Chatgpt (gpt-5.5). <https://chatgpt.com/>, 2026. Modelo de lenguaje utilizado para asistencia técnica y redacción. Accedido: 7 de mayo de 2026.
- [92] Anthropic. Claude (claude-sonnet-4-6). Herramienta de inteligencia artificial en línea, 2025. Modelo de lenguaje de gran escala. Accedido durante el desarrollo del proyecto en 2025. URL: <https://www.anthropic.com>.
- [93] Shinya Yamada. Repomix: Pack your codebase into ai-friendly formats. Sitio web oficial, 2025. Consultado: 21 de mayo de 2026. [En línea]. Disponible: <https://repomix.com/>.
- [94] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with CLIP latents. *arXiv preprint arXiv:2204.06125*, abs/2204.06125, abril 2022. URL: <https://arxiv.org/abs/2204.06125>, [arXiv:2204.06125](https://arxiv.org/abs/2204.06125).

- [95] Qodo. Qodo: Plataforma de inteligencia artificial para revisión, prueba y mejora de código. <https://www.qodo.ai/>, 2026. Accedido: 22-may-2026. Fundada en 2022 por Itamar Friedman y Dedy Kredo. Información complementaria obtenida de AWS Startups.
- [96] Amazon Web Services. Qodo ayuda a los desarrolladores a eliminar el código incorrecto con herramientas de ia que ahorran tiempo. <https://aws.amazon.com/startups/learn/qodo-helps-developers-banish-bad-code-with-ai-tools?lang=es>, 2026. AWS Startups. Accedido: 22-may-2026.
- [97] Qodo. Qodo ci example: Ejemplo de integración de qodo cover en github actions para generación automática de pruebas. <https://github.com/qodo-ai/qodo-ci-example>, 2026. Repositorio oficial de GitHub. Accedido: 22-may-2026.
- [98] Docker, Inc. Install docker desktop on windows, 2026. Accedido: 5 de mayo de 2026. URL: <https://docs.docker.com/desktop/setup/install/windows-install/>.
- [99] Git SCM Community. Install git on windows, 2026. Accedido: 5 de mayo de 2026. URL: <https://git-scm.com/install/windows>.
- [100] Git SCM Community. Install git on linux, 2026. Accedido: 5 de mayo de 2026. URL: <https://git-scm.com/install/linux>.
- [101] Postman, Inc. Postman api platform, 2026. Accedido: 5 de mayo de 2026. URL: <https://www.postman.com/>.
- [102] Bruno (UseBruno Project). Bruno - open source api client, 2026. Accedido: 5 de mayo de 2026. URL: <https://www.usebruno.com/>.